

Masterthesis

Untersuchung des Einsatzes klassischer und KI- basierter Verfahren zur Datenqualitätssicherung anhand eines prototypischen Use Cases in Microsoft Fabric

von

Elijah Andreas Stauß

zur Erlangung des akademischen Grades

Master of Science

an der Hochschule Konstanz für Technik, Wirtschaft und Gestaltung.

Name und Matrikel-Nr.

Elijah Andreas Stauß (312758)

Bearbeitungszeitraum

01.10.2025 - 31.03.2026

Kooperationspartner

Geberit Verwaltungs GmbH

Erstprüfer

Prof. Dr. Markus Eiglsperger

Zweitprüfer

Manfred Bantle

Tag der Ausgabe

1. Oktober 2025

Tag der Abgabe

31. März 2026

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich,

Elijah Andreas Stauß, geboren am 19.09.2003 in Überlingen,

die vorliegende Abschlussarbeit mit dem Titel "Untersuchung des Einsatzes klassischer und KI-basierter Verfahren zur Datenqualitätssicherung anhand eines prototypischen Use Cases in Microsoft Fabric" betreut von Prof. Dr. Markus Eiglsperger, selbständig und ohne unerlaubte Hilfe Dritter erbracht und noch keiner anderen Stelle zur Prüfung vorgelegt habe.

Ich habe ausschließlich die mit meinem Erstbetreuer vereinbarten KI-Werkzeuge eingesetzt und dies sowie den Umfang der Verwendung schriftlich dokumentiert.

Im Literaturverzeichnis sind alle verwendeten Quellen angegeben und ich habe keine anderen als die dort aufgelisteten Quellen und Hilfsmittel benutzt. Die Stellen der Arbeit (einschließlich Tabellen, Karten, Abbildungen etc.), die anderen Werken und Quellen (auch Internetquellen) dem Wortlaut oder dem Sinn nach entnommen sind, habe ich in jedem einzelnen Fall als Entlehnung mit exakter Quellenangabe kenntlich gemacht.

Mir ist bewusst, dass Täuschungen bzw. schon deren Versuch nach der für mich gültigen Studien- und Prüfungsordnung sanktioniert werden. Diese Sanktionen können neben dem Nichtbestehen der Prüfungsleistung weitreichende Folgen bis hin zur Ungültigkeit der Masterprüfung für mich haben.

Pfullendorf, 31. März 2026



Abstract

Die Qualität von Daten entwickelt sich für Unternehmen zunehmend zu einem kritischen Erfolgsfaktor, insbesondere im Kontext KI-basierter Anwendungen, die hohe Anforderungen an ihre zugrundeliegenden Daten stellen. Moderne Datenplattformen ermöglichen es, große Mengen strukturierter und unstrukturierter Daten aus unterschiedlichen Quellsystemen zentral zu speichern, zu verarbeiten und bereitzustellen, und integrieren dabei fortgeschrittene Technologien direkt in ihre Architektur.

Diese Arbeit untersucht in Zusammenarbeit mit Geberit, einem europäischen Marktführer im Bereich der Sanitärtechnik, wie Datenqualität innerhalb der Datenplattform Microsoft Fabric sichergestellt werden kann. Der Fokus liegt dabei auf der Identifikation geeigneter klassischer sowie KI-basierter Verfahren und deren Integration in die Plattform.

Auf Basis einer systematischen Literaturanalyse wird ein Bewertungsraster entwickelt, das eine strukturierte Einordnung von Methoden entlang datenqualitätsbezogener und architektonischer Anforderungen ermöglicht. Darauf aufbauend wird ein Methodenkatalog erstellt, aus dem ausgewählte Verfahren prototypisch für einen Use Case in Microsoft Fabric unter Verwendung von CRM-Daten implementiert und hinsichtlich ihrer praktischen Anwendbarkeit im Unternehmenskontext evaluiert.

Inhaltsverzeichnis

Abbildungsverzeichnis	VI
Tabellenverzeichnis	VII
Abkürzungsverzeichnis	VIII
1. Einleitung	1
1.1. Motivation	1
1.2. Forschungslücke	1
1.3. Zielsetzung	3
1.4. Aufbau der Arbeit	4
2. Theoretische Grundlagen	5
2.1. Entwicklung zu modernen Datenplattformen	5
2.2. Daten und Datenqualität	7
2.3. Typische Datenqualitätsprobleme und der Einfluss moderner Technologien . .	10
2.4. Architektur und Komponenten von Microsoft Fabric	16
3. Methodenkatalog zur Sicherstellung der Datenqualität	22
3.1. Forschungsdesign und Vorgehensweise	22
3.2. Kategorisierung und Gegenüberstellung der identifizierten Verfahren	24
3.3. Ableitung von Bewertungskriterien aus der Theorie	34
3.4. Entwicklung des Bewertungsrasters	41
3.5. Anwendung des Bewertungsrasters auf die Methodencluster	44
4. Prototypische Implementierung im Microsoft Fabric Use Case	56
4.1. Unternehmensspezifische Implementierung	56
4.2. Auswahl repräsentativer Methoden und Architekturweiterung	59
4.3. Festlegen von Metriken je Kategorie	62
4.4. Erhebung der Baseline	64
4.5. Deterministische und regelbasierte Verfahren zur Datenqualitätsprüfung	71
4.6. Intelligente und datengetriebene Verfahren zur Qualitätsbewertung	75
5. Evaluation	80
5.1. Bewertung der erweiterten Datenarchitektur	80
5.2. Ermittlung des initialen Datenqualitätszustands	82
5.3. Analyse der implementierten DQ-Komponenten für Bronze & Silver	84
5.4. Evaluation der experimentellen Untersuchung der intelligenten Verfahren	91
6. Schlussbetrachtung	101

6.1. Zusammenfassung der Ergebnisse	101
6.2. Kritische Würdigung und Reflexion	104
6.3. Ausblick und Handlungsempfehlungen	106
Literaturverzeichnis	109
A. Methodencluster	122
B. Bewertungsmatrix	125
C. Tabellenauflistung	127
D. Strukturanalyse der Produktiv-Tabellen	129
E. Ergebnisse Baseline Erhebung	135
F. Agent-Anweisungen	146
G. Data Agent Evaluation	147
H. LLM-Evaluation	162
I. Promptsammlung	174

Abbildungsverzeichnis

Abb. 1	Microsoft Fabric Architecture [49]	16
Abb. 2	Aktuelle Fabric Implementierung (eigene Darstellung)	57
Abb. 3	Datenworkflow im Workspace (eigene Darstellung)	58
Abb. 4	Use Case Erweiterung des CRM-Ladeprozesses (eigene Darstellung) . . .	61
Abb. 5	Konzeptionelle DQ-Zone Architektur (eigene Darstellung)	62
Abb. 6	Datenstruktur im DQ-Zone-Lakehouse für die Implementierung (eigene Darstellung)	64
Abb. 7	Semantisches Modell des Customer Data Objects (eigene Darstellung) . . .	78
Abb. 8	Auszug Ergebnisse Table Profiling (eigene Darstellung)	85
Abb. 9	Beispiel Schema Comparison nicht genutzte Spalten (eigene Darstellung) .	85
Abb. 10	Ausschnitt Ergebnisse Column Profiling (eigene Darstellung)	86
Abb. 11	Ausschnitt der Regelprüfungen im Bronze (eigene Darstellung)	87
Abb. 12	Beispiele Generic Silver Profiling (eigene Darstellung)	88
Abb. 13	Beispiele Generic Silver Records (eigene Darstellung)	89
Abb. 14	Beispiele Mail Violation (eigene Darstellung)	90
Abb. 15	Beispiele Phone Violation (eigene Darstellung)	90
Abb. 16	Beispiele falsch klassifizierte Anomalie (eigene Darstellung)	93
Abb. 17	Ergebnis des Profiling-Prompts an den Data Agent (eigene Darstellung) . .	94
Abb. 18	Ergebnis des Prompts zur Identifikation von Testdaten an den Data Agent (eigene Darstellung)	96

Tabellenverzeichnis

Tab. 1	Single-Source-DQ-Probleme [40]	11
Tab. 2	Multi-Source-DQ-Probleme [40]	13
Tab. 3	Taxonomie der Data Quality Dimensionen [48]	15
Tab. 4	Überblick über die Suchstrategie zu DQ-Methoden	23
Tab. 5	Ableitung der DQ-Dimensionen zur Methodenbewertung	35
Tab. 6	Ableitung der Problemtypen zur Methodenbewertung (eigene Darstellung)	37
Tab. 7	Ableitung der architekturbezogenen Anforderungen zur Methodenbewertung (eigene Darstellung)	39
Tab. 8	Bewertungsraster (eigene Darstellung)	43
Tab. 9	Auszug aus der Bewertungsmatrix (eigene Darstellung, Tab. B)	44
Tab. 10	Metriken aus dem Systematic Review (eigene Darstellung)	63
Tab. 11	Struktur Systemlandschaften (eigene Darstellung)	65
Tab. A	Methodencluster (eigene Darstellung)	122
Tab. B	Bewertungsmatrix (eigene Darstellung)	125
Tab. C	Tabellenauflistung (eigene Darstellung)	127
Tab. D	Strukturanalyse der Produktiv-Tabellen (eigene Darstellung)	129
Tab. E	Baseline Vollständigkeit & Datenmenge (eigene Darstellung)	135
Tab. F	Baseline Konsistenz & Integrität (eigene Darstellung)	139
Tab. G	Baseline Korrektheit & Genauigkeit (eigene Darstellung)	141
Tab. H	Baseline Relevanz & Kontextbezug (eigene Darstellung)	142
Tab. I	Baseline Interpretierbarkeit & Verständlichkeit (eigene Darstellung)	144

Abkürzungsverzeichnis

ACID:	Atomicity, Consistency, Isolation, Durability
API:	Application Programming Interface
CDO:	Customer Data Object
CEP:	Complex Event Processing
CRM:	Customer Relationship Management
DQ:	Datenqualität
DQM:	Data Quality Management
DWH:	Data Warehouse
ELT:	Extract Load Transform
ETL:	Extract Transform Load
FD:	Functional Dependency
IF:	Isolation Forest
IoT:	Internet of Things
IQR:	Inter Quartile Range
KI:	Künstliche Intelligenz
LLM:	Large Language Model
LOF:	Local Outlier Factor
ML:	Machine Learning
PCA:	Principal Component Analysis
SaaS:	Software as a Service
SVM:	Support Vector Machine
TDQM:	Total Data Quality Management

1. Einleitung

1.1. Motivation

In der heutigen datengetriebenen Geschäftswelt entwickelt sich die Qualität der Daten zunehmend zu einem entscheidenden Faktor für den Geschäftserfolg. Unternehmen sind auf präzise, konsistente und vertrauenswürdige Daten angewiesen, um fundierte Entscheidungen zu treffen, betriebliche Effizienz zu steigern und Wettbewerbsvorteile zu realisieren. Die Sicherstellung einer hohen Datenqualität (DQ) entlang der gesamten Datenverarbeitungskette stellt eine komplexe Herausforderung dar. Klassische regelbasierte Verfahren stoßen angesichts wachsender Datenmengen und zunehmender Komplexität an ihre Grenzen, weshalb die Notwendigkeit innovativer, flexibler Ansätze zunimmt. [1], [2]

Neue Technologien, insbesondere Künstliche Intelligenz (KI), bieten vielversprechende Möglichkeiten zur Verbesserung der DQ in Datenverarbeitungsprozessen, etwa durch Automatisierung, Anomalieerkennung oder durch den Einsatz generativer Modelle. Solche Ansätze bieten das Potenzial, DQ-Prüfungen effizienter, intelligenter und dynamischer zu gestalten. [3]

Auch Geberit, ein führendes Unternehmen im Bereich der Sanitärtechnik, ist auf eine verlässliche Datengrundlage angewiesen, um datengetriebene Prozesse, Berichte und KI-gestützte Anwendungen zu ermöglichen. Die Einführung einer neuen Datenplattform, Microsoft Fabric, bietet eine zentrale, skalierbare Cloud-Plattform für die Integration, Verarbeitung und Analyse von Daten aus verschiedenen Quellsystemen.

Die Arbeit ist im Kontext eines Produktionsunternehmens verortet und konzentriert sich auf datengetriebene Prozesse in relevanten Bereichen wie Kundenmanagement und Materialstammdaten. Damit erfolgt eine Eingrenzung auf typische DQ-Probleme, wie sie in der Fertigungsindustrie auftreten, wodurch die gewonnenen Erkenntnisse sowohl praxisnah als auch branchenspezifisch fundiert sind.

1.2. Forschungslücke

Die Sicherstellung von DQ in Extract Transform Load (ETL)-Prozessen ist ein etabliertes Forschungsfeld, welches sich traditionell an klassischen Qualitätsmodellen orientiert.

Standards wie ISO/IEC 25012 [4] betonen die Bedeutung des Kontextbezugs als zentrale Dimension der DQ, jedoch wird dessen explizite Integration in ETL-Prozesse bislang unzureichend adressiert [4], [2].

Klassische Ansätze basieren überwiegend auf regelbasierten Verfahren, bei denen Geschäftsregeln zur Sicherstellung von Konsistenz, Vollständigkeit und Korrektheit implementiert werden. Diese sind jedoch mit hohem manuellen Aufwand verbunden und nur begrenzt skalierbar [1]. Metadatengetriebene Ansätze ermöglichen zwar eine teilautomatisierte Regelgenerierung und bessere Integration in ETL-Prozesse, bleiben jedoch weiterhin abhängig von Domänenwissen und wenig flexibel [5], [1]. Souibgui et al. [2] zeigen zudem, dass bestehende ETL-Ansätze häufig nur wenige Qualitätsdimensionen berücksichtigen und semantische Aspekte weitgehend vernachlässigen. In heterogenen Big-Data-Umgebungen entsteht daraus ein Bedarf an flexiblen, erweiterbaren Frameworks und Verfahren zur Echtzeit-Anomalieerkennung [2].

Vor diesem Hintergrund gewinnt der Einsatz von KI zunehmend an Bedeutung. Arbeiten wie [6] zeigen, dass KI-basierte Verfahren zur Datenbereinigung, Anomalieerkennung und Transformation zentrale Schwächen klassischer Ansätze adressieren können, jedoch weiterhin Herausforderungen hinsichtlich Erklärbarkeit und Integration bestehen. Neuere Studien belegen zudem, dass KI klassische ETL-Prozesse in Bezug auf Automatisierung, Skalierbarkeit und Qualitätssicherung deutlich verbessern kann [3].

Einen wichtigen aktuellen Beitrag liefern zudem Eswaraja et al. [7], die eines der ersten Konzepte zur Integration KI-basierter Assistenzfunktionen in moderne Cloud-Datenplattformen wie Microsoft Fabric untersuchen. Die Autoren zeigen anhand eines prototypischen Szenarios, dass generative KI genutzt werden kann, um Analyse- und Transformationsprozesse zu unterstützen. Im Gegensatz zu klassischen ETL-Ansätzen wird hierbei verstärkt auf natürliche Sprache und kontextuelle Interpretation gesetzt. Die Ergebnisse verdeutlichen das Potenzial solcher Systeme, weisen jedoch auch auf bestehende Herausforderungen hin, insbesondere hinsichtlich Zuverlässigkeit, Reproduzierbarkeit und der Abhängigkeit von zugrunde liegenden Datenmodellen. [7]

Parallel dazu finden sich erste praktische Umsetzungen der Konzepte in modernen Datenplattformen. Generative KI-basierte Assistenten wie Microsoft Copilot ermöglichen

die Erstellung und Dokumentation von Pipelines mittels natürlicher Sprache. Diese Systeme versprechen erhebliche Produktivitätssteigerungen im Data Engineering, sind jedoch derzeit noch durch Einschränkungen in Bezug auf Kontextverständnis, Komplexität der Transformationen und fehlende Persistenz von Wissen limitiert. [8]

Zusammenfassend zeigt der Stand der Forschung einen klaren Übergang von klassischen, regelbasierten Verfahren hin zu KI-gestützten Ansätzen zur Sicherstellung von DQ. Während bestehende Arbeiten das Potenzial dieser Methoden hervorheben, fehlt es bislang an einer systematischen Integration und praktischen Evaluation in modernen Cloud-Datenplattformen wie Microsoft Fabric, insbesondere im Hinblick auf Kontextbezug und semantische Aspekte.

1.3. Zielsetzung

Der Stand der Forschung zeigt, dass die Anwendung und Evaluation von DQ-Verfahren im Kontext moderner Datenplattformen und in Bezug auf die Lösung von Microsoft bisher nur eingeschränkt untersucht wurde. Daraus ergibt sich die zentrale Fragestellung: *Wie kann Datenqualität in der Cloud-Datenplattform Microsoft Fabric durch den Einsatz klassischer und KI-gestützter Methoden sichergestellt werden?* Diese Untersuchung im Rahmen der vorliegenden Arbeit umfasst folgende Forschungsfragen:

- **F1:** Welche Datenqualitätsdimensionen und -Probleme werden in der *Literatur* beschrieben und wie zeigen sie sich im Kontext von Microsoft Fabric?
- **F2:** Welche klassischen und KI-gestützten *Verfahren* zur Verbesserung der Datenqualität sind in der Literatur etabliert und welche sind davon für die Anwendung in *Microsoft Fabric* geeignet?
- **F3:** Wie können ausgewählte Verfahren in einem *prototypischen Use Case* in Microsoft Fabric implementiert und hinsichtlich ihrer praktischen Eignung evaluiert werden?
- **F4:** Welche *Handlungsempfehlungen* zur Sicherstellung der Datenqualität lassen sich aus der Evaluation der prototypischen Umsetzung ableiten und inwieweit sind die Ergebnisse auf andere Datenquellen und Anwendungsszenarien *übertragbar*?

Als Ergebnis ist die Entwicklung einer methodischen Vorgehensweise vorgesehen, die aufzeigt, wie regelbasierte und KI-gestützte Verfahren in den ETL-Prozessen des Unternehmens integriert werden können. Um konkrete Handlungsempfehlungen daraus abzuleiten, wird ein prototypischer Anwendungsfall in Fabric konzipiert und evaluiert.

1.4. Aufbau der Arbeit

Zur schrittweisen Beantwortung der Forschungsfragen werden zunächst in Kapitel 2 die theoretischen Grundlagen zu DQ sowie relevante Begriffe, Dimensionen und Problemstellungen im Kontext moderner Datenplattformen erläutert und grundlegend in Microsoft Fabric eingeführt.

Kapitel 3 stellt die zentrale Methodik der Arbeit zur Identifikation, Klassifikation und Bewertung der Verfahren dar. Nach einer kurzen Einführung in das Forschungsdesign werden die identifizierten Verfahren unter induktiv gebildeten Kategorien sortiert und diskutiert. Anschließend werden aus der erarbeiteten Theorie Bewertungskriterien konsolidiert, welche die Grundlage für die Bewertung der Verfahren aus Sicht der DQ und architektonischen Anforderungen bilden. Danach wird das Bewertungsraster auf den Methodenkatalog angewandt, um die Eignung der Methoden für den Use Case in Microsoft Fabric herauszuarbeiten.

Darauf aufbauend erfolgt in Kapitel 4 die Auswahl repräsentativer Methoden für die Implementierung. Weiter wird eine Architekturerweiterung konzipiert, um die DQ mit den im Use Case implementierten Komponenten langfristig in die aktuellen Prozesse einzubeziehen. Im Use Case wird dafür zunächst explorativ ein Baseline-Zustand der DQ des zugrundeliegenden Customer Relationship Management (CRM)-Systems erhoben, bevor die ausgewählten Methoden entlang der Medallion-Architektur der Datenplattform integriert werden.

Kapitel 5 beinhaltet die Evaluation der entwickelten Lösung. Dabei werden sowohl die Architekturerweiterung, als auch die implementierten Verfahren hinsichtlich ihrer Wirksamkeit, Integrationsfähigkeit und Übertragbarkeit analysiert, wofür erneut Bezug auf die theoriegeleiteten Anforderungen und unternehmensseitigen Spezifika genommen wird.

Anschließend werden in Kapitel 6 die Ergebnisse zusammengefasst, das Vorgehen hinsichtlich methodischen und technischen Limitationen kritisch reflektiert sowie Implikationen für zukünftige Entwicklungsmöglichkeiten für Praxis und Forschung abgeleitet.

2. Theoretische Grundlagen

2.1. Entwicklung zu modernen Datenplattformen

Data Warehouse

Klassische Data Warehouse (DWH)-Systeme entstanden in den 1990er-Jahren als zentrale, strukturierte Plattformen zur Integration und Analyse von Unternehmensdaten. Sie zeichnen sich durch eine fachorientierte, integrierte, historisierte und stabile Datenhaltung aus, wodurch eine konsistente Grundlage für analytische Anwendungen geschaffen wird [9], [10]. Im Gegensatz zu operativen Systemen werden Daten aus verschiedenen Quellsystemen zusammengeführt, bereinigt und in ein einheitliches Schema überführt. Dieser Integrationsprozess erfolgt typischerweise über ETL-Prozesse, bei denen Daten extrahiert, transformiert und anschließend in das Zielsystem geladen werden. Insbesondere die Transformationsphase dient dabei der Vereinheitlichung und Qualitätssicherung der Daten [11], [12]. Ein wesentliches Merkmal klassischer DWH-Architekturen ist der Schema-on-Write-Ansatz. Daten müssen vor der Speicherung in ein fest definiertes Schema überführt werden, was eine hohe Datenkonsistenz ermöglicht, jedoch gleichzeitig die Flexibilität im Umgang mit heterogenen und unstrukturierten Daten einschränkt [13].

Data Lake

Mit dem zunehmenden Aufkommen großer und heterogener Datenmengen im Kontext von Big Data entstanden Data Lakes als alternative Speicher- und Verarbeitungsparadigmen. Sie ermöglichen die Speicherung von Daten in ihrem Rohformat unabhängig von Struktur oder Schema und bieten damit eine deutlich höhere Flexibilität als klassische DWH-Systeme [14], [15]. Im Gegensatz zum ETL-Ansatz folgen Data Lakes häufig dem Extract Load Transform (ELT)-Prinzip, bei dem Daten zunächst unverändert gespeichert und erst bei Bedarf transformiert werden (Schema-on-Read) [13]. Dadurch können sowohl strukturierte als auch semi- und unstrukturierte Daten effizient verarbeitet werden, was insbesondere für datengetriebene und explorative Analysen relevant ist. Die hohe Flexibilität geht jedoch mit Herausforderungen einher. Insbesondere fehlende Governance-Mechanismen und unzureichendes Metadatenmanagement können dazu führen, dass Data Lakes an Struktur und Nachvollziehbarkeit verlieren und sich zu Data Swamps entwickeln, in denen Herkunft, Qualität und Bedeutung der Daten nicht mehr eindeutig bestimmbar sind [13], [16], [17].

Lakehouse

Die Lakehouse-Architektur stellt eine Weiterentwicklung dar, die darauf abzielt, die Vorteile klassischer DWH-Systeme mit der Flexibilität von Data Lakes zu verbinden. Ausgangspunkt ist die Beobachtung, dass die parallele Nutzung beider Architekturen mit erheblichem Mehraufwand verbunden ist. Insbesondere führen getrennte Data-Lake- und Warehouse-Schichten zu redundanter Datenhaltung, komplexen ETL-Prozessen sowie erhöhtem Wartungsaufwand und Fehleranfälligkeit [18]. Zudem sind klassische DWH-Architekturen häufig durch batchorientierte Verarbeitung geprägt, wodurch Daten erst verzögert für Analysen zur Verfügung stehen und Echtzeitanwendungen eingeschränkt werden. Gleichzeitig können sie den wachsenden Anteil unstrukturierter Daten lediglich begrenzt verarbeiten. Data Lakes hingegen bieten eine hohe Flexibilität und ermöglichen den direkten Zugriff auf Rohdaten, weisen jedoch Defizite hinsichtlich Konsistenz, Transaktionssicherheit und Governance auf, was insbesondere im Kontext von DQ kritisch ist [18].

Die Lakehouse-Architektur adressiert diese Herausforderungen durch die Kombination eines offenen, kostengünstigen Speichers mit einer transaktionalen Metadatenebene. Technologien wie Delta Lake erweitern Data Lakes um Atomicity, Consistency, Isolation, Durability (ACID)-Transaktionen, Versionierung und konsistente Schreiboperationen, wodurch eine zuverlässige und nachvollziehbare Datenbasis geschaffen wird [19]. Darüber hinaus ermöglichen Mechanismen wie Schema-Enforcement sowie kontrollierte Schema-Evolution die Sicherstellung struktureller DQ, während Optimierungsverfahren die effiziente Ausführung analytischer Abfragen unterstützen. Auf diese Weise vereint das Lakehouse-Paradigma die Anforderungen klassischer Business-Intelligence-Anwendungen mit den Bedürfnissen moderner datengetriebener und Machine Learning (ML)-basierter Analysen auf einer einheitlichen Plattform [18].

Neben dem Lakehouse-Ansatz existieren weitere Architekturparadigmen wie Data Mesh, Data Fabric oder moderne Data-Warehouse-Konzepte. Diese werden im Rahmen dieser Arbeit jedoch nicht vertieft betrachtet, da sie entweder primär organisatorische Aspekte adressieren oder konzeptionell auf ähnlichen technologischen Grundlagen aufbauen [20], [21], [22].

2.2. Daten und Datenqualität

Daten und ihre Bedeutung

Die Definition von Daten änderte sich im Verlauf der Zeit. Wurden Daten anfangs lediglich mathematisch als Informationen codiert in Bits betrachtet, erweiterte sich ihr Verständnis mit der Zeit in Richtung beigemessener Bedeutung. So beschreibt Russell [23] eine Hierarchie von Daten als Symbolen ohne Bedeutung, die erst durch Kontext, Information und Zweck zu Informationen und schließlich zu Wissen führt. [23], [24] Auch Floridi [25] stellt sich dieser Frage und argumentiert, dass Informationen nicht bloß syntaktische Daten sind, sondern semantische Daten mit Bedeutung [25]. Dagegen verändern jüngere Entwicklungen wie Big Data erneut die Bedeutung und Definition von Daten, worauf in Kapitel 2.3 vertieft Bezug genommen wird.

Häufig werden Daten grundlegend wie folgt unterschieden:

- **Strukturierte Daten:** Besitzen ein vordefiniertes Schema und sind z. B. in Tabellenform relational organisiert. Bildet die Grundlage für relationale Datenbanken. [26], [27]
- **Semi-strukturierte Daten:** Verfügen über kein explizites Schema und sind flexibler als strukturierte Daten. Sie tragen Strukturinformationen mit sich selbst durch Schlüssel-Wert-Paare, Verschachtelungen und Variationen. Eine Syntax gibt eine gewisse Struktur vor, in welcher Variationen vorkommen. [27]
- **Unstrukturierte Daten:** Daten, die frei von Form oder schwer zu kategorisieren sind. Sie reichen von natürlicher Sprache über Text und Audio bis hin zu Videos [28].
- **Metadaten:** Werden verwendet, um andere Daten in einem bestimmten Kontext (Zweck, Umstand, Perspektive) zu beschreiben [29].

In der industriellen Forschung haben sich funktionale Typologien etabliert, um Daten kategorial zu unterscheiden. Morshedzadeh et al. [30] unterscheiden in ihrer Arbeit über Daten in Product Lifecycle Management Systemen (PLM) beispielsweise in Produkt-bezogene, Prozess-bezogene, Ressourcen-bezogene und externe Daten [30]. Tao et al. [31] stellen dagegen eine um unternehmensübergreifende und datengetriebene Perspektive erweiterte Klassifikation in folgende fünf Kategorien von Fertigungsdaten vor: [31].

- **Management Data:** Stammt aus unterschiedlichen Informationssystemen wie ERP, CRM, SCM und umfasst Daten zu Material und Produktionsplanung, Lager, Vertrieb und Kundenservice.

- **Equipment Data:** Beschreibt Maschinenzustände, Performance und Wartung und stammt bspw. aus Sensoren.
- **User Data:** Nutzerdaten wie Kundenpräferenzen, Such- und Kaufverhalten aus E-Commerce- und Social-Media-Plattformen.
- **Product Data:** Stammt aus Product-Service-Systemen und umfasst zum Beispiel Produktleistung, Nutzungsumfeld und Umgebungsbedingungen.
- **Public Data:** Öffentlich zugängliche Daten aus Regierungs- und Forschungsdatenbanken.

Daten bilden heute die Grundlage vieler betrieblicher Entscheidungen. Redman zufolge führt eine schlechte DQ zu operativen Problemen wie sinkender Kundenzufriedenheit, hohen Fehlerkorrekturkosten und unzufriedenen Mitarbeitern bis hin zu taktischen Herausforderungen wie erschwerte Entscheidungsfindung, gescheiterten Projekten oder Misstrauen zwischen Abteilungen. Auf strategischer Ebene hindert eine schlechte DQ die Entwicklung und Umsetzung von Strategien, verstärkt Konflikte um Datenhoheit und erschwert die Ausrichtung der Organisation. Diese Auswirkungen spiegeln sich auch in empirischen Befunden wider, die für Unternehmen Datenfeld-Fehlerraten zwischen 1-5 % zeigen. [32]

Dimensionen

DQ selbst beschreibt den Grad, in dem Daten den Anforderungen ihrer Nutzung entsprechen. Abhängig vom Verwendungszweck kann DQ demnach flexibel bewertet werden. [33] Wang und Strong [34] definieren DQ bereits 1996 als *“data that are fit for use by data consumers”*. In einer zwei-stufigen Studie sammeln sie Datenattribute und die Bewertung derer Wichtigkeit. Darauf aufbauend stellen sie ein Framework vor, das 15 verschiedene Dimensionen von DQ unter vier Kategorien einordnet. Eine Dimension besteht dabei aus einer Reihe von Attributen, die einzelne Aspekte darstellen. Die vier gebildeten Kategorien charakterisieren sich wie folgt [34]:

- **Intrinsic (intrinsische) DQ:** Intrinsische DQ widmet sich der Frage, ob die Daten selbst korrekt und vertrauenswürdig sind, mit der Idee, dass sie unabhängig von ihrem Nutzungskontext oder Format verlässlich sind. Geeignete Dimensionen hierfür sind Genauigkeit, Objektivität, Glaubwürdigkeit oder Reputation. Folglich sollte ein Geburtsdatum im System stimmen, unabhängig davon ob es gerade benötigt wird.

- **Contextual (kontextbezogene) DQ:** Die kontextbezogene DQ erweitert das Verständnis dagegen um den Nutzungskontext oder die Verwendung der Daten und legt nahe, dass Daten eine gute Qualität haben, wenn sie für die jeweilige spezifische Aufgabe nützlich sind. Beispielsweise reicht eine Liste von Kundenmails für Marketingzwecke aus, jedoch nicht für eine Kreditprüfung. Dimensionen sind Wertbeitrag, Relevanz, Aktualität, Vollständigkeit und angemessene Datenmenge.
- **Representational (repräsentative) DQ:** Die Idee hinter der repräsentativen DQ ist eine möglichst leichte Verständlichkeit und korrekte Interpretierbarkeit der Daten, durch Dimensionen wie Konsistenz der Darstellung, Kompaktheit und auch Interpretierbarkeit und Verständlichkeit selbst. Beispiele hierfür sind einheitliche Währungsangaben oder konsistente Formate für Datumsangaben.
- **Accessibility (zugängliche) DQ:** Durch die Betrachtung der Dimensionen Verfügbarkeit, Zugänglichkeit und Sicherheit wird gewährleistet, dass Daten rechtzeitig und mit vertretbarem Aufwand erhältlich sind. Hintergrund ist der Gedanke, dass perfekte DQ nichts nützt, wenn die Daten unerreichbar in einem abgeschotteten System liegen. Daraus lassen sich die *Wahrheit* der Daten, deren *Eignung im Nutzungskontext*, eine *verständliche und konsistente Darstellung* sowie *Zugriff und Verfügbarkeit* als die unterschiedlichen Aspekte von DQ festhalten.

Die bisher beschriebene Perspektive betrachtet DQ überwiegend aus Sicht der Komponenten nach ihrem Nutzen (fit for use). Eine weitere Perspektive erweitert dies auf den Produktionsprozess von Daten. Zentral ist dabei der Gedanke, dass Information wie ein Produkt behandelt werden muss. Analog zur produzierenden Industrie entstehen demnach aus Rohdaten in Informationssystemen in einem Herstellungsprozess Informationsprodukte. Wang [35] hebt hervor, dass Qualitätsprobleme nicht erst beim Konsumenten sichtbar werden, sondern im Produktionsprozess abgefangen werden müssen und stellt dafür den Total Data Quality Management (TDQM)-Zyklus vor. Er beschreibt damit ein Vorgehen aus vier wiederkehrenden Schritten, von der **Definition** relevanter Informationsprodukte und Qualitätsdimensionen zur Entwicklung und **Messung** von Metriken wie Genauigkeit, Vollständigkeit und Aktualität bis zur **Analyse** von Ursachen von Qualitätsproblemen, der Bewertung ihrer Auswirkungen und schließlich Ableitung von **Verbesserungsmaßnahmen**. Dafür werden im TDQM-Framework die vier Kategorien von Wang und Strong [34] erneut aufgegriffen und in die Qualitätsanforderungen an Informationsprodukte integriert. Mit dem Framework liegt demnach ein methodisch fundiertes

Vorgehen vor, um DQ systematisch zu definieren, messen, analysieren und verbessern. [35]

Auf ihrem ursprünglichen Werk bauen Wang und Strong [36] empirisch auf und bestätigen ihre bisherigen Untersuchungen. Aus der Beobachtung, dass DQ meist intrinsisch betrachtet wird, legen sie den Fokus auf den gesamten Kontext von Informationssystemen, die die DQ prägen. DQ-Management muss demnach prozessorientiert und konsumentenfokussiert betrieben werden, wobei die Qualität gespeicherter Daten mit der Qualität von Produktions- und Nutzungsketten kombiniert werden soll. Die Autoren schlagen weiter vor, DQ-Management-Verfahren und organisatorische Routinen zu etablieren, um die Leistungsfähigkeit von Informationssystemen in Bezug auf eine verbesserte DQ zu etablieren. Eine ebenso wichtige Rolle spielen dabei die Muster der identifizierten DQ-Probleme, welche die Entwicklung geeigneter Lösungen erfordern. [36]

2.3. Typische Datenqualitätsprobleme und der Einfluss moderner Technologien

Datenqualitätsprobleme

Unternehmen sehen sich häufig mit redundanten, inkonsistenten und unvollständigen Daten konfrontiert, wodurch die Nutzung von Daten für operative und analytische Zwecke erschwert wird. DQ-Probleme können dabei die inhaltliche Korrektheit, die Vollständigkeit und Aktualität von Daten und auch deren Struktur und Interpretierbarkeit betreffen [32].

Wang und Strong [36] identifizieren entlang ihrer DQ-Dimensionen typische Problemkonstellationen. Intrinsische Probleme entstehen häufig durch Inkonsistenzen zwischen verschiedenen Datenquellen oder durch subjektive Einflüsse im Datenproduktionsprozess, was die Glaubwürdigkeit und Reputation von Daten beeinträchtigen kann. Gleichzeitig können fehlende Transparenz über Datenherkunft und -verarbeitung das Vertrauen der Datenkonsumenten weiter reduzieren [36].

Auch im Bereich der Zugänglichkeit treten charakteristische Probleme auf. Neben technischen Einschränkungen durch unzureichende Infrastruktur oder Zugriffsbeschränkungen können insbesondere mangelnde Interpretierbarkeit sowie große Datenvolumina die effektive Nutzung von Daten erschweren. Daten sind zwar vorhanden, jedoch nicht in einer Form verfügbar, die eine effiziente Analyse ermöglicht. [36]

Kontextbezogene DQ-Probleme ergeben sich vor allem dann, wenn Daten die Anforderungen ihrer Nutzung nicht erfüllen. Ursachen hierfür liegen sowohl in unvollständiger oder fehlerhafter Datenerhebung als auch in sich verändernden Anforderungen der Datenkonsumenten. Insbesondere in verteilten Systemen führen Integrationsprobleme dazu, dass Daten aus unterschiedlichen Quellen nicht konsistent zusammengeführt werden können, wodurch ihre Nutzbarkeit weiter eingeschränkt wird. [36]

Eine zentrale technische Einordnung erfolgt durch die Unterscheidung zwischen Single-Source- und Multi-Source-Problemen sowie zwischen schema- und instanzbezogenen Fehlern. Rahm und Do (2000) zeigen, dass schema-bezogene Probleme insbesondere aus unzureichenden Datenmodellen und fehlenden Integritätsbedingungen resultieren, während auf Instanzebene Inkonsistenzen, Duplikate oder fehlerhafte Werte auftreten. Bei der Integration mehrerer Datenquellen entstehen zusätzliche Konflikte durch unterschiedliche Schemata, Formate und semantische Interpretationen, was die Sicherstellung von Konsistenz erheblich erschwert. [37]

Ergänzend lassen sich DQ-Probleme auch nach ihrer Ausprägung klassifizieren. Kim et al. [38] unterscheiden zwischen fehlenden Daten, fehlerhaften Daten und solchen Daten, die zwar korrekt sind, jedoch im gegebenen Kontext nicht sinnvoll genutzt werden können. Müller und Freytag [39] differenzieren darüber hinaus zwischen syntaktischen, semantischen und Coverage-Anomalien, wodurch sowohl strukturelle als auch inhaltliche und vollständigkeitsbezogene Probleme adressiert werden. [38], [39]

Zur systematischen Einordnung dieser Problemtypen stellen Oliveira et al. [40] umfassende Taxonomien für Single- und Multi-Source-DQ-Probleme bereit, die eine strukturierte Klassifikation entlang von Attribut-, Tupel- und Relationenebene ermöglichen [40]. Diese sind zur Orientierung in Tab. 1 und Tab. 2 dargestellt.

PROBLEMTYP	Beschreibung	Beispiel
MISSING VALUE	Pflichtattribut ohne Wert	Kunde ohne Name
SYNTAX VIOLATION	Wert entspricht nicht Format	13/12/2004, 2004/12/13
OUTDATED VIOLATION	Wert ist veraltet	Alte Kundenadresse
INTERVAL VIOLATION	Zahlenwert außerhalb gültigen Bereichs	-5 in Anzahl
SET VIOLATION	Wert nicht im erlaubten Wertebereich	West statt North/South

PROBLEMTYP	Beschreibung	Beispiel
MISSPELLED ERROR	Schreibfehler im Textwert	Gamdra, Gandra
INADEQUATE VALUE TO THE ATTRIBUTE CONTEXT	Wert passt semantisch nicht zum Attribut	Adress = Kundenname
VALUE ITEMS BEYOND THE ATTRIBUTE CONTEXT	Mehrere Werte in einem Feld	<i>Main St 123, 12345</i>
MEANINGLESS VALUE	Wert im Kontext nicht sinnvoll	Address = XYZ
VALUE WITH IMPRECISE OR DOUBTFUL MEANING	Abkürzung mit Mehrdeutigkeit	<i>Ant=Anthony/Antonia</i>
DOMAIN CONSTRAINT VIOLATION	Regel des Fachbereichs verletzt	Name > 2 Wörter
UNIQUENESS VALUE VIOLATION	Doppelter Wert, der eindeutig sein sollte	2 Kunden gleiche TaxID
SYNONYMS EXISTENCE	Verschiedene Begriffe mit gleicher Bedeutung	Professor, Lehrer
DOMAIN CONSTRAINT VIOLATION	Regelverletzung über mehrere Werte	Sortierung Datum
SEMI-EMPTY TUPLE	Großteil der Felder ist leer	>60% fehlen
INCONSISTENCY AMONG ATTRIBUTE VALUES	Abhängigkeit zwischen Werten verletzt	Total != Quantity * Preis
DOMAIN CONSTRAINT VIOLATION	Regelverletzung zwischen Attributen	Versand durch Kunden, keine Versandkosten
REDUNDANCY ABOUT AN ENTITY	Gleiche Entität mehrfach vorhanden	Smith Barney, S. Barney
INCONSISTENCY ABOUT AN ENTITY	Widersprüchliche Werte derselben Entität	Flower St, Sun St
DOMAIN CONSTRAINT VIOLATION	Regelverletzung auf Tabellenebene	>10 Produktfamilien
REFERENTIAL INTEGRITY VIOLATION	Fremdschlüssel verweist ins Leere	<i>Zip_Code=4415 n.v.</i>
OUTDATED REFERENCE	Verweis formal korrekt, jedoch veraltet	Alte Referenz
SYNTAX INCONSISTENCY	Unterschiedliche Formate in verknüpften Relationen	dd/mm/yyyy, yyyy/mm/dd
INCONSISTENCY AMONG RELATED ATTRIBUTE VALUES	Werte widersprechen sich zwischen Relationen	Total=100, Einzelsumme=90
CIRCULARITY OF TUPLES IN SELF-RELATIONSHIP	Zyklus in Selbstbeziehung	Produkt X -> Y -> X
DOMAIN CONSTRAINT VIOLATION	Fachregel über mehrere Relationen verletzt	Fakturist nicht Sales Dept.

Tab. 1: Single-Source-DQ-Probleme [40]

PROBLEMTYP	Beschreibung	Beispiel
SYNTAX INCONSISTENCY	Gleicher Attributtyp, unterschiedliche Darstellungsformate	dd/mm/yyyy yyyy/mm/dd
DIFFERENT MEASURE UNITS	Gleiche Attribute, verschiedene Maßeinheiten	Euro/Dollar
REPRESENTATION INCONSISTENCY	Unterschiedliche Werte/Kodierung für gleiche Sachverhalte	F/M, 0/1
DIFFERENT AGGREGATION LEVELS	Unterschiedliche Detaillierungsgrade	Verkäufe nach Produkt / -typ
SYNONYMS EXISTENCE	Unterschiedliche Bezeichnungen, gleiche Bedeutung	Professor, Lehrer
HOMONYMS EXISTENCE	Gleiche Bezeichnung, unterschiedliche Bedeutung	Maus
REDUNDANCY ABOUT AN ENTRY	Entität mehrfach oder leicht unterschiedlich vorhanden	Smith Barney, Smith B.
INCONSISTENCY ABOUT AN ENTRY	Widersprüchliche Attributwerte für dieselbe Entität	Flower Street, Sun Street
DOMAIN CONSTRAINT VIOLATION	Zusammengenommene Quellen verletzen eine Regel, die einzeln eingehalten wird	Produktanzahl: $7 + 8 = 15 > 10$

Tab. 2: Multi-Source-DQ-Probleme [40]

Insgesamt zeigt sich, dass DQ-Probleme sowohl auf technischer als auch auf kontextueller Ebene entstehen und sich insbesondere bei der Integration heterogener Datenquellen verstärken. Gleichzeitig existieren zahlreiche Klassifikationsansätze, die unterschiedliche Perspektiven auf das Problemfeld bieten. Dies unterstreicht die Notwendigkeit einer integrierten Betrachtung, die diese Perspektiven zusammenführt und als Grundlage für die im weiteren Verlauf der Arbeit entwickelte Systematisierung dient.

Big Data

Der Begriff Big Data bezeichnet Datensätze, deren Volumen, Geschwindigkeit und Vielfalt eine Verarbeitung mit klassischen Datenbanksystemen erschweren oder unmöglich machen [41]. Zur Charakterisierung haben sich insbesondere die sogenannten V-Dimensionen etabliert. Neben **Volume**, **Variety** und **Velocity** werden häufig weitere Eigenschaften wie **Variability** und **Veracity** ergänzt, wobei letztere explizit die Zuverlässigkeit und Vertrauenswürdigkeit der Daten adressiert und damit eine direkte Verbindung zur DQ herstellt [42], [43], [44].

Diese Merkmale bewirken einen Wandel von monolithischen hin zu verteilten und skalierbaren Architekturen. Insbesondere Cloud Computing stellt hierbei die technologische

Grundlage dar, um große, heterogene Datenmengen effizient zu speichern und zu verarbeiten [41], [45]. Gleichzeitig fungieren Cloud-Plattformen als Integrationsschicht für Daten aus unterschiedlichen Quellen, Formaten und Kontexten, wodurch strukturelle Heterogenität und Inkonsistenzen zunehmen. Die Sicherstellung von DQ verschiebt sich damit zunehmend auf die Konsistenz über mehrere Datenquellen hinweg [45].

Die Eigenschaften von Big Data verstärken bestehende DQ-Probleme und führen zugleich zu neuen Herausforderungen. Große Datenmengen erschweren die Skalierung von Prüf- und Bereinigungsverfahren, während hohe Verarbeitungsgeschwindigkeiten eine Echtzeitvalidierung erforderlich machen. Die Vielfalt heterogener Datenquellen führt zu fehlender oder inkonsistenter Semantik, wodurch die Interpretation und Integration der Daten zusätzlich erschwert wird. Die daraus resultierenden DQ-Probleme reichen von klassischen Multi-Source-Konflikten über Redundanz und Inkonsistenzen bis hin zu spezifischen Herausforderungen moderner Plattformen, etwa fehlender Strukturvalidität in semi-strukturierten Daten, zeitlicher Inkonsistenz durch asynchrone Aktualisierung sowie statistischen Verzerrungen in datengetriebenen Verfahren [44].

Cai und Zhu [46] heben ergänzend hervor, dass insbesondere fehlende globale Standards sowie die zunehmende Verteilung und Heterogenität von Daten die Bewertung und Sicherstellung von DQ erschweren. Zudem führen zeitliche Volatilität und fehlende Echtzeitverarbeitung zu Risiken für die Aktualität und Verlässlichkeit von Analyseergebnissen [46]. Taleb et al. [47] erweitern diese Perspektive um eine End-to-End-Betrachtung und argumentieren, dass Data Quality Management (DQM) nicht mehr isoliert in einzelnen Verarbeitungsschritten erfolgen kann. Vielmehr ist eine durchgängige Qualitätssicherung über den gesamten Datenlebenszyklus erforderlich, da Fehler entlang der gesamten Verarbeitungskette entstehen können - von der Datenerfassung über die Transformation bis hin zur Analyse [47].

Zur Einordnung dieser Herausforderungen greifen bestehende Arbeiten auf etablierte DQ-Dimensionen zurück, wie sie beispielsweise in der Norm ISO 25012 [4] definiert sind. Dabei wird zwischen inhärenter und systemabhängiger DQ unterschieden, wodurch sowohl die Eigenschaften der Daten selbst als auch deren Verarbeitung im System berücksichtigt werden [4].

Neuere Ansätze betonen darüber hinaus die Notwendigkeit einer kontext- und domänenspezifischen Bewertung von DQ. Ridzuan und Zainon [48] zeigen, dass klassische Dimensionen weiterhin relevant sind, jedoch um Aspekte erweitert werden müssen, die sich aus großen, dynamischen und heterogenen Datenumgebungen ergeben, etwa zunehmende Unsicherheit von Datenquellen, beschleunigter Informationsverfall und steigende Anforderungen an Aktualität und Konsistenz. Die konkrete Ausgestaltung der erweiterten Dimensionen ist in Tab. 3 dargestellt. [48]

KATEGORIE	Dimension	Definition
ACCESSIBILITY	Accessibility	<i>Leichter Zugriff auf Daten und einfache Abrufbarkeit</i>
	Availability	<i>Daten sind verfügbar und einsatzbereit</i>
	Security	<i>Schutz der Daten vor unbefugtem Zugriff</i>
CONTEXTUAL	Appropriate Amount of Data	<i>Datenmenge ist passend zum vorgesehenen Zweck</i>
	Currency	<i>Aktualität der Daten</i>
	Relevance	<i>Daten sind direkt anwendbar und sinnvoll</i>
	Validity	<i>Genauigkeit der Daten in Bezug auf das dargestellte Konzept</i>
	Value Added	<i>Zusätzlicher Mehrwert oder Erkenntnisse aus den Daten</i>
	Volatility	<i>Häufigkeit der Datenaktualisierung</i>
	Completeness	<i>Daten enthalten alle notwendigen Elemente</i>
	Timeliness	<i>Daten sind innerhalb des gewünschten Zeitrahmens nutzbar</i>
INTRINSIC	Correctness	<i>Entspricht der realen Welt</i>
	Redundancy	<i>Keine doppelten Daten im Datensatz</i>
	Reputation	<i>Vertrauenswürdigkeit bzw. Zuverlässigkeit der Datenquelle</i>
	Trust	<i>Grad des Vertrauens in die Daten</i>
	Accuracy	<i>Korrekte Abbildung der Realität</i>
REPRESENTATIONAL	Interpretability	<i>Daten sind leicht interpretierbar (Bedeutung verständlich)</i>
	Readability	<i>Gute Lesbarkeit der Darstellung</i>
	Understandability	<i>Nutzer können die Bedeutung der Daten verstehen</i>
	Consistent	<i>Einheitliche Darstellung der Daten über verschiedene Quellen hinweg</i>

Tab. 3: Taxonomie der Data Quality Dimensionen [48]

Insgesamt wird deutlich, dass Big Data neben der Skalierung bestehender DQ-Probleme auch neue Anforderungen an deren Bewertung und Sicherstellung stellt. Die Kombination aus Volumen, Heterogenität und Dynamik erfordert Ansätze, die DQ über den gesamten Datenlebenszyklus und im Kontext moderner Plattformen berücksichtigen. Die bisher dargestellten Ansätze verdeutlichen, dass DQ weder isoliert noch statisch betrachtet werden kann, sondern vielmehr im Zusammenspiel von Daten, Prozessen und Architekturen entsteht.

2.4. Architektur und Komponenten von Microsoft Fabric

Microsoft Fabric

Mit Microsoft Fabric stellt Microsoft eine cloudbasierte Software as a Service (SaaS)-Plattform für Datenintegration, -verarbeitung und -analyse bereit, die den gesamten Datenlebenszyklus in einer einheitlichen Umgebung abdeckt. Ziel ist es, bislang fragmentierte Datenwerkzeuge zu konsolidieren und eine durchgängige, rollenübergreifende Nutzung zu ermöglichen [49]. Die Plattform basiert technologisch auf offenen Standards wie Apache Spark und Delta Lake und kombiniert verschiedene analytische und operative Workloads auf einer gemeinsamen Datenbasis [50]. Abbildung Abb. 1 zeigt die zentrale Architektur von Microsoft Fabric.



Abb. 1: Microsoft Fabric Architecture [49]

Zentraler Bestandteil der Plattform ist die Integration verschiedener Workloads, die unterschiedliche Phasen des Datenprozesses adressieren. Dazu zählen insbesondere die Datenintegration (Data Factory), die Datenaufbereitung und -verarbeitung (Data Engineering), analytische Speicherstrukturen (DWH), datengetriebene Modellierung (Data Science) sowie Echtzeitanalysen (Real-Time Intelligence). Ergänzt werden diese durch Visualisierungsmöglichkeiten in Power BI sowie Governance-Funktionalitäten durch Microsoft Purview [49]. Anstatt isolierter Tools verfolgt Fabric damit einen integrativen Ansatz, bei dem Daten einmal bereitgestellt und anschließend über verschiedene Workloads hinweg genutzt werden können.

OneLake als zentrale Datenbasis

Grundlage dieser Integration ist der sogenannte OneLake, der als zentraler Data Lake für alle Fabric-Komponenten fungiert. Er basiert auf Azure Data Lake Storage Gen2 und

stellt eine tenantweite, logisch einheitliche Speicherstruktur bereit [49]. Daten werden dabei standardisiert im Delta-Parquet-Format abgelegt und können unabhängig von der verwendeten Engine (z. B. SQL oder Spark) genutzt werden. Dies ermöglicht es, Daten ohne physische Duplikation in unterschiedlichen Verarbeitungskontexten einzusetzen. Durch die Organisation in Workspaces sowie die Unterstützung von Zugriffskontrolle und Datenfreigabe wird gleichzeitig eine dezentrale Nutzung bei zentraler Governance ermöglicht. Zudem erlauben sogenannte Shortcuts die Einbindung externer Datenquellen, ohne diese replizieren zu müssen [51]. Insgesamt schafft OneLake damit die Grundlage für eine konsistente, organisationsweite Datenbasis im Sinne einer Single Source of Truth.

Unterstützung durch generative KI (Copilot)

Ergänzend zu den klassischen Datenverarbeitungsfunktionen integriert Fabric generative KI in Form des Copilot, der auf Large Language Model (LLM)-Technologie basiert [8]. Dieser unterstützt verschiedene Rollen entlang des Datenprozesses, z. B. bei der Erstellung von Datenpipelines, der Transformation von Daten, der Generierung von Code oder der Entwicklung analytischer Modelle. Auch die Interaktion mit Daten über natürliche Sprache sowie die Erstellung von Berichten und Visualisierungen wird unterstützt. Dadurch wird die Zugänglichkeit der Plattform erhöht und gleichzeitig die Effizienz in datengetriebenen Arbeitsprozessen gesteigert. [8]

Zusammenfassend lässt sich Microsoft Fabric als integrierte Datenplattform verstehen, die durch die Kombination aus einheitlicher Datenbasis, vielfältigen Workloads und KI-gestützter Unterstützung die Grundlage für moderne datengetriebene Anwendungen bildet. Diese Architektur schafft zugleich die Voraussetzung für weiterführende Konzepte wie datenproduktorientierte Ansätze oder agentenbasierte Interaktionen, die im weiteren Verlauf der Arbeit aufgegriffen werden.

Data Products

Die Sicht auf Daten hat sich von isolierten Datenextrakten und adhoc erzeugten Reports hin zu einer produktorientierten Perspektive entwickelt [35]. Daten werden dabei als bewusst gestaltete, konsumierbare Artefakte verstanden, die wiederkehrende Informationsbedürfnisse adressieren und einen konkreten Mehrwert für definierte Nutzergruppen liefern [52]. Legner und Hasan [52] definieren ein Data Product als *“managed artifact*

that satisfies recurring information needs and creates value through transforming and packaging relevant data elements into consumable form” [52]. Zentrale Merkmale sind dabei insbesondere die klare Ausrichtung auf Nutzung und Wertbeitrag, die Aufbereitung in konsumierbarer Form sowie eine definierte Ownership-Struktur [52].

In der Literatur werden Data Products häufig entlang ihres Abstraktionsgrades unterschieden: Von basisnahen Daten (Basic), über analytisch angereicherte Daten (Analytical) bis hin zu komplexen, entscheidungsunterstützenden Anwendungen (Advanced Analytical). Diese bauen konzeptionell aufeinander auf und verdeutlichen, dass hochwertige Datenprodukte schrittweise entwickelt werden. [52]

Mit dem Prinzip *Data as a Product* wird diese Perspektive weiter konkretisiert. Datenprodukte müssen demnach Eigenschaften wie Auffindbarkeit, Vertrauenswürdigkeit, Interoperabilität und Sicherheit erfüllen, um tatsächlich nutzbar zu sein [53]. Gleichzeitig erfordert ihre Umsetzung neben technischen auch organisatorische Strukturen, insbesondere klare Verantwortlichkeiten und Governance-Mechanismen [54].

Im Kontext von Microsoft Fabric lassen sich Data Products als wiederverwendbare, qualitätsgesicherte Datensätze interpretieren, die auf einer gemeinsamen Datenbasis bereitgestellt werden. Eine zentrale Rolle spielt dabei das Semantic Model, das Daten in einer für unterschiedliche Nutzergruppen verständlichen und konsumierbaren Form abstrahiert und zugänglich macht [50].

Damit fungiert das Semantic Model als Schnittstelle zwischen technischer Datenhaltung und fachlicher Nutzung und bildet die Grundlage für die wiederverwendbare Bereitstellung von Datenprodukten.

Data Agents

Während Data Products strukturierte und konsumierbare Daten bereitstellen, entsteht mit zunehmender Komplexität und Datenmenge die Herausforderung, diese effizient zugänglich zu machen. Hier setzen agentenbasierte Ansätze an, die eine natürlichsprachliche Interaktion mit Daten ermöglichen und damit eine zusätzliche Nutzungsschicht über Data Products legen.

Ein Agent wird klassisch als System beschrieben, das seine Umgebung wahrnimmt und autonom darauf reagiert [55]. Moderne agentische Systeme zeichnen sich darüber hinaus durch die Fähigkeit aus, komplexe Ziele eigenständig zu verfolgen, in natürlicher Sprache gesteuert zu werden und externe Tools in ihre Entscheidungslogik einzubinden [56]. Im Kontext aktueller Entwicklungen gewinnen insbesondere auf LLM basierende Agenten an Bedeutung, da sie unterschiedliche Komponenten wie Planung, Speicher und Tool-Nutzung kombinieren und so über reine Modellinferenz hinausgehen [57], [58].

Vor diesem Hintergrund lässt sich der Data Agent in Microsoft Fabric als Erweiterung der Nutzungsschicht von Data Products verstehen. Er basiert auf generativer KI und ermöglicht es, über natürliche Sprache auf strukturierte Daten zuzugreifen und diese zu analysieren. Technisch greift der Data Agent auf semantische Modelle und deren Metadaten zu, interpretiert Nutzeranfragen, generiert geeignete Abfragen und liefert die Ergebnisse in verständlicher Form zurück. Damit abstrahiert er die technische Komplexität der Datenzugriffe und erweitert die Zugänglichkeit für nicht-technische Nutzer. [59]

Data Agents sind somit nicht als isolierte Komponente zu betrachten, sondern bauen direkt auf den zuvor beschriebenen Datenprodukten auf und nutzen diese als kontextuelle Grundlage. Insgesamt entsteht dadurch eine mehrschichtige Architektur. Während Data Products eine strukturierte und qualitätsgesicherte Datenbasis bereitstellen, ermöglichen Data Agents eine flexible und nutzerzentrierte Interaktion mit diesen Daten.

Medallion-Architektur

Nachdem Data Products als konsumierbare, qualitätsgesicherte Datenartefakte und Data Agents als Zugriffsschicht darauf eingeführt wurden, rückt die systematische Erzeugung und Qualitätssicherung dieser Datenprodukte in den Fokus. Eine verbreitete Antwort darauf ist die Medallion-Architektur, die ein logisches Design-Pattern zur schrittweisen Aufbereitung von Daten in Lakehouse-Umgebungen beschreibt [60].

Die Architektur unterteilt Daten in mehrere Schichten, in denen DQ, Struktur und Nutzbarkeit sukzessive erhöht werden. Ziel ist es, eine konsistente und nachvollziehbare Datenbasis zu schaffen, die als Grundlage für vertrauenswürdige Data Products dient. Gleichzeitig unterstützt sie die Umsetzung einer Single Source of Truth durch versionierte, nachvollziehbare Transformationen unter Einhaltung der ACID-Prinzipien. [61]

Bronze (Raw Data): Im Bronze-Layer werden Rohdaten aus unterschiedlichen Quellsystemen in ihrer ursprünglichen Form gespeichert. Ziel ist es, einen vollständigen und unveränderten Datenbestand bereitzustellen, der als historisch nachvollziehbare Grundlage dient [50]. Die Daten werden typischerweise inkrementell geladen und um technische Metadaten ergänzt, um Herkunft und Ladezeitpunkte nachvollziehbar zu machen. Der Fokus liegt auf der Sicherstellung der Datenverfügbarkeit und Integrität, während Transformationen und Validierungen auf ein Minimum beschränkt bleiben [60]. Damit bildet der Bronze-Layer die Grundlage für Lineage, Reproduzierbarkeit und Wiederverarbeitung.

Silver (Cleaned Data): Im Silver-Layer werden die Rohdaten bereinigt, standardisiert und integriert. Ziel ist es, eine konsistente und qualitativ verbesserte Datenbasis zu schaffen, die für analytische Zwecke nutzbar ist. Typische Verarbeitungsschritte umfassen Datenbereinigung, Deduplizierung, Normalisierung sowie erste Qualitätsprüfungen [61]. Darüber hinaus werden Daten aus unterschiedlichen Quellen zusammengeführt und in eine unternehmensweite Sicht überführt. Dadurch entsteht eine strukturierte, query-optimierte Datenbasis, die als Ausgangspunkt für weiterführende Analysen und Datenprodukte dient [50].

Gold (Consumer-ready Data): Der Gold-Layer stellt die konsumfertige Ebene dar, in der Daten gezielt für spezifische Anwendungsfälle aufbereitet werden. Durch Aggregationen, Anreicherungen und modellbasierte Transformationen entstehen hier hochkurierte Datenstrukturen für Reporting, Analytics und datengetriebene Entscheidungen [50]. Typisch sind dabei denormalisierte und leseoptimierte Modelle, wie beispielsweise Star-Schemata oder Data Marts, die eine effiziente Nutzung in Dashboards und analytischen Anwendungen ermöglichen [60]. Damit entspricht der Gold-Layer in vielen Fällen der konkreten Implementierung von Data Products, da hier Daten in einer für Endnutzer verständlichen und direkt nutzbaren Form bereitgestellt werden.

Ergänzend kann eine vorgelagerte **Landing Zone** eingesetzt werden, in der Daten initial aufgenommen und für die Weiterverarbeitung vorbereitet werden, bevor sie in den Bronze-Layer überführt werden [50].

Insgesamt lässt sich die Medallion-Architektur als strukturierender Rahmen verstehen, der die schrittweise Transformation von Rohdaten hin zu konsumierbaren Data Products ermöglicht. Während die unteren Schichten primär technische Anforderungen wie Integration und DQ adressieren, stellt der Gold-Layer die Brücke zur fachlichen Nutzung dar und bildet die Grundlage für semantische Modelle und darauf aufbauende Data Agents.

3. Methodenkatalog zur Sicherstellung der Datenqualität

3.1. Forschungsdesign und Vorgehensweise

Dieses Kapitel ist zentraler Bestandteil der Arbeit und umfasst die Identifikation, Kategorisierung, Bewertung und Auswahl von Methoden und Verfahren zur Sicherstellung von DQ. Es bildet damit die Grundlage für die Implementierung im prototypischen Use Case in Microsoft Fabric.

Zur Identifikation relevanter Verfahren und Methoden wird eine reduzierte Version eines Systematic Review durchgeführt. Ziel ist es dabei geeignete klassische und KI-basierte Verfahren für die anschließende Bewertung und Implementierung zu identifizieren. Um Transparenz zu gewährleisten sind das Vorgehen und die Ergebnisse der unterschiedlichen Schritte in Tabelle Tab. 4 dokumentiert. Zunächst wird folgender Suchstring definiert, welcher sowohl den Hauptfokus des Themengebiets DQ, als auch Bezeichnungen für Methoden und Verfahren abdeckt und dies mit dem Technologiekontext um die Datenplattform Microsoft Fabric verbindet.

“data quality AND (validation OR cleansing OR monitoring OR detection) AND (data lake OR big data OR data platform OR lakehouse OR microsoft fabric)”

Dieser Suchstring wird anschließend in unterschiedlichen wissenschaftlichen Datenbanken und Research Tools verwendet, um potenziell relevante Literatur zu ermitteln. Verwendet werden IEEE Xplore Command Search, ACM Digital Library, Springer Link und Science Direct. Ursprünglich wurde neben diesen Datenbanken ebenfalls die Nutzung von Google Scholar als Tool in Betracht gezogen, welches zunächst 567 Treffer lieferte. Eine weitere Analyse wurde jedoch unterlassen, da nur eine geringe Anzahl von 24 Quellen nach Titel-Abstract Screening als relevant erachtet wurde und darunter jedoch eine hohe Redundanz mit den Ergebnissen der anderen Tools festzustellen war. Daher wird eine weitere Verwendung des Tools für den Methodenkatalog nur für ergänzende Suchen in Betracht gezogen.

Tab. 4 zeigt die Ergebnisse der Recherche aufbereitet nach Tool. Zunächst werden die erzielten Treffer einem Titel- und Abstract Screening unterzogen, um zu beurteilen, ob die Quellen für eine weitere Nutzung infrage kommen. Kriterien dafür sind: Es sollte sich um eine klar beschriebene, eigenständige Methode handeln, eine Systematisierung von

Verfahren liefern, expliziten Bezug zu DQ-Dimensionen und -Problemen herstellen und diese gegebenenfalls mit Metriken quantifizieren. Gegen eine weitere Nutzung spricht, wenn es sich um keines der vorherigen Kriterien oder um ein rein organisatorisches Framework handelt.

TOOL/ DATENBANK	Treffer gesamt	Zeitraum	Nach Titel-/ Abstract Screening	Nach textanalyse	Voll- Beitrag zu Methode (explizit)	Beitrag zu Metriken	Sonstiger argumenta- tiver Beitrag
IEEE XPLORE COMMAND SEARCH	113	2015-2025	42	30	30 (11)	5	1
	5	2024-2025	2	2	1 (1)	1	-
ACM DIGI- TAL LIBRARY	14	2019-2024	8	8	5 (3)	3	5
SPRINGER LINK	180	2012-2026	22	15	15 (7)	-	2
SCIENCE DI- RECT	111	2014-2026	9	9	8 (2)	-	3

Tab. 4: Überblick über die Suchstrategie zu DQ-Methoden

Diese reduzierte Menge wird anschließend einer detaillierteren Inhaltsanalyse unterzogen. Unter Nutzung eines dedizierten Prompts (vgl. Kapitel I) wird der Volltext KI-gestützt zusammengefasst und die zurückgegebenen Kernaussagen, sofern sie für eine weitere Betrachtung nach den zuvor genannten Kriterien relevant sind, im Originaltext gegengeprüft. An dieser Analyse anknüpfend wird der Text nach seinem Wertbeitrag (Eigenständige Methode, Systematisierung, Metrik, Sonstiger argumentativer Beitrag) klassifiziert.

Zudem ist in Tab. 4 bei IEEE Xplore eine zweite Zeile mit fünf Treffern vorhanden. Diese beruht auf einer gezielten Suche nach dem eingeschränkteren Suchstring *“data quality AND microsoft fabric”*. In den anderen Tools wurden dafür keine Ergebnisse erzielt, weshalb der Ansatz hier nicht weiterverfolgt wurde. Eine Aufnahme in die Tabelle erfolgte dennoch, da die identifizierten Paper einen inhaltlichen Mehrwert darstellen.

Als Ergebnis der Methodenrecherche liegen diese unstrukturiert vor. Auffällig ist die niedrige Repräsentanz von Verfahren im Themenfeld der agentischen oder generativen KI, während z. B. Methoden aus dem klassischen ML häufiger genannt werden. Ausgeschlossen wurde Literatur, welche rein organisatorische oder governance-orientierte

Arbeiten ohne technische Methodik darstellt. Letztlich müssen solche Verfahren grundsätzlich für eine Datenplattform und unabhängig von der konkreten Lösung geeignet sein, worauf später in der Bewertung und Auswahl der Verfahren eingegangen wird.

Im weiteren Vorgehen werden die gesammelten Methoden strukturiert, in eine kategorisierte Form überführt und diskutiert (Kapitel 3.2). Anschließend werden aus dem Theoriekapitel Kapitel 2 Kategorien konsolidiert (Kapitel 3.3) und in der Entwicklung eines Bewertungsrasters (Kapitel 3.4) zusammengeführt, was die Bewertung der Methoden hinsichtlich technischer Umsetzbarkeit, architektonischer Anforderungen und theoretischer Relevanz ermöglicht. Durch die darauffolgende Anwendung des Bewertungsrasters auf den Methodenkatalog entsteht eine priorisierte Auflistung der Methodengruppen, welche als Grundlage zur Auswahl relevanter Verfahren für den Use Case dient.

3.2. Kategorisierung und Gegenüberstellung der identifizierten Verfahren

Nach erfolgter Literaturrecherche durch das Systematic Review liegen die Informationen über Verfahren zur DQ-Messung und Verbesserung als Longlist vor. Um nachfolgend eine Bewertung vorzunehmen, gilt es die Vielzahl der Informationen aus der Literatur in ein strukturiertes Format zu überführen. Wie bereits im vorangegangenen Kapitel dargestellt eignen sich die identifizierten Quellen für unterschiedliche Argumentationsgruppen. So enthalten einige die Beschreibungen konkreter technischer Methoden oder Algorithmen mit denen sich die DQ messen oder verbessern lässt, während andere Verfahren nicht explizit beschreiben, lediglich unterschiedliche Arten nennen oder als Umfrage einen Überblick über verschiedene übliche Ansätze gewähren.

Die Kategorien werden zunächst induktiv auf Basis der im systematischen Review identifizierten Verfahren gebildet. Zur theoretischen Einordnung kann das grundlegende Werk zur Kategorienbildung von Mayring [\[62\]](#) herangezogen werden, auch wenn dessen konkrete Methodik hier keine direkte Anwendung findet.

Zentral für die Kategorienbildung ist eine klare Abgrenzung von den theoriegeleiteten Bewertungsdimensionen vorzunehmen. Dadurch bleibt die methodische Trennung zwischen der Strukturierungsebene (Einteilung der Verfahren) und der Analyseebene (Bewertung anhand definierter Kriterien) gewährleistet.

Dafür orientiert sich die primäre Strukturierungslogik an dem dominanten technischen bzw. methodischen Wirkmechanismus der Methoden, da dieser maßgeblich für den Implementierungsaufwand und die Integrationsfähigkeit ist. Ergänzend wurden eigene Kategorien gebildet, wenn Implementierungsanforderungen signifikant variieren. So machen spezifische technologische Kontexte (bspw. Streaming-Verarbeitung oder integrative Gesamtlösungen) hier eine Differenzierung erforderlich.

Nachfolgend wird ein Überblick über die identifizierten Methoden-Cluster gegeben und neben einer kurzen Beschreibung der Kategorie repräsentative Verfahren näher erläutert, um den Wirkmechanismus auf unterschiedliche DQ-Dimensionen zu unterstreichen. Der zugehörige Methodenkatalog liegt in Tab. A bei.

Rule-based Data Validation

Diese Kategorie umfasst Verfahren, die DQ durch explizit definierte Regeln und formale Prüfbedingungen sicherstellen. Im Zentrum stehen syntaktische und strukturelle Validierungen. Neben manuell definierten Regeln (z. B. domänenspezifische Regeln oder Integrity-Constraints) existieren auch Ansätze zur automatisierten Regelentdeckung, etwa metadatengetrieben oder über eine Regex-Erzeugung. Charakteristisch ist der deterministische Prüfmechanismus. Daten werden anhand klar formulierter Soll-Bedingungen validiert, weshalb sich die Verfahren insbesondere für präventive Qualitätssicherung, Compliance-Anforderungen oder strukturierte Datenumgebungen mit klarer Semantik eignen. Im Gegensatz zu anderen Kategorien existieren diese Methoden vergleichsweise lang und stellen explizite Normerfüllung in den Vordergrund.

Xie et al. [63] greifen auf eine Checkliste zurück. Eine Auswahl der Regeln sind z. B. übliche Prüfungen auf Datentypen, Wertebereich, Nullwerte, Datumsformate, Duplikate oder Korrektheit der Postleitzahl. Auch spezifischere Checks über die Zuordnung zu anderen Quellen werden genutzt [63]. In einer anderen Herangehensweise greifen Karri et al. [64] auf YAML als Grundlage zurück. Sie definieren statische und dynamische Regeln, die gegen jeden anderen Datensatz geprüft werden. Dynamisch werden z. B. alle CustomerIDs auf ihre Eindeutigkeit überprüft und Typ, Format, Länge und die jeweilige Tabellenspalte angegeben. Statisch prüft dagegen, ob der Datentyp einer Transaktion korrekt ist und die Werte innerhalb des Wertebereichs liegen [64].

Song et al. [65] legen Datenabhängigkeiten als Grundlage regelbasierter Verfahren und geben dafür einen systematischen Überblick. Ausgangspunkt bilden klassische funktionale Abhängigkeiten. Grundlegende Annahme dafür ist, wenn zwei Tupel die gleichen Werte für X haben, müssen sie auch denselben Wert für Y haben. Davon gehen Erweiterungen wie Conditional, Approximate oder Matching Dependencies hervor, die DQ-Regeln unter Bedingungen wie Datenheterogenität, Unsicherheit oder numerischen Beziehungen formulieren. [65]

Neben aufwändiger manueller Definition existiert die automatische Discovery solcher Abhängigkeit aus den Daten, deren Berechnung jedoch häufig hohe algorithmische Komplexität aufweist, die mit der Anzahl der Attribute exponentiell zunimmt [66], [65]. Ausgehend von strukturierten Textattributen (z. B. ID, Postleitzahl) beschreibt Mahajan [67] ein Verfahren zur automatisierten Qualitätsprüfung, bei dem syntaktische Regeln in Form regulärer Ausdrücke (Regex) automatisch aus Beispieldaten generiert werden. Der Regex wird über eine DFA (Deterministic Finite Automaton)-basierte Methode abgeleitet und dient als Validierungsregel für neue Daten. Allerdings bleibt hier die Aussagekraft auf Pattern-basierte Validität beschränkt und die Qualität hängt stark von der vorhandenen Daten ab [67].

Statistical Data Profiling

Statistical Data Profiling umfasst explorative und deskriptiv-statistische Verfahren zur quantitativen Charakterisierung von Datensätzen. Ziel ist die Identifikation auffälliger Verteilungen, Abhängigkeiten oder struktureller Inkonsistenzen mittels Kennzahlen, Verteilungsanalysen und multivariater Verfahren (z. B. Inter Quartile Range (IQR), Korrelationsanalysen, Principal Component Analysis (PCA)). Neben klassischen Outlier-Methoden kommen auch Dependency-Discovery und Sampling-basierte Analysen zum Einsatz. Der Fokus liegt hierbei weniger auf der direkten Fehlerkorrektur als auf Transparenz und quantitativer Zustandsbeschreibung. Statistical Profiling dient häufig als Grundlage für Schwellenwertdefinitionen oder weiterführende Anomalieerkennung und unterscheidet sich von regelbasierten Verfahren darin, dass die Prüfmechanismen hier datengetrieben und distributionsabhängig sind.

Juddo [66] beschreibt, dass viele DQ-Dimensionen in Form von Verhältnissen zwischen

fehlerhaften und gesamten Datenwerten gemessen werden können, wobei diese Metrik D typischerweise zwischen 0 (vollständig fehlerhaft) und 1 (vollständig korrekt) liegt.

$$D = 1 - \frac{N_i}{N_t}$$

Weiter wird Data Profiling als Prozess zur Erzeugung statistischer Informationen über Daten beschrieben, welcher ebensolche Ratio-based Metriken (z. B. Anzahl NULL-Werte, Fehlerquote, Anzahl unterschiedlicher Werte) liefert, welche als Grundlage für DQ-Regeln und Bereinigungen dienen können [66]. Moloïou et al. [68] unterstreichen die Bedeutung von Ratio-based Metriken und quantifiziert die Dimension Completeness damit als Anteil vorhandener Datenwerte. Ein anderes Beispiel ist der Attribute Value Quality Index, der eine aggregierte Qualitätsmetrik als Kombination mehrerer Fehlerarten darstellt [68].

Shrivastava et al. [69] beschreiben im Rahmen des DQ Advisor Frameworks statistische Validatoren zur automatisierten Analyse. Zentral sind hier verschiedene Korrelationsanalysen zur Analyse der statistischen Beziehungen zwischen Attributen wie Pearson-, Spearman-, Distance- und MIC (Maximal Information Coefficient)-Korrelation. Diese Verfahren ermöglichen es Abweichungen von erwarteten Zusammenhängen zu identifizieren, die auf DQ-Probleme oder strukturelle Inkonsistenzen hinweisen können. Bspw. kann keine Korrelation zwischen Temperatur und Energieverbrauch auf Sensorfehler oder fehlerhafte Transformationen hindeuten. Um die Struktur eines Datensatzes näher zu beschreiben erzeugen Statistical Summary Verfahren grundlegende Kennzahlen wie Mittelwert, Median, Minimum, Maximum oder Standardabweichung. Diese aggregierten Kennzahlen dienen als Basis für explorative Analysen und ermöglichen die frühzeitige Identifikation potenzieller DQ-Probleme. Auch Daten mit potenziell wenig Informationsgehalt wie fehlerhafte Datenerfassung oder redundante Attribute können durch Low-Variance-Analysen identifiziert werden.[69]

Eine relevante statistische Vorverarbeitung für die nachfolgende Kategorie stellt die PCA dar. So nutzen Elouataoui et al. [70] PCA zur Reduktion hochdimensionaler Datensätze, um Anomalieerkennung effizienter zu gestalten. Auch Afriliana et al. [71] beschreiben PCA im Kontext von Clustering- und Outlier-Detection-Verfahren, bei denen die Daten zunächst auf eine reduzierte Repräsentation transformiert werden, um strukturelle Auffälligkeiten besser identifizieren zu können. [70], [71]

Anomaly Detection

Diese Kategorie bündelt Verfahren zur Identifikation ungewöhnlicher und distributionsabweichender Datenpunkte. Im Gegensatz zum Profiling zielen diese Methoden explizit auf die Detektion potenzieller Fehlerzustände ab. Charakteristisch ist die Annahme, dass Qualitätsprobleme sich als Abweichungen vom Normalverhalten manifestieren. Zum Einsatz kommen dafür statistische Verfahren (Z-Score), dichte- oder nachbarschaftsbasierte Ansätze (Local Outlier Factor (LOF), KLOF), Clustering-Algorithmen (K-Means, DBSCAN, Isolation Forest), sowie moderne neuronale Modelle (Autoencoder). Besonders geeignet sind diese Ansätze für große, komplexe, oder hochdimensionale Datensätze, bei denen explizite Regeln nicht vollständig formulierbar sind.

Eine statistische Methode zur Ausreißererkennung ist der Z-Score. Moloiu et al. [68] nutzen den Wert für die Bewertung der DQ einzelner Attribute. Der Score misst dabei, wie viele Standardabweichungen ein Wert vom Mittelwert entfernt ist. Zusätzlich definieren die Autoren Schwellenwerte und attestieren bei einem Wert zwischen 0 und 1 den normalen Bereich, zwischen 3 und 4 dagegen schon eine starke Abweichung [68]. Encinas et al. [72] weisen zum Entfernen von Messfehlern und extremen Werten auf den IQR hin. Definiert ist er als Differenz zwischen dem 75. und 25. Perzentil. Den Autoren zufolge liegen Ausreißer außerhalb des 1,5-fachen Interquartilabstands. [72]

$$IQR = P_{75} - P_{25}$$

$$Lower = P_{25} - 1.5 * IQR$$

$$Upper = P_{75} + 1.5 * IQR$$

Für die IQR-Methode spricht ebenfalls, dass sie weniger empfindlich gegenüber Extremwerten ist, keine Annahme über eine Normalverteilung der Daten erfordert und robust gegenüber Messrauschen ist [73].

Der Isolation Forest (IF)-Algorithmus beruht als unsupervised Verfahren auf der Idee Datenpunkte durch zufällige Partitionierung des Datenraums zu isolieren. Dazu wird wiederholt ein Attribut zufällig ausgewählt und ein Split-Wert zwischen Minimal- und Maximalwert des Attributs bestimmt. Durch diese rekursive Aufteilung entsteht eine Art Baum. Die Grundidee ist, dass Anomalien aufgrund ihrer Entfernung zur Hauptverteilung schneller als normale Punkte isoliert werden, wofür weniger Splits benötigt werden. Entsprechend bildet die Pfadlänge ein Maß für die Anomaliewahrscheinlichkeit. Im

Gegensatz zu LOF-Verfahren, die auf der Abweichung eines Datenpunkts von der Dichte der Nachbarschaft beruhen, eignet sich der IF aufgrund seiner geringen Speicher- und Zeitkomplexität besonders für große Datenmengen. [74]

Afriliana et al. [71] nennen Autoencoder-basierte Modelle als Deep-Learning-Ansatz zur DQ-Bewertung. Insbesondere LSTM-basierte Autoencoder können genutzt werden, um semantische oder kontextreiche Daten zu analysieren, indem sie typische Datenmuster rekonstruieren und Abweichungen über Rekonstruktionsfehler identifizieren. [71]

Matching, Deduplication

Diese Kategorie adressiert die Identifikation redundanter, inkonsistenter oder semantisch identischer Datensätze. Verfahren kombinieren Ähnlichkeitsmetriken, Schwellenwertlogiken oder ML-basierte Near-Duplicate-Detection zur Zusammenführung von Datensätzen. Neben regelbasierten Similarity-Checks kommen probabilistische Modelle und lernbasierte Deduplication-Ansätze zum Einsatz. Charakteristisch ist der Fokus auf semantische Identität trotz syntaktischer Unterschiede (z. B. Schreibvarianten), was sich durch einfache regelbasierte Verfahren nicht einfach umsetzen lässt. Diese Methoden spielen eine zentrale Rolle bei Stammdatenmanagement oder Multi-Source-Integrationen. Im Unterschied zur Anomalieerkennung steht hier nicht die Abweichung vom Normalzustand, sondern die Identifikation redundanter Repräsentationen im Vordergrund.

Ngueilbaye et al. [75] beschreiben Similarity Matching als Ansatz zur Erkennung ähnlicher Datensatzwerte auf Basis von Distanzmaßen. Zwei Attributwerte gelten dabei als ähnlich, wenn die berechnete Distanz einen definierten Schwellenwert unterschreitet. Die Ähnlichkeit kann über verschiedene Attribute hinweg durch sogenannte Similarity Rules kombiniert werden, wodurch mehrere Feldvergleiche gemeinsam zur Identifikation potenzieller Duplikate genutzt werden. Aufbauend darauf erfolgt die eigentliche Dublettenbehandlung typischerweise in zwei Schritten. Zunächst identifiziert ein Match-Verfahren ähnliche Datensätze, die dasselbe reale Objekt repräsentieren, anschließend werden diese durch ein Merge-Verfahren zu einem konsolidierten Datensatz zusammengeführt. Diese Kombination bildet eine grundlegende Technik im Kontext von Entity Resolution und Deduplication. [75]

Automatisiertes Repairing

Automatisiertes Repairing umfasst Verfahren zur aktiven Korrektur identifizierter Qualitätsmängel. Dazu zählen neben regelbasierter Bereinigung und statistischer Imputation auch ML-gestützte Missing-Value-Prediction oder modellbasierte Rekonstruktion (z. B. Bayesian Regression). Während einfache Methoden wie Forward Fill oder Interpolation deterministisch arbeiten, ermöglichen probabilistische und lernbasierte Verfahren kontextabhängige Schätzungen. Charakteristisch ist die Transformation fehlerhafter oder fehlender Werte in plausibilisierte Ersatzwerte.

Bei regelbasierter Bereinigung werden Daten anhand vordefinierter Validierungs- oder Konsistenzregeln korrigiert. Systeme wie SparkDQ nutzen definierte DQ-Regeln und Functional Dependency (FD)s, um fehlerhafte Datensätze zu filtern und ungültige Werte zu ersetzen [76]. Häufig werden syntaktische und semantische Regeln eingesetzt, um Korrekturen hinsichtlich Vollständigkeit, Konsistenz oder Validität vorzunehmen [77].

Statistische bzw. probabilistische Imputationsverfahren werden insbesondere zur Behandlung fehlender Werte eingesetzt, indem fehlende Daten durch geschätzte Werte ersetzt werden, die aus vorhandenen Beobachtungen abgeleitet sind. Beispiele sind die Verwendung vorheriger Messwerte, Mittelwert- oder Modus-Imputation. In Zeitreihen- oder Sensordaten werden darüber hinaus häufig lineare, quadratische oder polynomiale Interpolationsverfahren eingesetzt oder der zuletzt gültige Wert fortgeschrieben, um eine automatisierte Ergänzung zu ermöglichen. Dennoch können durch Imputation potenziell neue Fehler eingeführt werden. [78]

Shi et al. [79] betrachten fehlende Werte als ein Prognoseproblem, bei dem die Lücken auf Grundlage vorheriger Beobachtungen geschätzt werden. Dafür nutzen sie eine Support Vector Machine (SVM) [79]. Mit Random Forest wird von Liu et al. [80] eine weitere ML-Technologie beschrieben, um multivariate Beziehungen zur Rekonstruktion fehlender Werte zu nutzen [80]. Gritsch et al. [81] zeigen in ihrer Literaturübersicht, dass auch zunehmend generative Deep-Learning-Modelle wie Variational Autoencoder (VAE) oder Generative Adversarial Network (GAN) eingesetzt werden. Sie lernen eine latente Repräsentation der Daten und können daraus fehlende Werte rekonstruieren, um auch komplexe Strukturen zu modellieren [81].

Gesamtansätze

Als gesonderte Kategorie integrieren Gesamtansätze mehrere DQ-Mechanismen in ein konsistentes Framework-Konzept. Diese Ansätze verbinden Profiling, Validierung, Monitoring und teilweise automatisierte Korrektur in einem übergeordneten Design. Charakteristisch ist neben der ganzheitlichen Orchestrierung von DQ-Prozessen der hohe Integrationsgrad sowie Einbettung in spezifische Architekturen.

Ein anschauliches Beispiel liefert Eswararaj et al. [7] mit einem Adaptive Hybrid Framework. Das Paper stellt ein hybrides Framework zur Sicherstellung der DQ in modernen Big-Data-Umgebungen vor, indem sie regelbasierte Datenvalidierung mit KI-basierter semantischer Analyse kombinieren. Die Autoren argumentieren, dass hybride Architekturen eine vielversprechende Lösung für moderne Data-Lakehouse-Umgebungen (Microsoft Fabric) darstellen. Sie greifen dafür auf eine zweistufige Pipeline zu, bei der zunächst Daten durch PySpark-basierte Regeln auf syntaktische Fehler geprüft und mittels heuristischer Regeln markiert werden. Anschließend werden diese fehlerhaften Datensätze an ein LLM für eine kontextuelle Analyse weitergeleitet. So können semantische Anomalien unter selektiv reduziertem Rechenaufwand erkannt werden. [7]

Evaluiert wurde der Ansatz anhand zweier realer Datensätze (Twitter Airline Sentiment und Amazon Produktbewertungen) in dem vier Qualitätsprobleme (Nullwerte, Formatverletzung, semantische Inkonsistenzen und Sentiment-Drift) untersucht wurden. Ihr Framework beschreiben sie als Quality Gate direkt nach der Data Ingestion vor jeglicher Verarbeitung. Als geeignete Datengrundlage werden strukturierte Tabellen mit natürlicher Sprache mit deskriptiven Metadaten als geeignet empfohlen. Sie greifen für die PySpark-basierte Validation auf Spark SQL, user-defined functions und "IS NULL Checks" auf kritischen Spalten zurück, sowie Regex für Pattern Violations oder leichtgewichtete Schlüsselwort-Heuristik auf bspw. "amazing". Die KI-getriebene Validation wird ihrerseits über zwei Prompts an ein BERT-Modell umgesetzt. Dabei prüfen sie einen heuristisch abgeleiteten Wert des Sentiments und verweisen auf eine Referenzverteilung mit einem spezifizierten Threshold. Das im November 2025 nach dem Beginn der Bearbeitung der vorliegenden Arbeit veröffentlichte Paper verdeutlicht somit die Relevanz und das Potenzial zur Kombination unterschiedlicher Ansätze Sicherstellung von DQ in modernen Datenplattformen wie Fabric. Jedoch zeigt es, dass sowohl die verwendeten rule-based

als auch LLM-getriebene Validierung domänenspezifisch bzw. konfigurationsabhängig sind und auf eine andere Datengrundlage spezifiziert sind. [7]

Dem gegenüber stellt das Framework SparkDQ von Gu et al. [76] ein generisches, verteiltes DQM-System auf Basis von Apache Spark vor, das Detektion und Reparatur in Big-Data-Umgebungen unterstützt. Zunächst ermöglichen Application Programming Interface (API)s die Implementierung von DQ-Tasks zur Unterstützung verschiedener Operationen. Damit werden unter anderem IF-basierte Outlier Detection, FD-Detection, Entity Matching sowie Naive-Bayes-basierte Imputation ermöglicht. Zusätzlich optimieren Multi-Task Scheduling und Data-State-Caching die Ausführungszeit. Ein Spark-Cluster mit 10 Mio. Datensätzen unterstreicht die Anwendbarkeit auf große Datensätze. Semantische und kontextbezogene Probleme werden lediglich eingeschränkt berücksichtigt [76]. Ein Ansatz der die regelbasierten Verfahren erweitert ist LUCARIO, welcher automatisch Regeln für Spalten (z. B. Pattern und Wertebereich) inferiert. [82].

GenAI, agentisch

Verfahren dieser Kategorie nutzen große Sprachmodelle oder kontextadaptive KI-Systeme. Dazu zählen LLM-gestützte Rule Generation, semantische Typdetektion oder kontextbasierte Fehlerklassifikation. Im Unterschied zu klassischen ML-Modellen steht hier nicht die Erkennung von Mustern, sondern das semantische Verständnis im Vordergrund. Diese Ansätze ermöglichen flexible, domänenübergreifende Qualitätssicherung bei initialer Regeldefinition. Charakteristisch ist die Fähigkeit zur kontextsensitiven Interpretation unstrukturierter oder semistrukturierter Daten.

Ein Beispiel liefern Tamm & Nikforova [83]. Sie stellen fest, dass nur ein kleiner Teil verschiedener DQ-Tools automatische Verfahren zur Rule Detection implementiert (meist Metadata oder ML-basiert) und weisen daher auf das Potenzial von LLMs hin DQ-Regeln zu generieren, semantische Beziehungen zu entdecken und Anomalien zusammenzufassen. Dies geht jedoch mit Herausforderungen wie mangelnde Explainability, Datenschutz & Compliance Risiken und Schwierigkeiten bei domänenspezifischem Wissen einher. [83]

Die Anwendung wird auch an dem zuvor beschriebenen Beispiel deutlich, das BERT-LLM zur Validierung zuvor als fehlerhaft klassifizierter Datensätze zu nutzen, indem zwei

Prompts zur Identifikation semantischer und kontextueller Probleme verwendet werden [7]. Andere Quellen unterstreichen die Bedeutung der Kontext- und Semantikbasierten Prüfungen (bspw. über Attributnamen und Metadaten) [84]. Jedoch finden sich kaum explizite LLM oder Agentenbasierte Verfahren im durchgeführten Review.

IoT, Streaming

Diese Kategorie umfasst Verfahren zur Qualitätssicherung in der kontinuierlichen, zeitkritischen Verarbeitung von Datenströmen. Im Gegensatz zu Batch-Ansätzen müssen hier Latenzanforderungen, Speicherrestriktionen und inkrementelle Berechnung berücksichtigt werden. Methoden wie Time-relevant k-means adressieren Ausreißererkennung in dynamischen Datenumgebungen. Diese Kategorie unterscheidet sich primär durch ihren architektonischen und technologischen Kontext.

Für Streaming-Verarbeitung stellt Yang et al. [85] ein Framework vor, DQ-Indikatoren auf Datenströmen mithilfe von Complex Event Processing (CEP) zu berechnen. Bei CEP werden mehrere Streams kombiniert, um Muster und Ereignisse zu erkennen. Typische Indikatoren sind fehlende oder redundante Werte und asynchrone Zeitstempel als binäre Regel pro Datenelement [85].

Zur Berechnung von DQ-Indikatoren in Streams werden meist Windowing- und Batching-Techniken verwendet, was einerseits eine inkrementelle Verarbeitung auf Segmenten, andererseits die Möglichkeit zur Anomalieerkennung bietet [86],[87],[88]. Rettig et al. [87] behandelt Anomaly Detection als Konsistenzprüfung auf mehreren Big Data-Streams, wozu diese gemeinsam auf ihre Konsistenz und Synchronisation analysiert werden [87].

ML-based Classification/Prediction

Hier werden überwachte Lernverfahren zur Identifikation, Klassifikation und Vorhersage von DQ-Problemen gebündelt. Klassische Modelle wie Random Forest oder Support Vector Machine (SVM) werden zur Fehlerklassifikation eingesetzt, während Regressions- oder Sequenzmodelle Missing-Value Prediction und Outlier Correction ermöglichen. Auch textbasierte Ansätze wie Out-of-Distribution Detection fallen in diese Gruppe. Maßgeblich ist die Nutzung gelabelter Trainingsdaten zur Modellbildung. Diese Verfahren grenzen sich von der klassischen Anomalieerkennung dadurch ab, dass sie meist supervised und zielgerichtet auf spezifische Fehlerklassen arbeiten.

Hossen et al. [89] nutzen ein ML-basiertes Data-Cleaning-Framework zur Behandlung von DQ-Problemen. Zunächst werden mittels Random Forest relevante Attribute bestimmt und darauf zwei Klassifikationsmodelle (Random Forest Classifier und Linear SVM) trainiert, um fehlerhafte bzw. unvollständige Datensätze zu identifizieren und fehlende Werte vorherzusagen [89]. Shi et al. [79] modellieren fehlende Sensor-Daten als Zeitreihen-Vorhersageproblem und nutzen SVMs zur Vorhersage. Ziel ist es, den fehlenden Wert aus multivariaten vorherigen Beobachtungen zu schätzen [79].

Elouataoui et al. [70] stellen in ihrem Active-Metadata Framework andere Verfahren vor. Nach der Identifikation von Anomalien und Daten-Nachbarschaften über IF und Metadaten trainieren sie ein Entscheidungsbaum-basiertes XGBoost-Modell auf den Nachbardaten, um korrigierte Werte für die Anomalien vorherzusagen. [70]

3.3. Ableitung von Bewertungskriterien aus der Theorie

In diesem Kapitel werden die unterschiedlichen DQ-Dimensionen und Problemtypen auf eine einheitliche Ebene konsolidiert. Ergänzt wird dies um die Betrachtung architektonischer Anforderungen an die Verfahren, welche sich aus der Theorie zu Microsoft Fabric und dessen technologischer Grundlage ergeben. Diese Ebene dient als Grundlage für die anschließende Erstellung des Bewertungsrasters (Kapitel 3.4).

Für die spätere Beurteilung, welche Verfahren für die Implementierung in Microsoft Fabric geeignet sind, ist die Definition auf welche DQ-Dimensionen sich die Methode auswirkt und welchen DQ-Problemen damit entgegnet werden kann, notwendig. Die bisherige Literaturarbeit zeigt, dass Dimensionen und Probleme zusammenhängen. Da ein Problemtyp mehrere Dimensionen verletzen und eine Dimension durch verschiedene Fehlerarten beeinträchtigt werden kann, ist eine getrennte Bewertung notwendig. Die Implementierung von DQ-Verfahren gelingt allerdings nur, wenn sie mit den Eigenschaften der Zielarchitektur kompatibel ist. Da dies in dieser Arbeit in einer Lakehouse-Architektur und konkret in Microsoft Fabric umgesetzt wird, müssen die ausgewählten Methoden nicht nur fachlich sinnvoll, sondern auch technisch realisierbar sein. Ein architekturbezogenes Bewertungskriterium ist daher erforderlich.

Datenqualitätsdimensionen

Um aus der Vielzahl theoretisch beschriebener Qualitätsmerkmale ein bewertbares Raster abzuleiten, wurden die identifizierten DQ-Dimensionen systematisch zusammengeführt. Die Konsolidierung folgt dem Ziel, unterschiedliche Begrifflichkeiten, Synonyme und Überschneidungen in thematischen Hauptdimensionen zu bündeln. Während etablierte Klassifikationen wie die von Wang & Strong [34] oder ISO 25012 [4] bereits eine robuste Grundlage bilden und sich in vielen Werken in ähnlicher Form wiederfinden, zeigt sich in jüngeren Publikationen zunehmend, dass moderne Datenumgebungen zusätzliche Anforderungen und Perspektiven erfordern, die klassische Modelle sinnvoll ergänzen.

Die resultierenden Hauptdimensionen entstehen somit nicht durch die Übernahme eines Frameworks, sondern aus einer Aggregation von Merkmalsgruppen. Die Tabelle Tab. 5 fasst diese Ableitung zusammen, zeigt die zugehörigen reduzierten und zusammengefassten Unterdimensionen und vermerkt ihre Literaturlbasis. Damit bilden die neuen Dimensionen den Ausgangspunkt für die spätere Bewertung in Kapitel 3.4.

HAUPTDIMENSION		Zugeordnete Dimensionen	Quellen	Ableitung
KORREKTHEIT & GENAUIGKEIT (1)		Accuracy, Correctness, Validity, Precision	[4], [34], [36], [35], [32], [37], [38], [39], [47], [48]	<i>Bündelt Merkmale, die inhaltliche Richtigkeit und formale Genauigkeit beschreiben.</i>
VOLLSTÄNDIGKEIT & DATENMENGE (2)		Completeness, Appropriate amount of data, Redundancy, Volatility	[4], [34], [36], [35], [32], [38], [39], [47], [48]	<i>Aspekte, die den Umfang der Informationen betreffen, ob genügend, nicht zu viele und nicht widersprüchliche Daten in stabiler Form vorliegen.</i>
RELEVANZ & KONTEXTBEZUG (3)		Relevancy, Value-added, Contextual, Granularity, Detail Level, Currency, Timeliness	[4], [34], [36], [35], [32], [39], [46], [47], [48]	<i>Inhaltliche Passung der Daten zum Nutzungskontext. Zweckdienlichkeit, sowohl im Detaillierungsgrad und zeitlich betrachtet.</i>
KONSISTENZ & INTEGRITÄT (4)		Consistency, Representational Consistency, Multi-Source Consistency, Integrity, Source-related	[4], [34], [36], [35], [32], [39], [45], [47], [48], [44]	<i>Innere Widerspruchsfreiheit und strukturelle Einheitlichkeit der Daten über Attribute und Quellen hinweg.</i>
INTERPRETIERBARKEIT & VERSTÄNDLICHKEIT (5)		Interpretability, Understandability, Readability,	[4], [34], [36], [35], [32], [37], [38], [39], [46], [47], [48]	<i>Merkmale, die die kognitive Erschließung der Daten unterstützen. Beschreiben</i>

HAUPTDIMENSION	Zugeordnete Dimensionen	Quellen	Ableitung
	Presentation Quality, Consistency of representation		<i>wie gut Daten präsentiert, erklärbar und intuitiv erfassbar sind.</i>
ZUGÄNGLICHKEIT & VERFÜGBARKEIT (6)	Accessibility, Availability, Usability, Portability, Recoverability	[4], [34], [36], [35], [38], [46], [47], [48]	<i>Nutzbarkeit im Sinne von Zugriff, Übertragung und Bereitstellung.</i>
SICHERHEIT & VERTRAUEN (7)	Security, Confidentiality, Access Security, Compliance, Reputation, Believability, Credibility, Ownership, Reliability	[4], [34], [36], [35], [32], [47], [48]	<i>Umfasst die sichere Behandlung und Vertrauenswürdigkeit.</i>

Tab. 5: Ableitung der DQ-Dimensionen zur Methodenbewertung

Probleme der Datenqualität

Analog zur Konsolidierung der DQ-Dimensionen wurden auch die in der Literatur identifizierten DQ-Probleme systematisch analysiert, zusammengeführt und in übergeordnete Fehlerkategorien abstrahiert. Ziel ist hierbei, aus der Vielzahl der Problembezeichnungen ein konsistentes und bewertbares Raster zu entwickeln.

In etablierten Klassifikationen der Literatur wie Oliveira et al. [40], Müller und Freytag [39] oder Rahm und Do [37] findet sich eine weitgehende Übereinstimmung in der Unterscheidung inhaltlicher, struktureller und semantischer Fehler. Gleichzeitig wird in jüngeren Publikationen deutlich, dass moderne Datenumgebungen zusätzliche Herausforderungen erzeugen. Arbeiten wie Saha und Srivastava [44], Hashem et al. [45], Cai & Zhu [46] oder Taleb et al. [47] verweisen insbesondere auf Probleme durch hohe Datenvolumina, zeitliche Volatilität, Heterogenität, verteilte Systemarchitekturen oder begrenzte Skalierbarkeit hinsichtlich DQ-Maßnahmen. Diese Aspekte erweitern klassische Fehlerkataloge gezielt auf die in dieser Arbeit relevante Architektur.

Ein wiederkehrendes Muster in der Literatur betrifft Fehler, die im Zusammenhang mit mehreren Systemen stehen. Obwohl Multi-Source-Probleme eine wesentliche Rolle spielen, wurde im Rahmen dieser Arbeit bewusst darauf verzichtet, eine eigene Fehlerkategorie dafür zu definieren, da die entstehenden Fehlerarten z. B. widersprüchliche Werte, strukturelle Heterogenität oder Schema-Konflikte, anderen bestehenden Kategorien wie *Konsistenz & Struktur*, *Semantik* oder *Vollständigkeit* zugeordnet werden können. Die Multi-Source-Perspektive stellt somit weniger eine Kategorie als vielmehr

einen kontextuellen Auslöser dar, der sich durch alle Fehlergruppen zieht und deren Ausprägung beeinflusst.

Eine ähnliche Entscheidung betrifft die Kategorie *Sicherheit & Verfügbarkeit*, die neben klassischen Aspekten wie Zugriffsbeschränkungen und technischer Verfügbarkeit auch Merkmale zu Glaubwürdigkeit und Reputation berücksichtigt. Diese umfangreiche Erweiterung wurde vorgenommen, da im Rahmen moderner Architekturen technische und infrastrukturelle Maßnahmen oftmals den Umgang mit solchen Fehlern bestimmen. Diese Kategorie wurde stärker zusammengefasst, da der Gestaltungsspielraum in diesen Problemtypen durch die im Unternehmenskontext spezifizierte Systemlandschaft limitiert ist.

Tabelle Tab. 6 fasst die konsolidierten Fehlerkategorien zusammen, zeigt die jeweils zugeordneten Problemtypen sowie ihre maßgeblichen Quellen und erläutert die zugrundeliegende Ableitung.

Die resultierenden Fehlerkategorien erheben keinen Anspruch auf Vollständigkeit. Dazu weisen einige der identifizierten Problemtypen Merkmalsüberschneidungen zu mehreren Fehlerkategorien auf. So können veraltete Werte als Inhalts- und Konsistenzproblem interpretiert werden, während Format-Anomalien strukturelle und semantische Dimensionen zugleich berühren. In dieser Arbeit wird auf eine Mehrfachklassifikation bewusst verzichtet und stattdessen die jeweils prägende Kategorie gewählt.

FEHLERKATEGORIE	Zugeordnete Problemtypen	Quellen	Ableitung
INHALTLICH (1)	Falscher Wert/Abweichung, Integritätsverletzung (Referenziell, Eindeutigkeit, Constraints), fehlerhafte Transformation/Sensordaten, Erfassungs- oder menschliche Fehler, statistische Ausreißer, temporale/räumliche/kategorische Abweichung, Intervallfehler	[36], [37], [38], [39], [40], [44], [47]	<i>Fehler, bei denen Daten inhaltlich falsch, ungültig oder veraltet sind. Umfasst klassische Genauigkeits- und Korrektheitsprobleme.</i>
VOLLSTÄNDIGKEIT (2)	Fehlende Werte, unvollständige Datensätze/Erfassung, Coverage-Anomalien/unvollständige Quellen, Verlustdaten durch Verarbeitung/Latenz	[36], [38], [39], [40]	<i>Fehlende oder unzureichend erfasste Daten, die die Abdeckung des relevanten Informationsraums beeinträchtigen.</i>
KONSISTENZ & STRUKTUR (3)	Syntaktische Anomalien (Domain-Format, Typ, Syntax), uneinheitliche Kodierung, inkompatible/überlappende/di-	[36], [37], [38], [39], [40], [45], [44], [46], [47]	<i>Betreffen nicht den semantischen Inhalt, sondern die Form der</i>

FEHLERKATEGORIE	Zugeordnete Problemtypen	Quellen	Ableitung
	vergierende Formate/Schemata, unterschiedliche Repräsentation, Namenskonflikte/Synonyme, Widersprüchliche Werte, Redundanz, Duplikate, zirkuläre Selbstbeziehung, zeitliche Inkonsistenz, Trigger/Transaktions/Fremdschlüsselproblem, Referenzen, fehlende Standards		<i>Daten über syntaktische und strukturelle Inkonsistenzen. Entstehen im Rahmen des Integrations- und Transformationsprozesses in Single- und Multi-Source-Systemen.</i>
SEMANTIK (4)	Ungeeignet für Kontext/Zweck, bedeutungsloser/zweifelhafter Wert, falscher Detailgrad/Granularität, Mehrdeutigkeiten (Abkürzung, fehlender Semantikkontext), Probleme bei Relevanz/Abbildung Realität, nicht aggregierbare Daten, fehlender/unzureichender Kontext/Lesbarkeit/Interpretierbarkeit, fehlende Metadaten, konflikthafte/uneinheitliche Darstellung/Repräsentation, erforderliches Domänenwissen	[36], [37], [38], [39], [40], [40]	<i>Daten können formal korrekt sein, aber im Nutzungskontext unpassend, nicht eindeutig interpretierbar, semantisch inkohärent oder liegen ohne Metadaten vor.</i>
SICHERHEIT & VERFÜGBARKEIT (5)	Zugriffsbeschränkungen, mangelnde Glaubwürdigkeit/Verlässlichkeit, Outdated Referenz, Fehlende benötigte Daten, Datenvolumina (Verarbeitungszeiten/Fehlverarbeitung), zeitliche Volatilität, technische Verfügbarkeit (Ausfälle, Rechenleistung), instabile Kommunikationskanäle, verteilte Speicherorte/heterogene Schnittstellen, Skalierungsprobleme/fehlende Qualitätssicherung	[36], [40], [44], [46], [47]	<i>Risiken im Umgang mit Datenquellen, etwa systemische, architekturelle oder prozessuale Einschränkungen, die Sicherheit, Aktualität, Datenzugriff oder Bereitstellung beeinträchtigen.</i>

Tab. 6: Ableitung der Problemtypen zur Methodenbewertung (eigene Darstellung)

Architekturbezogene Anforderungen

Um die Kompatibilität der DQ-Verfahren mit der Zielarchitektur bewerten zu können, werden zunächst die in Kapitel 2.1 dargestellten architektonischen Eigenschaften moderner Lakehouse-Architekturen sowie die spezifischen Komponenten und Funktionsweisen von Microsoft Fabric (Kapitel 2.4) herangezogen.

Diese Sammlung architekturerelevanter Eigenschaften wird anschließend einer thematischen Gruppierung unterzogen. Die daraus entstehenden Gruppen werden unter folgender Fragestellung betrachtet: *„Welche Implikationen besitzt die jeweilige architektonische Eigenschaft für die Umsetzung und Wirksamkeit von Datenqualitätsverfahren?“*

Auf dieser Grundlage werden Anforderungen an DQ-Verfahren abgeleitet, die deren technische Eignung für die Architektur beschreiben.

In einem weiteren Schritt werden inhaltlich ähnliche Anforderungen konsolidiert und auf ihre praktische Bewertbarkeit überprüft. Das Ergebnis ist eine reduzierte, konsistente Menge architekturbezogener Anforderungen, die als Grundlage für die Bewertung der Architekturkompatibilität der Verfahren dient. Die Operationalisierung dieser Anforderungen erfolgt im Bewertungsraster in Kapitel 3.4.

Das Ergebnis dieses Vorgehens ist die nachfolgend in Tab. 7 dargestellte tabellarische Übersicht. Darin werden den identifizierten Eigenschaften ihre zugehörigen Kernaussagen gegenübergestellt, jeweils ergänzt um die entsprechenden Quellen aus den vorangegangenen Kapiteln sowie der daraus abgeleiteten Anforderung.

ARCHITEKTURMERKMAL	Kernaussagen	Quellen	Abgeleitete Anforderung
SKALIERBARKEIT UND VERTEILTE AUSFÜHRUNG (1)	• Verarbeitung großer Datenmengen • Heterogene Datenquellen und -domänenspezifische • Parallele, verteilte Architekturen • BigData- und Cloud-Computing als Antwort auf Skalierungsanforderungen • Apache Spark als verteilte Compute-Engine	[90], [41], [43], [45], [46], [50]	Das Verfahren muss in einer verteilten Compute-Umgebung skalierbar einsetzbar sein und mit großen Datenvolumina umgehen können.
KOMPATIBILITÄT MIT SPEICHERFORMATEN (2)	• Offene Tabellenformate (Parquet) • Delta Lake als offenes Speichersystem • ACID-Transaktionen, Versionierung, Upsert, MERGE • Transaktionale Metadaten-Schicht • OneLake als Single Source of Truth	[49], [18], [19], [51]	Datenqualitätsverfahren müssen mit offenen, dateibasierten Speicherformaten wie Delta/Parquet kompatibel sein und Schema-on-Read sowie Schema-Evolution unterstützen.
EINORDNUNG MEDAILLON-ARCHITEKTUR (3)	• Medallion-Architektur, • End-to-End-Qualitätssicherung über Datenlebenszyklus • Data as a Product (Delta Table/Semantic Model)	[9], [47], [50], [52], [91], [92], [61]	Datenqualitätsverfahren sollten klar einer oder mehreren Stufen der Medallion-Architektur zuzuordnen sein.
UNTERSTÜTZUNG VERARBEITUNGSMODI (4)	• Batch- und Streaming-Verarbeitung • (Near-)Realtime-Use Cases, • Strukturierte, semi-strukturierte und unstrukturierte Daten	[49], [19], [43], [44]	Datenqualitätsverfahren müssen mindestens batchfähig sein und sollten auch in Streaming- oder Near-Realtime-Szenarien einsetzbar sein, um den Anforderungen moderner Datenpipelines gerecht zu werden.

ARCHITEKTURMERKMAL	Kernaussagen	Quellen	Abgeleitete Anforderung
INTEGRATION IN MICROSOFT FABRIC KOMPONENTEN (5)	• Einheitliche Plattform für unterschiedliche Workloads und Datenquellen • Verschiedene Fabric-Komponenten • Spark, SQL, Notebooks, Pipelines • Copilot- und Agent-Unterstützung	[49], [90], [93], [94], [8], [59]	Datenqualitätsverfahren müssen sich technisch in die Komponentenlandschaft von Microsoft Fabric integrieren lassen, z. B. als Notebook, Pipeline-Schritt, SQL-Logik, ML-Workflow oder Agent.
AUTOMATISIERBARKEIT UND PIPELINE-FÄHIGKEIT (6)	• ETL- und ELT-Pipelines, Orchestrierung • Regelmäßige, reproduzierbare Ausführung, Scheduling • Adaptive, datengetriebene Qualitätsregeln • Inkrementelle Anpassung und dezentrale Fehlerkorrektur	[90], [44], [46], [93], [8]	Datenqualitätsverfahren müssen automatisierbar und reproduzierbar in Pipelines eingebunden werden können.

Tab. 7: Ableitung der architekturbezogenen Anforderungen zur Methodenbewertung (eigene Darstellung)

Ein zentrales Merkmal moderner Datenplattformen ist ihre Ausrichtung auf *verteilte und skalierbare Verarbeitung*, um große und heterogene Datenmengen effizient zu verarbeiten. Daraus ergibt sich die Anforderung, dass DQ-Verfahren parallelisierbar sind und ohne zentrale Engpässe in verteilten Compute-Umgebungen ausgeführt werden können. Stark sequenzielle Verfahren sind in diesem Kontext nur eingeschränkt geeignet.

Eng damit verbunden ist die Nutzung *offener, dateibasierter Speicherformate* (z. B. Parquet) in Lakehouse-Architekturen. DQ-Verfahren müssen daher mit Eigenschaften wie Schema-on-Read, Schema-Evolution und transaktionalen Metadaten umgehen können, um ohne zusätzliche Transformationsschichten einsetzbar zu sein.

Die *Medallion-Architektur* strukturiert den Datenlebenszyklus in klar getrennte Verarbeitungsstufen. Für DQ-Methoden ist daher entscheidend, ob sie sich diesen Stufen zuordnen lassen, was sowohl die technische Umsetzung als auch die konzeptionelle Einbettung entlang des Datenlebenszyklus unterstützt.

Darüber hinaus müssen Verfahren *unterschiedliche Verarbeitungsmodi* unterstützen, insbesondere Batch- und Streaming- beziehungsweise Near-Realtime-Szenarien, und mindestens für einen dieser Modi geeignet sein.

Im Kontext von Microsoft Fabric ist zudem die *Integrationsfähigkeit in bestehende Plattformkomponenten* wesentlich. DQ-Verfahren müssen sich in die vorhandene Architektur einfügen sowie *automatisierbar und pipelinefähig* sein, da DQ kontinuierlich und reproduzierbar überwacht werden muss.

Insgesamt bilden diese Anforderungen eine technisch motivierte Bewertungsgrundlage, um die praktische Eignung von DQ-Verfahren unter den Bedingungen einer Lakehouse-Architektur und Microsoft Fabric zu beurteilen.

3.4. Entwicklung des Bewertungsrasters

Das Ziel des folgenden Abschnitts ist die Entwicklung des Bewertungsrasters, um die im Methodenkatalog identifizierten Verfahren transparent und theoriegeleitet (Kapitel 2) bewerten zu können.

DQ-Verfahren verfolgen unterschiedliche Zielsetzungen und wirken auf verschiedene Qualitätsdimensionen ein. Die aus der Theorie abgeleiteten DQ-Dimensionen (Tab. 5) und Problemtypen (Tab. 6) ermöglichen eine valide Beurteilung der fachlichen Wirksamkeit einer Methode. Ergänzt wird dies durch die Betrachtung von Anforderungen, welche sich aus dem Implementierungskontext und der Zielarchitektur ergeben (Tab. 7). Ebenfalls sollte die Umsetzbarkeit im Use Case hinsichtlich des Implementierungsaufwands in die Betrachtung mit einfließen.

Datenqualitäts-Dimensionen

Das Ziel der Verfahren ist die Sicherstellung oder Verbesserung von DQ. Dafür ist es notwendig, die Methoden nach ihrer fachlichen Wirksamkeit zu beurteilen. Dies lässt sich entlang der in Kapitel 2.2 erarbeiteten DQ-Dimensionen erfassen. Grundlage dafür bildet Tab. 5, worin die unterschiedlichen in der Literatur etablierten Dimensionen erfasst und konsolidiert wurden. Für die Beurteilung der Methodengruppen eignet sich die Fragestellung "*Welche Dimensionen werden adressiert?*" und "*In welchem Umfang wird diese beeinflusst?*". Dafür wird folgende Score-Skala definiert:

- **0:** Die Methode stellt keinen Bezug auf die Dimension her.
- **1:** Die Methode weist eine schwache oder indirekte Wirkung auf die Dimension auf.
- **2:** Die Methode beeinflusst die Dimension gezielt.

Zur Vergleichbarkeit der Cluster wird zusätzlich das arithmetische Mittel der Dimensions-Scores als Gesamtscore ausgewiesen.

Problemtypen der Datenqualität

Analog werden die in Kapitel 2.3 zusammengetragenen Problemkategorien im Methodenkontext betrachtet. Die Fragestellung *“Inwiefern trägt die Methode zur Behandlung eines DQ-Problems bei?”* ist ausschlaggebend für die ergänzende Bewertung der Wirksamkeit eines Verfahrens. Zur Berechnung des Gesamtscores werden die einzelnen Problemkategorien (Tab. 6) ebenfalls mit einem Score zwischen 0 und 2 bewertet.

- **0:** Das Problem wird von der Methode nicht adressiert.
- **1:** Der Problemtyp wird durch die Methode erkannt.
- **2:** Die Methode ermöglicht die Prävention oder Korrektur von Problemen.

Diese Kategorie wird ebenfalls mit Gesamt- und Einzelscores in das Bewertungsraster aufgenommen und bildet mit den Dimensionen die fachliche Wirksamkeit ab.

Architektur

Diese Dimension basiert auf den architektonischen Anforderungen (Tab. 7) und bewertet, wie gut entsprechende DQ-Verfahren in der Zielarchitektur umgesetzt werden können.

Für die Architekturkompatibilität wird folgender Score definiert:

- **0:** Ungeeignet. Die Methode lässt sich nicht auf die Anforderung abbilden und erfüllt sie nicht.
- **1:** Eingeschränkt geeignet. Die Methode ist grundsätzlich mit den architektonischen Rahmenbedingungen vereinbar, weist jedoch Unstimmigkeiten in Teilen oder andere Bedenken auf (ist z. B. nicht nativ mit vorhandenen Komponenten umsetzbar).
- **2:** Geeignet. Lässt sich mit moderatem Aufwand über Fabric-Komponenten integrieren und erfüllt die Fabric-spezifischen Anforderungen.

Die Darstellung erfolgt analog zur fachlichen Betrachtung in Gesamt- und Einzelscores.

Komplexität

Ein weiterer, die Eignung des Use Cases ebenfalls beeinflussender Aspekt ist die Komplexität der Methode und der damit verbundene Implementierungsaufwand. Hierbei wird die Komplexität der Methode selbst, sowie der Aufwand diese in Microsoft Fabric unter Nutzung der vorhandenen Komponenten (nativ, Workaround) zu implementieren, betrachtet. Die Komplexität wird dabei nach der Frage *“Wie hoch ist der Aufwand zur*

Implementierung der Methoden in Microsoft Fabric?“ bewertet und jeweils mit einer begründeten Einschätzung von niedrig bis hoch ausgewiesen.

Skalierbarkeit

Ergänzend wird eine Einschätzung gegeben, wie leicht Verfahren skalierbar sind, um mit großen Datenmengen und veränderter Datengrundlage umzugehen. Dies ermöglicht auch, in Betracht zu ziehen, wie eine generalisierte längerfristige DQ-Sicherung durch die Methoden gewährleistet werden kann. Die Leitfrage *“Wie skalierbar sind die Verfahren und eignen sie sich für eine kontextunabhängige Übertragung auf andere Datengrundlagen?”* wird dabei ergänzend zur Komplexitätsbetrachtung als eigene Kategorie mit niedrig, mittel oder hoch begründet.

Eignung im Use Case

Letztlich fasst diese Dimension die praktische Umsetzbarkeit und den potenziellen Nutzen der Methoden zusammen und dient als Grundlage für die Auswahl repräsentativer Methoden des Clusters für die praktische Anwendung im Use Case. Dafür werden die Scores der Dimensionen, Problemtypen und Architektur-Anforderungen gemittelt, um einen aggregierten Gesamtscore für das Methodencluster zu erhalten. Für die spätere Auswahl und Priorisierung werden Komplexität und Skalierbarkeit in die Auswahlbegründung aufgenommen.

Auf Basis der einzeln abgeleiteten Bewertungsaspekte ist die Struktur des Bewertungsrasters nachfolgend in Tab. 8 übersichtlich dargestellt. Dies wird in Kapitel 3.2 auf den Methodenkatalog angewandt.

KRITERIUM	Bedeutung	Score-Skala	Grundlage
DQ-DIMENSIONEN (1)	Welche Dimensionen werden adressiert?	0-2	Kapitel 2.2
DQ-PROBLEMTYPEN (2)	Inwiefern trägt die Methode zur Behandlung eines DQ-Problems bei?	0-2	Kapitel 2.3
ARCHITEKTURKOMPATIBILITÄT (3)	Ist die Methode für Lakehouses geeignet und lässt sie sich mit Microsoft Fabric Komponenten umsetzen?	0-2	Kapitel 2.4, Kapitel 3.3
KOMPLEXITÄT (4)	Wie hoch ist der Implementierungsaufwand?	Niedrig - Hoch	Eigene Einordnung
SKALIERBARKEIT (5)	Wie geeignet ist die Methode für zukünftige Skalierung oder Generalisierung?	Niedrig - Hoch	Eigene Einordnung

KRITERIUM	Bedeutung	Score-Skala	Grundlage
EIGNUNG IM USE CASE (6)	Wie geeignet ist das Verfahren für eine Integration in die Use Cases unter Gesamtbetrachtung der Kriterien?	Durchschnitt (1)-(3)	Score: (1)-(3), Einordnung (4), (5)

Tab. 8: Bewertungsraster (eigene Darstellung)

3.5. Anwendung des Bewertungsrasters auf die Methodencluster

Die Anwendung des Bewertungsrasters (Tab. 8) erfolgt auf der Ebene der gebildeten Methodencluster, da die innerhalb eines Clusters zusammengefassten Verfahren auf einem vergleichbaren Wirkprinzip basieren und ähnliche Anforderungen hinsichtlich Implementierung, Datenvoraussetzungen und Architekturintegration aufweisen. Eine Einzelbewertung jedes Verfahrens würde daher keine signifikant differenzierenden Erkenntnisse liefern.

Tabelle Tab. 9 zeigt exemplarisch die Bewertung dreier Methodencluster aus der Bewertungsmatrix (Kapitel B). Die einzelnen Kriterien der DQ-Dimensionen (Tab. 5), DQ-Problemtypen (Tab. 6) und architekturbezogenen Anforderungen (Tab. 7) werden zur Übersichtlichkeit in verkürzter Weise mit ihrer Indexnummer dargestellt. Die einzelnen Aspekte werden daraufhin anhand des definierten Scores (Kapitel 3.4) bewertet und daraus ein Durchschnittswert gebildet. Die Bewertung erfolgt als analytische Einschätzung im Kontext der untersuchten Microsoft Fabric-Umgebung und basiert auf den zuvor dargestellten Wirkmechanismen der Verfahren (Kapitel 3.2).

METHOD- ENKATEGORIE	DQ-Dimensio- nen	DQ-Problem- typen	Architek- turkompatibil- ität Fabric	Komplexität	Skalierbarkeit	Eignung im Use Case
RULE-BASED DATA VALIDA- TION	• (1): 2 • (2): 2 • (3): 1 • (4): 2 • (5): 1 • (6): 0 • (7): 1 1,29	• (1): 2 • (2): 2 • (3): 2 • (4): 1 • (5): 0 1,4	• (1): 2 • (2): 2 • (3): 2 • (4): 2 • (5): 2 • (6): 2 2	Niedrig-Mittel	Mittel-Hoch	$(1,29 + 1,4 + 2) / 3 = 1,56$
STATISTICAL DATA PROFIL- ING	• (1): 1 • (2): 2 • (3): 2 • (4): 2 • (5): 1	• (1): 2 • (2): 1 • (3): 2 • (4): 2 • (5): 0	• (1): 2 • (2): 2 • (3): 2 • (4): 1 • (5): 2	Mittel	Hoch	$(1,29 + 1,4 + 1,83) / 3 = 1,51$

METHOD- ENKATEGORIE	DQ-Dimensio- nen	DQ-Problem- typen	Architek- turkompatibil- ität Fabric	Komplexität	Skalierbarkeit	Eignung im Use Case
	• (6): 0 • (7): 1 1,29	1,4	• (6): 2 1,83			
ANOMALY DE- TECTION	• (1): 2 • (2): 0 • (3): 2 • (4): 2 • (5): 1 • (6): 0 • (7): 1 1,14	• (1): 2 • (2): 1 • (3): 2 • (4): 1 • (5): 0 1,2	• (1): 2 • (2): 2 • (3): 2 • (4): 1 • (5): 2 • (6): 1 1,67	Mittel	Mittel-Hoch	$(1,14 + 1,2 + 1,67) / 3 = 1,34$

Tab. 9: Auszug aus der Bewertungsmatrix (eigene Darstellung, Tab. B)

Rule-based Data Validation

Regelbasierte Verfahren adressieren primär formale und strukturelle DQ-Aspekte, indem Datenwerte gegen explizit definierte Soll-Bedingungen geprüft werden. Sie eignen sich insbesondere zur Identifikation fehlerhafter Werte und Integritätsverletzungen sowie zur Sicherstellung von Vollständigkeit und struktureller Konsistenz, etwa durch Pflichtfeld- oder Formatprüfungen.

Ihre Wirkung bleibt jedoch auf klar definierte Regelräume beschränkt. Semantische Probleme wie inhaltliche Widersprüche oder kontextabhängige Interpretationsfragen können nur insoweit erfasst werden, wie sie formalisiert sind. Entsprechend ist die Wirkung auf semantische Problemtypen indirekt und begrenzt. Sicherheits- und verfügbarkeitsbezogene Aspekte liegen grundsätzlich außerhalb ihres Anwendungsbereichs.

Bezogen auf die DQ-Dimensionen zeigen regelbasierte Verfahren eine direkte Wirkung auf Korrektheit & Genauigkeit, Vollständigkeit & Datenmenge sowie Konsistenz & Integrität, da diese durch deterministische Prüfregeln unmittelbar beeinflusst werden. Indirekte Effekte bestehen für Relevanz & Kontextbezug, etwa durch zeit- oder domänenspezifische Regeln, sowie für Interpretierbarkeit & Verständlichkeit durch Standardisierung von Formaten und Typen. Zugänglichkeit & Verfügbarkeit bleiben unberührt, während Sicherheit & Vertrauen durch konsistente und regelkonforme Daten zumindest mittelbar gestärkt werden. Insgesamt sind regelbasierte Ansätze damit besonders

geeignet für klar definierte, formale und strukturelle Qualitätsprobleme, während semantische und infrastrukturelle Fragestellungen nur eingeschränkt adressiert werden.

Hinsichtlich architektonischer Anforderungen erfüllen sie einen Großteil der Kriterien. Sie lassen sich gut in bestehende Fabric-Komponenten integrieren, sind pipelinefähig und können entlang der gesamten Medallion-Architektur, von quellsystemnahen Prüfungen bis in den Gold-Layer eingesetzt werden. Zudem sind sie mit gängigen Speicherformaten kompatibel und sowohl für Batch- als auch für Streaming-Szenarien geeignet.

Der Implementierungsaufwand ist insgesamt Gering bis Mittel, da viele Validierungen (z. B. Datentyp- oder Formatprüfungen) generisch und mit überschaubarem Aufwand umsetzbar sind. Mit zunehmender Domänenspezifität steigt neben der Wirksamkeit jedoch die Komplexität, da fachliche Regeln eine präzise Formalisierung sowie Abstimmung und kontinuierliche Pflege erfordern. Auch bei der Skalierbarkeit gilt es zu differenzieren. Technisch sind regelbasierte Verfahren gut skalierbar, da sie deterministisch und parallelisierbar sind. Die fachliche Skalierbarkeit nimmt jedoch mit wachsender Domänenspezifität ab, was zu einer Einordnung im Bereich mittel bis hoch führt.

Die Eignung für den Use Case wird anhand der Mittelung der Scores mit **1,56** bewertet.

Statistical Data Profiling

Im Gegensatz zu regelbasierten Verfahren definiert Profiling keine expliziten Soll-Zustände, sondern dient der explorativen und quantitativen Charakterisierung von Daten anhand deskriptiver Kennzahlen, Verteilungen und Abhängigkeiten. Ziel ist die systematische Transparenz über Datenstruktur, Werteverteilungen und potenzielle Auffälligkeiten.

In Bezug auf DQ-Probleme adressiert statistisches Profiling vor allem inhaltliche Auffälligkeiten wie Ausreißer oder ungewöhnliche Kategorien sowie strukturelle und konsistenzbezogene Probleme (unerwartete Verteilungen oder Abhängigkeiten). Semantische Auffälligkeiten lassen sich dabei nur indirekt erkennen, bspw. über dominante Kategorien oder Korrelationen. Vollständigkeitsprobleme werden primär sichtbar gemacht, jedoch nicht aktiv verbessert, während Sicherheits- und verfügbarkeitsbezogene Aspekte unberührt bleiben.

Bezogen auf die DQ-Dimensionen zeigt Profiling eine analytische Wirkung auf Vollständigkeit & Datenmenge, da fehlende Werte, Verteilungen und Datenvolumina systematisch quantifiziert werden. Ebenso unterstützt es die Beurteilung von Konsistenz & Integrität durch die Identifikation struktureller Unregelmäßigkeiten. Eine indirekte Wirkung besteht auf Korrektheit & Genauigkeit, da potenziell fehlerhafte Werte erkannt, jedoch nicht bewertet werden. Für Relevanz & Kontextbezug ergeben sich gezielte Hinweise durch Zeitreihen- oder Korrelationsanalysen, die auf veraltete oder fachlich ungewöhnliche Daten hinweisen. Die Interpretierbarkeit & Verständlichkeit wird durch erhöhte Transparenz unterstützt, ohne jedoch automatisch semantische Klarheit zu schaffen. Sicherheit & Vertrauen können durch die Nachvollziehbarkeit des Datenzustands gestärkt werden, während Zugänglichkeit & Verfügbarkeit unbeeinflusst bleiben. Insgesamt fungiert Statistical Data Profiling primär als analytisches Instrument zur Quantifizierung und Visualisierung von DQ-Zuständen.

Hinsichtlich der architektonischen Anforderungen ergibt sich ein ähnliches Bild wie bei regelbasierten Verfahren. Profiling lässt sich gut in verteilten Umgebungen auf (Spark-DataFrame-Ebene) skalierbar umsetzen und entlang der Medallion-Architektur einsetzen, sowohl zur initialen Analyse als auch zur Überprüfung nach Transformationen. Die Integration in Pipelines ist problemlos über Notebook-basierte Verarbeitungsschritte möglich. Einschränkungen bestehen bei Streaming-Szenarien, da Profiling auf aggregationsbasierte Batch-Analysen ausgerichtet ist. Streaming-Ansätze (z. B. über Windowing) sind technisch möglich, im Kontext dieser Arbeit jedoch nur eingeschränkt praktikabel.

Die Komplexität wird insgesamt als mittel eingestuft. Während einfache Kennzahlen wie Null-Raten oder Basisstatistiken mit geringem Aufwand umsetzbar sind, steigt der Implementierungs- und Interpretationsaufwand bei multivariaten Verfahren oder Abhängigkeitsanalysen deutlich und erfordert teilweise Domänenwissen. Die Skalierbarkeit ist hingegen Hoch, da die Verfahren überwiegend auf Aggregationen und Verteilungen basieren und sich in verteilten Architekturen effizient parallelisieren lassen, insbesondere durch native Unterstützung in Spark.

Die Eignung für den Use Case wird mit **1,51** bewertet.

Anomaly Detection

Anomaly-Detection-Verfahren zielen auf die Identifikation von Datenpunkten und Mustern, die signifikant vom erwarteten Normalverhalten abweichen. Ähnlich wie Statistical Data Profiling sind sie datengetrieben und basieren auf Modellen oder Nachbarschaftsstrukturen. Auffälligkeiten werden dabei relativ zur vorhandenen Datenstruktur bestimmt und nicht explizit vorgegeben.

Sie adressieren insbesondere inhaltliche Auffälligkeiten, indem ungewöhnliche Werte oder inkonsistente Attributkombinationen erkannt werden. Auch konsistenz- und strukturebezogene Probleme, etwa Verletzungen von Attributbeziehungen oder unplausible Clusterzugehörigkeiten, lassen sich identifizieren. Vollständigkeits- und semantische Probleme werden hingegen nur indirekt erfasst, da fehlende Werte oder Bedeutungsabweichungen nur dann auffallen, wenn sie sich als Abweichung im Datenmuster manifestieren.

Bezogen auf die DQ-Dimensionen zeigen Anomaly-Detection-Verfahren eine direkte Wirkung auf Korrektheit & Genauigkeit sowie Konsistenz & Integrität, da unplausible oder fehlerhafte Datenpunkte identifiziert werden. Auch Relevanz & Kontextbezug können durch die Erkennung zeitlicher oder struktureller Abweichungen unterstützt werden. Die Interpretierbarkeit bleibt jedoch eingeschränkt, da die Klassifikation als Anomalie oft nur begrenzt nachvollziehbar ist. Zugänglichkeit & Verfügbarkeit werden nicht beeinflusst und Sicherheit & Vertrauen können indirekt durch die frühzeitige Erkennung auffälliger Datenzustände gestärkt werden.

Hinsichtlich der Architektur sind die Verfahren grundsätzlich gut integrierbar, wenn auch mit höherem Aufwand als einfache, aggregationsbasierte Ansätze. Viele Verfahren lassen sich parallelisieren und auf strukturierten Daten anwenden (z. B. Distanzbasierte). In der Medallion-Architektur bieten sie sich vor allem im Silver-Layer an, etwa für Monitoring auf bereits transformierten Daten. Die Integration in Microsoft Fabric ist über Spark-Notebooks oder ML-Funktionen möglich und unterstützt prinzipiell Automatisierung, erfordert jedoch zusätzlichen Implementierungsaufwand. Streaming-basierte Ansätze existieren, sind jedoch weniger verbreitet und analog zum Profiling mit erhöhtem Aufwand verbunden.

Die Komplexität wird insgesamt als mittel eingestuft, da neben der technischen Umsetzung insbesondere Modellwahl, Parametrisierung und Schwellenwertdefinition zusätzlichen Aufwand verursachen. Während einfache statistische Verfahren zur Ausreißererkennung vergleichsweise leicht implementierbar sind, steigt die Komplexität bei clustering- oder ML-basierten Ansätzen deutlich. Die Skalierbarkeit wird als mittel bis hoch bewertet, da viele Verfahren parallelisierbar und für verteilte Systeme geeignet sind, jedoch bei hoher Dimensionalität oder komplexen Modellen ressourcenintensiv werden können.

Im Gesamtscore ergibt sich für diese Kategorie eine Bewertung von **1,34**.

Matching, Deduplication

Verfahren dieser Kategorie zielen auf die Identifikation und Zusammenführung redundanter oder semantisch identischer Datensätze ab. Im Fokus steht dabei nicht die Prüfung einzelner Werte, sondern die Auflösung mehrfacher Repräsentationen derselben Entität, innerhalb oder über verschiedene Datenquellen hinweg. Entsprechend wirken sie primär auf strukturelle und identitätsbezogene Qualitätsaspekte wie Konsistenz & Struktur sowie semantische Problemfelder. Unterschiedliche Schreibweisen, Attributkombinationen oder Repräsentationen werden zusammengeführt und vereinheitlicht. Eine Verbesserung für Vollständigkeitsprobleme ergibt sich dabei nur indirekt durch die Konsolidierung verteilter Informationen. Inhaltliche Fehler einzelner Werte werden hingegen nicht gezielt erkannt und können sich im Zuge der Zusammenführung fortpflanzen.

Bezogen auf die DQ-Dimensionen entfalten die Verfahren ihre stärkste Wirkung auf Vollständigkeit & Datenmenge sowie Konsistenz & Integrität, da redundante Daten reduziert und verteilte Informationen zusammengeführt werden. Indirekte Effekte bestehen auf Relevanz & Kontextbezug sowie Interpretierbarkeit & Verständlichkeit, da konsolidierte Datenbestände eine höhere inhaltliche Dichte und geringere Redundanz aufweisen. Zugänglichkeit & Verfügbarkeit bleiben unberührt und Sicherheit & Vertrauen können erneut lediglich indirekt durch eine konsistente Datenbasis gestärkt werden.

Aus architektonischer Sicht sind Matching- und Deduplication-Verfahren aufwendiger als die zuvor betrachteten Kategorien. Sie erfordern häufig globale Vergleiche, komplexe

Join-Operationen sowie mehrstufige Ähnlichkeits- und Blocking-Verfahren. Zwar lassen sich einzelne Schritte parallelisieren, der Ressourcenbedarf steigt jedoch insbesondere bei paarweisen Vergleichen deutlich an. Die Integration in Lakehouse-Architekturen ist grundsätzlich möglich, da auf tabellarischen Daten gearbeitet wird und Ergebnisse persistiert werden können. Typischerweise erfolgt der Einsatz vom Silver- zum Gold-Layer, um eine konsolidierte Entitätssicht für nachgelagerte Analysen bereitzustellen. Einschränkungen bestehen insbesondere bei Streaming-Szenarien, da kontinuierliches Matching zusätzliche Mechanismen wie Windowing oder State Management erfordert. Auch die Integration in Microsoft Fabric ist prinzipiell möglich, jedoch stärker durch kontextspezifische Logik, Custom Code und erhöhten Orchestrierungsaufwand geprägt, was die Automatisierbarkeit einschränkt.

Die Komplexität wird insgesamt als mittel bis hoch bewertet, da neben der technischen Umsetzung insbesondere die Definition von Ähnlichkeitsmetriken, Blocking-Strategien, Schwellenwerten und fachlichen Regeln (z. B. Golden Records, Konfliktauflösung) einen hohen konzeptionellen und operativen Aufwand erfordert. Die Skalierbarkeit ist hingegen eingeschränkt, da trotz möglicher Parallelisierung paarweise Vergleiche, große Vergleichsräume und hohe Kardinalitäten zu überproportional steigendem Rechenaufwand führen. Zudem ist die Übertragbarkeit auf andere Datensätze aufgrund der starken Domänenspezifik begrenzt.

Die Eignung für Matching und Deduplication wird mit **1,19** bewertet.

Automatisiertes Repairing

Verfahren dieser Kategorie unterscheiden sich dadurch, dass sie aktiv Korrekturen an Daten vornehmen und damit identifizierte Fehler voraussetzen. Sie greifen direkt in die Datenstruktur ein, indem fehlende Werte ergänzt, fehlerhafte ersetzt und inkonsistente Einträge transformiert werden. Dadurch wirken sie vor allem auf Vollständigkeit und inhaltliche Plausibilität. Konsistenz- und Strukturprobleme werden hingegen nur eingeschränkt adressiert, da die Verfahren meist auf Attributebene operieren und grundlegende Integritätsprobleme auf Schemaebene kaum berücksichtigen. Auch semantische Aspekte bleiben weitgehend unberührt, da die Korrekturen auf statistischen Mustern oder vordefinierten Regeln basieren und kein echtes Kontextverständnis vorliegt. Sicherheits- und verfügbarkeitsbezogene Aspekte werden nicht adressiert.

Bezogen auf DQ-Dimensionen zeigen Repairing-Verfahren eine direkte Wirkung auf Korrektheit & Genauigkeit sowie Vollständigkeit & Datenmenge durch Korrektur und Ergänzung. Indirekte Effekte können auf Konsistenz & Integrität entstehen, etwa durch die Harmonisierung widersprüchlicher Attributwerte, ohne jedoch strukturelle Integrität sicherzustellen. Relevanz & Kontextbezug bleiben weitgehend unbeeinflusst, ebenso die Interpretierbarkeit, da imputierte Werte nicht zwingend transparenter sind. Zugänglichkeit & Verfügbarkeit bleiben unberührt, während Sicherheit & Vertrauen zwar durch konsistentere Daten gestärkt werden können, gleichzeitig besteht jedoch ein Risiko von Fehlkorrekturen.

Hinsichtlich Microsoft Fabric ist die Implementierung grundsätzlich möglich, jedoch mit erhöhtem Aufwand verbunden. Einfache, regelbasierte Transformationen lassen sich gut parallelisieren, modellbasierte Ansätze verursachen jedoch einen deutlich höheren Rechenaufwand. Die Verfahren operieren auf tabellarischen Daten und sind typischerweise im Silver-Layer einzuordnen, da sie auf bereits integrierten Daten aufsetzen und diese für nachgelagerte Nutzung vorbereiten. Eine Integration ist über Notebooks, Dataflows oder ML-Komponenten möglich, erfolgt jedoch häufig über individuelle Logik oder Modelle, was die Automatisierbarkeit und Pipeline-Fähigkeit einschränkt. Streaming-Szenarien sind nur eingeschränkt geeignet, da sie vereinfachte oder inkrementelle Ansätze erfordern und den Implementierungsaufwand erhöhen.

Die Komplexität wird insgesamt als mittel bis hoch bewertet, da neben der technischen Umsetzung insbesondere die Definition geeigneter Korrekturstrategien anspruchsvoll ist und fehlerhafte Annahmen zu Verzerrungen führen können. Entsprechend sind Wartung und Validierung aufwendiger als bei rein detektierenden Verfahren. Die Skalierbarkeit wird hingegen als niedrig eingestuft, da Reparaturstrategien (abgesehen von Forward Fill) häufig stark domänenspezifisch und nur begrenzt übertragbar sind.

Dadurch ergibt sich ein Score von **1,12**.

Gesamtansätze

Diese Kategorie umfasst konzeptionelle und architekturübergreifende Modelle aus der Literatur, die verschiedene DQ-Mechanismen systematisch kombinieren. In Bezug auf

DQ-Dimensionen entfalten Gesamtansätze eine breite, jedoch überwiegend indirekte Wirkung. Aspekte wie Korrektheit, Vollständigkeit, Konsistenz, Relevanz und Interpretierbarkeit werden nicht durch konkrete Verfahren adressiert, sondern durch Koordination, Standardisierung und die Integration bestehender Methoden. Die Wirkung ist daher weniger gezielt als vielmehr unterstützend und abhängig von der jeweiligen Ausgestaltung des Ansatzes.

Ein ähnliches Bild zeigt sich bei den Problemtypen. Inhaltliche, strukturelle, semantische und vollständigkeitsbezogene Probleme werden konzeptionell berücksichtigt, jedoch nicht unmittelbar durch den Gesamtansatz selbst gelöst. Stattdessen definieren diese Modelle Rahmenbedingungen und Prozessschritte, innerhalb derer konkrete Verfahren implementiert werden müssen. Sicherheits- und Verfügbarkeitsaspekte liegen meist außerhalb des methodischen Kerns und werden eher auf Governance-Ebene adressiert.

Architektonisch sind Gesamtansätze grundsätzlich in moderne Datenplattformen integrierbar, da sie häufig eine strukturierte Einbettung entlang von Verarbeitungsphasen (vergleichbar mit der Medallion-Logik) vorsehen. Eine direkte, architektur-native Implementierung ist jedoch selten gegeben. Auch die Unterstützung spezifischer Plattformfunktionen oder Verarbeitungsmodi hängt stark von der konkreten Ausprägung ab und ist daher nur eingeschränkt bewertbar.

Die Komplexität ist insgesamt als hoch einzustufen, da mehrere Methoden, Prozessschritte und häufig auch übergreifendes DQM koordiniert werden müssen. Die Skalierbarkeit wird als niedrig bis mittel bewertet, da die Modelle zwar konzeptionell übertragbar sind, ihre praktische Umsetzung jedoch stark kontextabhängig ist.

Für die Kategorie der Gesamtansätze ergibt sich damit ein Score von **0,94**.

IoT, Streaming

Im Gegensatz zu Batch-orientierten Verfahren arbeiten diese Ansätze inkrementell und berücksichtigen zeitliche Abfolgen und Ereignisreihen. Als Sammelkategorie entfalten sie primär eine indirekte Wirkung auf DQ-Problemtypen. So können unplausible Sensormessungen, abrupte Werteänderungen oder inkonsistente Ereignisfolgen über zeitabhängige Muster erkannt werden. Auch Inkonsistenzen zwischen Datenströmen lassen sich

identifizieren. Vollständigkeitsprobleme werden hingegen nur sichtbar, wenn erwartete Ereignisse ausbleiben. Semantische sowie Sicherheits- und Verfügbarkeitsaspekte bleiben weitgehend unberührt.

Bezogen auf DQ-Dimensionen liegt der Schwerpunkt auf Korrektheit & Genauigkeit, etwa durch Filterung fehlerhafter Messwerte. Konsistenz & Integrität können hinsichtlich zeitlicher und stromübergreifender Zusammenhänge überprüft werden, während strukturelle Integrität auf Schemaebene nur eingeschränkt adressiert wird. Die Wirkung auf Vollständigkeit & Datenmenge ist meist an Zeitfenster gebunden und Relevanz & Kontextbezug bleiben aufgrund fehlenden semantischen Verständnisses begrenzt. Auch die Interpretierbarkeit ist eingeschränkt, insbesondere bei komplexeren Streaming-Logiken. Zuletzt werden Zugänglichkeit & Verfügbarkeit sowie Sicherheit & Vertrauen nicht direkt beeinflusst, können jedoch durch kontinuierliches Monitoring indirekt profitieren.

Architektonisch lassen sich Streaming-Verfahren gut in moderne Cloud-Umgebungen integrieren, da sie für verteilte und skalierbare Verarbeitung geeignet sind und zeitbasierte Aggregationen und Windowing parallelisiert werden können. Die Unterstützung von Batch-Szenarien ist hingegen begrenzt sinnvoll. Die Einordnung erfolgt typischerweise im Übergang von Bronze zu Silver, etwa zur frühzeitigen Qualitätsprüfung oder zum kontinuierlichen Monitoring. Durch ihre Ausführung als laufende Prozesse sind Automatisierung und Pipeline-Integration grundsätzlich gegeben.

Die Komplexität wird als mittel bewertet. Einfache Verfahren wie Sliding Windows oder Micro-Batch-Analysen sind gut handhabbar, während Multi-Stream-Logik, Schemaänderungen oder Ausfallbehandlung den Aufwand erhöhen. Die Skalierbarkeit ist ebenfalls mittel, da die technische Verarbeitung gut skaliert, Robustheit und Übertragbarkeit jedoch stark vom konkreten Anwendungsfall abhängen.

Für IoT- und Streaming-basierte Verfahren ergibt sich ein Score von **1,23**.

GenAI, agentisch

GenAI- und agentische Ansätze nutzen große Sprachmodelle und kontextadaptive KI-Systeme, um DQ-Probleme semantisch zu analysieren, Regeln abzuleiten oder Konflikte kontextsensitiv zu bewerten. Ihre Stärke liegt insbesondere auf strukturellen und

semantischen Problemtypen. Sie können inkonsistente Attributkombinationen erkennen, widersprüchliche Einträge kontextuell einordnen und unterschiedliche Repräsentationen fachlich zusammenführen. Vor allem bei heterogenen oder unstrukturierten Daten ermöglichen sie die Interpretation von Zusammenhängen, die statistische Verfahren nur eingeschränkt erfassen. Inhaltliche Fehler werden hingegen kaum adressiert, da die Bewertung auf probabilistischen Modellen basiert. Vollständigkeitsprobleme können zwar kontextuell ergänzt werden, jedoch ohne verlässliche Validierung.

Entsprechend zeigt sich die Wirkung entlang der DQ-Dimensionen bei Relevanz & Kontextbezug sowie Konsistenz & Integrität durch semantische Interpretation oder die Generierung von Regelvorschlägen. Auch Interpretierbarkeit & Verständlichkeit können durch automatische Beschreibung oder Klassifikation verbessert werden. Die Wirkung auf Korrektheit & Genauigkeit sowie Vollständigkeit & Datenmenge bleibt indirekt, da die erzeugten Ergebnisse plausibel, aber nicht deterministisch korrekt sind, während Zugänglichkeit & Verfügbarkeit nicht beeinflusst und sicherheitsbezogene Aspekte nicht adressiert werden oder durch nicht-deterministische Modellantworten sogar neue Unsicherheiten entstehen können.

Architektonisch ist die Integration deutlich komplexer. Skalierbarkeit und verteilte Ausführung sind eingeschränkt, da zentrale Schritte häufig über Endpunkte erfolgen und sich nur begrenzt parallelisieren lassen. Auch die Kompatibilität mit Speicherformaten ist indirekt, da meist Vorverarbeitungsschritte erforderlich sind und Ergebnisse in unterschiedlichen Formaten zurückgegeben werden. Eine klare Einordnung in die Medallion-Architektur ist nicht möglich, da sie eher als Querschnittsfunktion fungieren. Die Integration ist über Notebooks oder den Einsatz von Data Agents umsetzbar. Automatisierbarkeit und Pipeline-Fähigkeit sind jedoch eingeschränkt, da agentische Logiken iterative Abläufe, externe Abhängigkeiten und nicht-deterministische Ergebnisse aufweisen.

Die Komplexität wird insgesamt als hoch bewertet, insbesondere aufgrund zusätzlicher Anforderungen wie Prompt Engineering, Evaluation und Monitoring. Die Skalierbarkeit wird als niedrig eingestuft, da die Verfahren stark vom Anwendungskontext, verfügbaren Metadaten und der Modellwahl abhängen und nur begrenzt übertragbar sind.

Für diese Kategorie ergibt sich ein Score von **1,05**.

ML-based Classification/Prediction

Diese überwachten Verfahren zielen darauf ab, Datenzustände zu klassifizieren und Fehler vorherzusagen. Die Wirkung ist dabei stark daten- und modellgetrieben, sodass vor allem solche Fehlerarten erkannt werden, die im Trainingsprozess repräsentiert sind. Hinsichtlich der DQ-Problemtypen zeigt sich dadurch eine Wirkung auf inhaltliche, strukturelle und teilweise semantische Auffälligkeiten, etwa durch die Klassifikation von Fehlern oder die Prognose fehlender Werte. Die Erkennung bleibt jedoch auf systematische Muster beschränkt und ist stark von der Qualität der Trainingsdaten abhängig.

Bezogen auf die DQ-Dimensionen liegt der Schwerpunkt auf Korrektheit & Genauigkeit und Vollständigkeit & Datenmenge, insbesondere durch Missing-Value-Prediction oder Fehlerklassifikation. Konsistenz & Integrität werden nur indirekt beeinflusst, da Modelle keine expliziten Integritätsregeln prüfen. Beiträge zu Relevanz & Kontextbezug sowie Interpretierbarkeit & Verständlichkeit entstehen durch die Erkennung kontextabhängiger Muster, bleiben jedoch datengetrieben und ohne fachliches Verständnis. Zugänglichkeit & Verfügbarkeit sowie sicherheitsbezogene Aspekte werden nicht adressiert.

Architektonisch ist eine einheitliche Einordnung aufgrund der Heterogenität nur eingeschränkt möglich. Skalierbarkeit und verteilte Ausführung sind begrenzt, da Training und Inferenz häufig ressourcenintensiv sind. Zwar können Trainingsdaten aus gängigen Speicherformaten genutzt werden, Modelle werden jedoch als separate Artefakte verwaltet, wodurch zusätzliche Komplexität entsteht. In der Medallion-Architektur sind solche Verfahren im Silver- oder Gold-Layer einzuordnen, jedoch ergänzend und nicht als integrierte Transformationsschritte. Die Verarbeitung erfolgt überwiegend batch-orientiert. Eine Integration in Microsoft Fabric ist für Inferenz gut möglich, Training erfordert jedoch eigenständige Workflows, was die Automatisierbarkeit einschränkt.

Die Komplexität wird insgesamt als hoch bewertet, da neben der technischen Umsetzung auch Training, Feature-Engineering, Modellauswahl und kontinuierliche Validierung erforderlich sind. Die Skalierbarkeit ist hingegen niedrig, da Modelle stark domänenspezifisch sind und nur begrenzt auf andere Datenkontexte übertragbar sind.

Entsprechend ergibt sich ein Score von **0,82**.

4. Prototypische Implementierung im Microsoft Fabric Use Case

Dieses Kapitel untersucht die Anwendbarkeit verschiedener technischer Verfahren innerhalb der Datenplattform Microsoft Fabric. Zunächst wird dafür grundlegend in das Kooperationsunternehmen sowie dessen spezifische Implementierung von Microsoft Fabric eingeführt. Daraufhin werden repräsentative Methoden aus den Ergebnissen der Anwendung des Bewertungsrasters ausgewählt (Kapitel 3.5). Der aktuelle unternehmensseitige Datenintegrationsprozess wird daraufhin konzeptionell um qualitätsbezogene Komponenten erweitert, welche die ausgewählten Verfahren enthalten und die Grundlage für die technische Implementierung bilden. Zur Evaluierung wird zunächst eine Baseline über den aktuellen Stand der DQ in den CRM-Daten erhoben, bevor die Verfahren in den einzelnen Komponenten implementiert werden.

4.1. Unternehmensspezifische Implementierung

Die börsennotierte Geberit Gruppe mit Hauptsitz in Rapperswil-Jona in der Schweiz ist weltweit mit rund 11.000 Mitarbeitern in 50 Ländern aktiv. Im Geschäftsjahr 2025 erzielte die Gruppe einen Nettoumsatz von 3,163 Milliarden CHF [95]. Spezialisiert hat sich der Konzern auf Sanitärtechnik, wodurch er eine marktführende Stellung im europäischen Raum erlangt. 1955 wurde der Standort in Pfullendorf gegründet, an welchem heute ca. 1930 Mitarbeiter tätig sind. Der Standort umfasst neben der Verwaltungstätigkeit das größte Produktionswerk und die größte Vertriebsgesellschaft der Gruppe und ist zusätzlich mit der Logistik als zentrale Distributionseinheit für ganz Europa verantwortlich. [96]

Am Standort in Pfullendorf ist neben der Betriebsinformatik auch der Großteil der Gruppen-IT des Konzerns angesiedelt. Dem Finanzbereich untergeordnet teilt sich die Gruppen-IT nach ihrem Einsatzgebiet gegenüber unterschiedlichen Geschäftsbereichen beispielsweise in die Bereiche Operations Applications, Sales and Marketing Applications und IT Application Services auf. Letzterer verantwortet die Bereitstellung grundlegender IT-Infrastruktur und zentrale Plattformen und Systeme, wo auch die für das Thema dieser Arbeit zuständige Abteilung SAP Development organisiert ist. Neben der eigenen SAP Entwicklung in Technologien wie Fiori ist auch das zuletzt abgeschlossene Projekt der Einführung der neuen Datenplattform Microsoft Fabric, sowie die weitere Ausgestaltung und Migration bisheriger Lösungen verantwortlich.

Abbildung Abb. 2 zeigt die aktuelle Systemlandschaft um die Implementierung. Fabric fungiert dabei als zentrale Datenplattform, die mehrere heterogene Quellsysteme anbindet, um Daten unterschiedlicher fachlicher Domänen zu verarbeiten, abzuspeichern und bereitzustellen. Zusätzlich werden öffentlich verfügbare Daten von Eurostat und dem statistischen Bundesamt (Destatis) über deren APIs bezogen und dienen primär analytischen und berichtsbezogenen Zwecken.

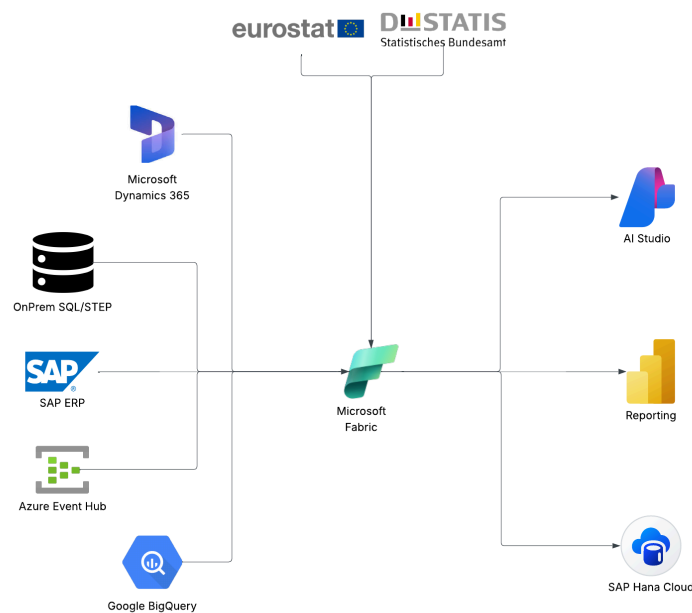


Abb. 2: Aktuelle Fabric Implementierung (eigene Darstellung)

Nachfolgend sind die Inbound-Systeme mit einer kurzen Datenbeschreibung aufgelistet:

- **Microsoft Dynamics 365 (CRM):** Enthält strukturierte Kunden-, Vertriebs- und Marketingdaten, wie Kontakte, Leads, Opportunities und Kampagneninformationen.
- **STEP:** Dient als zentrales Master-Data-Management-System zur Pflege und Bereitstellung konsistenter Stamm- und Referenzdaten.
- **SAP:** Liefert strukturierte operative Unternehmensdaten, insbesondere aus den Bereichen Finanzen, Logistik, Materialwirtschaft und Auftragsabwicklung.
- **Azure Event Hub:** Stellt ereignisbasierte, hochfrequente Streaming-Daten bereit, beispielsweise aus Anwendungen, Sensoren oder Systemlogs der eigenen Produkte.
- **Google BigQuery:** Fungiert als externe analytische Datenquelle und enthält Informationen über Nutzerverhalten auf den Firmen-Websites.
- **Azure Blob Storage:** Beinhaltet semi-strukturierte und unstrukturierte Daten wie Dateien, Exporte oder Rohdaten, die als Input für weitere Verarbeitungsschritte dienen.

- **On-Prem SQL-Server:** Enthält relationale, strukturierte Daten aus lokalen Anwendungssystemen, die periodisch in die Cloud-Datenplattform integriert werden.

Aufbereitete Daten werden im OneLake der Datenplattform persistiert und logisch im Lakehouse separiert. Wie bereits im Theoriekapitel (Kapitel 2.1) erwähnt eignet sich eine Lakehouse-Architektur zur Unterstützung von Reporting, KI- und Self-Service Use Cases. Auf die genauere Bereitstellung für diese Zwecke wird in Betrachtung des Workflows in Abb. 3 genauer eingegangen.

Im Gegensatz zur vorherigen Betrachtung zeigt Abbildung Abb. 3 den Datenworkflow eines einzelnen Workspaces, bzw. der Ablaufschritte des Datenintegrationsprozesses mit verschiedenen Fabric-Komponenten ausgehend von unterschiedlichen Integrationsmöglichkeiten durch die Medallion-Architektur. Zur Abbildung der Medallion-Architektur wird der Workspace in ein einzelnes Lakehouse je Zone (Landing, Bronze, Silver, Gold) untergliedert.

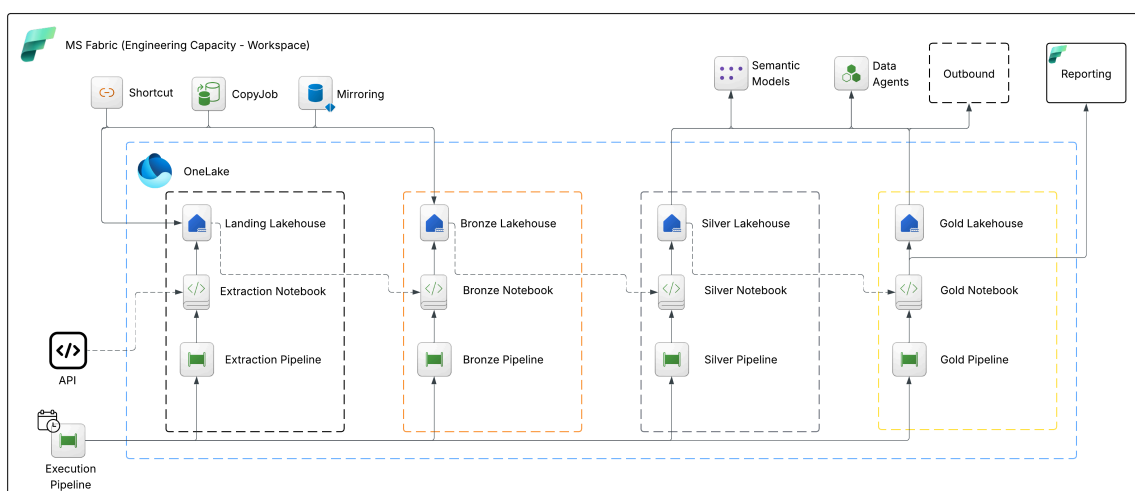


Abb. 3: Datenworkflow im Workspace (eigene Darstellung)

Zu Beginn des Datenintegrationsprozesses liegen die Daten in den Quellsystemen aus Abb. 2 vor. Die Einbindung in den OneLake lässt sich über vielfältige Integrationsoptionen vornehmen. Klassische Kopier- und Ladeprozesse über CopyJobs und Pipelines ermöglichen es Daten in ein Lakehouse zu schreiben. Daneben bietet Microsoft Fabric die Möglichkeit, Daten über Shortcuts zu referenzieren, welche einen Zugriff auf die Datenbasis geben, ohne diese physisch zu replizieren. Ergänzend kann Mirroring Daten kontinuierlich synchronisieren oder APIs angebunden werden, um sowohl batch- als auch ereignis- und schnittstellengetriebene Integrationsszenarien abzudecken.

In der Abbildung Abb. 3 sind die Integrationskomponenten sowohl mit Landing- als auch dem Bronze-Lakehouse verbunden. Während die Landing-Zone die zentrale Einstiegsstelle für Daten jeglichen Formates darstellt, liegen sie im Bronze-Layer bereits in verwendbarer Form vor. Das CRM-System liefert bspw. per Shortcut Tabellen im Delta-Parquet-Format, sodass diese nicht zuerst konstruiert und für weiteren Zugriff ins Lakehouse aufgenommen werden müssen. Dementsprechend gilt es hier je Datenquelle abzuwägen, ob die Integration in das Landing- oder Bronze-Lakehouse erfolgt. Das Landing-Layer dient dabei zur Speicherung unstrukturierter Daten, die dann im Prozess auf das Bronze-Layer strukturiert werden.

Der Integrationsprozess selbst wird durch eine Execution-Pipeline gesteuert, die eine regelmäßig terminierte Ausführung gewährleistet. Sie umfasst mehrere untergeordnete Pipelines, die Transport und Transformation durch die unterschiedlichen Ebenen der Architektur koordinieren. Eine Pipeline greift dafür auf unterschiedliche Komponenten zu, um Modifikationen oder Lade- und Schreibprozesse vorzunehmen. Vorzugsweise wird in der vorliegenden Implementierung mit Notebooks gearbeitet, die Daten aus der vorangegangenen Zone laden, transformieren und in die eigene Stufe (oder extern) schreiben.

Der Zugriff auf die Daten wird architekturell sowohl im Silver- als auch im Gold-Lakehouse ermöglicht, um je nach Use Case bereinigte, standardisierte oder kuratierte und aggregierte Daten als Tabellen zur Verfügung zu stellen. Anschließend können Semantic Models sowohl auf Silver als auch auf Gold-Tabellen definiert werden, die als Grundlage von Data Agents dienen. Auch in externe Systeme wird in der vorliegenden Architektur geschrieben. Daten für KI-Use Cases werden aktuell über den SQL-Endpoint des Silver-Lakehouses und per Data Agent abgefragt.

4.2. Auswahl repräsentativer Methoden und Architekturerweiterung

Wie bereits in Kapitel 3.5 gezeigt, eignen sich unterschiedliche Methodenkategorien an unterschiedlichen Stellen im Datenintegrationsprozess. Nach der Bewertung der Methodencluster anhand des Bewertungsrasters liegt ein Gesamtscore für die technische Wirkung und architektonische Stimmigkeit der Methodenkategorien vor. Diesem wird nachfolgend die Einordnung in die unterschiedlichen Layer der oben genannten Medallion-Architektur beigelegt, womit sich folgendes Ranking abzeichnet:

1. Rule-based Data Validation (**1.56**): Bronze, Silver

2. Statistical Data Profiling (**1.51**): Bronze, Silver
3. Anomaly Detection (**1.34**): Silver
4. IoT, Streaming (**1.23**): Bronze, Silver
5. Matching, Deduplication (**1.19**), Silver
6. Automatisiertes Repairing (**1.12**): Silver
7. GenAI, Agentisch (**1.05**): Querschnittsfunktion
8. Gesamtansätze (**0.94**): Ganzheitlich
9. ML-gestützte Klassifikation/Prediction (**0.82**): Silver, Gold

Die prototypische Implementierung im Use Case zielt darauf ab unterschiedliche Verfahren explorativ am gesamten Datenintegrationsprozess zu evaluieren. Inhärent spiegelt das Ranking bereits die Abfolge der Datenverarbeitung wider und reflektiert sowohl die Zunahme der Komplexität, als auch die Abnahme der Skalierbarkeit.

Rule-based Data Validation und **Statistical Data Profiling** weisen in obigem Ranking den höchsten Score auf und stellen etablierte Verfahren dar. Sie bilden eine deterministische und deskriptive Grundlage innerhalb der Bronze und Silver Schicht und sind aufgrund ihrer guten technischen Integrierbarkeit geeignet für eine Basissicherung von Qualitätsanforderungen. Daher werden diese Kategorien zunächst zur Implementierung ausgewählt um die Grundlage für weitere Verfahren zu schaffen. Welche Regeln und Statistiken aus dem Data Profiling implementiert werden, wird in Kapitel 4.4 ausgeführt.

Als dritte Kategorie wird **Anomaly-Detection** gewählt. Sie bringt eine datengetriebene, adaptive Perspektive für das Silver Layer ein und ermöglicht die Identifikation bislang nicht explizit definierter Ungewohntheiten und erweitert damit den Erkennungsbereich über klassische Prüfmechanismen hinaus. Für die Implementierung werden der statistische Z-Score und IQR Outlier Detection als Methoden mit vergleichsweise geringer Komplexität sowie ein aufwändigerer ML-orientierter Algorithmus gewählt.

Des Weiteren wird **GenAI und Agentische Verfahren** trotz des vergleichsweise geringen Scores für den Use Case herangezogen. Die Stärke dieser Kategorie liegt darin, kontextuelle und semantische Problemstellungen zu adressieren, die durch die bisherigen strukturellen und statistischen Verfahren nicht ausreichend erfasst werden können. Ziel ist es hierbei explorativ zu untersuchen, inwiefern solche Modelle eine ergänzende Rolle in der DQ-Analyse einnehmen können.

Zuletzt unterscheidet sich **Automatisiertes Repairing** von den anderen Kategorien durch seinen einzigartigen Charakter in DQ aktiv (korrigierend) einzugreifen. Jedoch wird diese Kategorie aufgrund mangelnden Domänenwissens und der Beschaffenheit der CRM-Daten als Stammdaten von der Implementierung im Use Case ausgeschlossen.

Das Design des Use Cases basiert auf den ausgewählten Methoden und ist in Abb. 4 abgebildet. Dies folgt der im Kapitel zur Methodendiskussion (Kapitel 3.2) beschriebenen Trade-Off zwischen generischen Verfahren, die eine skalierbare und unternehmensweit einsetzbare Grundlage schaffen und spezifischen Komponenten, deren Prüfungen fachliche Besonderheiten miteinbeziehen und damit die Effektivität erhöhen. Dies ist in der Abbildung als Fabric-Notebook Komponenten dargestellt.

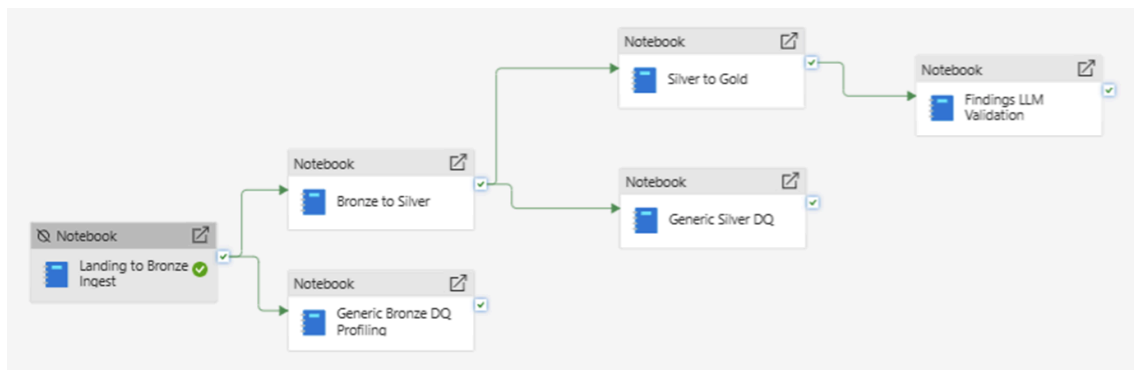


Abb. 4: Use Case Erweiterung des CRM-Ladeprozesses (eigene Darstellung)

Der Use Case erweitert dabei den bestehenden Ladeprozess der CRM-Daten. Während entsprechend der Priorisierung der Fokus der quellnahen Bronze-Schicht primär auf Profiling liegt, wird das Silver-Layer um Methoden aus der ML erweitert und dort explorativ GenAI untersucht. Anstelle des zuvor beschriebenen Ausschlusses von Repairing-Verfahren aufgrund fachlicher Einschränkungen wird bewusst ein Quality Gate in Form einer Quarantäne-Funktion etabliert, die auffällige Daten zur manuellen (oder zukünftig automatisierten) Prüfung separiert und die bisherigen Analyseergebnisse ergänzt.

Abb. 5 erweitert den bisherigen Prozess (Abb. 3) um die eben dargelegte Vorgehensweise. Zentral wird hier das DQ-Zone-Lakehouse eingeführt, um die Daten der Reports sowie die Quarantäne-Daten zu speichern. Entsprechend verändert sich der Datenfluss (in rot hervorgehoben) zwischen den Medallion-Ebenen und wirkt nun direkt oder indirekt (nicht-durchgezogene Linien) auf Ebene der DQ-Zone.

Für die konkrete Implementierung im Use Case wird zunächst eine Baseline über die CRM-Datenquelle erhoben. Darauf aufbauend werden die Methoden in den zuvor beschriebenen Komponenten als PySpark-Notebooks implementiert, um maximale Flexibilität durch konfigurierbares Custom Coding zu gewährleisten.

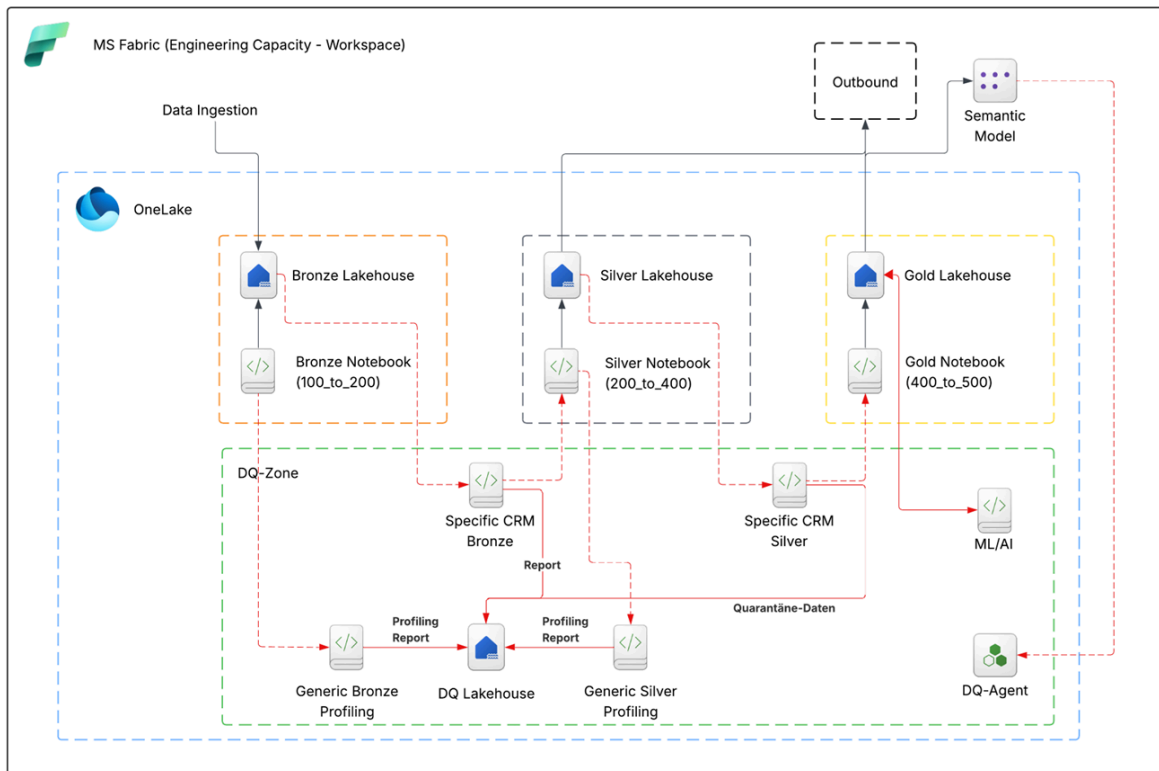


Abb. 5: Konzeptionelle DQ-Zone Architektur (eigene Darstellung)

4.3. Festlegen von Metriken je Kategorie

Um die DQ in der initialen Baseline-Erhebung zu messen, bedarf es Kennzahlen. Tab. 10 enthält eine Sammlung unterschiedlicher Metriken für die DQ-Dimensionen, welche in der Literaturarbeit identifiziert wurden (Tab. 5). Im Rahmen des Systematic Review zu DQ-Methoden wurden neun Quellen identifiziert, die für die Bewertung von DQ Informationen, Kategorien und Metriken enthalten. Diese wurden aus den Quellen extrahiert und den Dimensionen zugeordnet. Dabei gilt zu beachten, dass sich Metriken teilweise mehreren Dimensionen zuordnen lassen.

Aus dieser Metrikensammlung werden im weiteren Verlauf der Arbeit exemplarisch Kennzahlen für die Baseline-Erhebung und Evaluation einzelner Verfahren ausgewählt. Die Auswahl selbst erfolgt dabei kontext- und verfahrensabhängig. Auffällig ist, dass sich

für die Dimension *Interpretierbarkeit & Verständlichkeit* nur einzelne und bei *Sicherheit & Vertrauen* bzw. *Zugänglichkeit & Verfügbarkeit* in den untersuchten Quellen keine konkret beschriebenen Kennzahlen primär zuordnen lassen. Hierbei spiegelt sich auch der Fokus auf technisch überprüfbare Qualitätsaspekte wider, was sich bereits in der Bewertung der Methodencluster anhand der DQ-Dimensionen angedeutet hat (siehe Kapitel 3.5).

DIMENSIONS-KATEGORIE	Metriken	Quellen
VOLLSTÄNDIGKEIT & DATENMENGE	• Null-Rate/Not-Null-Rate • (Not) Missing Rows Percentage, (zb je Kunde/Fenster) • (Not) Missing Fields • Pflichtfeldregeln, • Anteil eindeutiger Zeitstempel, • Vollständige Rows/Attribute, • Kombinierte Integrität, • Redundanz- und Duplikatfreiheit,	[97] , [98] , [99] [100] , [101] , [102] [103] , [104] , [105]
KONSISTENZ & INTEGRITÄT	• Distinct-Rate, • (Not) Duplicate-Checks./Ratio (auch über Zeit) • Cross-Field-Checks, • Regel/Type-verletzungsrate, • Cross-Source /Attribut/Record Checks/Regeln, • Datenüberflüssigkeit, • PK/FK-Regeln,	[97] , [98] , [99] [101] , [102] , [103] [105]
KORREKTHEIT & GENAUIGKEIT	• Referenzabgleich, • Range-Checks, • Einheit/Skalenprüfung, • Formatregeln, • Regelbasierte/statistische Plausibilitätsprüfung, • Anteil gültiger/plausibler Datenwerte, • Regelverletzung, • Drift in Mittelwert/Varianz, • Werte innerhalb erlaubter Grenzen (80, 95, 99 % CI),	[97] , [98] , [99] [100] , [101] , [102] [103] , [104] , [105]
RELEVANZ & KONTEXTBEZUG	• Zeitfenster-Checks, Anteil zulässiger Abstände • Verzögerung (Event/Ingest), • Nutzungshäufigkeit, • Beitrag zur Entscheidungsqualität/Aggregation-sniveau, • Prüfung funktionaler Abhängigkeiten, • FK-/PK-Beziehungen, Veränderungen von Korrelationen • chronologische Ordnung • Wahrscheinlichkeit Aktualität,	[97] , [98] , [99] [100] , [101] , [102] [103] , [104]

DIMENSIONS-KATEGORIE	Metriken	Quellen
	• Shannon-Entropie über Attributwerte (Informationsgehalt), • Abdeckung thematischer Kategorien (Breite & Tiefe Kundenprofil) • Alter der Daten (a current - Update), • Gültigkeitsdauer (b storage-update/total), • Kontextabhängige Aktualität (1 - Alter der Daten/ Gültigkeitsdauer),	
INTERPRETIERBARKEIT & VERSTÄNDLICHKEIT	• Symbolfreiheit • Metadaten/Semantikverfügbarkeit	[97], [98], [104]
SICHERHEIT & VERTRAUEN	-	-
ZUGÄNGLICHKEIT & VERFÜGBARKEIT	-	-

Tab. 10: Metriken aus dem Systematic Review (eigene Darstellung)

4.4. Erhebung der Baseline

Die Erhebung eines Baseline-Zustands hat zum Ziel, eine initiale Analyse der Qualität der Datengrundlage und mögliche Probleme zu identifizieren. Es ist nicht von Bedeutung, eine vollständige Datenprofilierung zu erstellen, sondern lediglich zur Identifikation potenzieller Qualitätsprobleme zu dienen. Wie in Abb. 3 gezeigt werden die CRM-Daten aktuell über einen Shortcut in den OneLake von Microsoft Fabric gespiegelt. Um eine Vergleichbarkeit und Reproduzierbarkeit der Ergebnisse sicherzustellen, ist es zwingend notwendig eine Kopie der aktuellen CRM-Datenlandschaft zu erstellen.

Dafür werden die Shortcuts der unterschiedlichen Instanzen des Systems (Dev, QA, Wave-Test, Prod) zum Zeitpunkt des **09. Februar 2026** (14:57) in das lh_300_dqzone-Lakehouse als eigene Schemas mit darunter liegenden Delta-Tabellen gespeichert (Abb. 6). Es ist abzugrenzen, dass die Datenstruktur wie in Abb. 6 dargestellt lediglich für die Implementierung im prototypischen Use Case der Arbeit vorgesehen ist. Darüber hinaus gilt die bisherige Datenstruktur, bei der nur in den Prozess eingegriffen wird (Abb. 5). Alle Berechnungen und Operationen dieser Arbeit werden auf einer Capacity der Größe F16 und F32 in der Region West Europe durchgeführt.

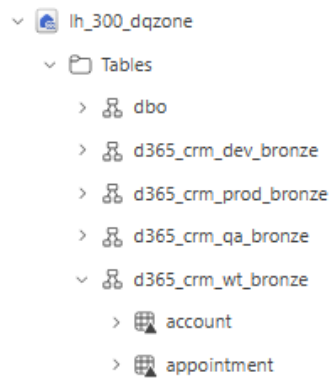


Abb. 6: Datenstruktur im DQ-Zone-Lakehouse für die Implementierung (eigene Darstellung)

Die Erhebung selbst gliedert sich in zwei Schritte. Zuerst werden allgemeine Kennzahlen für alle Systeme erhoben und dann die Daten des Produktivsystems genauer analysiert, da sie die tatsächliche Datengrundlage für die Geschäftsprozesse liefern und damit am relevantesten für die Implementierung und Untersuchung der ausgewählten Methoden im Use Case sind.

Zum Start der Erhebung werden zunächst die unterschiedlichen Systemlandschaften oberflächlich auf ihre Struktur hin analysiert. Die Anzahl der im Use Case betrachteten Tabellen liegt konstant bei 28. Die nachfolgende Tabelle Tab. 11 zeigt andere Dimensionen, in welchen sich die Umgebungen unterscheiden.

UMGEBUNG	Anzahl Zeilen	Anzahl Spalten	Anzahl Felder	Null-Rate	Null-Rate Schlüssel	Duplikate Schlüssel
DEV	679.066	4.327	135.067.476	78,44 % (105.947.435)	0,00 % (11)	0,00 % (1, incident)
WAVE-TEST	5.974.788	4.310	2.078.227.131	69,98 % (1.454.240.531)	0,099 % (5.933)	0,00 % (1, appointment)
QA	5.650.428	4.304	2.078.227.131	70,23 % (1.361.920.331)	0,093 % (5.240)	0
PROD	15.058.853	4.298	4.308.898.317	66,94 % (2.884.569.855)	0,14 % (20.975)	0

Tab. 11: Struktur Systemlandschaften (eigene Darstellung)

Aus Tabelle Tab. 11 wird ersichtlich, dass Dev- und Testsysteme hinsichtlich der Datenmenge deutlich unter dem Produktivsystem liegen. Die Spaltenanzahl ist weitgehend konstant, mit einem leichten Rückgang vom Dev- zum Produktivsystem. Die Anzahl der Felder wächst entsprechend mit der Zeilenzahl.

Die Null-Rate wird als Anteil leerer Strings ("") bzw. mittels `isNull()` identifizierter Nullwerte an allen Feldern berechnet. Sie ist in allen Systemen hoch, mit dem niedrigsten Wert im Produktivsystem (66,94 %) und dem höchsten im Dev-System (78,44 %). Ursache ist das Wide-Format des CRM-Systems, bei dem viele spezialisierte Attribute nicht für alle Datensätze befüllt sind. Auf Schlüsselspalten ist die Null-Rate hingegen sehr gering (max. 0,14 % im Produktivsystem) und deutet dort klar auf Datenfehler hin.

Die Duplikatsprüfung mittels `isDuplicate()` zeigt keine Auffälligkeiten in Produktiv- und QA-System, jedoch jeweils einen Eintrag im Dev- (incident) und Wave-Test-System (appointment). Die betroffenen Datensätze unterscheiden sich lediglich in mehreren zeitstempelbasierten Attributen, was auf Integrations- oder Migrationsprobleme hindeutet.

Für den Use Case sind primär die produktiven Daten relevant, da sie die reale Datengrundlage abbilden. Allerdings werden im bestehenden Integrationsprozess nur 612 Spalten als fachlich relevant genutzt. Eine vollständige Auflistung befindet sich im Anhang (Tab C).

Bei Beschränkung auf diese Spalten bleibt die Zeilenzahl unverändert, die Feldanzahl reduziert sich jedoch im Produktivsystem auf 239.300.304 (ca. 5 %). Entsprechend sinkt die Null-Rate auf 35,36 %. In den anderen Systemen bleibt sie höher (Dev: 80,24 %, Wave-Test: 52,69 %, QA: 57,20 %), was auf deren jeweiligen Einsatzzweck zurückzuführen ist. Die Anzahl der Duplikate auf Schlüsselspalten bleibt unverändert.

Im nächsten Schritt wird die strukturelle Beschaffenheit der produktiven Daten analysiert. Eine Übersicht der Tabellen, inklusive Zeilen- und Attributanzahl, relevanter Spalten, Datentypverteilung sowie zeitlicher Abdeckung und Änderungsdichte, ist in Tab. D dargestellt. Die betrachteten Tabellen umfassen sowohl Stamm- (z. B. account, contact), Transaktions- (z. B. appointment, incident) als auch Metadaten. Teilweise werden nur ausgewählte Spalten genutzt, was sich in einer geringeren Anzahl relevanter Attribute widerspiegelt.

Die weitere Analyse erfolgt entlang der in Kapitel 3.3 definierten DQ-Dimensionen (vgl. Tab. 5). Hierzu werden repräsentative Metriken aus Tab. 10 ausgewählt, um

Qualitätsprobleme zu identifizieren und zu quantifizieren. Die Dimensionen *Sicherheit & Vertrauen* sowie *Zugänglichkeit & Verfügbarkeit* werden ausgeschlossen, da hierfür keine geeigneten Metriken identifiziert wurden und sie primär Governance-Aspekte betreffen, die nicht im Fokus der Arbeit stehen.

Vollständigkeit & Datenmenge

Für diese Dimension werden zunächst ratio-basierte Metriken wie die Null-Rate relevanter Attribute sowie ein Completion-Ratio erhoben, das den Anteil vollständig befüllter Datensätze beschreibt. Ergänzend werden kombinierte Integrität und die Ähnlichkeit von Datensätzen analysiert (vgl. Tab. E).

Im Fokus stehen die Tabellen *account* und *contact*, da sie zentrale Stammdaten im CRM darstellen. Die Null-Raten zeigen ein differenziertes Bild: Während Schlüssel- und Pflichtfelder nahezu vollständig befüllt sind, weisen typische Business-Attribute mittlere Null-Raten auf, was auf nicht verpflichtende Pflege hinweist. Andere Attribute sind häufig oder nahezu vollständig leer, was auf geringe Nutzung, spezifische Anwendungsfälle oder fehlende fachliche Relevanz hindeutet. Das Completion-Ratio bestätigt diese Beobachtung: Nur ein geringer Anteil der Datensätze ist vollständig befüllt (*contact*: 8,37 %, *account*: 58,38 %; mit optionalen Feldern deutlich geringer). Gleichzeitig sind zentrale Identifikations- und Kontaktinformationen in den meisten Fällen vorhanden. Kritische Datensätze ohne grundlegende Informationen treten nur selten auf.

Die Analyse der referenziellen Integrität zeigt keine inkonsistenten Verknüpfungen. Zwar ist etwa die Zuordnung von Kontakten zu Accounts häufig nicht gesetzt, dies ist jedoch fachlich zulässig. Andere Beziehungen, wie über *primarycontactid*, werden hingegen nur selten genutzt.

Zur Bewertung potenzieller Redundanzen wird eine Stichprobe von 1 % der Datensätze betrachtet und eine attributbasierte Ähnlichkeit berechnet. Die Ergebnisse zeigen insgesamt nur einen sehr geringen Anteil stark ähnlicher Datensatzpaare (< 0,05 %), sodass keine systematischen Dublettenstrukturen erkennbar sind. Höhere Ähnlichkeiten treten lediglich in kleinen Metadatentabellen auf und sind dort strukturell bedingt.

Insgesamt zeigt die Analyse eine geringe vollständige Attributabdeckung bei gleichzeitig

stabiler Versorgung mit zentralen Informationen sowie nur gering ausgeprägten Redundanzen.

Konsistenz & Integrität

Für die Dimension Konsistenz & Integrität werden die Eindeutigkeit der Primärschlüssel sowie die referenzielle Konsistenz ausgewählter Fremdschlüsselbeziehungen untersucht (vgl. Tab. F).

Die Ergebnisse zeigen zunächst eine durchgehend eindeutige Vergabe der Primärschlüssel. In keiner der betrachteten Prod-Tabellen treten Duplikate auf, wodurch die Identifizierbarkeit der Datensätze systemseitig sichergestellt ist. Auch die referenzielle Integrität ist überwiegend hoch. Zentrale Beziehungen, etwa zwischen contact und account, weisen durchgehend gültige Referenzen ohne verwaiste Einträge auf. Dies spricht für stabile strukturelle Beziehungen zwischen den wesentlichen Geschäftsobjekten.

Abweichungen treten vereinzelt bei Attributen auf, die nicht strikt als Fremdschlüssel modelliert sind. So können bspw. Referenzen in Tabellen wie appointment, incident oder email nicht immer eindeutig auf eine Entität abgebildet werden. Dies ist jedoch meist strukturell bedingt, etwa durch polymorphe Referenzen oder frei befüllbare Felder im CRM-System, und stellt daher kein eindeutiges Integritätsproblem dar.

Insgesamt zeigt die Analyse eine weitgehend konsistente Datenstruktur mit stabilen Beziehungen und nur wenigen, kontextuell erklärbaren Abweichungen.

Korrektheit & Genauigkeit

Die Bewertung dieser Dimension erfordert in der Regel eine externe Referenz oder fachlich definierte Soll-Werte. Da eine solche Ground-Truth im Use Case nicht vorliegt, sind weitergehende Plausibilitäts- oder Regelprüfungen nur eingeschränkt möglich. Die Analyse beschränkt sich daher auf Formatprüfungen standardisierter Attribute wie E-Mail-Adressen und Telefonnummern (vgl. Tab. G).

Die dafür verwendeten Regex-Ausdrücke wurden iterativ angepasst, um eine realistische Balance zwischen Erkennungsrate und falsch-positiven Ergebnissen zu erreichen.

Bspw. filtern sie E-Mail-Adressen heraus, die nicht der erwarteten syntaktischen Struktur (lokaler Teil, @-Zeichen, Domain) entsprechen. Dennoch ist zu beachten, dass aufgrund der Vielfalt möglicher Formate und mangelnden Domänenwissens keine vollständige Abdeckung gewährleistet werden kann.

Die Ergebnisse zeigen insgesamt nur geringe Abweichungen von erwarteten Formaten. E-Mail-Adressen weisen sehr niedrige Fehlerraten auf (z. B. < 0,5 %), was auf systemseitige Validierungen und standardisierte Eingaben hindeutet. Auch bei Telefonnummern sind die Abweichungen gering (ca. 1-2 %), was angesichts freier Eingabeformate als erwartbar einzustufen ist. Eine Auffälligkeit zeigt sich in der Tabelle email, in der ein Teil der Datensätze doppelte Empfänger enthält. Dies deutet jedoch weniger auf ein Korrektheitsproblem als auf Redundanzen in Kommunikationsprozessen hin.

Insgesamt lassen die Ergebnisse darauf schließen, dass formale Fehler in den untersuchten Attributen nur in geringem Umfang auftreten und keine systematischen Qualitätsprobleme hinsichtlich der Datenformate vorliegen. Dennoch ist die vergleichsweise schwache Gestaltung der Regex-Ausdrücke zu beachten.

Relevanz & Kontextbezug

Zur Bewertung dieser Dimension werden zeit- und kontextbezogene Eigenschaften der Daten analysiert, insbesondere das Alter der Datensätze, deren Änderungsverhalten sowie ausgewählte funktionale Abhängigkeiten (vgl. Tab. H).

Die Altersanalyse zeigt eine deutliche Differenz zwischen Stammdaten und transaktionalen Daten. Während zentrale Tabellen wie account und contact sehr lange Lebenszyklen aufweisen, teils mit auffällig hohen Alterswerten (126 Jahre), die auf Migrationen zurückzuführen sein könnten, bewegen sich Transaktionsdaten wie appointment oder incident in realistischeren Zeiträumen (mehrere Jahre). Dies entspricht der typischen Nutzung, dass Stammdaten langfristig bestehen, während Transaktionsdaten ereignisbezogen erstellt werden.

Auch die Änderungsdichte der letzten 30 Tage verdeutlicht diese Trennung. Stammdaten, insbesondere contact und account, weisen vergleichsweise hohe Änderungsraten auf (51,75 %, 11,72 %) und sind damit aktiv im operativen Einsatz. Transaktionsdaten

hingegen werden nach ihrer Erstellung nur selten angepasst und dienen primär der historischen Dokumentation (z. B. incident 2.49 %). Referenz- und Metadatentabellen zeigen erwartungsgemäß kaum Änderungen, was ihre stabile, unterstützende Rolle widerspiegelt.

Die Analyse von FDs zeigt nur geringe Abweichungen, etwa bei alternativen Identifikationsattributen in der account-Tabelle, was auf eine weitgehend konsistente Datenpflege hinweist. So wird die Beziehung `geb_sapnumber -> accountid` mit 0,43 % (2265) nur sehr selten verletzt. Auch zeitliche Inkonsistenzen konnten nahezu nicht festgestellt werden, da Erstellungs- und Änderungszeitpunkte konsistent gesetzt sind.

Insgesamt bestätigen die Ergebnisse, dass sich die Daten entsprechend ihres funktionalen Kontexts verhalten. Stammdaten sind langlebig und aktiv gepflegt, Transaktionsdaten überwiegend statisch, und Referenzdaten strukturell stabil. Dies spricht für eine insgesamt kontextgerechte Nutzung der Daten im CRM-System.

Interpretierbarkeit & Verständlichkeit

Zur Bewertung dieser Dimension wird untersucht, inwieweit Daten semantisch nachvollziehbar und konsistent interpretierbar sind. Betrachtet werden dabei insbesondere Placeholder-Werte, die Konsistenz von Optionset-Metadaten sowie potenzielle semantische Auffälligkeiten in Stringattributen (vgl. Tab. I).

Die Analyse generischer Placeholder-Werte (z. B. "n/a", "test", "0") zeigt insgesamt nur geringe Auffälligkeiten. In der Tabelle `contact` liegt der Anteil bei Telefonnummern bei etwa 0,02 % und bei E-Mail-Adressen bei rund 0,004 %. Auch in `account` treten solche Werte nur vereinzelt auf. Dies deutet darauf hin, dass Textfelder überwiegend mit inhaltlich sinnvollen Werten befüllt sind und generische Platzhalter kaum genutzt werden. Die zur Identifikation solcher Werte verwendeten Listen wurden heuristisch zusammengestellt und LLM-gestützt erzeugt und stellen daher keine vollständige Abbildung dar.

Hinsichtlich der Metadatenkonsistenz zeigt sich, dass die meisten verwendeten Optionset-Werte eindeutig auf definierte Kategorien zurückgeführt werden können. Es treten jedoch nicht zuordenbare Werte auf, was auf Inkonsistenzen zwischen Datenbestand und Metadatendefinitionen hindeutet. Zusätzlich wurden Stringattribute auf potenzielle

Fehlverwendungen untersucht, etwa wenn Zeitstempel als Text gespeichert sind. Solche Fälle treten vereinzelt auf und lassen sich meist als semantische Fehlklassifikationen einzelner Felder einordnen. Demgegenüber zeigen die vier untersuchten Metadatatabellen konsistent eine als Datum interpretierbare Textspalte.

Insgesamt weist der Datensatz eine hohe semantische Interpretierbarkeit auf, mit geringen Einschränkungen durch einzelne Inkonsistenzen und uneinheitliche Attributnutzung.

Die Analyse zeigt insgesamt eine stabile Datenbasis mit konsistenter Struktur, geringer Redundanz und überwiegend korrekt erfassten Attributen. Gleichzeitig bestehen jedoch Einschränkungen hinsichtlich Vollständigkeit, semantischer Konsistenz und kontextabhängiger Interpretierbarkeit, insbesondere bei optional gepflegten oder selten genutzten Attributen. Auffälligkeiten treten dabei weniger in Form systematischer Fehler, sondern eher als inkonsistente oder unvollständige Nutzung auf. Es ist zudem zu berücksichtigen, dass sich die Analyse ausschließlich auf eine einzelne Domäne bzw. Datenquelle bezieht und somit keine allgemeingültige Aussage über den Qualitätszustand der gesamten Plattform getroffen werden kann.

4.5. Deterministische und regelbasierte Verfahren zur Datenqualitätsprüfung

Ein zentrales Spannungsfeld der DQ-Sicherung liegt zwischen standardisierten, wiederverwendbaren Prüfmechanismen und der Berücksichtigung domänenspezifischer Anforderungen. Während aus Unternehmenssicht eine effektive Anwendung auf einzelne Datendomänen notwendig ist, um relevante Fehler zu identifizieren, erfordert eine breite Systemlandschaft gleichzeitig Skalierbarkeit und Wiederverwendbarkeit. Im Use Case Design (Abb. 4) wird diesem Trade-Off daher durch die Kombination generischer und spezifischer Komponenten begegnet.

Da die Implementierung entlang der Medallion-Architektur erfolgt, wird zunächst das Bronze-Layer betrachtet. Besonders geeignet sind hier Rule-based Data Validation und Statistical Data Profiling, da sie die höchsten Scores aufweisen und sich für ingestion-nahe Prüfungen eignen. Aufgrund ihrer niedrigen bis mittleren Komplexität und hohen Skalierbarkeit bieten sie sich zudem für die Umsetzung in generischen Komponenten an und werden daher in diesem Schritt priorisiert.

Generic Bronze Profiling

Ziel der generischen Bronze Komponente ist es, ingestion-nah Transparenz über den Zustand der Rohdaten zu schaffen und erste Indikatoren zur Bewertung der DQ bereitzustellen. Der Fokus liegt dabei bewusst auf einer explorativen Analyse, nicht auf der Korrektur der Daten. Die generische Ausgestaltung ermöglicht es die Komponente skalierbar auf unterschiedliche Datensätze zu übertragen.

Das Notebook selbst gliedert sich in drei Profiling-Modi, die unterschiedliche Analyseebenen adressieren:

- **Table Profiling** liefert eine Übersicht über die Tabellenstruktur, einschließlich Zeilen- und Spaltenanzahl, Datentypen sowie Kennzahlen zur Aktualität wie Alter der Datensätze oder Zeit seit der letzten Änderung.
- **Column Profiling** analysiert einzelne Attribute detaillierter, etwa hinsichtlich Null-Rate, Distinct-Werten und Verteilungen. Je nach Datentyp werden zusätzliche Kennzahlen berechnet, z. B. Längenstatistiken für Strings, Zeitbereiche für Datumswerte oder Quantile für numerische Attribute.
- **Schema Comparison** vergleicht das aktuelle Schema mit dem im Silver-Layer definierten Referenzschema, um strukturelle Änderungen wie hinzugefügte, entfernte oder geänderte Attribute frühzeitig zu erkennen.

Die Ergebnisse werden in einem zentralen Profiling-Report zusammengeführt und im DQ-Lakehouse (lh_300_dqzone) gespeichert. Hierfür wird das Schema *profiling_results_bronze* genutzt, wobei die Ergebnisse je Quelle und Modus als Delta-Tabellen abgelegt werden. Ein Zeitstempel je Profiling-Lauf ermöglicht die Historisierung und Analyse der DQ-Entwicklung.

Methodisch basiert die Komponente primär auf statistischem Data Profiling, ergänzt um einfache regelbasierte Prüfungen und nutzt dafür z. B. Ratio-based Metriken. Sie adressiert insbesondere die Dimensionen Vollständigkeit, Aktualität und strukturelle Konsistenz, ist aufgrund ihrer generischen Auslegung jedoch bewusst nicht in der Lage, fachliche oder semantische Qualitätsaspekte abzubilden. Dennoch bildet sie eine skalierbare Grundlage für die Identifikation früher Qualitätsprobleme und nachgelagerte Verarbeitungsschritte.

Specific Bronze Profiling

Die spezifische Bronze-Komponente adressiert die Limitationen der generischen Variante, indem sie fachlogische Prüfungen integriert. Aufgrund technischer Einschränkungen der Plattform wird sie nicht als eigenständiges Notebook umgesetzt, sondern direkt in die Transformation von Bronze zu Silver eingebettet. So können die Prüfungen effizient innerhalb desselben Verarbeitungsschritts und mit den vorhandenen Ressourcen ausgeführt werden, ohne zusätzliche Zwischenspeicherungen zu benötigen.

Die Komponente ist prinzipiell als Template konzipiert und auf andere Datenobjekte übertragbar, erfordert jedoch eine domänenspezifische Definition der Prüfregeln. Im vorliegenden Use Case orientieren sich diese exemplarisch an den Analysen und Ergebnissen der Baseline-Analyse (Kapitel 4.4). Folgende Regex-basierte Formatprüfungen, Referenzprüfungen sowie einfache funktionale Abhängigkeiten werden durchgeführt.

- **INVALID_EMAIL/PHONE_FORMAT**: Verletzung definierter Formatregeln
- **INVALID_OPTIONSET/STATECODE**: Fehlende Referenz in Metadaten
- **INVALID_REFERENCE**: Fremdschlüsselverletzungen
- **MISSING_SAP_NUMBER**: Verletzung einer funktionalen Abhängigkeit

Wie bei der generischen Komponente erfolgt auch hier ausschließlich ein Profiling, jedoch mit stärker regelbasierter Ausrichtung. Die Ergebnisse werden im Schema *profiling_results_bronze* gespeichert und beeinflussen die eigentliche Datenverarbeitung bewusst nicht. Der Ablauf folgt einem einheitlichen Muster. Zuerst werden fachliche Regeln definiert, dann geschieht die Integration in den Transformationsprozess und anschließend eine separate Auswertung und Speicherung der Ergebnisse. Die Auswertung erfolgt auf aggregierter Ebene, um Performanceeinbußen zu vermeiden. Ergebnisse umfassen z. B.. Anzahl und Anteil der Regelverletzungen je Tabelle sowie betroffene Attribute und werden historisiert gespeichert.

Die spezifische Bronze-Komponente ergänzt somit das generische Profiling um fachliche Validierungen und ermöglicht eine gezieltere Identifikation von Inkonsistenzen, insbesondere in den Dimensionen Korrektheit & Genauigkeit sowie Konsistenz & Integrität. Gleichzeitig steigt der Implementierungs- und Pflegeaufwand durch die notwendige manuelle Regeldefinition. In Kombination bilden die Komponenten eine erste

mehrschichtige Qualitätsbewertung, die generische Muster und domänenspezifische Analysen verbindet und als Grundlage für DQ-Analysen und nachgelagerte Maßnahmen in der Medallion-Architektur dient.

Generic Silver Component

Die generische Silver-Komponente baut auf den Ergebnissen des Bronze-Profilings auf und erweitert diese um gezielte, weiterhin domänenunabhängige Prüfungen. Im Gegensatz zum explorativen Bronze-Layer basiert sie auf bereits gefilterten, fachlich relevanten Tabellen und Attributen, wodurch eine fokussiertere Bewertung der DQ ermöglicht wird.

Methodisch kombiniert die Komponente regelbasierte Plausibilitätsprüfungen mit statistischer Ausreißererkennung. Der Fokus liegt dabei nicht mehr auf der reinen Beschreibung der Daten, sondern auf der Identifikation potenzieller Auffälligkeiten. Die regelbasierten Prüfungen orientieren sich an allgemeinen, datentypbezogenen Annahmen, etwa negative Werte in numerischen Spalten, zukünftige Zeitstempel oder unerwartete Typinterpretationen von String-Werten. Ergänzend werden konfigurierbare Prüfungen, z. B. für Schlüssel- oder nullkritische Spalten, integriert. Zur Erweiterung dieser regelbasierten Sicht werden statistische Verfahren eingesetzt. Für numerische Attribute werden Z-Score (> 3) und ein IQR-basiertes Verfahren ($\pm 1.5 * IQR$) verwendet, um Ausreißer auf Record-Ebene zu identifizieren. Diese werden formal ebenfalls als Regeln behandelt und einheitlich ausgewertet.

Die Ergebnisse werden im Lakehouse im Schema *profiling_results_silver* gespeichert und enthalten neben aggregierten Kennzahlen (z. B. Anzahl und Anteil von Regelverletzungen je Tabelle und Spalte) auch betroffene Datensätze inklusive ihrer Schlüssel, Regel-ID und auffälligem Wert, in Form von zwei separaten Delta-Tabellen. Zeitstempel und Schwellenwerte werden ebenfalls gespeichert, um Nachvollziehbarkeit und zeitliche Analysen zu ermöglichen.

Die Komponente wirkt weiterhin analytisch-deskriptiv und verzichtet bewusst auf automatische Korrekturen, da insbesondere statistische Verfahren Unsicherheiten enthalten. Sie erweitert das Bronze-Profilings um eine gezielte Analyse möglicher Auffälligkeiten auf Tabellen- und Record-Ebene und bildet damit eine Zwischenstufe zu weiterführenden, datengetriebenen Verfahren. Durch ihre generische und konfigurierbare Ausgestaltung

bleibt sie skalierbar und unabhängig von tiefem Domänenwissen, mit Ausnahme grundlegender Informationen wie Schlüssel- oder kritischer Attribute. Einschränkungen bestehen insbesondere in der fehlenden Berücksichtigung semantischer Zusammenhänge sowie in der Abhängigkeit von gewählten Schwellenwerten.

4.6. Intelligente und datengetriebene Verfahren zur Qualitätsbewertung

Die im vorherigen Abschnitt beschriebenen deterministischen und regelbasierten Verfahren bilden eine skalierbare und robuste Grundlage zur initialen Bewertung der DQ, insbesondere zur Identifikation struktureller Auffälligkeiten und der Prüfung allgemeiner und domänenspezifischer Regeln entlang der Bronze- und Silver-Layer.

Gleichzeitig zeigen sich jedoch Limitationen dieser Ansätze. So setzen regelbasierte Verfahren eine explizite Definition erwarteter Zustände voraus, während statistische Methoden häufig auf vereinfachenden Annahmen über die Datenverteilung beruhen. In realen Unternehmensdaten, wie sie im vorliegenden Use Case betrachtet werden, sind Qualitätsprobleme jedoch oftmals nicht eindeutig durch einzelne Regeln oder univariate Ausreißer beschreibbar, sondern ergeben sich aus komplexen Interaktionen mehrerer Attribute oder aus implizitem Domänenwissen.

Vor diesem Hintergrund werden im folgenden Abschnitt datengetriebene und intelligente Verfahren zur Qualitätsbewertung betrachtet, welche über die deterministische Prüfung hinausgehen. Ziel ist es, Muster und Auffälligkeiten automatisiert zu erkennen, Zusammenhänge zwischen Attributen zu berücksichtigen sowie die Bewertung von DQ stärker kontext- und wissensbasiert zu gestalten.

Specific Silver Component

Die Implementierung erfolgt im Übergang von Silver zu Gold und greift dort aktiv in die Datenverarbeitung ein. Bisher enthält dieser Übergang die Transformation von Silber-Tabellen durch Joins mit anderen Entitäten und die Bildung einer zentralen Übersetzungstabelle aus den Optionsets. Die spezifische Silver-Komponente erweitert die zuvor beschriebenen generischen Anomalieprüfungen um datengetriebene Verfahren zur Identifikation komplexer Auffälligkeiten. Während die bisherigen Ansätze primär auf univariaten Betrachtungen sowie explizit definierten Annahmen basieren, zielt diese

Komponente darauf ab, multivariate Zusammenhänge zwischen den Attributen zu berücksichtigen und daraus potenzielle Abhängigkeiten abzuleiten.

Zur exemplarischen Demonstration wird die Account-Tabelle als Grundlage herangezogen, welche im Customer Data Object (CDO) Datenprodukt die zentrale Entität darstellt (siehe Abb. 7). Zur Identifikation von Auffälligkeiten wird der IF als Verfahren aus dem Bereich der unüberwachten Anomalieerkennung eingesetzt. Das Modell zielt darauf ab seltene und potenziell abweichende Datenpunkte in multivariaten Datensätzen zu identifizieren und eignet sich damit für die tabellarische Datengrundlage. Der IF wird auf ausgewählten numerischen und kodierten Spalten angewendet und weist jedem Datensatz einen Anomalie-Score, als Indikator für potenzielle Abweichungen zu. Die generische Komponente wird weiter für fachlogische Regeln genutzt, welche in den bisherigen Komponenten rein analytisch verwendet wurden.

In der Implementierung wird zunächst in die Verarbeitung der account Tabelle eingegriffen und betroffene Records zunächst intern im Code gespeichert. Es werden auf Basis des Profilings in der *Specific Bronze Component* (Kapitel 4.5) folgende Regeln definiert:

- **EMAIL_VIOLATION:** Adresse enthält ungültige Zeichen oder entspricht nicht dem Standardformat einer E-Mail (inkl. Entfernen versteckter Unicode-Zeichen).
- **PHONE_VIOLATION:** Telefonnummer enthält ungültige Zeichen oder entspricht nicht dem erwarteten internationalen Rufnummerformat (inkl. Entfernen nicht sichtbarer Unicode-Zeichen).
- **METADATA_NOT_AVAILABLE:** Die genutzte Option ist in keiner Metadatatabelle vorhanden.

Anschließend wird der IF auf den Daten für folgendes Szenario trainiert: *Es gibt nicht-produktive Datensätze (z. B. Test- und Entwicklungsdaten), die sich von produktiven Daten durch ihre Aktivität auf Account-Level unterscheiden.* Dafür werden zunächst relevante Features selektiert, die auf eine eingeschränkte Aktivität und unechte Daten hindeuten können. Diese stammen einerseits aus der Betrachtung der Baseline-Erhebung (has_contactinfo), andererseits umfassen sie Aggregationen mit anderen Entitäten auf Ebene des Datenprodukts.

- **days_since_modified:** Anzahl der Tage seit der letzten Änderung auf Record-Ebene.
- **has_contactinfo“:** Mindestens eine Kontaktmöglichkeit (E-Mail, Telefon, Fax).

- **email/phone_format_violation:** Das Format von Email/Telefon verletzt den Regex.
- **missing_rate:** Anteil der leeren Spalten an der Gesamtzahl der Attribute.
- **num_contacts/projects/appointments/incidents:** Die Anzahl dem Account zugeordneten weiteren Entitäten die für eine Aktivität sprechen.

Nach der Aggregation werden die numerischen Features logarithmisch transformiert, da diese Verteilungen meist rechtsschief sind und extremere Werte sonst vom IF isoliert werden, obwohl sie auf aktive Datensätze hindeuten könnten. Der Algorithmus wird aus der SynapseML Bibliothek genutzt mit den Parametern `contamination 0.001` (Erwartung von Anomalien in %), `num_estimators = 200` (Anzahl Isolation Trees) und `maxSamples = 256` (Anzahl Stichproben pro Baum) verwendet. Der niedrige `contamination` Wert reduziert False Positives, da er den Threshold zur Anomalieklassifikation bestimmt. Die Zahl von 200 Schätzern erhöht die Stabilität und glättet Zufallsvariation. `MaxSamples` zu verringern erhöht die Fähigkeit seltene Anomalien effizient zu isolieren, da IF auf Subsamples optimiert ist. Das Model gibt für einen Datensatz einen Anomaly-Score und eine Einschätzung ab, ob es sich um einen Ausreißer handelt.

Die Ausreißer werden mit Daten, die gegen Fachregeln verstoßen isoliert in das Schema *quarantine_silver* des DQ-Zone-Lakehouses gespeichert und von einer Weiterverarbeitung in Richtung Gold ausgeschlossen. Der Quarantäne-Mechanismus bildet damit ein Quality-Gate für das Gold Layer und dient als Ausgangspunkt für manuelle Prüfungen, DQ-Analysen und Regelmodifikationen.

Microsoft Fabric Data Agent:

Die Datenplattform Microsoft Fabric ermöglicht die Erstellung sogenannter Data Agents, die auf LLM-Technologie basieren und den Zugriff auf Datenquellen wie semantische Modelle über ein Chat-Interface erlauben. Nutzer können so in natürlicher Sprache mit den Daten interagieren.

Im Use Case wird diese Funktion hinsichtlich ihrer Eignung zur Unterstützung von DQ-Prüfungen untersucht. Ziel ist es zu evaluieren, inwiefern ein Data Agent zur Identifikation von DQ-Problemen beitragen kann und welche Grenzen sich im Kontext strukturierter Unternehmensdaten ergeben. Aufgrund seines interaktiven Charakters ist der Agent keiner klassischen Schicht der Medallion-Architektur zuzuordnen.

Grundlage dafür bildet das semantische Modell des CDO Datenprodukts, das die Business-relevanten CRM-Daten auf Gold-Layer-Ebene strukturiert abbildet und mit Metadaten anreichert (Abb. 7). Dieses Modell wird dem Data Agent (dq_crm_agent) als Datenquelle zugewiesen.

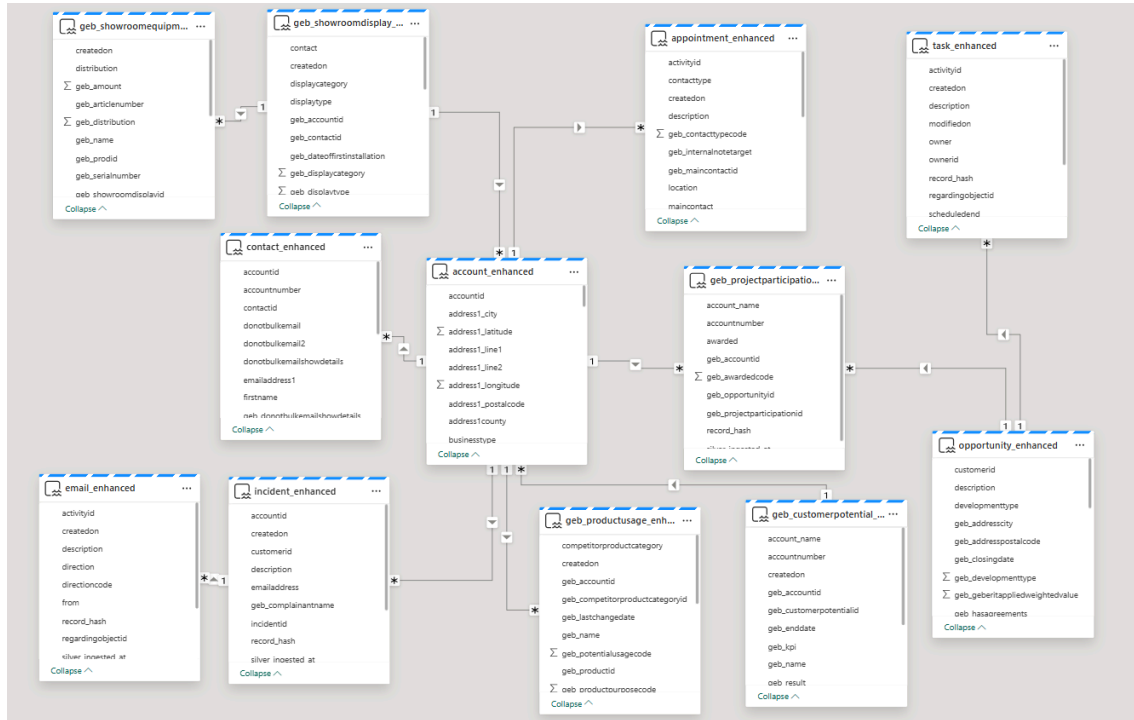


Abb. 7: Semantisches Modell des Customer Data Objects (eigene Darstellung)

Zur strukturierten Evaluation werden Testkategorien definiert, die sich an den zuvor abgeleiteten DQ-Dimensionen und den implementierten Verfahren orientieren. Neben klassischen Ansätzen wie Profiling, Regelprüfungen und Anomalieerkennung werden auch typische LLM-Fähigkeiten wie die Generierung von Regeln oder die Identifikation semantischer Zusammenhänge berücksichtigt, was in folgende Kategorien resultiert:

- **Vollständigkeit & Datenmenge:** Reproduktion einfacher deskriptiver Analysen
- **Konsistenz & Integrität:** Erkennung struktureller Zusammenhänge und Inkonsistenzen
- **Korrektheit & Genauigkeit:** Identifikation potenziell fehlerhafter Werte ohne explizite Regeln
- **Relevanz & Kontextbezug:** Bewertung kontextueller Auffälligkeiten
- **Interpretierbarkeit & Verständlichkeit:** Analyse semantischer Klarheit und Metadatenqualität

Auf dieser Basis werden spezifische Instruktionen für den Agenten definiert (Kapitel F) und die Evaluation systematisch über das Chat-Interface durchgeführt. Die vollständigen Prompts und Ergebnisse sind im Anhang dokumentiert (Kapitel G).

LLM-gestützte Rule Generation und Qualitätsprüfung:

Neben dem plattformeigenen Data Agent werden im Rahmen dieser Arbeit auch generische, API-basierte LLMs hinsichtlich ihrer Eignung zur DQ-Bewertung untersucht. Der Einsatz erfolgt dabei explizit als Gegenprobe zum Data Agent, um insbesondere dessen identifizierte Schwächen im Bereich semantischer Interpretation und kontextueller Bewertung systematisch zu untersuchen.

Während der Data Agent primär auf der Übersetzung natürlicher Sprache in datenbanknahe Abfragen basiert und damit vor allem strukturierte Analysen ermöglicht, liegt der Fokus der LLM-basierten Untersuchung auf der Interpretation von Daten im fachlichen Kontext. Ziel ist es zu evaluieren, inwiefern LLMs in der Lage sind, semantische Inkonsistenzen, implizite Zusammenhänge und kontextabhängige Auffälligkeiten zu erkennen, die über rein formale oder statistische Prüfungen hinausgehen.

Die technische Umsetzung erfolgt über explorative API-basierte Aufrufe in einem Notebook, in dem ausgewählte Datensätze an das LLM übergeben und die Antworten analysiert werden. Verwendet wird das Modell gpt-5.3-pro, bereitgestellt über Azure AI Foundry. Aufgrund technischer Einschränkungen wird die Analyse auf eine kleine Stichprobe von fünf exemplarischen Datensätzen beschränkt, die gezielt ausgewählt wurden, um unterschiedliche DQ-Probleme abzubilden.

Die Datensätze selbst werden vor der Verarbeitung in ein JSON-Objekt transformiert und zusammen mit spezifischen Prompts an das Modell übergeben. Diese Prompts orientieren sich an den zuvor definierten DQ-Dimensionen, adressieren jedoch bewusst Fragestellungen, die ein höheres Maß an Kontextverständnis erfordern. Dazu zählen insbesondere die Identifikation semantischer Inkonsistenzen innerhalb von Datensätzen, die Bewertung kontextuell unplausibler Attributkombinationen, die Einschätzung potenzieller Testdaten, die Analyse semantischer Ähnlichkeiten sowie die Ableitung fachlich begründeter DQ-Regeln.

Die vollständige Dokumentation der verwendeten Prompts sowie der generierten Antworten ist im Anhang (Kapitel H) enthalten und bildet die Grundlage für die anschließende Evaluation.

5. Evaluation

5.1. Bewertung der erweiterten Datenarchitektur

Die Bewertung der erweiterten Datenarchitektur erfolgt im Folgenden nicht hinsichtlich der inhaltlichen Qualität einzelner Verfahrensergebnisse, sondern mit Blick auf ihre technische Eignung zur Sicherstellung von DQ im Kontext der Microsoft Fabric-Umgebung. Grundlage bilden die zuvor abgeleiteten architekturbezogenen Anforderungen.

Die Erweiterung der Architektur zeigt sich konkret in Abb. 5 und Abb. 4, in denen der bestehende Datenintegrationsprozess um das DQ-Zone-Lakehouse sowie zusätzliche DQ-Komponenten entlang des Medallion-Layers ergänzt wird.

Hinsichtlich der **Skalierbarkeit und verteilten Ausführung** zeigt die Architektur insgesamt eine gute Eignung für den Einsatz in einer Lakehouse-Umgebung. Die generischen Bronze- und Silver-Komponenten basieren auf PySpark und verteilbaren Datenstrukturen, sodass Profiling, regelbasierte Prüfungen und aggregierte Ergebnisse skalierbar umgesetzt werden können. Unterschiede bestehen jedoch zwischen den Komponenten. Während generische Ansätze breit einsetzbar sind, ist die spezifische Silver-Komponente mit datengetriebenen Verfahren wie IF rechenintensiver und stärker datenabhängig. Noch eingeschränkter sind agentische und LLM-basierte Verfahren, da diese nicht auf verteilter Verarbeitung, sondern auf punktueller Inferenz basieren.

Auch hinsichtlich der **Kompatibilität mit den Speicherformaten** ist die Architektur positiv zu bewerten. Die DQ-Zone wurde als zusätzliches Lakehouse integriert und speichert Ergebnisse als Delta-Tabellen. Dadurch bleiben alle Artefakte im gleichen technologischen Paradigma, sind historisierbar und weiterverarbeitbar. Die Architektur nutzt damit konsequent die bestehenden Speicherstrukturen, ohne zusätzliche Systembrüche einzuführen.

Eine besondere Stärke liegt in der klaren Einordnung in die **Medallion-Architektur**. Die DQ-Komponenten sind entlang des Datenflusses positioniert. Im Bronze-Layer erfolgt eine frühe, deskriptive Bewertung, im Silver-Layer eine vertiefte Analyse durch Plausibilitäts- und Anomalieprüfungen, und am Übergang zu Gold wird ein Quality Gate über die Quarantäne-Logik etabliert. Ergänzt wird dies durch semantische und interaktive

Verfahren wie den Data Agent, die bewusst als Querschnittsfunktion außerhalb der Pipeline agieren. Insgesamt ermöglicht die Architektur eine durchgängige Betrachtung von DQ entlang des Lebenszyklus.

Eingeschränkt ist die Architektur jedoch bei der Unterstützung **unterschiedlicher Verarbeitungsmodi**. Die implementierten Komponenten sind primär batch-orientiert. Streaming- oder Near-Realtime-Szenarien wurden nicht umgesetzt. Dies schränkt die Übertragbarkeit auf zeitkritische Anwendungsfälle ein.

Die **Integration in Microsoft Fabric** ist insgesamt sehr stimmig. Die Umsetzung nutzt bestehende Komponenten wie Lakehouses, PySpark-Notebooks, Transformation-slogiken und das semantische Modell. Auch der Data Agent ist eingebettet, indem er auf dem Gold-Layer-Modell aufsetzt. Dadurch bleibt die Qualitätssicherung, abgesehen von den explorativen API-Calls, vollständig innerhalb der Plattform.

Differenzierter ist die Bewertung der **Automatisierbarkeit und Pipeline-Fähigkeit**. Generische und spezifische Prüfungen lassen sich gut in bestehende Prozesse integrieren und reproduzierbar ausführen. Gleichzeitig sind nicht alle Verfahren vollständig pipelinefähig. Der Data Agent ist interaktiv ausgelegt, und auch LLM-basierte Ansätze eignen sich eher für explorative oder teilautomatisierte Szenarien. Die Architektur kombiniert damit bewusst automatisierbare und nicht-automatisierbare Komponenten.

Ein weiterer wichtiger Aspekt ist die **Wiederverwendbarkeit**. Durch die Trennung in generische und spezifische Komponenten wird eine Balance zwischen Standardisierung und fachlicher Anpassung erreicht. Die generischen Komponenten konnten mit minimalen Konfigurationsanpassungen (z. B. Schema, Schlüsselattribute) auf eine weitere Datenquelle übertragen werden, was ihre Skalierbarkeit und praktische Einsetzbarkeit unterstreicht.

Insgesamt erweist sich die Architektur als geeignete Grundlage zur Integration von DQ-Verfahren in Microsoft Fabric. Stärken liegen in der engen Integration in die bestehende Architektur, der hohen Plattformkompatibilität sowie der Kombination generischer und spezifischer Komponenten. Einschränkungen bestehen insbesondere bei kontinuierlichen Verarbeitungsmodi und bei der Skalierbarkeit sowie Automatisierbarkeit

semantischer, LLM-basierter Verfahren. Dennoch stellt die entwickelte DQ-Zone eine architektonisch stimmige und praxisnahe Erweiterung des bestehenden Datenintegrationsprozesses dar.

5.2. Ermittlung des initialen Datenqualitätszustands

Die in Kapitel 4.4 durchgeführte Baseline-Erhebung dient im Rahmen der Evaluation als Referenzrahmen für die Bewertung der implementierten DQ-Verfahren.

Die Baseline basiert auf einem Snapshot der CRM-Datenlandschaft zum definierten Zeitpunkt und fokussiert insbesondere auf die Daten des Produktivsystems sowie auf im bisherigen Datenintegrationsprozess als business-relevant identifizierten Attribute. Dadurch wird eine konsistente und reproduzierbare Ausgangsbasis für die Evaluation geschaffen, die zugleich die tatsächlich im operativen Kontext genutzten Daten widerspiegelt.

Gleichzeitig ist die Aussagekraft der Baseline bewusst einzuordnen. Die Analyse wurde explorativ durchgeführt und verfolgt primär das Ziel, potenzielle Qualitätsprobleme indikativ zu identifizieren, nicht jedoch eine vollständige oder abschließende Bewertung der DQ vorzunehmen. Insbesondere für Dimensionen wie Korrektheit & Genauigkeit fehlt im betrachteten Kontext eine belastbare Grundwahrheit (Ground Truth) in Form externer Referenzdaten oder vollständig definierter fachlicher Regeln, wodurch entsprechende Aussagen nur eingeschränkt möglich sind. Darüber hinaus wurden einzelne Dimensionen, insbesondere Sicherheit & Vertrauen sowie Zugänglichkeit & Verfügbarkeit, nicht betrachtet, da sie überwiegend Governance-orientierte Aspekte adressieren und nicht im Fokus des Use Cases stehen. Auch semantische Fragestellungen oder komplexe fachliche Zusammenhänge können im Rahmen einer rein metrischen und regelbasierten Baseline nur begrenzt erfasst werden. Die Baseline ist somit als problemorientierter, jedoch bewusst nicht vollständiger Referenzrahmen zu verstehen.

Die zentralen Ergebnisse der Baseline lassen sich wie folgt zusammenfassen. Hinsichtlich der **Dimension Vollständigkeit & Datenmenge** zeigt sich eine insgesamt hohe Null-Rate im Ausgangssystem, die sich durch die Wide-Table-Struktur des CRM-Datenmodells erklären lässt. Nach Einschränkung auf business-relevante Attribute reduziert sich diese deutlich, bleibt jedoch für viele optionale Felder weiterhin signifikant. Gleichzeitig sind Identifikations- und Kontaktinformationen in der überwiegenden Mehrheit

der Datensätze vorhanden, sodass kritische Informationslücken nur in geringem Umfang auftreten.

In Bezug auf **Konsistenz & Integrität** weisen die Daten insgesamt eine stabile strukturelle Qualität auf. Primärschlüssel sind eindeutig vergeben, und referenzielle Beziehungen zwischen zentralen Entitäten sind weitgehend konsistent. Vereinzelt nicht auflösbare Referenzen lassen sich größtenteils durch die flexible, teilweise polymorphe Struktur einzelner CRM-Attribute erklären und stellen keine systematischen Integritätsprobleme dar.

Für die Dimension **Korrektheit & Genauigkeit** beschränkt sich die Analyse auf formale Prüfungen, insbesondere von E-Mail- und Telefonnummernformaten aufgrund ihres weitgehend standardisierten Charakters. Die Ergebnisse zeigen hierbei nur geringe Abweichungen, sodass keine grundlegenden systematischen Formatprobleme identifiziert werden konnten. Gleichzeitig bleibt festzuhalten, dass ohne domänenspezifische Referenzwerte oder validierte Geschäftsregeln keine belastbare Aussage über die inhaltliche Korrektheit der Daten getroffen werden kann.

Die Betrachtung von **Relevanz & Kontextbezug** verdeutlicht Unterschiede im Lebenszyklus und Nutzungsverhalten der Daten. Stammdaten weisen eine hohe Lebensdauer und teilweise regelmäßige Aktualisierungen auf, während Transaktionsdaten überwiegend historisch geprägt sind und nach ihrer Erstellung nur selten verändert werden. Ergänzend zeigen FDs zwischen ausgewählten Attributen eine hohe Stabilität, während zeitliche Inkonsistenzen praktisch nicht auftreten.

Schließlich zeigt die Analyse zur **Interpretierbarkeit & Verständlichkeit**, dass die überwiegende Mehrheit der Daten semantisch nachvollziehbar ist. Platzhalter treten nur in geringem Umfang auf, und kodierte Attributwerte lassen sich größtenteils über vorhandene Metadaten interpretieren. Einzelne Abweichungen deuten jedoch darauf hin, dass die semantische Dokumentation nicht in allen Fällen vollständig mit den tatsächlich verwendeten Datenwerten übereinstimmt.

Zusammenfassend zeigt die Baseline-Erhebung, dass die betrachtete Datengrundlage eine insgesamt stabile strukturelle Integrität aufweist, gleichzeitig jedoch deutliche

Unterschiede hinsichtlich Vollständigkeit, Pflegegrad sowie potenzieller Auffälligkeiten zwischen Attributen und Tabellen bestehen. Daraus ergibt sich ein geeigneter Referenzrahmen, um die im weiteren Verlauf betrachteten DQ-Verfahren hinsichtlich ihres Beitrags zur Identifikation, Bewertung und potenziellen Verbesserung dieser Aspekte einzuordnen.

5.3. Analyse der implementierten DQ-Komponenten für Bronze & Silver

Die im Use Case implementierten DQ-Komponenten für das Bronze- und Silver-Layer bilden die Grundlage der technischen DQ-Bewertung. Sie verfolgen primär einen deterministischen Ansatz, der auf statistischem Profiling sowie regelbasierter Validierung basiert. Aufbauend auf den in der Baseline-Erhebung identifizierten strukturellen Eigenschaften und potenziellen Qualitätsproblemen wird im Folgenden untersucht, inwiefern diese Komponenten zur systematischen Identifikation und Einordnung von DQ-Problemen beitragen können. Der Fokus liegt dabei insbesondere auf dem Erkenntnisgewinn z. B. für die weitere Operationalisierung, der Interpretierbarkeit der Ergebnisse sowie den methodischen und praktischen Grenzen der eingesetzten Verfahren.

Generic Bronze Component

Die generische Bronze-Komponente dient der initialen, ingestion-nahen Analyse des Datenzustands und generiert Profiling-Ergebnisse auf Tabellen-, Spalten- sowie Schemaebene. Die resultierenden Reports liefern einen umfassenden Überblick über strukturelle Eigenschaften der Daten, wie etwa Datentypverteilungen, Null-Raten, Wertebereiche oder Schemaabweichungen.

Im Kontext der Evaluation zeigt sich, dass diese Komponente insbesondere zur Erhöhung der Transparenz über den Datenzustand beiträgt. Die erzeugten Kennzahlen ermöglichen eine schnelle Identifikation grundlegender Muster, etwa stark variierende Null-Raten zwischen Attributen oder auffällige Verteilungen numerischer Werte. Damit bestätigt das Profiling in weiten Teilen die in der Baseline-Erhebung identifizierten Eigenschaften der Daten und stellt diese in reproduzierbarer Form bereit. Abb. 8 zeigt exemplarisch Ergebnisse des Bronze Table-Profilings.

Table view

	ABC table_name	ANY type_counts	12L row_count	12L min_agediff	12L max_agediff	12L min_updatediff	12L max_updatediff	12 avg_agediff	12 avg_updatediff
1	geb_geberitcom...	["numeric":8,"dat...	4005	63	4162	0	3748	1838.326092384...	1088.1083916083917
2	geb_federalstat...	["numeric":7,"dat...	892	526	4214	0	3115	2274.081838565...	1375.6356502242152
3	geb_district	["numeric":10,"d...	771	1941	3321	0	958	2660.533073929...	499.50194552529183
4	opportunity	["numeric":101,"...	417633	39	10592	0	9497	2071.742819652...	1262.3832213881205
5	incident	["numeric":50,"d...	1470765	39	5307	0	5141	1158.152712363...	810.2357216677092
6	contact	["numeric":96,"d...	2546994	39	46100	0	46061	2609.584875739...	2514.702328956234
7	email	["numeric":32,"d...	4083809	39	6997	0	6452	1184.136091575...	758.1329121968673
8	OptionsetMetad...	["numeric":2,"bo...	99064	NULL	NULL	NULL	NULL	NULL	NULL

Abb. 8: Auszug Ergebnisse Table Profiling (eigene Darstellung)

Gleichzeitig ist die Aussagekraft der Ergebnisse in Bezug auf konkrete DQ-Probleme begrenzt. Die generierten Statistiken sind primär deskriptiver Natur und erlauben ohne zusätzliche Kontextinformationen keine eindeutige Unterscheidung zwischen erwartbarem Datenverhalten und tatsächlichen Qualitätsproblemen. So lassen sich beispielsweise hohe Null-Raten zwar klar identifizieren, jedoch nicht ohne Weiteres als fehlerhaft interpretieren, da sie im CRM-Kontext häufig strukturell bedingt sind. Auch die Schema-Comparison liefert im betrachteten Snapshot-Szenario keine signifikanten Auffälligkeiten, da keine zeitliche Veränderung der Struktur vorliegt. Sie besitzt jedoch das Potenzial Ursachen und Zuständigkeiten für DQ-Probleme wie Schema Mismatches klar zu benennen.

Die Abbildung Abb. 8 zeigt exemplarisch die Ergebnisse der Schema Comparison zwischen Silver und Bronze des DCRM-Schemas. Der Screenshot zeigt die Spalten, welche im Transformationsprozess von Bronze auf Silver nicht berücksichtigt werden.

Table view

	ABC table_name	ABC column_name	ABC change_type	ABC bronze_data_type	ABC silver_data_type	0/1 bronze_nullable	0/1 silver_nullable	📅 profile_date
1	opportunity	modifiedby	removed_in_silver	string	NULL	true	NULL	2026-03-23
2	opportunity	geb_projecttype	removed_in_silver	bigint	NULL	true	NULL	2026-03-23
3	opportunity	geb_addresslinetwo	removed_in_silver	string	NULL	true	NULL	2026-03-23
4	opportunity	geb_geberitpecifiedweightedvalue_base	removed_in_silver	decimal(38,0)	NULL	true	NULL	2026-03-23
5	opportunity	geb_websiteurl	removed_in_silver	string	NULL	true	NULL	2026-03-23
6	opportunity	createdonbehalfofbyyominame	removed_in_silver	string	NULL	true	NULL	2026-03-23
7	opportunity	geb_responsibilityidname	removed_in_silver	string	NULL	true	NULL	2026-03-23
8	opportunity	totalitemdiscountamount	removed_in_silver	decimal(38,0)	NULL	true	NULL	2026-03-23

Abb. 9: Beispiel Schema Comparison nicht genutzte Spalten (eigene Darstellung)

Zuletzt zeigt Abb. 10 exemplarisch die Ergebnisse des Column Profilings, wobei die Spalten für Timestamp- und numerische Datentypen (min_len, max_len, avg_len, min_value, max_value, avg_value, p05, median_value, p95, p99, mindate und maxdate) ausgeblendet sind.

Table view

#	ARC table_name	ARC column_name	%L row_count	ARC data_type	ARC type_class	%L null_count	%L null_rate	%L empty_string_count	%L distinct_count	%L distinct_rate
1	geb_productusa...	owneridomina...	934063	string	string	934063	1.0	0	0	0.0
2	geb_productusa...	owningbusinessun...	934063	string	string	1052	0.0011262623...	0	26	2.783538155349...
3	geb_productusa...	timezoneruleversio...	934063	bigint	numeric	53901	0.0577059577...	NULL	5	5.352957991056...
4	geb_productusa...	utcconversiontime...	934063	bigint	numeric	934063	1.0	NULL	0	0.0
5	geb_productusa...	versionnumber	934063	bigint	numeric	0	0.0	NULL	871139	0.932634094274...
6	geb_productusa...	isDelete	934063	boolean	boolean	933011	0.9988737376...	NULL	1	1.070591598211...
7	geb_productusa...	Partitionid	934063	string	string	0	0.0	0	3	3.211774794633...
8	geb_productusa...	geb_productpurpo...	934063	bigint	numeric	1052	0.0011262623...	NULL	29	3.104715634812...
9	geb_productusa...	geb_usageposition...	934063	bigint	numeric	269095	0.2880908461...	NULL	3	3.211774794633...
10	geb_productusa...	geb_fromexternals...	934063	boolean	boolean	1052	0.0011262623...	NULL	1	1.070591598211...
11	geb_productusa...	createdby	934063	string	string	1052	0.0011262623...	0	608	6.509196917124...
12	geb_productusa...	createdby_entitytype	934063	string	string	1052	0.0011262623...	0	1	1.070591598211...
13	geb_productusa...	createdonbehalfof	934063	string	string	926929	0.9923623995...	0	115	1.231180337942...
14	geb_productusa...	createdonbehalfof...	934063	string	string	926929	0.9923623995...	0	1	1.070591598211...

Abb. 10: Ausschnitt Ergebnisse Column Profiling (eigene Darstellung)

Insgesamt erweist sich die generische Bronze-Komponente somit als skalierbares und wiederverwendbares Werkzeug zur explorativen Analyse, dessen Stärke in der strukturierten Beschreibung des Datenzustands liegt, während die Fähigkeit zur konkreten Fehleridentifikation ohne ergänzende Verfahren eingeschränkt bleibt.

Specific Bronze Component

Die spezifische Bronze-Komponente erweitert das generische Profiling um fachlogisch definierte Prüfregelein. Die Ergebnisse werden in Form aggregierter Kennzahlen dargestellt, z. B: als Anzahl und Anteil betroffener Datensätze pro Regelverletzung.

Die Evaluation zeigt, dass diese Komponente einen deutlichen Mehrwert gegenüber dem rein deskriptiven Profiling bietet. Durch die explizite Definition von Regeln lassen sich konkrete Qualitätsprobleme identifizieren, etwa ungültige E-Mail-Formate, fehlerhafte Telefonnummern oder nicht auflösbare Referenzen. Im Gegensatz zur generischen Komponente sind die Ergebnisse somit direkt interpretierbar und können klar als Regelverstöße eingeordnet werden.

Gleichzeitig wird die Abhängigkeit von den zugrundeliegenden Regeldefinitionen deutlich. Insbesondere bei formatbasierten Prüfungen (E-Mail-Adressen oder Telefonnummern) hängt die Anzahl identifizierter Verstöße stark von der gewählten Regex-Definition ab. Dies führt dazu, dass abhängig von der Restriktivität der Regelformulierung sowohl False Positives als auch False Negatives auftreten können. Demgegenüber zeigen referenzielle Prüfungen oder funktionale Abhängigkeiten eine höhere Robustheit, da sie weniger interpretationsabhängig sind, jedoch explizite Formulierung voraussetzen.

Ein wesentlicher Aspekt ist die Aggregation der Ergebnisse auf Tabellenebene, wie sie in Abb. 11 dargestellt ist. Während dies eine performante und skalierbare Umsetzung ermöglicht, fehlt eine detaillierte Record-Level-Sicht, wodurch die direkte Nachverfolgbarkeit einzelner Fehler eingeschränkt ist. Dies ist im aktuellen Use Case darauf zurückzuführen, Performanceverluste im Produktivbetrieb weitestgehend zu verhindern, da die Auswertung in derselben technischen Komponente geschieht.

Insgesamt zeigt sich, dass die spezifische Bronze-Komponente insbesondere für klar definierbare, regelbasierte Qualitätsanforderungen geeignet ist, jedoch einen erhöhten Implementierungsaufwand erfordert und in ihrer Aussagekraft stark von der Qualität der definierten Regeln abhängt, sowie ihre Ausgestaltung aus Performance-Sicht bewertet werden muss.

Table view

	ARC table_name	ARC check_type	ARC column_name	%L affected_count	%L row_count	%L affected_ratio	ARC details
6	geb_showroomdisplay	INVALID_GLOBAL_OPTIONSET	geb_displaytype	0	113813	0.0	value not found in global optionset metadata
7	account	INVALID_GLOBAL_OPTIONSET	geb_buynestypecode	28457	529606	0.0537323972915...	value not found in global optionset metadata
8	account	INVALID_GLOBAL_OPTIONSET	geb_partnerstatuscode	0	529606	0.0	value not found in global optionset metadata
9	contact	INVALID_OPTIONSET	geb_donotbulkemailshowdetails	918037	2546994	0.3604394042545...	value not found in local optionset metadata
10	geb_showroomdisplay	INVALID_GLOBAL_OPTIONSET	geb_location	0	113813	0.0	value not found in global optionset metadata
11	geb_showroomequipments	INVALID_OPTIONSET	geb_distribution	0	474070	0.0	value not found in local optionset metadata
12	geb_showroomdisplay	INVALID_OPTIONSET	geb_displaycategory	0	113813	0.0	value not found in local optionset metadata
13	appointment	INVALID_GLOBAL_OPTIONSET	geb_contacttypecode	0	1822890	0.0	value not found in global optionset metadata
14	account	INVALID_OPTIONSET	geb_donotbulkemailshowdetails	91932	529606	0.1735856466882...	value not found in local optionset metadata
15	account	INVALID_OPTIONSET	donotbulkemail	529606	529606	1.0	value not found in local optionset metadata
16	account	INVALID_REFERENCE	geb_responsibilityid	527185	529606	0.9954286771675...	no contact found for responsibilityid
17	account	MISSING_SAP_NUMBER	geb_sapnumber	364681	529606	0.6885892531429...	sap number missing for accountnumber
18	account	INVALID_GLOBAL_OPTIONSET	geb_locationvisibilitycode	0	529606	0.0	value not found in global optionset metadata
19	account	INVALID_OPTIONSET	geb_rating	0	529606	0.0	value not found in local optionset metadata
20	account	INVALID_GLOBAL_OPTIONSET	geb_otcrating	0	529606	0.0	value not found in global optionset metadata
21	account	INVALID_GLOBAL_OPTIONSET	geb_showroomrating	0	529606	0.0	value not found in global optionset metadata
22	opportunity	INVALID_OPTIONSET	geb_developmenttype	0	417633	0.0	value not found in local optionset metadata

Abb. 11: Ausschnitt der Regelprüfungen im Bronze (eigene Darstellung)

Generic Silver Component

Die generische Silver-Komponente erweitert die Analyse um allgemeine Plausibilitätsprüfungen sowie statistische Verfahren zur Identifikation von Auffälligkeiten. Die Ergebnisse werden sowohl auf aggregierter Tabellenebene als auch auf Record-Ebene bereitgestellt, wodurch eine detailliertere Analyse ermöglicht wird.

Im Rahmen der Evaluation zeigt sich, dass diese Komponente eine wichtige Brücke zwischen deskriptivem Profiling und tatsächlicher Problemidentifikation darstellt. Die generischen Plausibilitätsannahmen liefern in vielen Fällen sinnvolle Hinweise auf potenzielle Qualitätsprobleme, etwa bei negativen Werten in numerischen Feldern oder bei zeitlichen Inkonsistenzen.

Die Integration statistischer Verfahren wie Z-Score oder IQR ermöglicht darüber hinaus die Identifikation von Ausreißern, die durch klassische Regeln nicht erfasst werden. So konnten beispielsweise auffällige Datensätze mit ungewöhnlich hohen Aktivitätswerten identifiziert werden, die auf besondere Nutzungsmuster oder potenziell fehlerhafte Daten hinweisen.

Gleichzeitig zeigt sich jedoch eine zentrale Limitation dieser Verfahren in ihrer Interpretierbarkeit. Statistische Auffälligkeiten sind nicht zwangsläufig gleichbedeutend mit DQ-Problemen. In der praktischen Anwendung treten daher auch Fehlinterpretationen auf, etwa wenn kategoriale Werte oder technisch kodierte Attribute fälschlicherweise als numerische Größen interpretiert werden und dadurch als Ausreißer erscheinen. Ebenso können String-Felder, die als Datum interpretierbar sind, zu scheinbar sinnvollen, jedoch fachlich irrelevanten Ergebnissen führen.

Table view

	ARC run_id	run_ts	ARC table_name ↑	ARC column_name	ARC rule_id	12L row_count	12L affected_count	12 affected_ratio	ARC threshold_value
3	e4e2152b-3...	2026-03-20...	StatusMetadata	createdonpartition	STRING_PARSEABLE_AS_DATE	18839	18839	1.0	parseable_as_date
4	e4e2152b-3...	2026-03-20...	StatusMetadata	EntityName	STRING_PARSEABLE_AS_BOOLEAN	18839	0	0.0	parseable_as_boolean
5	e4e2152b-3...	2026-03-20...	StatusMetadata	LocalizedLabel	STRING_PARSEABLE_AS_DATE	18839	0	0.0	parseable_as_date
6	e4e2152b-3...	2026-03-20...	StatusMetadata	EntityName	STRING_PARSEABLE_AS_DATE	18839	0	0.0	parseable_as_date
7	e4e2152b-3...	2026-03-20...	StatusMetadata	Status	NEGATIVE_VALUE_SHARE	18839	0	0.0	value < 0
8	e4e2152b-3...	2026-03-20...	StatusMetadata	Status	IQR_OUTLIER	18839	3935	0.2088752056903...	1.5*IQR
9	e4e2152b-3...	2026-03-20...	StatusMetadata	Status	ZSCORE_OUTLIER	18839	272	0.0144381336588...	3
10	e4e2152b-3...	2026-03-20...	account	accountid	NULL_CRITICAL	529606	0	0.0	critical_column_null...
11	e4e2152b-3...	2026-03-20...	account	telephone1	NULL_CRITICAL	529606	9685	0.0182871795259...	critical_column_null...
12	e4e2152b-3...	2026-03-20...	account	emailaddress1	NULL_CRITICAL	529606	217849	0.4113416388787...	critical_column_null...
13	e4e2152b-3...	2026-03-20...	account	accountid	BUSINESS_KEY_DUP	529606	0	0.0	duplicate_business...
14	e4e2152b-3...	2026-03-20...	account	address1_line1	STRING_PARSEABLE_AS_DATE	529606	0	0.0	parseable_as_date
15	e4e2152b-3...	2026-03-20...	account	address1_line2	STRING_PARSEABLE_AS_DATE	529606	0	0.0	parseable_as_date
16	e4e2152b-3...	2026-03-20...	account	address1_postalcode	STRING_PARSEABLE_AS_DATE	529606	2	3.7763922614169...	parseable_as_date
17	e4e2152b-3...	2026-03-20...	account	address1_city	STRING_PARSEABLE_AS_DATE	529606	0	0.0	parseable_as_date
18	e4e2152b-3...	2026-03-20...	account	geb_deviatinglocat...	STRING_PARSEABLE_AS_DATE	529606	0	0.0	parseable_as_date
19	e4e2152b-3...	2026-03-20...	account	geb_responsibilityid	STRING_PARSEABLE_AS_DATE	529606	0	0.0	parseable_as_date

Abb. 12: Beispiele Generic Silver Profiling (eigene Darstellung)

Beispielsweise wird das Feld *createdonpartition* der StatusMetadata Tabelle in Abb. 12 vollständig korrekt als Datumsfeld erkannt, jedoch dem Feld Status selbst Ausreißer beigemessen, obwohl es sich um kategoriale (OptionSet) Werte handelt. Auf Record Ebene zeigt Abb. 13 anschaulich, dass bspw. der Längen- und Breitengrad der Adresse durch die generische Negativ-Prüfung geloggt wird, was fachlich irrelevant ist, sowie die Übersetzbarkeit von *createdonpartition* als Datum.

Diese Beobachtungen verdeutlichen, dass die generische Silver-Komponente zwar eine erweiterte Erkennungsfähigkeit für potenzielle Auffälligkeiten bietet, ihre Ergebnisse jedoch einer sorgfältigen fachlichen Einordnung bedürfen. Ohne diese Kontextualisierung

besteht die Gefahr von Fehlinterpretationen und einer Überschätzung der tatsächlichen DQ-Probleme.

Table view

	ABC run_id	run_ts	ABC table_name	ABC record_key	ABC column_name	ABC rule_id	ABC observed_value
1	e4e2152b-3...	2026-03-20...	OptionsetMetad...	708a803fc062aa...	createdonpartition	STRING_PAR...	2026-02-09 02:21.35
2	e4e2152b-3...	2026-03-20...	OptionsetMetad...	104ee2f3ee78b...	createdonpartition	STRING_PAR...	2026-02-09 02:21.35
3	e4e2152b-3...	2026-03-20...	OptionsetMetad...	2b0873a465eaf...	createdonpartition	STRING_PAR...	2026-02-09 02:21.35
4	e4e2152b-3...	2026-03-20...	OptionsetMetad...	30e7649da2e36...	createdonpartition	STRING_PAR...	2026-02-09 02:21.35
5	e4e2152b-3...	2026-03-20...	OptionsetMetad...	eff2fac3206955...	createdonpartition	STRING_PAR...	2026-02-09 02:21.35
6	e4e2152b-3...	2026-03-20...	account	55ce61d138689...	emailaddress1	NULL_CRITI...	NULL
7	e4e2152b-3...	2026-03-20...	account	2d3ee6912b209...	emailaddress1	NULL_CRITI...	NULL
8	e4e2152b-3...	2026-03-20...	account	605a2e1e5af34...	emailaddress1	NULL_CRITI...	NULL
9	e4e2152b-3...	2026-03-20...	account	5046507a3e4c6...	emailaddress1	NULL_CRITI...	NULL
10	e4e2152b-3...	2026-03-20...	account	af2922acfa17a7...	emailaddress1	NULL_CRITI...	NULL
11	e4e2152b-3...	2026-03-20...	account	c3206df759aaca...	address1_longitude	NEGATIVE_V...	-3.89953
12	e4e2152b-3...	2026-03-20...	account	e0f5bb8529013...	address1_longitude	NEGATIVE_V...	-3.76148
13	e4e2152b-3...	2026-03-20...	account	8ab37a2214893...	address1_longitude	NEGATIVE_V...	-2.14761
14	e4e2152b-3...	2026-03-20...	account	2ea6a42b5ba4f...	address1_longitude	NEGATIVE_V...	-0.45996
15	e4e2152b-3...	2026-03-20...	account	6ab7d2849ee1f...	address1_longitude	NEGATIVE_V...	-1.70505
16	e4e2152b-3...	2026-03-20...	account	d3de385cc8646...	address1_longitude	NEGATIVE_V...	-9.1414
17	e4e2152b-3...	2026-03-20...	account	76e235eb703d9...	address1_longitude	NEGATIVE_V...	-6.36723

Abb. 13: Beispiele Generic Silver Records (eigene Darstellung)

Specific Silver Component

Die spezifische Silver-Komponente stellt die konsequenteste Weiterentwicklung der regelbasierten Ansätze dar, indem sie identifizierte Regelverstöße nicht nur analysiert, sondern aktiv in die Datenverarbeitung eingreift. Datensätze, die definierte Anforderungen nicht erfüllen, werden isoliert und von der Weiterverarbeitung ausgeschlossen. Durch diese Quarantäne-Logik wird zudem ein explizites Quality Gate für das Gold-Layer geschaffen, wodurch sichergestellt werden kann, dass bestimmte Mindestanforderungen an die DQ eingehalten werden, was einen wesentlichen Unterschied zu den zuvor betrachteten primär analytisch-deskriptiven Komponenten darstellt.

Die Evaluation zeigt, dass diese Komponente insbesondere für klar definierte Qualitätskriterien einen hohen praktischen Nutzen bietet. So können beispielsweise Datensätze mit ungültigen E-Mail-Adressen oder fehlerhaften Telefonnummern zuverlässig identifiziert und isoliert werden. Die in Abb. 14 und Abb. 15 bereitgestellten Beispiele verdeutlichen, dass es sich hierbei nicht nur um theoretische Auffälligkeiten, sondern um tatsächlich problematische Daten handelt.

Table view

	ABC accountid	ABC emailaddress1
56	f647fdae-45a5...	@ap.jil.com
57	fd0708f2-ca46...	info@keye-hamburg
58	0405e3c5-0ec...	pershads!arc@yahoo.co.in
59	ba5daf6d-21ff...	duum@duum.pl.am@duum.pl
60	da9f4a15-164...	@watsonbatty.com
61	663c1d52-134...	info@alexsmurrayconstruction.co.u
62	b7b04f88-000...	juraj.karen@gmail.com,karen@i4u.sk
63	05c38cf7-340f...	sekretariat@homola.as
64	a9112981-15b...	???@gmail.com
65	aaf2f209-c991...	info@messner-architektur.d
66	bfe77a9e-f3d6...	bueror@galliard@bluewin.ch
67	300c1d1f-7f96...	idrotermicaevangelista@gmail.com;idrotermica.evangelista@gmail.com
68	75e4c693-3fde...	domkeramiki@i.ua.stebako031213@gmail.com,
69	3f609246-0bc0...	c.lasorogeeka@gmail.com,
70	7fc81e51-14c0...	amit@spcdelhi.com,
71	a3d61329-46a...	@mijadesigngroup.com
72	d7ecfa2f-20ff...	brzezny_pawel@poczta.onet.brzezny.pawel@neostrada.pl,brzezny.pawel@i...

Abb. 14: Beispiele Mail Violation (eigene Darstellung)

Table view

	ABC accountid	ABC telephone1
16	1c724d60-f2d...	-
17	1c886725-544...	+49 851 41061 ...
18	1d48066e-96a...	0049
19	243d9d98-a58...	0
20	250ee8fd-cdd...	031
21	258c9c68-514...	+49
22	2739b763-e82...	0 nur noch Handy
23	2e5bb957-f00...	0049
24	2f10d024-01b...	09941 1247 o. 7...
25	3600f273-fd53...	0
26	3811c627-cf58...	0
27	3e287ab8-3c1...	0
28	40cb6583-c4a...	08463-247 u. 326
29	41338b32-e3d...	031
30	49b41cf5-d98...	+49
31	4cc3c33b-be9...	0
32	54ce55f3-262a...	0049

Abb. 15: Beispiele Phone Violation (eigene Darstellung)

Im betrachteten Use Case werden durch die definierten Domain-Regeln 36.176 Datensätze von 529.606 Daten der Account-Tabelle als Verstöße identifiziert und nicht in das Gold-Layer fortgeschrieben, sondern im Quarantäne-Schema des DQ-Zone-Lakehouses isoliert. Davon entfallen 1.116 Regelverletzungen auf den E-Mail-Check, 7.013 auf Telefonnummern und 28.271 Datensätze nutzen Optionsets, denen keine Definition in den Metadaten zugeordnet werden kann. Entsprechend müssen diese Daten manuell überprüft und freigegeben werden.

Gleichzeitig ist auch diese Komponente mit Einschränkungen verbunden. Die Definition geeigneter Regeln erfordert fachliches Wissen sowie eine kontinuierliche Pflege und

Anpassung. Zudem besteht die Gefahr, dass zu restriktive Regeln valide Daten ausschließen, während zu großzügige Definitionen problematische Datensätze weiterhin durchlassen. Insgesamt zeigt sich, dass die spezifische Silver-Komponente insbesondere für geschäftskritische Qualitätsanforderungen geeignet ist, jedoch eine sorgfältige Balance zwischen Strenge und Flexibilität erfordert.

Die Evaluation der Bronze- und Silver-Komponenten zeigt, dass sich die eingesetzten Verfahren sinnvoll ergänzen. Während das generische Profiling im Bronze-Layer eine skalierbare Transparenz über den Datenzustand schafft, ermöglichen regelbasierte Verfahren die gezielte Identifikation konkreter Qualitätsprobleme. Die Erweiterung um statistische Methoden im Silver-Layer erhöht die Erkennungsfähigkeit für nicht explizit definierte Auffälligkeiten, bringt jedoch zusätzliche Interpretationsunsicherheiten mit sich. Keines der Verfahren ist dabei isoliert ausreichend. Erst die Kombination aus deskriptiven, regelbasierten und statistischen Ansätzen ermöglicht eine differenzierte Bewertung der DQ. Gleichzeitig wird deutlich, dass die Interpretation der Ergebnisse sowie die Definition geeigneter Regeln zentrale Aufgaben bleiben und nicht vollständig automatisierbar sind.

Diese Erkenntnisse bilden die Grundlage für die anschließende Evaluation datengetriebener und intelligenter Verfahren, die insbesondere auf die adressierten Limitationen in Bezug auf Kontext, Semantik und Mustererkennung abzielen.

5.4. Evaluation der experimentellen Untersuchung der intelligenten Verfahren

Isolation Forest zur Anomalieerkennung

Im Rahmen der Evaluation datengetriebener Verfahren wurde der Einsatz eines IF-Modells zur Identifikation potenziell auffälliger Datensätze untersucht. Ziel war es zu prüfen, inwiefern sich nicht-produktive oder ungewöhnliche Datensätze, wie z. B. Test- oder Entwicklungsdaten, auf Basis von Nutzungsmustern automatisiert erkennen lassen.

Im ersten Versuch wurde ein Feature-Set definiert, das strukturelle und aktivitätsbezogene Eigenschaften der Datensätze abbildet, darunter Null-Rate, vorhandene Kontaktinformationen, Zeit seit letzter Aktualisierung sowie Aktivitätskennzahlen wie die Anzahl

von Projekten oder Kontakten. Ziel war es, inaktive oder unvollständige Datensätze zu identifizieren, die sich durch niedrige Ausprägungen dieser Merkmale auszeichnen.

Das Modell wurde mit einer Kontamination von 1 % trainiert und klassifizierte entsprechend etwa 1 % der Datensätze als Anomalien. Die Ergebnisse zeigen jedoch, dass nicht die erwarteten "schwachen" Datensätze erkannt wurden, sondern vor allem solche mit besonders hoher Aktivität. Dies wird durch die stark überdurchschnittlichen Z-Score-Werte zentraler Aktivitätsmerkmale deutlich, während beispielsweise Null-Rate oder Aktualität eher unterdurchschnittliche Werte aufweisen. Auch die Log-Transformation zeigte keine ausreichende Wirkung zur Dämpfung extremer Werte. Die Verteilung der Statuswerte bestätigt dieses Bild. Ein Großteil der als anomal identifizierten Datensätze gehört zu aktiven Accounts. Insgesamt erkennt das Modell somit primär statistische Extremwerte, unabhängig von ihrer fachlichen Relevanz.

Der erste Versuch verdeutlicht damit eine zentrale Limitation datengetriebener Verfahren: Ohne geeignete Feature-Auswahl und klare Zieldefinition werden zwar Ausreißer identifiziert, diese entsprechen jedoch nicht zwangsläufig fachlich relevanten Anomalien.

Aufbauend darauf wurde im zweiten Versuch ein angepasstes Feature-Set verwendet, das stärker auf den Nutzungskontext ausgerichtet ist. Statt direkter Aktivitätsmetriken wurden aggregierte und besser interpretierbare Kennzahlen wie die Anzahl verknüpfter Entitäten, die Zeit seit der letzten Änderung sowie eine vereinfachte Missing-Rate genutzt. Zusätzlich wurde die Kontamination auf 0,1 % reduziert und die Modellparameter angepasst, um stabilere Ergebnisse zu erzielen.

Die Ergebnisse zeigen weiterhin, dass die identifizierten Anomalien durch extreme Ausprägungen einzelner Features geprägt sind. Insbesondere weisen diese Datensätze hohe Aktivitätswerte bei gleichzeitig niedriger Missing-Rate und aktueller Pflege auf. Auch hier dominieren somit statistische Extremwerte die Klassifikation. Im Vergleich zum ersten Versuch sind die Ergebnisse zwar stabiler, die grundlegende Charakteristik bleibt jedoch unverändert.

- avg(days_since_modified_z) -1.2948407358595047
- avg(has_contactinfo_z) -0.05020016020026232
- avg(missing_rate_z) -2.672427287411258

- avg(num_contacts_z) 4.5828224152400505
- avg(num_projects_z) 6.321333423184734
- avg(num_appointments_z) 3.7033819339806366
- avg(num_incidents_z) 9.442533404606849

Insgesamt zeigen die Experimente, dass der IF geeignet ist, auffällige Muster in hochdimensionalen Daten zu identifizieren, insbesondere seltene oder extreme Kombinationen von Attributwerten. Im Use Case wurden 530 von 529.606 Datensätzen als Anomalien isoliert und von der Weiterverarbeitung ausgeschlossen. Die Analyse zeigt jedoch, dass es sich dabei nicht zwangsläufig um fachlich problematische Daten handelt, sondern häufig um besonders aktive und gut gepflegte Datensätze (siehe Abb. 16).

abc_run_id	abc_name	abc_geb_nametwo	abc_geb_accountnumber	abc_address1_line1	abc_emailaddress1	abc_telephone1	modifiedon	12_has_contactinfo	123_days_since_modified	124_num_contacts
8358f877-5...	Otto Cobo...	Heizung - Lüftung...	AC-100014124	Lübecker Landstraß...	info@cobobes.de	+49 4521 70490	2026-01-30 06:5...	1.0	53	23
8358f877-5...	August The...	Sanitär - Heizung	AC-100021142	Nordring 228	info@august-theb...	+49 2861 924580	2026-01-29 08:4...	1.0	54	20
8358f877-5...	D. F. Liedelt	Heizungs-u. Sanitä...	AC-100029699	Robert-Koch-Str. 23	info@liedelt-direkt...	+49 40 529020	2026-02-01 21:2...	1.0	51	58
8358f877-5...	Friedrich L...	Sanitär-Großhandel	AC-100029658	Bornkampsweg 58	info@friedrich-lan...	+49 40 8996630	2026-02-08 21:5...	1.0	44	40
8358f877-5...	Mosecker ...	Vertriebsbereich U...	AC-100029739	Beckstrasse 26	info.lingen@mose...	+49 591 97500	2026-02-01 21:2...	1.0	51	39
8358f877-5...	Tschantre AG	NULL	AC-100033263	Birsfelderstrasse 44	info@tschantre.com	+41 61 338 30 30	2026-02-09 09:1...	1.0	43	50
8358f877-5...	Hassmann ...	Fachgroßhandel fü...	AC-100029579	Leopoldstr. 50	kontakt.hassmann...	+49 9281 6080	2026-02-01 15:5...	1.0	51	68
8358f877-5...	Gétaz-Mia...	Appareils sanitaires	AC-100034436	avenue de la Prallie...	info@getaz-roman...	+41 22 918 15 00	2026-02-01 22:0...	1.0	51	30
8358f877-5...	Blank & Se...	Thür. Fachgroßh. f...	AC-100029647	Österholzstr. 1	info.blank@gc-gru...	+49 3643 5600	2026-02-01 17:5...	1.0	51	89
8358f877-5...	Fritz Pogen...	Sanitär-Großhandel	AC-100029806	Königskamp 1	info@pogenwisch...	+49 2461 97160	2026-01-22 07:0...	1.0	61	33

Abb. 16: Beispiele falsch klassifizierte Anomalie (eigene Darstellung)

Die Aussagekraft der Ergebnisse hängt somit stark von der Feature-Auswahl und der fachlichen Definition von Anomalien ab. Ohne diese identifiziert das Modell primär statistische Ausreißer, deren Relevanz für die DQ begrenzt ist. Zudem ist die Interpretierbarkeit eingeschränkt, da Anomalie-Scores eine nachgelagerte fachliche Einordnung erfordern.

Insgesamt stellt der IF eine sinnvolle explorative Ergänzung dar, kann jedoch nicht isoliert zur DQ-Bewertung eingesetzt werden. Sein Mehrwert liegt insbesondere in der Identifikation potenzieller Auffälligkeiten, die anschließend durch regelbasierte oder fachlich fundierte Verfahren weiter untersucht werden müssen.

Data Agent zur Qualitätsbewertung

Im Rahmen der experimentellen Untersuchung intelligenter Verfahren wurde der Data Agent auf seine Eignung zur Unterstützung von DQ-Prüfungen evaluiert. Ziel war es zu untersuchen, inwiefern ein auf einem semantischen Modell basierender Agent typische Fragestellungen der DQ-Bewertung beantworten kann und welche Grenzen sich dabei im Kontext der strukturierten Unternehmensdaten zeigen. Die Untersuchung orientiert sich an den zuvor definierten DQ-Dimensionen Vollständigkeit & Datenmenge, Konsistenz

& Integrität, Korrektheit & Genauigkeit, Relevanz & Kontextbezug, Interpretierbarkeit & Verständlichkeit sowie ergänzend an expliziten LLM-bezogenen Fragestellungen wie Regelgenerierung, funktionalen Abhängigkeiten, Ähnlichkeitsbewertung und potenziellen Dubletten. Für den Agenten wurden Anweisungen als System-Prompt definiert, die bei jeder Interaktion automatisch mitgeschickt werden (Kapitel F). Des Weiteren findet sich eine Auflistung der Prompts und die Antworten des Agenten im Anhang (vgl. Kapitel G).

Hier ist ein oberflächliches Data Profiling der Account-Tabelle (account_enhanced) – ohne Rückgriff auf frühere Analysen:

Spaltenname	Datentyp	Anzahl Zeilen	Nicht-leer	Nicht-leer (%)	Distinct Werte	Auffälligkeiten
accountid	Text	532.293	532.293	100%	532.293	Vermutlich eindeutiger Schlüssel
address1_city	Text	532.293	532.218	99,99%	72.928	—
address1_line1	Text	532.293	532.207	99,98%	469.218	—
address1_line2	Text	532.293	53.468	10,04%	43.747	Sehr viele fehlende Werte
address1_postalcode	Text	532.293	532.099	99,96%	112.576	—
address1_county	Text	532.293	299.318	56,23%	741	—
district	Text	532.293	131.619	24,73%	770	Hoher Anteil fehlender Werte
geb_accountnumber	Text	532.293	532.293	100%	531.867	—
geb_address1_countyid	Text	532.293	299.318	56,23%	742	—
geb_countrycode	Integer	532.293	532.293	100%	165	—
geb_district	Text	532.293	131.619	24,73%	770	Hoher Anteil fehlender Werte
geb_maintypeid	Text	532.293	532.050	99,95%	648	—
geb_namethree	Text	532.293	6.911	1,30%	5.107	Extrem viele fehlende Werte
geb_nametwo	Text	532.293	128.324	24,10%	57.601	Hoher Anteil fehlender Werte
maintype	Text	532.293	532.108	99,97%	31	—
name	Text	532.293	532.292	~100%	486.014	—
statecode	Integer	532.293	532.293	100%	2	Wenig Wertevielfalt (Status, Flag?)
statuscode	Integer	532.293	532.293	100%	9	Wenig Wertevielfalt (Status/Phasen)

Zusammenfassung und Auffälligkeiten:

- Primär- und Schlüsselattribute (accountid, geb_accountnumber, name, statuscode, statecode, geb_countrycode): Immer gefüllt, sehr hohe Wertevielfalt bei IDs.
- Adresse: address1_city, address1_line1, address1_postalcode beinahe vollständig, address1_line2 hingegen bei 90% der Datensätze leer.
- Zusatzattribute wie district, geb_district, address1_county, geb_address1_countyid, geb_nametwo, geb_namethree haben niedrige Füllgrade (1-56%), werden also entweder selten benötigt oder nur bei Sonderfällen gepflegt.
- Attributwertemengen: Teilweise sehr hoch (wie bei accountid), teilweise gering (statuscode, statecode = wenige vorgegebene Werte).

Bemerkung: Das Profiling zeigt eine typische Stammdatenstruktur mit vollständigen Kernfeldern und teils sehr lückenhaften Rand- oder Zusatzfeldern. Die Datenqualität ist bei Pflichtfeldern sehr gut, bei optionalen Attributen schwach ausgeprägt. Besondere Aufmerksamkeit verdienen Felder mit extrem niedrigem Füllgrad, da hier entweder ein Optimierungspotenzial oder eine bewusste Systemlogik vorliegt.

1 Schritt abgeschlossen  ⌚ Antwortzeit: 71 Sek

Abb. 17: Ergebnis des Profiling-Prompts an den Data Agent (eigene Darstellung)

Die Ergebnisse zeigen zunächst, dass der Agent bei einfachen deskriptiven Analysen eine grundsätzlich brauchbare Unterstützung bietet. So konnte er fehlende Werte einzelner Spalten der Account-Tabelle mit plausiblen Anteilen benennen und die Verteilung der Datenmenge über Attribute in nachvollziehbarer Form zusammenfassen. Auch ein

oberflächliches Data Profiling der Account-Tabelle gelang nach einem ersten fehlerhaften Anlauf in insgesamt brauchbarer Form. Der Agent stellte dabei Datentypen, Füllgrade, Distinct-Werte und erste Auffälligkeiten strukturiert dar. Abb. 17 zeigt exemplarisch ein solches Ergebnis. Die Antwort ist zwar nicht mit einem dedizierten Profiling-Notebook gleichzusetzen, verdeutlicht aber, dass der Agent einfache Profiling-Fragestellungen auf Basis des semantischen Modells und tabellarischer Abfragen grundsätzlich angemessen bearbeiten kann. In diesem Bereich reproduziert der Agent damit Teile klassischer Profiling-Ergebnisse in natürlicher Sprache und kann als exploratives Interface für einfache DQ-Fragen dienen.

Darüber hinaus zeigt sich eine gewisse Stärke des Agenten bei strukturell naheliegenden Konsistenz- und Integritätsfragen, sofern diese in den Daten oder im semantischen Modell bereits implizit angelegt sind. So identifiziert der Agent Redundanzen zwischen verwandten Attributen, verweist auf mögliche Inkonsistenzen zwischen redundanten Feldern oder benennt unvollständige Referenzen zwischen Accounts und Kontakten. Auch bei der Generierung möglicher tabellenübergreifender Regeln oder der Benennung potenzieller Constraints formuliert er inhaltlich plausible und fachlich nachvollziehbare Vorschläge. Diese Ergebnisse deuten darauf hin, dass der Agent dort am besten funktioniert, wo sich Fragestellungen relativ direkt in tabellarische Abfragen übersetzen lassen und der Kontext bereits durch sprechende Attributnamen, Relationen und Beschreibungen gestützt wird.

Ein besonders interessanter Befund ergibt sich bei der Erkennung potenzieller Test- und Entwicklungsdaten im Produktivbestand. Hier lieferte der Agent vergleichsweise überzeugende Ergebnisse und konnte typische Muster wie Testbegriffe im Namen, Platzhalterwerte, generische Telefonnummern oder auffällige Adressangaben identifizieren. Abb. 18 zeigt ein entsprechendes Beispiel. Gleichzeitig ist die Leistungsfähigkeit des Agenten an dieser Stelle jedoch einzuordnen: Die Erkennung erfolgte nicht auf Basis eines tieferen kontextuellen Verständnisses der Geschäftsobjekte, sondern ersichtlich über heuristische Muster, die in den generierten Abfragen abgebildet wurden, etwa durch die Suche nach typischen Begriffen wie “test”, “demo” oder “dummy” in Namensfeldern. Der Agent war in diesem Fall damit durchaus nützlich, allerdings eher als flexibler heuristischer Suchmechanismus denn als System mit belastbarem semantischem Reasoning.

Gerade für die explorative Untersuchung solcher Muster kann dies dennoch einen praktischen Mehrwert bieten.

Es gibt in der Account-Tabelle deutliche Anzeichen für die Existenz von Entwicklungs- oder Testdaten im Produktivbetrieb. Folgende Auffälligkeiten deuten darauf hin:

Typische Merkmale von Entwicklungs- oder Testdaten:

- Namen mit Test-/Dummy-Begriffen oder Sonderzeichen**
 - Präfixe wie #, TEST, DEMO, Dummy in Namen (z.B. "#prefab test Account PL", "#Piping test-New Employee Testing 1").
 - Namen, die auf generische oder fiktive Unternehmen hinweisen ("Default CRM - MÅ IKKE BENYTTES").
- Unrealistische oder offensichtliche Platzhalterwerte**
 - Straßen wie „Plot no 245“ oder sehr generisch („ul. Zachodnia 12“).
 - Telefonnummern wie „0“, „0000“ oder +48 22222222.
 - Städte, die nicht existieren, oder generische Adressen („Teststadt“, „Ho Chi Minh“ mit westlichem Adressformat).
 - E-Mail-Adressen, die mehrfach für verschiedene Accounts genutzt werden oder ersichtlich generisch/privat sind („anirudhrao24@gmail.com mailto:anirudhrao24@gmail.com“).
- Leere, gleichförmige oder unplausible Kombinationsfelder**
 - Viele leere Felder wie Adresse, Kontaktinfo, PLZ.
 - Firmierung kombiniert mit Namen von Personen („Mr. Manoj Agarwal“ als Accountname).
- Prägnante Serienmuster oder Platzhalter-Postleitzahlen**
 - „99-400“, „70000“, „500016“ u.a., die in der Realität selten oder regional auffällig serienmäßig auftreten.
 - Telefonnummern und Websites ausschließlich mit Default- oder Testwerten.

Wie häufig kommen diese Auffälligkeiten vor?

- Die Zahl solcher Datensätze ist nicht auf wenige Einzelfälle begrenzt. Die Beispiele enthalten zahlreiche Accounts mit oben genannten Auffälligkeiten.
- Namen mit Präfix „#“ oder Begriffen wie „test“, „Default CRM“, „prefab“ oder „Employee Testing“ treten vielfach auf.
- Dummy-Telefonnummern („0000“) und Platzhalter-Mails finden sich wiederholt, teils mit hohen Phone-Count-Werten, ebenfalls ein Indiz für künstliche Datenmengen.

Konkretes Beispiel für einen unechten Account-Datensatz

Account Name	Straße	Stadt	PLZ	Telefon	Auffälligkeit
#prefab test Account PL	Podrzeczna	Łowicz	99-400	+48 22222222	Sowohl Name als auch Telefonnummer und PLZ auffällig
#Piping test-New Employee Testing 1	11 Doan Van Bo District 4 Ho Chi Minh	Ho Chi Minh	70000	+84 90804685	Testbegriff, PLZ und Telefonnummer
Default CRM - MÅ IKKE BENYTTES	-	-	-	0000	Name, Adresse, Telefon — alles Platzhalterwerte

Zusammenfassung

- Entwicklungs-/Testdaten sind im produktiven Datenbestand mehrfach nachweisbar.
- Typische Indizien: Testpräfixe, Dummy-Namen, generische/Platzhalter-Adressdaten und Kontaktdaten.
- Das Vorkommen ist systematisch und betrifft mehrere Dutzend bis hunderte Fälle, teils mit klaren Mustern und hohen Wiederholungsraten.

Empfehlung:
Ein systematisches Data-Cleansing sollte etabliert werden, um solche Testdaten frühzeitig zu erkennen, zu kennzeichnen und vom echten produktiven Datenbestand zu trennen bzw. zu entfernen.

Abb. 18: Ergebnis des Prompts zur Identifikation von Testdaten an den Data Agent (eigene Darstellung)

Gleichzeitig zeigen die Antworten auch deutliche Grenzen. Eine zentrale Einschränkung besteht darin, dass der Agent Semantik und Kontext nur eingeschränkt aus der Datenstruktur selbst inferieren kann. Dies wird besonders dort sichtbar, wo fachliche Interpretation oder tieferes kontextuelles Verständnis erforderlich wäre. So werden z. B. numerische Ländercodes oder andere kodierte Attribute zwar als potenziell auffällig beschrieben, ihre tatsächliche Bedeutung kann ohne zusätzliche Mapping-Logik jedoch nicht belastbar eingeordnet werden. Ähnlich verhält es sich bei funktionalen Abhängigkeiten oder Duplikaten. Die vom Agent formulierten Aussagen sind häufig plausibel, bleiben aber auf einem generischen Niveau und orientieren sich stark an typischen CRM-Mustern, statt belastbar aus dem konkreten Datenbestand abgeleitet zu sein.

Besonders deutlich wird diese Limitation bei komplexeren oder mehrstufigen Fragestellungen, die stärkeres Reasoning erfordern würden. Der Data Agent scheint Prompts

überwiegend direkt in Abfragen zu übersetzen und die Antworten aus deren Ergebnissen zu synthetisieren, anstatt komplexere Hypothesen explizit schrittweise zu prüfen. Dadurch entstehen zwar oft schnell formulierte Antworten, deren analytische Tiefe jedoch begrenzt bleibt. Dies zeigt sich auch daran, dass einige Aufgabenstellungen mehrere Anläufe benötigten, bis ein sinnvoll interpretierbares Ergebnis vorlag. So führte etwa die Berechnung eines Similarity Scores zunächst zu wenig aussagekräftigen Resultaten, da triviale Felder mit extrem geringer Varianz oder binäre Verwaltungsattribute dominieren können und damit vor allem Werte nahe 0 oder 1 erzeugen. Erst durch wiederholte Präzisierung der Fragestellung entstand ein Ergebnis, das für die Dublettenbewertung tatsächlich interpretierbar war. Dies verdeutlicht, dass die Qualität der Antworten stark von Promptformulierung, Auswahl geeigneter Attribute und impliziter Vorstrukturierung abhängt.

Hinzu kommen technische Limitationen. Mehrere Anfragen benötigten vergleichsweise lange Laufzeiten, einzelne Antworten wurden laut Agent aufgrund technischer Fehler oder Timeouts abgebrochen. Dies deutet darauf hin, dass der Einsatz des Agenten gegenüber größeren Datenmengen, komplexeren Query-Generierungen oder ungünstig formulierten Prompts empfindlich ist. Ein weiterer Einflussfaktor liegt im noch vergleichsweise geringen Reifegrad des Data Agents, der im Verlauf der Arbeit von einer Preview-Phase in die allgemeine Verfügbarkeit (General Availability) überführt wurde. Für einen explorativen Chat-basierten Zugang ist dies grundsätzlich tolerierbar, für eine belastbare und insbesondere reproduzierbare DQ-Prüfung stellt es jedoch eine relevante Einschränkung dar. Diese Beobachtung bestätigt zugleich die architektonisch getroffene Einordnung des Agenten als interaktive Ergänzung und nicht als automatisierbaren Bestandteil einer produktiven DQ-Pipeline.

Ein gemischtes Bild ergibt sich auch bei den explizit LLM-bezogenen Aufgaben. Die Generierung einfacher DQ-Regeln, tabellenübergreifender Prüfvorschläge oder generischer funktionaler Abhängigkeiten gelingt schnell und liefert zum Teil brauchbare Ideen für weiterführende Prüfungen. Gleichzeitig bleiben diese Vorschläge häufig relativ naheliegend und sind nur begrenzt auf die Besonderheiten des konkreten Datenmodells zugeschnitten. Bei stärker datengetriebenen Aufgaben, wie bei der Berechnung von Similarity Scores oder der Identifikation potenziell widersprüchlicher Dubletten in den Daten, entstehen zwar plausible Ergebnisse, jedoch ist die Nachvollziehbarkeit der internen

Berechnungslogik begrenzt. Dadurch eignet sich der Agent eher zur Hypothesengenerierung und zur explorativen Unterstützung als zur belastbaren Ableitung formaler DQ-Regeln.

Insgesamt zeigt die Evaluation, dass der Data Agent im Kontext der DQ-Bewertung vor allem als exploratives Assistenzwerkzeug einzuordnen ist. Seine Stärken liegen in der natürlichsprachlichen Aufbereitung einfacher Profiling- und Strukturfragen sowie in der heuristischen Identifikation auffälliger Muster, etwa bei potenziellen Testdaten. Seine Schwächen zeigen sich dagegen bei tieferem semantischem Verständnis, belastbarer Inferenz aus komplexen Datenstrukturen, technischer Stabilität und reproduzierbarem Reasoning. Für eine systematische, verlässliche und automatisierbare DQ-Prüfung ist der Agent daher allein nicht ausreichend. Sein größter Nutzen liegt vielmehr darin, klassische DQ-Verfahren ergänzend zu unterstützen, etwa indem er explorative Analysen erleichtert, erste Hypothesen formuliert oder auffällige Muster zur weiteren Prüfung sichtbar macht.

LLM-basierte Experimente

Ergänzend zum Data Agent wurden API-basierte LLM eingesetzt, um ausgewählte Datensätze des CRM-Systems explorativ auf semantische Auffälligkeiten und potenzielle DQ-Probleme zu untersuchen. Aufgrund technischer Einschränkungen erfolgte die Analyse nicht auf vollständigen Tabellen, sondern auf einzelnen Datensätzen beziehungsweise kleinen Stichproben, die strukturiert (z. B. als JSON-Objekte) an das Modell übergeben wurden. Ziel war es insbesondere zu evaluieren, inwieweit LLMs die beim Data Agent identifizierten Schwächen im Bereich semantischer Interpretation, kontextueller Bewertung und Regelableitung adressieren können. Die Prompts und Antworten des Modells sind im Anhang Kapitel H beigefügt.

Die Ergebnisse zeigen zunächst, dass LLMs semantische Inkonsistenzen und kontextuelle Auffälligkeiten auf Datensatzebene deutlich differenzierter erkennen als der Data Agent. So wurden innerhalb einzelner Datensätze Widersprüche zwischen Attributen identifiziert, etwa inkonsistente Kombinationen von Aktivitätskennzahlen (z. B. vorhandene Besuche bei gleichzeitig fehlenden Kontakten), unplausible Aufteilungen von Attributen (z. B. fragmentierte Namensfelder) oder semantisch unpassende Werte wie Statusinformationen im Namensfeld. Strukturelle Auffälligkeiten wie die Speicherung

mehrerer Werte in einem Attribut (z. B. semikolon-separierte Business-Typen) wurden erkannt und als potenzielle Modellierungsprobleme eingeordnet.

Besonders hervorzuheben ist die Fähigkeit des Modells, nicht nur einzelne Auffälligkeiten zu benennen, sondern diese auch zu begründen und in einen fachlichen Kontext einzuordnen. Dies zeigt sich beispielsweise bei der Bewertung einzelner Datensätze, bei der das LLM mehrere Indikatoren kombiniert, etwa Adresskonsistenz, Aktivitätskennzahlen und Attributstruktur, und daraus eine abgestufte Einschätzung der DQ ableitet. Im Gegensatz zum Data Agent, der primär query-basierte Ergebnisse liefert, entsteht hier eine stärker interpretative und argumentativ gestützte Analyse.

Ein weiterer Mehrwert zeigt sich in der Bewertung potenzieller Testdaten. Während der Data Agent Testdaten vor allem über explizite Muster wie "test" im Namen identifiziert, erweitert das LLM diese Perspektive um zusätzliche Indikatoren. In der Analyse wurden neben solchen offensichtlichen Merkmalen auch indirekte Hinweise berücksichtigt, etwa fehlende Aktivität (z. B. null Kontaktkennzahlen), ungewöhnliche Kombinationen von Attributwerten oder inkonsistente Statusausprägungen. Auffällig ist dabei, dass das Modell nicht nur eine Klassifikation vornimmt, sondern explizit Argumente für und gegen die Einordnung als Testdatensatz formuliert. Diese abwägende Betrachtung stellt einen Unterschied zum Data Agent dar, dessen Vorgehen stärker heuristisch ist.

Auch im Bereich der Dubletten- und Ähnlichkeitsanalyse zeigt sich eine Stärke der LLMs. Während klassische Verfahren und der Data Agent primär auf strukturierte Metriken angewiesen sind, kann das Modell potenzielle Zusammenhänge zwischen Datensätzen anhand mehrerer Attribute interpretieren. So wurden beispielsweise Datensätze mit identischen Adressen, Telefonnummern und ähnlichen Namensbestandteilen als potenziell gleiche Entität oder zumindest eng verbundene Organisationen identifiziert und entsprechend eingeordnet. Diese Bewertung erfolgt dabei nicht binär, sondern als abgestufte Wahrscheinlichkeit, was insbesondere für explorative Analysen hilfreich ist.

Darüber hinaus konnten aus den bereitgestellten Daten kontextbezogene DQ-Regeln abgeleitet werden, die über triviale Prüfungen hinausgehen. Beispiele hierfür sind Regeln zur Konsistenz zwischen Länderkennzeichen und Telefonnummern, zur Plausibilität von Zeitstempeln (z. B. keine zukünftigen Änderungsdaten) oder zur logischen Beziehung

zwischen Aktivitätskennzahlen. Auch tabellenübergreifende Regeln, etwa zur Konsistenz zwischen Accounts und zugehörigen Kontakten oder Aktivitäten, wurden formuliert. Diese Ergebnisse zeigen, dass LLMs insbesondere zur Hypothesengenerierung und explorativen Regelableitung geeignet sind.

Gleichzeitig treten jedoch klare Grenzen zutage. Eine zentrale Einschränkung besteht darin, dass die Modelle ihre Einschätzungen nicht ausschließlich aus den bereitgestellten Daten ableiten, sondern diese mit allgemeinem Weltwissen oder typischen Annahmen über CRM-Systeme anreichern. Dies wird bei der Bewertung von Länder- oder Postleitzahlencodes deutlich, die wiederholt mit externen Standards (z. B. ISO-Codes) verglichen werden, obwohl es sich im konkreten Fall um interne Codierungen handelt. Ähnliche Effekte zeigen sich bei der Bewertung von Formaten oder Aktivitätskennzahlen, die unter Annahmen interpretiert werden, die nicht im Datensatz selbst begründet sind.

Auch die Ableitung funktionaler Abhängigkeiten ist kritisch zu betrachten. Die vom Modell formulierten Abhängigkeiten entsprechen häufig eher plausiblen fachlichen Annahmen als empirisch nachgewiesenen funktionalen Abhängigkeiten im datenbanktheoretischen Sinn. Damit eignen sich diese Ergebnisse eher als Ausgangspunkt für weiterführende Analysen, jedoch nicht als direkt übernehmbare Constraints.

Im direkten Vergleich zum Data Agent zeigt sich somit ein komplementäres Bild. Der Data Agent liefert reproduzierbare, datenbanknahe Ergebnisse und eignet sich insbesondere für strukturierte Analysen und aggregierte Auswertungen, bleibt jedoch in seiner Fähigkeit zur semantischen Interpretation begrenzt. Die LLMs hingegen ermöglichen eine tiefere inhaltliche Analyse einzelner Datensätze, erkennen komplexe Zusammenhänge und unterstützen die Ableitung kontextbezogener Regeln, sind jedoch in ihrer Skalierbarkeit, Reproduzierbarkeit und Verlässlichkeit eingeschränkt.

Die Ergebnisse zeigen, dass LLMs insbesondere für explorative Analysen, die Identifikation semantischer Auffälligkeiten und die Ableitung potenzieller Prüfregeln einen Mehrwert bieten. Für eine systematische und automatisierte Sicherstellung der DQ sind sie im vorliegenden Setup jedoch nicht geeignet. Ihr Einsatz ist daher vor allem als ergänzendes Werkzeug zu verstehen, das die bisherigen Ansätze erweitert, jedoch nicht ersetzt.

6. Schlussbetrachtung

6.1. Zusammenfassung der Ergebnisse

Im Rahmen dieser Arbeit wurde ein ganzheitlicher Ansatz zur Sicherstellung von DQ in modernen Datenplattformen am Beispiel von Microsoft Fabric entwickelt, prototypisch implementiert und evaluiert. Ziel war es, sowohl theoretische Grundlagen als auch praktische Verfahren systematisch zu verbinden und deren Eignung im Kontext eines realitätsnahen Use Cases zu untersuchen.

Die Ergebnisse zeigen, dass DQ in modernen Lakehouse-Architekturen nicht durch einzelne isolierte Maßnahmen sichergestellt werden kann, sondern eine mehrschichtige, architektonisch eingebettete Herangehensweise erfordert. Insbesondere die Kombination aus deskriptivem Profiling, regelbasierter Validierung, statistischer Auffälligkeitserkennung sowie ergänzenden datengetriebenen und KI-basierten Verfahren ermöglicht eine differenzierte Bewertung entlang verschiedener DQ-Dimensionen.

Die entwickelte Erweiterung der Medallion-Architektur um eine dedizierte Data-Quality-Zone erweist sich dabei als technisch stimmige und praktikable Lösung, um DQ-Verfahren systematisch entlang des Datenlebenszyklus zu verankern. Ihre Stärken liegen insbesondere in der Integration in bestehende Microsoft Fabric-Komponenten, der Nutzung verteilter Verarbeitung sowie der klaren Trennung zwischen generischen und spezifischen Qualitätsprüfungen. Gleichzeitig zeigen sich Einschränkungen insbesondere bei der Unterstützung kontinuierlicher Verarbeitungsmodi sowie bei der Einbindung semantisch anspruchsvoller KI-basierter Verfahren.

Ein zentrales Ergebnis der Evaluation ist, dass klassische Verfahren, insbesondere regelbasierte Validierung und statistisches Profiling, im betrachteten Kontext die höchste Robustheit, Interpretierbarkeit und praktische Einsetzbarkeit aufweisen. Demgegenüber liefern datengetriebene und agentische Verfahren zwar wertvolle zusätzliche Erkenntnisse, sind jedoch stark abhängig von Parametrisierung, Feature-Auswahl und Kontextwissen und daher aktuell primär als explorative Ergänzung einzuordnen.

Darüber hinaus zeigt die Arbeit, dass DQ nicht ausschließlich als technisches Problem verstanden werden kann. Vielmehr erfordert insbesondere die Interpretation von Auffälligkeiten sowie die Definition geeigneter Prüfregele ein domänenspezifisches und

zunehmend organisatorisches Kontextverständnis, das nicht vollständig automatisiert abgebildet werden kann.

F1: Welche Datenqualitätsdimensionen und -Probleme werden in der Literatur beschrieben und wie zeigen sie sich im Kontext von Microsoft Fabric?

Die Analyse der Literatur zeigt, dass DQ typischerweise entlang mehrerer Dimensionen wie Vollständigkeit, Konsistenz, Korrektheit, Relevanz sowie Interpretierbarkeit betrachtet wird. Diese abstrakten Konzepte lassen sich im Kontext von Microsoft Fabric konkret in Form struktureller und dateninhärenter Probleme beobachten. Im untersuchten Use Case manifestieren sich insbesondere Herausforderungen in Form hoher Null-Raten aufgrund der Wide-Table-Struktur, heterogener Pflegezustände von Attributen, vereinzelter referenzieller Inkonsistenzen sowie potenzieller semantischer Unklarheiten. Gleichzeitig zeigt sich, dass zentrale strukturelle Integritätsanforderungen, wie eindeutige Primärschlüssel oder stabile Beziehungen zwischen Entitäten, weitgehend erfüllt sind. Die Ergebnisse verdeutlichen, dass sich die in der Literatur beschriebenen DQ-Probleme grundsätzlich auf moderne Datenplattformen übertragen lassen, ihre konkrete Ausprägung jedoch stark durch das jeweilige Datenmodell und Nutzungskontext geprägt ist.

F2: Welche klassischen und KI-gestützten Verfahren zur Verbesserung der Datenqualität sind in der Literatur etabliert und welche sind davon für die Anwendung in Microsoft Fabric geeignet?

Die systematische Literaturrecherche zeigt ein breites Spektrum etablierter Verfahren, das von regelbasierter Validierung und statistischem Profiling über klassische Machine-Learning-Ansätze bis hin zu neueren KI- und agentischen Verfahren reicht. Durch die Strukturierung in Methodencluster sowie die Anwendung eines theoriegeleiteten Bewertungsrasters konnte eine differenzierte Einordnung dieser Verfahren hinsichtlich ihrer fachlichen Wirksamkeit und technischen Eignung vorgenommen werden. Für den Einsatz in Microsoft Fabric erweisen sich insbesondere solche Verfahren als geeignet, die sich in verteilten, tabellarischen Datenverarbeitungskontexten umsetzen lassen und eine hohe Integration in bestehende Pipeline-Strukturen ermöglichen. Dazu zählen vor allem regelbasierte Validierungen sowie statistische Profiling- und Anomalieerkennungsverfahren. Demgegenüber zeigen KI-basierte und agentische Ansätze zwar Potenziale, insbesondere im Hinblick auf explorative Analysen, Regelgenerierung und semantische

Unterstützung, sind jedoch aktuell nur eingeschränkt für eine robuste, skalierbare und automatisierbare Integration in produktive Datenpipelines geeignet.

F3: Wie können ausgewählte Verfahren in einem prototypischen Use Case in Microsoft Fabric implementiert und hinsichtlich ihrer praktischen Eignung evaluiert werden?

Die prototypische Implementierung erfolgte durch die Erweiterung einer bestehenden Medallion-Architektur um dedizierte DQ-Komponenten in Bronze-, Silver- und Gold-Layer. Dabei wurden repräsentative Verfahren aus unterschiedlichen Methodenclustern ausgewählt und entlang des Datenflusses integriert. Die Evaluation basiert auf einer explorativen Baseline-Erhebung sowie der anschließenden Anwendung der implementierten Verfahren auf reale Datenbestände. Dabei zeigt sich, dass die einzelnen Komponenten unterschiedliche Beiträge zur DQ-Bewertung leisten. Während das generische Profiling primär Transparenz über den Datenzustand schafft, ermöglichen regelbasierte Verfahren die gezielte Identifikation konkreter Qualitätsprobleme. Statistische Methoden erweitern die Erkennung um bislang nicht definierte Auffälligkeiten, erfordern jedoch eine sorgfältige Interpretation. Datengetriebene und KI-basierte Verfahren, wie IF oder der Data Agent, konnten ergänzende Erkenntnisse liefern, zeigten jedoch deutliche Grenzen hinsichtlich Interpretierbarkeit, Stabilität und fachlicher Aussagekraft. Insgesamt wird deutlich, dass ein sinnvoller Vergleich der Verfahren nur im Zusammenspiel ihrer jeweiligen Stärken und Limitationen erfolgen kann und stark vom jeweiligen Anwendungskontext abhängt.

F4: Welche Handlungsempfehlungen zur Sicherstellung der Datenqualität lassen sich aus der Evaluation der prototypischen Umsetzung ableiten und inwieweit sind die Ergebnisse auf andere Datenquellen und Anwendungsszenarien übertragbar?

Aus der Evaluation lassen sich mehrere zentrale Handlungsempfehlungen ableiten. Erstens sollte DQ entlang des gesamten Datenlebenszyklus systematisch verankert werden, idealerweise in Form einer mehrschichtigen Architektur, die unterschiedliche Verfahren kombiniert. Zweitens bilden regelbasierte und statistische Verfahren eine robuste Grundlage für die operative DQ-Prüfung, während datengetriebene und KI-basierte Ansätze insbesondere für explorative Analysen und die Generierung neuer Hypothesen geeignet sind. Drittens zeigt sich die Notwendigkeit einer engen Verzahnung von technischen Verfahren mit fachlichem Kontextwissen, insbesondere bei der Definition

von Regeln und der Interpretation von Auffälligkeiten. Viertens sollten DQ-Prozesse so gestaltet werden, dass sie sowohl skalierbar als auch nachvollziehbar und reproduzierbar sind. Hinsichtlich der Generalisierbarkeit lässt sich festhalten, dass insbesondere die entwickelte Architektur sowie die methodische Vorgehensweise zur Auswahl und Bewertung von Verfahren auf andere Datenquellen und Anwendungsszenarien übertragbar sind. Einschränkungen bestehen hingegen bei der direkten Übertragbarkeit konkreter Regeln, Schwellenwerte und datengetriebener Modelle, da diese stark von der jeweiligen Datenstruktur und dem fachlichen Kontext abhängen.

6.2. Kritische Würdigung und Reflexion

Die vorliegende Arbeit verfolgt das Ziel, Verfahren zur Sicherstellung von DQ systematisch zu analysieren, in einer modernen Datenplattform zu verankern und im Rahmen eines prototypischen Use Cases zu evaluieren. Die erzielten Ergebnisse liefern dabei wertvolle Erkenntnisse hinsichtlich der praktischen Einsetzbarkeit verschiedener Ansätze, sind jedoch zugleich im Kontext methodischer, technischer und konzeptioneller Grenzen zu betrachten.

Methodische Grenzen

Eine zentrale methodische Einschränkung der Arbeit liegt in der fehlenden Verfügbarkeit einer belastbaren Ground Truth. Insbesondere für DQ-Dimensionen wie Korrektheit und Genauigkeit existieren im betrachteten Use Case keine extern validierten Referenzdaten oder vollständig formalisierten Geschäftsregeln. Eine Quantifizierte Bewertung ist daher erschwert möglich. Dadurch ist eine objektive Bewertung der tatsächlichen DQ nur eingeschränkt möglich. Die Evaluation basiert stattdessen auf Indikatoren, Plausibilitätsannahmen und regelbasierten Prüfungen, die potenzielle Qualitätsprobleme aufzeigen, jedoch keine abschließende Beurteilung erlauben.

Darüber hinaus ist das entwickelte Bewertungsraster trotz seiner theoriegeleiteten Herleitung mit Subjektivität behaftet. Die Bewertung der Methodencluster erfolgt auf Basis analytischer Einschätzungen hinsichtlich ihrer Wirksamkeit, Umsetzbarkeit und architektonischen Eignung im Kontext von Microsoft Fabric. Auch wenn diese Bewertung systematisch durchgeführt wurde, bleibt sie in Teilen interpretationsabhängig und könnte in anderen Anwendungskontexten zu abweichenden Ergebnissen führen.

Ein weiterer methodischer Aspekt betrifft das gewählte Evaluationsdesign. Die Untersuchung verfolgt einen explorativen Ansatz, bei dem verschiedene Verfahren exemplarisch implementiert und hinsichtlich ihres Beitrags zur DQ-Bewertung analysiert werden. Ein strikter, quantitativer Vergleich im Sinne eines kontrollierten Experiments ist dabei nur eingeschränkt möglich, da die Verfahren unterschiedliche Zielsetzungen verfolgen und auf verschiedenen Abstraktionsebenen wirken. Die Ergebnisse sind daher primär qualitativ zu interpretieren und dienen der Einordnung von Stärken und Schwächen, nicht der belastbaren Bestimmung eines einzelnen priorisierten Verfahrens.

Technische Grenzen

Auf technischer Ebene zeigen sich insbesondere Einschränkungen im Zusammenhang mit der Skalierbarkeit und Robustheit einzelner Verfahren. Während klassische Profiling- und regelbasierte Ansätze gut in die verteilte Verarbeitung von Microsoft Fabric integrierbar sind, gilt dies nur eingeschränkt für datengetriebene und KI-basierte Verfahren.

Die Experimente mit dem IF-Modell verdeutlichen, dass dessen Ergebnisse stark von der Feature-Auswahl und Parametrisierung abhängen. Ohne eine klare fachliche Definition von Anomalien identifiziert das Modell primär statistische Ausreißer, deren Relevanz für die DQ nicht unmittelbar gegeben ist. Dies limitiert die praktische Nutzbarkeit als eigenständiges Verfahren zur Qualitätsbewertung.

Noch deutlicher treten technische Einschränkungen beim Einsatz des Data Agents zutage. Neben begrenzter Stabilität in Form von Timeouts oder abgebrochenen Abfragen zeigt sich, dass der Agent komplexere Fragestellungen nur eingeschränkt zuverlässig verarbeiten kann. Insbesondere bei umfangreichen Datenmengen oder mehrstufigen analytischen Aufgaben stößt der Ansatz an praktische Grenzen. Diese Beobachtungen unterstreichen, dass agentische und LLM-basierte Verfahren derzeit nicht in gleicher Weise skalierbar und reproduzierbar sind wie klassische datenverarbeitende Ansätze.

Ein weiterer technischer Aspekt betrifft die Ausrichtung der Architektur auf Batch-Verarbeitung. Streaming- oder Near-Realtime-Szenarien wurden im Rahmen des Use Cases nicht umgesetzt, wodurch die Übertragbarkeit auf zeitkritische Datenflüsse eingeschränkt ist. Für eine produktive Nutzung in solchen Szenarien wären zusätzliche architektonische Erweiterungen erforderlich.

Konzeptuelle Grenzen

Neben methodischen und technischen Einschränkungen ergeben sich auch konzeptionelle Grenzen, die grundlegend für das Verständnis von DQ sind. DQ ist in hohem Maße kontextabhängig und kann nicht vollständig unabhängig vom fachlichen Nutzungskontext bewertet werden. Viele der identifizierten Auffälligkeiten lassen sich erst durch domänenspezifisches Wissen sinnvoll einordnen.

Dies zeigt sich insbesondere bei statistischen Verfahren, deren Ergebnisse ohne fachliche Interpretation leicht fehlgedeutet werden können. Auffällige Werte oder Ausreißer sind nicht zwangsläufig als Qualitätsprobleme zu interpretieren, sondern können auch legitime Ausprägungen des Datenbestands darstellen.

Auch regelbasierte Verfahren unterliegen konzeptionellen Grenzen. Die Definition geeigneter Prüfregele erfordert ein tiefes Verständnis der zugrundeliegenden Geschäftslogik. Gleichzeitig besteht ein inhärenter Zielkonflikt zwischen zu restriktiven und zu großzügigen Regeln, der sich nicht vollständig automatisiert auflösen lässt.

Im Kontext KI-basierter Verfahren wird zudem deutlich, dass ein belastbares semantisches Verständnis der Daten nicht allein aus strukturellen Informationen abgeleitet werden kann. Der Data Agent zeigt zwar Potenziale bei der heuristischen Mustererkennung und der Generierung von Hypothesen, bleibt jedoch in seiner Fähigkeit zur fundierten semantischen Interpretation und zur Ableitung belastbarer Schlussfolgerungen begrenzt.

6.3. Ausblick und Handlungsempfehlungen

Wissenschaftlicher Beitrag:

Die vorliegende Arbeit leistet einen Beitrag zur systematischen Einordnung und praktischen Anwendbarkeit von DQ-Verfahren im Kontext moderner Datenplattformen. Insbesondere wird deutlich, dass bestehende Forschungsarbeiten zwar eine Vielzahl einzelner Methoden beschreiben, jedoch häufig eine integrierte Betrachtung fehlt, die sowohl methodische, architektonische als auch praktische Aspekte zusammenführt.

Ein zentraler wissenschaftlicher Beitrag liegt daher in der Entwicklung einer übertragbaren methodischen Vorgehensweise, die von der strukturierten Literaturrecherche über die Kategorisierung von Verfahren bis hin zur Bewertung der Verfahren anhand

eines theoriegeleiteten Rasters und der prototypischen Implementierung reicht. Diese Vorgehensweise ermöglicht es, DQ-Verfahren nicht isoliert zu betrachten, sondern systematisch im Kontext konkreter Datenplattformen zu bewerten und auszuwählen.

Darüber hinaus zeigt die Arbeit, dass insbesondere im Bereich KI- und agentenbasierter Ansätze weiterer Forschungsbedarf besteht. Während erste Ansätze vielversprechende Ergebnisse liefern, fehlen bislang belastbare Konzepte für deren stabile, skalierbare und reproduzierbare Integration in Datenverarbeitungsprozesse. Zukünftige Forschungsarbeiten könnten hier ansetzen und untersuchen, wie solche Verfahren stärker in automatisierte Datenpipelines eingebunden werden können, etwa durch verbesserte Prompt-Strategien, hybride Ansätze oder die Kombination mit regelbasierten Systemen.

Ein weiterer Forschungsbedarf ergibt sich hinsichtlich der Evaluation von DQ selbst. Die fehlende Ground Truth erschwert eine objektive Bewertung der Wirksamkeit von Verfahren erheblich. Zukünftige Arbeiten könnten sich daher mit der Entwicklung standardisierter Benchmark-Datensätze oder Evaluationsmetriken befassen, die eine besser vergleichbare und reproduzierbare Bewertung ermöglichen.

Auch die Übertragbarkeit der Ergebnisse stellt ein relevantes Forschungsfeld dar. Während die entwickelte Architektur und Methodik grundsätzlich auf andere Kontexte übertragbar sind, bleibt zu untersuchen, in welchem Umfang dies für unterschiedliche Domänen, Datenmodelle oder Plattformen gilt. Insbesondere der Einfluss von Datenstruktur, Domänenwissen und organisatorischen Rahmenbedingungen auf die Wirksamkeit von DQ-Verfahren bietet hier Potenzial für weiterführende Untersuchungen.

Praktische Implikationen:

Für die praktische Umsetzung von DQ-Maßnahmen in modernen Datenplattformen lassen sich aus den Ergebnissen mehrere zentrale Handlungsempfehlungen ableiten. Zunächst zeigt sich, dass DQ frühzeitig und entlang des gesamten Datenlebenszyklus berücksichtigt werden sollte. Eine rein nachgelagerte Prüfung im Sinne eines abschließenden Quality Gates ist nicht ausreichend. Stattdessen empfiehlt sich eine mehrschichtige Integration von DQ-Verfahren, wie sie im Rahmen der Arbeit durch die Einbettung in die Medallion-Architektur realisiert wurde.

Darüber hinaus sollten Organisationen auf eine Kombination unterschiedlicher Verfahren setzen. Regelbasierte Validierungen und statistisches Profiling bilden dabei eine robuste Grundlage für die systematische Identifikation von DQ-Problemen. Ergänzend können datengetriebene und KI-basierte Verfahren eingesetzt werden, um zusätzliche Auffälligkeiten zu identifizieren oder explorative Analysen zu unterstützen. Diese sollten jedoch bewusst nicht als alleinige Lösung betrachtet werden.

Ein weiterer zentraler Aspekt ist die Verankerung von fachlichem Kontextwissen in den DQ-Prozessen. Die Definition geeigneter Regeln sowie die Interpretation von Auffälligkeiten erfordern ein tiefes Verständnis der zugrundeliegenden Geschäftsprozesse. Entsprechend sollten klare Verantwortlichkeiten, etwa in Form von Data Ownership oder Data Stewardship, etabliert werden. Zudem ist auf eine Balance zwischen Strenge und Flexibilität bei der Definition von Qualitätsregeln zu achten. Zu restriktive Regeln können valide Daten ausschließen, während zu großzügige Definitionen problematische Datensätze nicht erfassen. Eine kontinuierliche Überprüfung und Anpassung der Regeln ist daher essenziell.

Aus technischer Sicht empfiehlt sich die Nutzung plattformnaher, skalierbarer Lösungen, die sich nahtlos in bestehende Datenverarbeitungsprozesse integrieren lassen. Die Speicherung von Profiling-Ergebnissen, Regelverstößen und Quarantäne-Daten in zentralen, nachvollziehbaren Strukturen unterstützt dabei die Transparenz und Nachvollziehbarkeit der DQ-Bewertung.

Schließlich sollte der Einsatz von KI- und agentenbasierten Verfahren bewusst als ergänzendes Werkzeug verstanden werden. Ihr größter Nutzen liegt aktuell in der Unterstützung explorativer Analysen, der Generierung von Hypothesen sowie der Identifikation potenzieller Muster. Für eine vollständig automatisierte und verlässliche DQ-Prüfung sind sie derzeit jedoch noch nicht ausreichend ausgereift.

DQ erweist sich nicht als isoliertes technisches Problem, sondern als Zusammenspiel aus Architektur, Methodik und organisatorischem Kontext. Die Arbeit zeigt, dass insbesondere in modernen Datenplattformen wie Microsoft Fabric eine systematische und mehrschichtige Herangehensweise erforderlich ist, um nachhaltige Verbesserungen zu erzielen.

Literaturverzeichnis

- [1] T. T. Pham Thi and M. Helfert, "Discovering dynamic integrity rules with a rules-based tool for data quality analyzing," in *Proceedings of the 11th International Conference on Computer Systems and Technologies and Workshop for PhD Students in Computing on International Conference on Computer Systems and Technologies*, 2010, pp. 89–94.
- [2] M. Souibgui, F. Atigui, S. Zammali, S. Cherfi, and S. B. Yahia, "Data quality in ETL process: A preliminary study," *Procedia Computer Science*, vol. 159, pp. 676–687, 2019, doi: 10.1016/j.procs.2019.09.223.
- [3] P. S. S. Koneru, "The evolution of data integration: AI-driven ETL and modern data lakes," *World Journal of Advanced Engineering Technology and Sciences*, vol. 15, no. 1, pp. 788–794, 2025.
- [4] International Organization for Standardization, "ISO/IEC 25012:2024 - Software engineering – Software product Quality Requirements and Evaluation (SQuaRE) – Data quality model." [Online]. Available: <https://www.iso.org/standard/35736.html>
- [5] J. Rodic and M. Baranovic, "Generating data quality rules and integration into ETL process," in *Proceedings of the ACM twelfth international workshop on Data warehousing and OLAP*, 2009, pp. 65–72.
- [6] H. Gadde, "AI-Enhanced Data Warehousing: Optimizing ETL Processes for Real-Time Analytics," *Revista de Inteligencia Artificial en Medicina*, vol. 11, no. 1, pp. 300–327, 2020.
- [7] D. Eswararaj, V. Kollati, L. R. Koppada, R. S. Bodala, and A. B. Nellipudi, "A Hybrid Framework for Scalable Data Quality: Comparing PySpark and AI-Powered Validation in Microsoft Fabric," *IEEE Access*, vol. 13, pp. 205854–205864, 2025, doi: 10.1109/ACCESS.2025.3638723.
- [8] Microsoft, "Overview of Copilot in Fabric - Microsoft Fabric." Accessed: Nov. 13, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-fabric-overview>
- [9] W. H. Inmon, *Building the Data Warehouse Fourth Edition*, 4th edition. Indianapolis, Indiana: John Wiley & Sons, 2005.
- [10] A. Bauer and H. Günzel, *Data-Warehouse-Systeme: Architektur, Entwicklung, Anwendung*. dpunkt. verlag, 2013.

- [11] R. Kimball and J. Caserta, *The data warehouse ETL toolkit: practical techniques for extracting, cleaning, conforming, and delivering data*. in Safari Books Online. Indianapolis, IN: Wiley, 2004.
- [12] P. Ponniah, *Data Warehousing Fundamentals for IT Professionals, Second Edition*, 2. ed. John Wiley & Sons, 2010.
- [13] P. P. Khine and Z. S. Wang, "Data lake: a new ideology in big data era," *ITM Web of Conferences*, vol. 17, p. 3025, 2018, doi: 10.1051/itmconf/20181703025.
- [14] C. Madera and A. Laurent, "The Next Information Architecture Evolution: The Data Lake Wave," 2016.
- [15] P. Sawadogo and J. Darmont, "On data lake architectures and metadata management," *Journal of Intelligent Information Systems*, vol. 56, no. 1, pp. 97–120, Feb. 2021, doi: 10.1007/s10844-020-00608-7.
- [16] C. Walker and H. Alrehamy, "Personal Data Lake with Data Gravity Pull," in *2015 IEEE Fifth International Conference on Big Data and Cloud Computing*, Aug. 2015, pp. 160–167. doi: 10.1109/BDCloud.2015.62.
- [17] M. Chessell, F. Scheepers, N. Nguyen, and R. van Kessel, "Governing and Managing Big Data for Analytics and Decision Makers," no. 252, 2014.
- [18] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," 2021.
- [19] M. Armbrust *et al.*, "Delta lake: high-performance ACID table storage over cloud object stores," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, Aug. 2020, doi: 10.14778/3415478.3415560.
- [20] J. Serra and F. Langenau, *Datenarchitekturen: Modern Data Warehouse, Data Fabric, Data Lakehouse und Data Mesh Richtig Einsetzen*. Heidelberg, GERMANY: O'Reilly Verlag GmbH & Co. KG, 2024. Accessed: Nov. 03, 2025. [Online]. Available: <http://ebookcentral.proquest.com/lib/htwg-konstanz/detail.action?docID=31745120>
- [21] A. Nambiar and D. Mundra, "An Overview of Data Warehouse and Data Lake in Modern Enterprise Data Management," *Big Data and Cognitive Computing*, vol. 6, no. 4, p. 132, Dec. 2022, doi: 10.3390/bdcc6040132.
- [22] Z. Dehghani, *Data Mesh: Delivering Data-driven Value at Scale*. O'Reilly, 2022. [Online]. Available: <https://books.google.de/books?id=M5J5zgEACAAJ>
- [23] R. L. Ackoff, "From data to wisdom," *Journal of applied systems analysis*, vol. 16, no. 1, pp. 3–9, 1989.

- [24] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, Jul. 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x.
- [25] L. Floridi, "Is Semantic Information Meaningful Data?," *Philosophy and Phenomenological Research*, vol. 70, no. 2, pp. 351–370, 2005, doi: 10.1111/j.1933-1592.2005.tb00531.x.
- [26] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.
- [27] S. Abiteboul, P. Buneman, and D. Suciu, *Data on the Web: From Relations to Semistructured Data and XML*. Morgan Kaufmann, 2000.
- [28] P. Russom, "Big Data Analytics," *TDWI best practices report, fourth quarter*, vol. 19, no. 4, pp. 1–34, 2011.
- [29] "ISO/IEC 11179-1:2023 — Information technology — Metadata registries (MDR) — Part 1: Framework." [Online]. Available: <https://www.vde-verlag.de/iec-normen/251510/iso-iec-11179-1-2023.html>
- [30] I. Morshedzadeh, A. H. Ng, and M. Jeusfeld, "Managing manufacturing data and information in product lifecycle management systems considering changes and revisions," *International Journal of Product Lifecycle Management*, vol. 13, no. 3, pp. 244–264, Jan. 2021, doi: 10.1504/IJPLM.2021.118041.
- [31] F. Tao, Q. Qi, A. Liu, and A. Kusiak, "Data-driven smart manufacturing," *Journal of Manufacturing Systems*, vol. 48, pp. 157–169, Jul. 2018, doi: 10.1016/j.jmsy.2018.01.006.
- [32] T. C. Redman, "The impact of poor data quality on the typical enterprise," *Communications of the ACM*, vol. 41, no. 2, pp. 79–82, Feb. 1998, doi: 10.1145/269012.269025.
- [33] International Organization for Standardization, "ISO/IEC 8000-2:2022 Information technology – Master data: Vocabulary," Geneva, Switzerland, 2024.
- [34] R. Y. Wang and D. M. Strong, "Beyond Accuracy: What Data Quality Means to Data Consumers," *Journal of Management Information Systems*, vol. 12, no. 4, pp. 5–33, Mar. 1996, doi: 10.1080/07421222.1996.11518099.
- [35] R. Y. Wang, "A product perspective on total data quality management," *Communications of the ACM*, vol. 41, no. 2, pp. 58–65, Feb. 1998, doi: 10.1145/269012.269022.

- [36] D. M. Strong, Y. W. Lee, and R. Y. Wang, "Data quality in context," *Commun. ACM*, vol. 40, no. 5, pp. 103–110, May 1997, doi: 10.1145/253769.253804.
- [37] Erhard Rahm and H. H. Do, "Data Cleaning: Problems and Current Approaches," *IEEE Data Eng. Bull.*, vol. 23, no. 4, pp. 3–13, 2000.
- [38] W. Kim, B.-J. Choi, E.-K. Hong, S.-K. Kim, and D. Lee, "A Taxonomy of Dirty Data," *Data Mining and Knowledge Discovery*, vol. 7, no. 1, pp. 81–99, Jan. 2003, doi: 10.1023/A:1021564703268.
- [39] H. Müller and J.-C. Freytag, "Problems, Methods, and Challenges in Comprehensive Data Cleansing," 2003.
- [40] P. Oliveira, F. Rodrigues, P. Rangel Henriques, and H. Galhardas, "A Taxonomy of Data Quality Problems," *Journal of Data and Information Quality - JDIQ*, Jan. 2005.
- [41] M. Chen, S. Mao, and Y. Liu, "Big Data: A Survey," *Mobile Networks and Applications*, vol. 19, no. 2, pp. 171–209, Apr. 2014, doi: 10.1007/s11036-013-0489-0.
- [42] J. Manyika *et al.*, "Big data: The next frontier for innovation, competition, and productivity," 2011.
- [43] NIST Big Data Public Working Group Definitions and Taxonomies Subgroup, "NIST Big Data Interoperability Framework: Volume 1, Definitions," Oct. 2015. doi: 10.6028/NIST.SP.1500-1.
- [44] B. Saha and D. Srivastava, "Data quality: The other face of Big Data," in *2014 IEEE 30th International Conference on Data Engineering*, Mar. 2014, pp. 1294–1297. doi: 10.1109/ICDE.2014.6816764.
- [45] I. A. T. Hashem, I. Yaqoob, N. B. Anuar, S. Mokhtar, A. Gani, and S. Ullah Khan, "The rise of "big data" on cloud computing: Review and open research issues," *Information Systems*, vol. 47, pp. 98–115, Jan. 2015, doi: 10.1016/j.is.2014.07.006.
- [46] L. Cai and Y. Zhu, "The Challenges of Data Quality and Data Quality Assessment in the Big Data Era," *Data Science Journal*, vol. 14, p. 2, May 2015, doi: 10.5334/dsj-2015-002.
- [47] I. Taleb, M. A. Serhani, and R. Dssouli, "Big Data Quality: A Survey," in *2018 IEEE International Congress on Big Data (BigData Congress)*, Jul. 2018, pp. 166–173. doi: 10.1109/BigDataCongress.2018.00029.
- [48] F. Ridzuan and W. M. N. W. Zainon, "A Review on Data Quality Dimensions for Big Data," *Procedia Computer Science*, vol. 234, pp. 341–348, Jan. 2024, doi: 10.1016/j.procs.2024.03.008.

- [49] Microsoft, "Grundlegendes zu Microsoft Fabric - Was ist Microsoft Fabric?." Accessed: Nov. 09, 2025. [Online]. Available: <https://learn.microsoft.com/de-de/fabric/fundamentals/microsoft-fabric-overview>
- [50] P. Strengtholt, *Building Medallion Architectures: Designing with Delta Lake and Spark*. "O'Reilly Media, Inc.", 2025.
- [51] Microsoft, "OneLake, the OneDrive for Data - Microsoft Fabric." Accessed: Nov. 13, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/fabric/onelake/onelake-overview>
- [52] C. Legner and M. R. Hasan, "Understanding Data Products: Motivations, Definition, and Categories," Jun. 2023.
- [53] I. A. Machado, C. Costa, and M. Y. Santos, "Data Mesh: Concepts and Principles of a Paradigm Shift in Data Architectures," *Procedia Computer Science*, vol. 196, pp. 263–271, 2022, doi: 10.1016/j.procs.2021.12.013.
- [54] Z. Dehghani, "Data Mesh Principles and Logical Architecture." Dec. 2020.
- [55] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. in Prentice Hall Series in Artificial Intelligence. Upper Saddle River: Prentice Hall, 1995.
- [56] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "AI Agents That Matter," no. arXiv:2407.01502. arXiv, Jul. 2024. doi: 10.48550/arXiv.2407.01502.
- [57] L. Wang *et al.*, "A Survey on Large Language Model Based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, Mar. 2024, doi: 10.1007/s11704-024-40231-1.
- [58] "posts." Accessed: Nov. 20, 2025. [Online]. Available: <https://lilianweng.github.io/posts/2023-06-23-agent/>
- [59] Microsoft, "Fabric Data Agent - What Is the Fabric Data Agent?."
- [60] D. Inc., "What Is a Medallion Architecture?." Accessed: Nov. 24, 2025. [Online]. Available: <https://www.databricks.com/glossary/medallion-architecture>
- [61] Microsoft, "What Is the Medallion Lakehouse Architecture?." Accessed: Nov. 24, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/azure/databricks/lakehouse/medallion>
- [62] P. Mayring, *Qualitative Inhaltsanalyse: Grundlagen und Techniken*, 13., aktualisierte Auflage. in Beltz Pädagogik. Weinheim: Beltz, 2022. [Online]. Available: <https://www.beltz.de/fachmedien/paedagogik/produkte/details/48632-qualitative-inhaltsanalyse.html>

- [63] C. Xie, J. Gao, and C. Tao, "Big Data Validation Case Study," in *2017 IEEE Third International Conference on Big Data Computing Service and Applications (BigDataService)*, Apr. 2017, pp. 281–286. doi: 10.1109/BigDataService.2017.44.
- [64] S. B. R. Karri, V. K. Devalla, R. K. Bojja, and M. S. Pandey, "An Architecture for Model Monitoring System with Automated Data Validation and Failure Handling," in *2025 3rd International Conference on Communication, Security, and Artificial Intelligence (ICCSAI)*, Apr. 2025, pp. 1960–1966. doi: 10.1109/ICCSAI64074.2025.11064092.
- [65] S. Song, F. Gao, R. Huang, and C. Wang, "Data Dependencies Extended for Variety and Veracity: A Family Tree," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 10, pp. 4717–4736, Oct. 2022, doi: 10.1109/TKDE.2020.3046443.
- [66] S. Juddoo, "Overview of Data Quality Challenges in the Context of Big Data," in *2015 International Conference on Computing, Communication and Security (ICCCS)*, Dec. 2015, pp. 1–9. doi: 10.1109/CCCS.2015.7374131.
- [67] P. Mahajan, "Textual Data Quality at Scale for High Dimensionality Data," in *2022 International Conference on Data Science, Agents & Artificial Intelligence (ICDSAAI)*, Dec. 2022, pp. 1–4. doi: 10.1109/ICDSAAI55433.2022.10028903.
- [68] A.-S. Moloiu, G. Albeanu, H. Madsen, and F. Popențiu-Vădicescu, "Data Quality Assessment for ML Decision-Making," *Applications in Reliability and Statistical Computing*. Springer International Publishing, Cham, pp. 163–178, 2023. doi: 10.1007/978-3-031-21232-1_8.
- [69] S. Shrivastava *et al.*, "DQA: Scalable, Automated and Interactive Data Quality Advisor," in *2019 IEEE International Conference on Big Data (Big Data)*, Dec. 2019, pp. 2913–2922. doi: 10.1109/BigData47090.2019.9006187.
- [70] W. Elouataoui, S. El Mendili, and Y. Gahi, "Active Metadata and Machine Learning Based Framework for Enhancing Big Data Quality," in *Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security*, in NISS '24. New York, NY, USA: Association for Computing Machinery, Aug. 2024, pp. 1–8. doi: 10.1145/3659677.3659707.
- [71] N. Afrilliana, D. Król, and F. L. Gaol, "Computational Intelligence Techniques for Assessing Data Quality: Towards Knowledge-Driven Processing," in *Computational Science – ICCS 2021*, M. Paszynski, D. Kranzlmüller, V. V. Krzhizhanovskaya, J.

- J. Dongarra, and P. M. Sloot, Eds., Cham: Springer International Publishing, 2021, pp. 392–405. doi: 10.1007/978-3-030-77967-2_33.
- [72] M. A. Encinas, A. T. Tunkiel, and D. Sui, “Downhole Data Correction for Data-Driven Rate of Penetration Prediction Modeling,” *Journal of Petroleum Science and Engineering*, vol. 210, p. 109904, Mar. 2022, doi: 10.1016/j.petrol.2021.109904.
- [73] S. C. H. Geekiyanage, A. Tunkiel, and D. Sui, “Drilling Data Quality Improvement and Information Extraction with Case Studies,” *Journal of Petroleum Exploration and Production*, vol. 11, no. 2, pp. 819–837, Feb. 2021, doi: 10.1007/s13202-020-01024-x.
- [74] V. V. Keskar, J. Yadav, and A. Kumar, “Enhancing Data Quality by Detecting and Repairing Inconsistencies in Big Data,” in *Proceedings of Second International Conference in Mechanical and Energy Technology*, S. Yadav, A. Haleem, P. K. Arora, and H. Kumar, Eds., Singapore: Springer Nature, 2023, pp. 185–197. doi: 10.1007/978-981-19-0108-9_20.
- [75] A. Nguetilbaye, H. Wang, D. A. Mahamat, and R. Madadjim, “Implicit Dedupe Learning Method on Contextual Data Quality Problems,” in *Advances in Data Science and Information Engineering*, R. Stahlbock, G. M. Weiss, M. Abou-Nasr, C.-Y. Yang, H. R. Arabnia, and L. Deligiannidis, Eds., Cham: Springer International Publishing, 2021, pp. 343–358. doi: 10.1007/978-3-030-71704-9_22.
- [76] R. Gu *et al.*, “SparkDQ: Efficient Generic Big Data Quality Management on Distributed Data-Parallel Computation,” *Journal of Parallel and Distributed Computing*, vol. 156, pp. 132–147, Oct. 2021, doi: 10.1016/j.jpdc.2021.05.012.
- [77] T. Nguyen, H.-T. Nguyen, and T.-A. Nguyen-Hoang, “Data Quality Management in Big Data: Strategies, Tools, and Educational Implications,” *Journal of Parallel and Distributed Computing*, vol. 200, p. 105067, Jun. 2025, doi: 10.1016/j.jpdc.2025.105067.
- [78] S. Juddoo and C. George, “A Qualitative Assessment of Machine Learning Support for Detecting Data Completeness and Accuracy Issues to Improve Data Analytics in Big Data for the Healthcare Industry,” in *2020 3rd International Conference on Emerging Trends in Electrical, Electronic and Communications Engineering (ELECOM)*, Nov. 2020, pp. 58–66. doi: 10.1109/ELECOM49001.2020.9297009.
- [79] W. Shi *et al.*, “Improving Power Grid Monitoring Data Quality: An Efficient Machine Learning Framework for Missing Data Prediction,” in *2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE*

- 7th International Symposium on Cyberspace Safety and Security, and 2015 IEEE 12th International Conference on Embedded Software and Systems*, Aug. 2015, pp. 417–422. doi: 10.1109/HPCC-CSS-ICSS.2015.16.
- [80] J. Liu, Y. Cao, Y. Li, Y. Guo, and W. Deng, “Big Data Cleaning Based on Improved CLOF and Random Forest for Distribution Networks,” *CSEE Journal of Power and Energy Systems*, vol. 10, no. 6, pp. 2528–2538, Nov. 2024, doi: 10.17775/CSEEJPES.2020.04080.
 - [81] C. Gritsch, A. Tick, and P. Rosenberger, “Data Cleansing Methods for Big Data: A Systematic Review,” in *2025 IEEE 29th International Conference on Intelligent Engineering Systems (INES)*, Jun. 2025, pp. 27–32. doi: 10.1109/INES67149.2025.11078189.
 - [82] Q. Qin, H. Li, and E. Merlo, “Unsupervised, Robust, and Lightweight Detection of Data Pattern Anomalies and Outliers,” *Information and Software Technology*, vol. 192, p. 108035, Apr. 2026, doi: 10.1016/j.infsof.2026.108035.
 - [83] H. C. Tamm and A. Nikiforova, “From Data Quality for AI to AI for Data Quality: A Systematic Review of Tools for AI-Augmented Data Quality Management in Data Warehouses,” in *Perspectives in Business Informatics Research*, R. Deneckère, M. Kirikova, and J. Grabis, Eds., Cham: Springer Nature Switzerland, 2026, pp. 37–53. doi: 10.1007/978-3-032-04375-7_3.
 - [84] M. V. Silva, H. Herrmann, and V. Maxville, “Attribute-Based Semantic Type Detection and Data Quality Assessment,” in *2024 IEEE/ACM International Conference on Big Data Computing, Applications and Technologies (BDCAT)*, Dec. 2024, pp. 119–124. doi: 10.1109/BDCAT63179.2024.00030.
 - [85] W. Yang, A. Da Silva, and M.-L. Picard, “Computing Data Quality Indicators on Big Data Streams Using a CEP,” in *2015 International Workshop on Computational Intelligence for Multimedia Understanding (IWCIM)*, Oct. 2015, pp. 1–5. doi: 10.1109/IWCIM.2015.7347061.
 - [86] X. Xu, Y. Lei, and Z. Li, “An Incorrect Data Detection Method for Big Data Cleaning of Machinery Condition Monitoring,” *IEEE Transactions on Industrial Electronics*, vol. 67, no. 3, pp. 2326–2336, Mar. 2020, doi: 10.1109/TIE.2019.2903774.
 - [87] L. Rettig, M. Khayati, P. Cudré-Mauroux, and M. Piórkowski, “Online Anomaly Detection over Big Data Streams,” in *2015 IEEE International Conference on Big Data (Big Data)*, Oct. 2015, pp. 1113–1122. doi: 10.1109/BigData.2015.7363865.

- [88] A. Goknil *et al.*, “A Systematic Review of Data Quality in CPS and IoT for Industry 4.0,” *ACM Comput. Surv.*, vol. 55, no. 14s, pp. 1–38, Jul. 2023, doi: 10.1145/3593043.
- [89] J. Hossen, M. Z. Jesmeen H, and S. Sayeed, “Modifying Cleaning Method in Big Data Analytics Process Using Random Forest Classifier,” in *2018 7th International Conference on Computer and Communication Engineering (ICCCE)*, Sep. 2018, pp. 208–213. doi: 10.1109/ICCCE.2018.8539254.
- [90] B. Berisha, E. Mëziu, and I. Shabani, “Big data analytics in Cloud computing: an overview,” *Journal of Cloud Computing*, vol. 11, no. 1, p. 24, Aug. 2022, doi: 10.1186/s13677-022-00301-w.
- [91] Z. Dehghani, *Data Mesh: eine dezentrale Datenarchitektur entwerfen*, 1. Auflage. in Animals. Heidelberg: O'Reilly, 2023.
- [92] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*, 2. ed., [Nachdr.]. in Kimball's Data Warehouse Toolkit Classics. New York, NY: Wiley, 2009.
- [93] Microsoft, “Was ist Data Factory? - Microsoft Fabric.” Accessed: Nov. 13, 2025. [Online]. Available: <https://learn.microsoft.com/de-de/fabric/data-factory/data-factory-overview>
- [94] Microsoft, “What Is Data Engineering in Microsoft Fabric? - Microsoft Fabric.” Accessed: Nov. 13, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/fabric/data-engineering/data-engineering-overview>
- [95] “Geschäftsbericht | Geberit AG.”
- [96] Geberit Vertriebs GmbH, “Das Unternehmen Geberit.” Accessed: Nov. 25, 2025. [Online]. Available: <https://www.geberit.de/ueber-uns/>
- [97] H. Fadlallah *et al.*, “BIGQA: Declarative Big Data Quality Assessment,” *J. Data and Information Quality*, vol. 15, no. 3, pp. 1–30, Aug. 2023, doi: 10.1145/3603706.
- [98] H. Fadlallah *et al.*, “Context-Aware Big Data Quality Assessment: A Scoping Review,” *J. Data and Information Quality*, vol. 15, no. 3, pp. 1–33, Aug. 2023, doi: 10.1145/3603707.
- [99] Z. Zhu, Y. Tian, and H. Yang, “Research on Power Data Quality Analysis Method Based on Verification Rules in Big Data Environment,” in *Proceedings of the 5th International Conference on Computer Information and Big Data Applications*, in CIBDA '24. New York, NY, USA: Association for Computing Machinery, Jul. 2024, pp. 1192–1196. doi: 10.1145/3671151.3671359.

- [100] Y. Suh, "Exploring the Impact of Data Quality on Business Performance in CRM Systems for Home Appliance Business," *IEEE Access*, vol. 11, pp. 116076–116089, 2023, doi: 10.1109/ACCESS.2023.3325892.
- [101] M. A. Serhani, H. T. El Kassabi, I. Taleb, and A. Nujum, "An Hybrid Approach to Quality Evaluation across Big Data Value Chain," in *2016 IEEE International Congress on Big Data (BigData Congress)*, Jun. 2016, pp. 418–425. doi: 10.1109/BigDataCongress.2016.65.
- [102] N. Guo, Q. Shen, and C. Liu, "Power Data Quality Assessment Based on Entropy Weight Method and Grey System Theory," in *2024 IEEE 11th International Conference on Cyber Security and Cloud Computing (CSCloud)*, Jun. 2024, pp. 79–83. doi: 10.1109/CSCloud62866.2024.00021.
- [103] A. Iyengar, D. Patel, S. Shrivastava, N. Zhou, and A. Bhamidipaty, "Real-Time Data Quality Analysis," in *2020 IEEE Second International Conference on Cognitive Machine Intelligence (CogMI)*, Oct. 2020, pp. 101–108. doi: 10.1109/CogMI50398.2020.00022.
- [104] G.-O. Meritxell, B. Sierra, and S. Ferreira, "On the Evaluation, Management and Improvement of Data Quality in Streaming Time Series," *IEEE Access*, vol. 10, pp. 81458–81475, 2022, doi: 10.1109/ACCESS.2022.3195338.
- [105] R. Ji, H. Hou, G. Sheng, and X. Jiang, "Data Quality Assessment for Electrical Equipment Condition Monitoring," in *2022 9th International Conference on Condition Monitoring and Diagnosis (CMD)*, Nov. 2022, pp. 1–4. doi: 10.23919/CMD54214.2022.9991385.
- [106] R. He, T. Xiao, S. Qiu, J. Gu, M. Wei, and P. Xu, "A Rule-Based Data Preprocessing Framework for Chiller Rooms Inspired by the Analysis of Engineering Big Data," *Energy and Buildings*, vol. 273, p. 112372, Oct. 2022, doi: 10.1016/j.enbuild.2022.112372.
- [107] H. Liu, X. Wang, S. Lei, X. Zhang, W. Liu, and M. Qin, "A Rule Based Data Quality Assessment Architecture and Application for Electrical Data," in *Proceedings of the International Conference on Artificial Intelligence, Information Processing and Cloud Computing*, in AIIPCC '19. New York, NY, USA: Association for Computing Machinery, Dec. 2019, pp. 1–6. doi: 10.1145/3371425.3371435.
- [108] W. Elouataoui, S. E. Mendili, and Y. Gahi, "Big Data Quality Anomaly Scoring Framework Using Artificial Intelligence," in *2023 7th IEEE Congress on Infor-*

- mation Science and Technology (CiSt)*, Dec. 2023, pp. 87–92. doi: 10.1109/CiSt56084.2023.10409909.
- [109] V. Keskar, J. Yadav, and A. Kumar, “Perspective of Anomaly Detection in Big Data for Data Quality Improvement,” *Materials Today: Proceedings*, vol. 51, pp. 532–537, Jan. 2022, doi: 10.1016/j.matpr.2021.05.597.
 - [110] D. Tu *et al.*, “Auto-Validate by-History: Auto-Program Data Quality Constraints to Validate Recurring Data Pipelines,” in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, in KDD '23. New York, NY, USA: Association for Computing Machinery, Aug. 2023, pp. 4991–5003. doi: 10.1145/3580305.3599776.
 - [111] N. Nadjeh, S. Abdellaoui, and F. Nader, “Big-MDC: A Multi-Dimensions Cleaning Framework to Improve Big Data Quality,” *International Journal of Data Science and Analytics*, vol. 20, no. 6, pp. 5869–5893, Nov. 2025, doi: 10.1007/s41060-025-00804-0.
 - [112] F. Ridzuan and W. M. N. Wan Zainon, “A Review on Data Cleansing Methods for Big Data,” *Procedia Computer Science*, vol. 161, pp. 731–738, Jan. 2019, doi: 10.1016/j.procs.2019.11.177.
 - [113] S. Sachi, R. Kiran Pagidi, S. Karunakaran, S. K. Gupta, S. Dharmapuram, and O. Goel, “Data Lake Validation Strategies: Ensuring Quality in Data Warehousing Pipelines,” in *2025 International Conference on Intelligent and Secure Engineering Solutions (CISES)*, Aug. 2025, pp. 918–922. doi: 10.1109/CISES66934.2025.11265447.
 - [114] V. Mishra and H. Darade, “Advancements in Data Quality Management in the Big Data Era: A Comprehensive Review,” in *2024 Third International Conference on Artificial Intelligence, Computational Electronics and Communication System (AICECS)*, Dec. 2024, pp. 1–6. doi: 10.1109/AICECS63354.2024.10956447.
 - [115] G. Zhang, “A Data Traceability Method to Improve Data Quality in a Big Data Environment,” in *2020 IEEE Fifth International Conference on Data Science in Cyberspace (DSC)*, Jul. 2020, pp. 290–294. doi: 10.1109/DSC50466.2020.00051.
 - [116] T. Hongxun *et al.*, “Data Quality Assessment for On-Line Monitoring and Measuring System of Power Quality Based on Big Data and Data Provenance Theory,” in *2018 IEEE 3rd International Conference on Cloud Computing and Big Data Analysis (ICCCBDA)*, Apr. 2018, pp. 248–252. doi: 10.1109/ICCCBDA.2018.8386521.

- [117] I. Taleb, R. Dssouli, and M. A. Serhani, "Big Data Pre-processing: A Quality Framework," in *2015 IEEE International Congress on Big Data*, Jun. 2015, pp. 191–198. doi: 10.1109/BigDataCongress.2015.35.
- [118] J. Zhao, C. Li, and L. Wang, "Hadoop-Based Power Grid Data Quality Verification and Monitoring Method," *Journal of Electrical Engineering & Technology*, vol. 18, no. 1, pp. 89–97, Jan. 2023, doi: 10.1007/s42835-022-01171-7.
- [119] A. Zaveri and A. Rula, "Data Quality and Data Cleansing of Semantic Data," *Encyclopedia of Big Data Technologies*. Springer, Cham, pp. 573–579, 2019. doi: 10.1007/978-3-319-77525-8_289.
- [120] W. Dai, K. Yoshigoe, and W. Parsley, "Improving Data Quality Through Deep Learning and Statistical Models," in *Information Technology - New Generations*, S. Latifi, Ed., Cham: Springer International Publishing, 2018, pp. 515–522. doi: 10.1007/978-3-319-54978-1_66.
- [121] H. Zaidi, Y. Pollet, F. Boufarès, and N. Kraiem, "Semantic of Data Dependencies to Improve the Data Quality," in *Model and Data Engineering*, L. Bellatreche and Y. Manolopoulos, Eds., Cham: Springer International Publishing, 2015, pp. 53–61. doi: 10.1007/978-3-319-23781-7_5.
- [122] C. Lettner, R. Stumptner, and K.-H. Bokesch, "An Approach on ETL Attached Data Quality Management," in *Data Warehousing and Knowledge Discovery*, L. Bellatreche and M. K. Mohania, Eds., Cham: Springer International Publishing, 2014, pp. 1–8. doi: 10.1007/978-3-319-10160-6_1.
- [123] X. Xu, Y. Lei, and X. Zhou, "A LOF-Based Method for Abnormal Segment Detection in Machinery Condition Monitoring," in *2018 Prognostics and System Health Management Conference (PHM-Chongqing)*, Oct. 2018, pp. 125–128. doi: 10.1109/PHM-Chongqing.2018.00027.
- [124] M. Ali, P. Scandurra, F. Moretti, and H. H. R. Sherazi, "Anomaly Detection in Public Street Lighting Data Using Unsupervised Clustering," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 4524–4535, Feb. 2024, doi: 10.1109/TCE.2024.3354189.
- [125] S. Li, H. He, P. Zhao, and S. Cheng, "Data Cleaning and Restoring Method for Vehicle Battery Big Data Platform," *Applied Energy*, vol. 320, p. 119292, Aug. 2022, doi: 10.1016/j.apenergy.2022.119292.
- [126] G. Kaliyaperumal, A. Gatla, S. Lakhanpal, K. J. Reddy, R. Najm, and K. B. Raju, "Adaptive Framework for Comprehensive Quality Assessment in Un-

- structured Big Data,” in *2024 7th International Conference on Contemporary Computing and Informatics (IC3I)*, Sep. 2024, pp. 1522–1528. doi: 10.1109/IC3I61595.2024.10829192.
- [127] S. Geerkens, C. Sieberichs, A. Braun, and T. Waschulzik, “QI²: An Interactive Tool for Data Quality Assurance,” *AI and Ethics*, vol. 4, no. 1, pp. 141–149, Feb. 2024, doi: 10.1007/s43681-023-00390-6.
- [128] P. S. Niranjana and K. S. Rekha, “Data Deduplication System Using Machine Learning,” in *2025 Annual International Conference on Data Science, Machine Learning and Blockchain Technology (AICDMB)*, Jun. 2025, pp. 1–7. doi: 10.1109/AICDMB64359.2025.11277461.
- [129] L. Caruccio and S. Cirillo, “Incremental Discovery of Imprecise Functional Dependencies,” *J. Data and Information Quality*, vol. 12, no. 4, pp. 1–25, Oct. 2020, doi: 10.1145/3397462.
- [130] K. M. Sunderland *et al.*, “The Utility of Multivariate Outlier Detection Techniques for Data Quality Evaluation in Large Studies: An Application within the ONDRI Project,” *BMC Medical Research Methodology*, vol. 19, no. 1, p. 102, May 2019, doi: 10.1186/s12874-019-0737-5.
- [131] R. D. Pandey and I. Snigdha, “Validity as a Measure of Data Quality in Internet of Things Systems,” *Wireless Personal Communications*, vol. 126, no. 1, pp. 933–948, Sep. 2022, doi: 10.1007/s11277-022-09777-w.
- [132] T. Ouyang and T. Amagasa, “Textual Out-of-Distribution Data Detection Based on Granular Dictionary,” in *2024 IEEE International Conference on Big Data (Big-Data)*, Dec. 2024, pp. 8333–8340. doi: 10.1109/BigData62323.2024.10825751.

A. Methodencluster

CLUSTER	Methoden	Quellen
RULE-BASED DATA VALIDATION	• Sämtliche Checks • Regex / Pattern based Preprocessing • Domain specific validity rules • Rule / Integrity-Constraint Discovery • Ontology-Consistency Checking • Hash-based Transformation • Expert guided Thresholding • Dictionary based • LUHN-Algorithm	[106] , [76] , [98] , [97] , [107] , [108] , [82] , [109] , [110] , [7] , [67] , [69] , [83] , [111] , [73] , [75] , [77] , [112] , [99] , [88] , [101] , [113] , [64] , [84] , [114] , [78] , [103] , [115] , [116] , [63] , [66] , [117] , [118] , [68] , [119] , [120] , [121] , [122] , [65]
STATISTICAL DATA PROFILING	• IQR Outlier Detection • Median/Moving Average Filter • Korrelationsanalysen (Pearson Spearman, MIC) • Consistency Rate • Ratio-based Quality Metrics • Local Outlier Factor (LOF) • Post-Cleaning-Statistical Validation • Fuzzy-Methods • Graph-based Data Quality Metrics • Dominant Category Detection • Semantic Validity Check • Dependency-Discovery • Low Variance Erkennung • Sampling based Profiling	[70] , [98] , [97] , [108] , [82] , [110] , [123] , [73] , [124] , [69] , [71] , [77] , [72] , [88] , [114] , [78] , [66] , [119] , [120] , [121] , [122] , [68]
ANOMALY DETECTION	• Z-Score-based, • Residual based, • Threshold based, • Autoencoder-based • Neighborhood-based • Clustering Algorithms (K-means, IF, DBSCAN, OPTICS), • Dimensionality Reduction, PCA • Pearson-/Korr-based Anomalieerkennung • LOF-based Abnormal Section Detection (KLOF) • CLOF Outlier Detection	[106] , [76] , [125] , [70] , [107] , [108] , [123] , [81] , [80] , [126] , [67] , [124] , [69] , [127] , [74] , [71] , [88] , [64] , [114] , [78] , [103] , [68] , [120] , [122]
MATCHING, DEDUPLICATION	• Similarity Rules, • Threshold-based, • Record Lineage, • Match/Merge Algorithms • Pattern Discovery	[128] , [81] , [67] , [69] , [75] , [119] , [121] , [76] , [65] , [83]

CLUSTER	Methoden	Quellen
	• ML-based Near-Duplicate-Detection/ Deduplication • Missing Pattern Clustering • Similarity-based Semantic/Block Entity Resolution • Text Similarity Matching	
AUTOMATISIERTES REPAIRING	• Lineare Interpolation, Extrapolation • Filtering-/Replacing-based Repairing • Clustering (K-NN), • Forward-Fill, • Regelbasierte, Probabilistische/Statistische Bereinigung/Imputation • ML-gestützte Imputation, Decoder-based Reconstruction • Similarity-based Value Standardization • Naive Bayes Missing Value Filling, Gaussian Simulation, • Bayesian/Isotonic Regression, • lp-norm regularized sparse Regression • Traceability-based Correction • Deletion/Standardization Rules	[125] , [76] , [81] , [80] , [89] , [111] , [73] , [71] , [77] , [72] , [113] , [114] , [78] , [103] , [115] , [66] , [79] , [117]
GESAMTANSÄTZE	• LUCARIO • SparkDQ • BIGQA • Active-Metadata • Auto-Validate-by-History • Relaxed Functional Dependencies • R-DQAA-EBA • Multivariate Outlier-based DQ Evaluation Process • Dedupe Learning Framework • Hybrid Fabric	[82] , [97] , [76] , [70] , [110] , [129] , [7] , [107] , [130] , [75]
GENAI, AGENTISCH	• NLP/LLM gestützte Rule Generation • Entity-/Conflict Resolution rules • Context aware Error Classification • Deep-LSTM-based Temporal Feature Extraction • GAN-based Data Cleaning • Attributbasierte semantische Typdetektion	[125] , [81] , [7] , [83] , [84] ,
IoT, STREAMING	• CEP-basierte Online-DQ Berechnung • Sliding/Microbatch-Windowing • Multistream Konsistenzprüfung • Time-Series Anomaly Detection • Time-relevant K-means, KNPTS	[104] , [86] , [85] , [87] , [88] , [107] , [131]

CLUSTER	Methoden	Quellen
ML-BASED CLASSIFICATION/PREDICTION	• Random Forest • SVM (Optimized, Refined) • Multivariate/Sequential Missing Data Prediction • NN-based Error Classification • Sequence-to-Sequence Regression • Predictive Outlier Correction (XGBoost) • TF-IDF, FAISS • Textuelle Out-of-Distribution Erkennung	[125] , [70] , [128] , [132] , [89] , [77] , ([112]) , [64] , [114] , [78] , [85] , [79] , [71]

Tab. A: Methodencluster (eigene Darstellung)

B. Bewertungsmatrix

METHOD- ENKATEGORIE	DQ-Dimensio- nen	DQ-Problem- typen	Architek- turkompatibil- ität Fabric	Komplexität	Skalierbarkeit	Eignung im Use Case
RULE-BASED DATA VALIDA- TION	• (1): 2 • (2): 2 • (3): 1 • (4): 2 • (5): 1 • (6): 0 • (7): 1 1,29	• (1): 2 • (2): 2 • (3): 2 • (4): 1 • (5): 0 1,4	• (1): 2 • (2): 2 • (3): 2 • (4): 2 • (5): 2 • (6): 2 2	Niedrig-Mittel	Mittel-Hoch	$(1,29 + 1,4 + 2) / 3 = 1,56$
STATISTICAL DATA PROFIL- ING	• (1): 1 • (2): 2 • (3): 2 • (4): 2 • (5): 1 • (6): 0 • (7): 1 1,29	• (1): 2 • (2): 1 • (3): 2 • (4): 2 • (5): 0 1,4	• (1): 2 • (2): 2 • (3): 2 • (4): 1 • (5): 2 • (6): 2 1,83	Mittel	Hoch	$(1,29 + 1,4 + 1,83) / 3 = 1,51$
ANOMALY DE- TECTION	• (1): 2 • (2): 0 • (3): 2 • (4): 2 • (5): 1 • (6): 0 • (7): 1 1,14	• (1): 2 • (2): 1 • (3): 2 • (4): 1 • (5): 0 1,2	• (1): 2 • (2): 2 • (3): 2 • (4): 1 • (5): 2 • (6): 1 1,67	Mittel	Mittel-Hoch	$(1,14 + 1,2 + 1,67) / 3 = 1,34$
MATCHING, DEDUPLICA- TION	• (1): 0 • (2): 2 • (3): 1 • (4): 2 • (5): 1 • (6): 0 • (7): 0 0,86	• (1): 0 • (2): 1 • (3): 2 • (4): 2 • (5): 1 1,2	• (1): 1 • (2): 2 • (3): 2 • (4): 1 • (5): 2 • (6): 1 1,5	Mittel-Hoch	Niedrig	$(0,86 + 1,2 + 1,5) / 3 = 1,19$
AUTOMA- TISIERTES RE- PAIRING	• (1): 2 • (2): 2 • (3): 0 • (4): 1 • (5): 0 • (6): 0 • (7): 1 0,86	• (1): 2 • (2): 2 • (3): 1 • (4): 0 • (5): 0 1	• (1): 1 • (2): 2 • (3): 2 • (4): 1 • (5): 2 • (6): 1 1,5	Mittel-Hoch	Niedrig	$(0,86 + 1 + 1,5) / 3 = 1,12$

METHOD- ENKATEGORIE	DQ-Dimensio- nen	DQ-Problem- typen	Architek- turkompatibil- ität Fabric	Komplexität	Skalierbarkeit	Eignung im Use Case
GESAMTAN- SÄTZE	• (1): 1 • (2): 1 • (3): 1 • (4): 1 • (5): 1 • (6): 0 • (7): 1 0,86	• (1): 1 • (2): 1 • (3): 1 • (4): 1 • (5): 0 0,8	• (1): 1 • (2): 1 • (3): 2 • (4): 1 • (5): 1 • (6): 1 1,17	Hoch	Niedrig-Mittel	$(0,86 + 0,8 + 1,17) / 3 = \mathbf{0,94}$
IoT, STREAM- ING	• (1): 2 • (2): 1 • (3): 1 • (4): 1 • (5): 1 • (6): 0 • (7): 0 0,86	• (1): 2 • (2): 2 • (3): 1 • (4): 0 • (5): 0 1	• (1): 2 • (2): 2 • (3): 2 • (4): 1 • (5): 2 • (6): 2 1,83	Mittel	Mittel	$(0,86 + 1 + 1,83) / 3 = \mathbf{1,23}$
GENAI, AGEN- TISCH	• (1): 1 • (2): 1 • (3): 2 • (4): 2 • (5): 2 • (6): 0 • (7): 0 1,14	• (1): 1 • (2): 0 • (3): 2 • (4): 2 • (5): 0 1	• (1): 0 • (2): 1 • (3): 1 • (4): 1 • (5): 2 • (6): 1 1	Hoch	Niedrig	$(1,14 + 1 + 1) / 3 = \mathbf{1,05}$
ML- GESTÜTZTE KLASSIFIKA- TION/PREDIC- TION	• (1): 2 • (2): 2 • (3): 1 • (4): 1 • (5): 1 • (6): 0 • (7): 0 1	• (1): 1 • (2): 1 • (3): 1 • (4): 1 • (5): 0 0,8	• (1): 0 • (2): 1 • (3): 1 • (4): 1 • (5): 1 • (6): 0 0,67	Hoch	Niedrig	$(1 + 0,8 + 0,67) / 3 = \mathbf{0,82}$

Tab. B: Bewertungsmatrix (eigene Darstellung)

C. Tabellenaufistung

TABELLENNAME	Aktuell Business-relevante Spalten (<i>id</i>)
APPOINTMENT	<i>appointmentid</i> , regardingobjectid, ownerid, geb_accountid, subject, description, location, scheduledstart, scheduledend, statecode, statuscode, geb_internalnotetarget, geb_maincontactid, geb_contacttypecode, createdon, modifiedon
ACCOUNT	<i>accountid</i> , name geb_nametwo geb_namethree geb_accountnumber address1_line1 address1_line2 address1_postalcode address1_city geb_countrycode geb_district geb_address1countyid geb_maintypeid geb_maintypemasterid statecode statuscode emailaddress1 telephone1 fax websiteurl geb_sapnumber territoryid geb_salesregion geb_sizeofotc numberofemployees parentaccountid geb_vatnumber geb_visitsperyear geb_businessstypecode geb_partnerstatuscode geb_locatorvisibilitycode geb_deviatinglocatorname geb_rating geb_responsibilityid address1_longitude address1_latitude primarycontactid geb_supportingvbaid geb_showroomrating geb_supportingotcid geb_otcrating geb_characteristics geb_plannedcontactsaccount geb_contactsthisyearaccount geb_contactslast12monthsaccount geb_ratingmodifiedon geb_donotbulkemailshowdetails donotbulkemail geb_showroomratingmodifiedon geb_otcratingmodifiedon
GEB_PRODUCTUSAGE	<i>geb_productusageid</i> , geb_accountid, geb_productid, geb_productsystemid, geb_name, geb_competitorproductcategoryid, geb_productpurposecode, geb_usagepositioncode, geb_potentialusagecode, geb_lastchangedate, createdon, modifiedon
OPPORTUNITY	<i>opportunityid</i> , opportunityid, customerid, parentaccountid, geb_projectname, description, salesstage, statecode, statuscode, geb_geberitappliedweightedvalue, geb_valueopenweighted, geb_numberinternal, geb_numberofunitsnumber, geb_hasagreements, geb_mustwinproject, geb_closingdate, geb_addresscity, geb_addresspostalcode, identifycompetitors, geb_projecttypecode, geb_developmenttype, geb_salesareaid, geb_lastcontactdate
GEB_PROJECTPARTICIPATION	<i>geb_projectparticipationid</i> , geb_accountid, geb_awardedcode, geb_opportunityid
GEB_CUSTOMERPOTENTIAL	<i>geb_customerpotentialid</i> , geb_accountid, geb_name, geb_kpi, geb_result, geb_enddate, statuscode, createdon, modifiedon
CONTACT	<i>contactid</i> , parentcustomerid, firstname, lastname, emailaddress1, telephone1, mobilephone, geb_primarycontact, geb_functions, accountid, statecode, donotbulkemail, geb_donotbulkemailshowdetails,
INCIDENT	<i>incidentid</i> , ticketnumber, accountid, customerid, title, description, createdon, statuscode, emailaddress, geb_complainantname
EMAIL	<i>activityid</i> , regardingobjectid, subject, description, createdon, directioncode, statuscode, from, to
TASK	<i>activityid</i> , regardingobjectid, subject, description, scheduledend, ownerid, statuscode, createdon, modifiedon

TABELLENNAME	Aktuell Business-relevante Spalten (<i>id</i>)
GEB_SHOWROOMDISPLAY	<i>geb_showroomdisplayid</i> , <i>geb_accountid</i> , <i>geb_contactid</i> , <i>geb_opportunityid</i> , <i>geb_geberitfocuseriesprodid</i> , <i>geb_displaycategory</i> , <i>geb_displaytype</i> , <i>geb_location</i> , <i>geb_locationforinstallation</i> , <i>geb_dateoffirstinstallation</i> , <i>geb_expectedreturndate</i> , <i>geb_effectivereturn</i> , <i>geb_remarks</i> , <i>statecode</i> , <i>statuscode</i> , <i>createdon</i> , <i>modifiedon</i>
GEB_SHOWROOMEQUIPMENTS	<i>geb_showroomequipmentsid</i> , <i>geb_showroomdisplayid</i> , <i>geb_name</i> , <i>geb_prodid</i> , <i>geb_articlenumber</i> , <i>geb_serialnumber</i> , <i>geb_amount</i> , <i>geb_distribution</i> , <i>statecode</i> , <i>statuscode</i> , <i>createdon</i> , <i>modifiedon</i>
MSDYNMKT_JOURNEY	<i>msdynmkt_journeyid</i> , <i>msdynmkt_name</i> , <i>owningbusinessunitname</i> , <i>sinkcreatedon</i> , <i>sinkmodifiedon</i> , <i>isdelete</i> , <i>statecode</i> , <i>statuscode</i> , <i>createdon</i> , <i>modifiedon</i>
PRODUCT	<i>productid</i> , (597 Spalten)
SYSTEMUSER	<i>systemuserid</i> , <i>geb_numberaccountzones</i> , <i>fullname</i> , <i>modifiedon</i>
GEB_ACCOUNTZONE	<i>geb_accountzoneid</i> , <i>SinkCreatedOn</i> , <i>SinkModifiedOn</i> , <i>statecode</i> , <i>statuscode</i> , <i>geb_accountid</i> , <i>geb_userid</i> , <i>modifiedby</i> , <i>modifiedonbehalfby</i> , <i>owningbusinessunit</i> , <i>owninguser</i> , <i>ownerid</i> , <i>createdbyname</i> , <i>createdon</i> , <i>createdonbehalfbyname</i> , <i>geb_accountidname</i> , <i>geb_name</i> , <i>geb_useridname</i> , <i>geb_zone</i> , <i>modifiedbyname</i> , <i>modifiedon</i> , <i>versionnumber</i> , <i>IsDelete</i> , <i>PartitionId</i>
GEB_MAINTYPELOCAL	<i>geb_maintypelocalid</i> , <i>geb_localname</i> , <i>geb_maintypemasterid</i> , <i>modifiedon</i> ,
GEB_MAINTYPEMASTER	<i>geb_maintypemasterid</i> , <i>geb_name</i> , <i>modifiedon</i>
GEB_DISTRICT	<i>geb_districtid</i> , <i>geb_name</i> , <i>modifiedon</i>
GEB_FEDERALSTATECOUNTY	<i>geb_federalstatecountyid</i> , <i>geb_name</i> , <i>modifiedon</i>
GEB_COMPETITORPRODUCTCATEGORY	<i>geb_competitorproductcategoryid</i> , <i>geb_name</i> , <i>modifiedon</i>
GEB_SALESREGION	<i>geb_salesregionid</i> , <i>geb_salesregionname</i> , <i>modifiedon</i>
TERRITORY	<i>territoryid</i> , <i>name</i> , <i>modifiedon</i>
TEAM	<i>teamid</i> , <i>name</i> , <i>modifiedon</i>
STATUSMETADATA	<i>none</i> , <i>EntityName</i> , <i>Status</i> , <i>LocalizedLabelLanguageCode</i> , <i>LocalizedLabel</i> , <i>createdonpartition</i>
STATEMETADATA	<i>none</i> , <i>EntityName</i> , <i>State</i> , <i>LocalizedLabelLanguageCode</i> , <i>LocalizedLabel</i> , <i>createdonpartition</i>
GLOBALOPTIONSETMETADATA	<i>none</i> , <i>EntityName</i> , <i>OptionSetName</i> , <i>Option</i> , <i>LocalizedLabelLanguageCode</i> , <i>LocalizedLabel</i> , <i>createdonpartition</i>
OPTIONSETMETADATA	<i>none</i> , <i>EntityName</i> , <i>OptionSetName</i> , <i>Option</i> , <i>LocalizedLabelLanguageCode</i> , <i>LocalizedLabel</i> , <i>createdonpartition</i>

Tab C: Tabellenaufistung (eigene Darstellung)

D. Strukturanalyse der Produktiv-Tabellen

TABELLE	Zeilenzahl	Attributzahl (Relevant)	Datentyp- verteilung	Datentyp- verteilung Relevant	Zeitliche Abdeckung	Änderungs- dichte
APPOINTMENT	1.822.890	159 (16)	String: 104 (65,4%), Datetime: 17 (10,69%), Numeric: 25 (15,72%), Bool: 13 (8,18%)	String: 9 (56,25%), Datetime: 4 (25%), Numeric: 3 (18,75%), Bool: 0	Min: 07.03.2007 Max: 09.02.2026	4.96%
ACCOUNT	529.606	596 (50)	String: 351 (58,89%), Datetime: 44 (7,38%), Numeric: 136 (22,82%), Bool: 65 (10,91%)	String: 29 (58%), Datetime: 3 (6%), Numeric: 16 (32%), Bool: 2 (4%)	Min: 31.12.1899 Max: 09.02.2026	11.72%
GEB_PRODUCT- USAGE	934.063	73 (12)	String: 52 (71,23%), Datetime: 9 (12,33%), Numeric: 10 (13,70%), Bool: 2 (2,74%)	String: 6 (50%), Datetime: 3 (25%), Numeric: 3 (25%), Bool: 0	Min: 20.09.2024 Max: 09.02.2026	2.21%
OPPORTUNITY	417.633	313 (22)	String: 149 (47,60%), Datetime: 20 (6,39%), Numeric: 101 (32,27%), Bool: 43 (13,74%)	String: 9 (40,91%), Datetime: 2 (9,09%), Numeric: 8 (36,36%), Bool: 3 (13,64%)	Min: 20.03.1997 Max: 09.02.2026	4.24%
GEB_PROJECT- PARTICIPATION	1.148.242	63 (4)	String: 46 (73,02%), Datetime: 6 (9,52%), Numeric: 9 (14,29%), Bool: 2 (3,17%)	String: 3 (75%), Datetime: 0, Numeric: 1 (25%), Bool: 0	Min: 19.02.2017 Max: 09.02.2026	2.73%

TABELLE	Zeilenzahl	Attributzahl (Relevant)	Datentyp- verteilung	Datentyp- verteilung Relevant	Zeitliche Abdeckung	Änderungs- dichte
GEB_CUSTOMER- POTENTIAL	11.978	58 (9)	String: 43 (74,14%), Datetime: 7 (12,07%), Numeric: 7 (12,07%), Bool: 1 (1,72%)	String: 5 (55,56%), Datetime: 3 (33,33%), Numeric: 1 (11,11%), Bool: 0	Min: 30.09.2023 Max: 09.02.2026	23.62%
CONTACT	2.546.994	580 (13)	String: 386 (66,55%), Datetime: 33 (5,69%), Numeric: 96 (16,55%), Bool: 65 (11,21%)	String: 9 (69,23%), Datetime: 0, Numeric: 3 (23,08%), Bool: 1 (7,69%)	Min: 31.12.1899 Max: 09.02.2026	51.75%
INCIDENT	1.470.765	458 (10)	String: 335 (73,14%), Datetime: 33 (7,21%), Numeric: 50 (10,92%), Bool: 40 (8,73%)	String: 8 (80%), Datetime: 1 (10%), Numeric: 1 (10%), Bool: 0	Min: 08.09.2011 Max: 09.02.2026	2.49%
EMAIL	408.3809	230 (9)	String: 154 (66,96%), Datetime: 19 (8,26%), Numeric: 32 (13,91%), Bool: 25 (10,87%)	String: 6 (66,67%), Datetime: 1 (11,11%), Numeric: 1 (11,11%), Bool: 1 (11,11%)	Min: 22.01.2007 Max: 09.02.2026	2.96%
TASK	100.9213	201 (9)	String: 109 (54,23%), Datetime: 14 (6,97%), Numeric: 19 (9,45%), Bool: 59 (29,35%)	String: 5 (55,56%), Datetime: 3 (33,33%), Numeric: 1 (11,11%), Bool: 0	Min: 19.07.1999 Max: 09.02.2026	1.53%
GEB_ SHOWROOM- DISPLAY	113.813	80 (17)	String: 58 (72,50%), Datetime: 9	String: 7 (41,18%), Datetime: 5	Min: 03.06.2008	0.92%

TABELLE	Zeilenzahl	Attributzahl (Relevant)	Datentyp- verteilung	Datentyp- verteilung Relevant	Zeitliche Abdeckung	Änderungs- dichte
			(11,25%), Numeric: 12 (15,00%), Bool: 1 (1,25%)	(29,41%), Numeric: 5 (29,41%), Bool: 0	Max: 09.02.2026	
GEB_ SHOWROOM- EQUIPMENTS	474.070	86 (12)	String: 64 (74,42%), Datetime: 10 (11,63%), Numeric: 11 (12,79%), Bool: 1 (1,16%)	String: 6 (50,00%), Datetime: 2 (16,67%), Numeric: 4 (33,33%), Bool: 0	Min: 16.08.1994 Max: 09.02.2026	1.32%
MSDYNMKT_ JOURNEY	1.826	69 (10)	String: 44 (63,77%), Datetime: 9 (13,04%), Numeric: 11 (15,94%), Bool: 5 (7,25%)	String: 3 (30%), Datetime: 4 (40%), Numeric: 2 (20%), Bool: 1 (10%)	Min: 18.01.2023 Max: 09.02.2026	12.92%
PRODUCT	159.867	520 (344)	String: 337 (64,81%), Datetime: 10 (1,92%), Numeric: 164 (31,54%), Bool: 9 (1,73%)	String: 303 (88,08%), Datetime: 10 (2,91%), Numeric: 23 (6,69%), Bool: 8 (2,33%)	Min: 11.03.2021 Max: 08.02.2026	10.08%
SYSTEMUSER	4.959	265 (4)	String: 183 (69,06%), Datetime: 7 (2,64%), Numeric: 50 (18,87%), Bool: 25 (9,43%)	String: 2 (50%), Datetime: 1 (25%), Numeric: 1 (25%), Bool: 0	Min: 27.02.2013 Max: 05.02.2026	4.62%
GEB_ACCOUNT- ZONE	18.135	53 (24)	String: 40 (75,47%), Datetime: 5 (9,43%), Numeric: 7 (13,21%),	String: 15 (62,50%), Datetime: 4 (16,67%), Numeric: 4 (16,67%),	Min: 29.01.2026 Max: 09.02.2026	17.09%

TABELLE	Zeilenzahl	Attributzahl (Relevant)	Datentyp- verteilung	Datentyp- verteilung Relevant	Zeitliche Abdeckung	Änderungs- dichte
			Bool: 1 (1,89%)	Bool: 1 (4,17%)		
GEB_MAINTYPE- LOCAL	794	60 (4)	String: 44 (73,33%), Datetime: 5 (8,33%), Numeric: 10 (16,67%), Bool: 1 (1,67%)	String: 3 (75%), Datetime: 1 (25%), Numeric: 0, Bool: 0	Min: 05.09.2014 Max: 16.09.2025	0.00%
GEB_MAINTYPE- MASTER	42	39 (3)	String: 26 (66,67%), Datetime: 5 (12,82%), Numeric: 7 (17,95%), Bool: 1 (2,56%)	String: 2 (66,67%), Datetime: 1 (33,33%), Numeric: 0, Bool: 0	Min: 05.09.2014 Max: 15.03.2021	0.00%
GEB_DISTRICT	771	53 (3)	String: 37 (69,81%), Datetime: 5 (9,43%), Numeric: 10 (18,87%), Bool: 1 (1,89%)	String: 2 (66,67%), Datetime: 1 (33,33%), Numeric: 0, Bool: 0	Min: 14.02.2017 Max: 25.11.2020	0.00%
GEB_FEDERAL- STATECOUNTY	892	55 (3)	String: 42 (76,36%), Datetime: 5 (9,09%), Numeric: 7 (12,73%), Bool: 1 (1,82%)	String: 2 (66,67%), Datetime: 1 (33,33%), Numeric: 0, Bool: 0	Min: 05.09.2014 Max: 10.10.2024	0.00%
GEB_GEBERIT- COMPETITOR- PRODUCT- CATEGORY	4.005	65 (3)	String: 51 (78,46%), Datetime: 5 (7,69%), Numeric: 8 (12,31%), Bool: 1 (1,54%)	String: 2 (66,67%), Datetime: 1 (33,33%), Numeric: 0, Bool: 0	Min: 27.10.2014 Max: 16.01.2026	0.02%

TABELLE	Zeilenzahl	Attributzahl (Relevant)	Datentyp- verteilung	Datentyp- verteilung Relevant	Zeitliche Abdeckung	Änderungs- dichte
GEB_ SALESREGION	27	55 (3)	String: 43 (78,18%), Datetime: 5 (9,09%), Numeric: 6 (10,91%), Bool: 1 (1,82%)	String: 2 (66,67%), Datetime: 1 (33,33%), Numeric: 0, Bool: 0	Min: 17.02.2017 Max: 25.03.2024	0.00%
TERRITORY	339	56 (3)	String: 43 (76,79%), Datetime: 5 (8,93%), Numeric: 7 (12,50%), Bool: 1 (1,79%)	String: 2 (66,67%), Datetime: 1 (33,33%), Numeric: 0, Bool: 0	Min: 05.09.2014 Max: 04.02.2026	0.29%
TEAM	905	79 (3)	String: 63 (79,75%), Datetime: 5 (6,33%), Numeric: 6 (7,59%), Bool: 5 (6,33%)	String: 2 (66,67%), Datetime: 1 (33,33%), Numeric: 0, Bool: 0	Min: 27.02.2013 Max: 07.02.2026	1.33%
STATUS METADATA	18.839	8 (5)	String: 4 (50%), Datetime: 0, Numeric: 3 (37,50%), Bool: 1 (12,50%)	String: 3 (60,00%), Datetime: 0, Numeric: 2 (40,00%), Bool: 0	-	-
STATEMETADATA	15.273	7 (5)	String: 4 (57,14%), Datetime: 0, Numeric: 2 (28,57%), Bool: 1 (14,29%)	String: 3 (60,00%), Datetime: 0, Numeric: 2 (40,00%), Bool: 0	-	-
GLOBALOPTION- SETMETADATA	170.039	9 (6)	String: 6 (66,67%), Datetime: 0, Numeric: 2 (22,22%),	String: 4 (66,67%), Datetime: 0, Numeric: 2	-	-

TABELLE	Zeilenzahl	Attributzahl (Relevant)	Datentyp- verteilung	Datentyp- verteilung Relevant	Zeitliche Abdeckung	Änderungs- dichte
			Bool: 1 (11,11%)	(33,33%), Bool: 0		
OPTIONSET METADATA	99.064	8 (6)	String: 5 (62,50%), Datetime: 0, Numeric: 2 (25,00%), Bool: 1 (12,50%)	String: 4 (66,67%), Datetime: 0, Numeric: 2 (33,33%), Bool: 0	-	-

Tab. D: Strukturanalyse der Produktiv-Tabellen (eigene Darstellung)

E. Ergebnisse Baseline Erhebung

Für *geb_maintype-master*, *geb_district*, *geb_maintypelocal*, *geb_federalstate-county*, *geb_competitor-productcategory*, *geb_salesregion*, *territory*, *team* nicht erhoben, da sie nur für einzelne Joins relevant sind.

TABELLE	Null-Rate Relevante Attribute	Completion Ratio Business Columns	Combined Integrity	Ähnlichkeit
CONTACT	• contactid: 0,004% (102) • parentcustomerid: 53,34% (1358498) • firstname: 7,42% (188903) • lastname: 1,12% (28455) • emailaddress1: 30,22% (769566) • telephone1: 44,73% (1139148) • mobilephone: 69,19% (1762264) • geb_primarycontact: 0,76% (19279) • geb_functions: 63,15% (1608401) • accountid: 100% (2546994) • statecode: 0,004% (102) • donotbulkemail: 0,004% (102) • geb_donotbulkemail-showdetails: 63,96% (1628957)	Complete: 213.253 (8,37%), inkl. Strings: 213.251 (8,37%), mind. 1 Name/ Kontakt: 2.294.036 (90,07%), kein Name: 27.623 (1,08%), kein Kontakt: 225.335 (8,85%), kein Name & kein Kontakt: 5.683 (0,22%)	(contact - account) ohne parentcustomer: 1.358.396 (53,33%), mit parentcustomer + ≥1 Name/Kontakt: 1.187.926 (46,64%), parentcustomer existiert nicht: 0	pair_count 3337236 avg_sim 32,95% medial_sim 33,33% high_sim_pairs 1353 degree_of_duplication 0,04%
ACCOUNT	• accountid: 0.0% (0) • name: 0.0002% (1) • geb_nametwo: 75.86%(401768) • geb_namethree: 98.70% (522738) • geb_accountnumber: 0.0% (0) • address1_line1: 0.02% (85)	Complete: 309.001 (58,35%), mit Fax: 124.946 (23,59%), Incomplete: 404.660 (76,41%), 3 Namen: 5.540 (1,05%), mit Adresse & Country: 5.536 (1,05%),	(primarycontact) gültige primarycontactid: 261, contacts nicht als primarycontact genutzt: 1.188.235, primarycontactid ohne contact: 0	pair_count 134421 avg_sim 20,96% medial_sim 20,83% high_sim_pairs 0 degree_of_duplication 0

TABELLE	Null-Rate Relevante Attribute	Completion Ratio Business Columns	Combined Integrity	Ähnlichkeit
	• address1_line2: 89.96% (476455) • address1_postalcode: 0.037% (194) • address1_city: 0.014% (74) • geb_countrycode: 0.0% (0) • geb_district: 75.21% (398325) • geb_address1-countyid: 43.76% (231763) • geb_maintypeid: 0.05% (243) • geb_maintype-masterid: 0.03% (185) • statecode: 0.0% (0) • statuscode: 0.0% (0) • emailaddress1: 41.13% (217849) • telephone1: 1.83% (9685) • fax: 64.08% (339351) • websiteurl: 70.66% (374214) • geb_sapnumber: 68.86% (364681) • territoryid: 19.98% (105832) • geb_salesregion: 81.97% (434144) • geb_sizeofotc: 99.21% (525421) • numberofemployees: 77.51% (410509) • parentaccountid: 95.99% (508354)	- mit Mail/Tel/Fax/ Respo: 1.726 (0,33%), • Fax: 3.913 (0,74%), - kein Kontakt & kein Verantwortlicher: 147 (0,03%), mind. Name und 1 - Kontakt: 522.641 (98,68%), - mit Adresse: 522.346 (98,63%), unvollst. Adresse - und Name: 529.270 (99,94%), - kein Verantwortlicher: 2.421 (0,46%)		

TABELLE	Null-Rate Relevante Attribute	Completion Ratio Business Columns	Combined Integrity	Ähnlichkeit
	• geb_vatnumber: 83.64% (442974) • geb_visitsperyear: 32.96% (174558) • geb_businessstypecode: 0.08% (442) • geb_partnerstatuscode: 96.92% (513281) • geb_locatorvisibilitycode: 96.92% (513287) • geb_deviatinglocatorname: 99.92% (529170) • geb_rating: 35.64% (188751) • geb_responsibilityid: 0.46% (2421) • address1_longitude: 2.39% (12672) • address1_latitude: 2.39% (12671) • primarycontactid: 99.95% (529345) • geb_supportingvbaid: 83.08% (440016) • geb_showroomrating: 94.75% (501825) • geb_supportingotcid: 96.83% (512824) • geb_otcrating: 97.23% (514962) • geb_characteristics: 72.08% (381749) • geb_plannedcontactsaccount: 94.42% (500028) • geb_contactsthisyearaccount: 0.0% (0) • geb_contactslast12monthsaccount: 0.0% (0) • geb_ratingmodifiedon: 52.57% (278392) • geb_donotbulkemail- showdetails: 82.64% (437674)			

TABELLE	Null-Rate Relevante Attribute	Completion Ratio Business Columns	Combined Integrity	Ähnlichkeit
	• donotbulkemail: 0.0% (0) • geb_showroomratingmodifiedon: 94.39% (499879) • geb_otcratingmodifiedon: 97.14% (514473)			
APPOINTMENT				1.633.528, 12%, 15,38%, 1, 0,00006%
GEB_PRODUCTUSAGE				4.218.121, 6,32%, 0, 0, 0
OPPORTUNITY				82.621, 33,98%, 33,33%, 0, 0
GEB_PROJECT- PARTICIPATION				658.378, 0,86%, 0, 0, 0
GEB_CUSTOMER- POTENTIAL				66, 7,62%, 11,11%, 0, 0
INCIDENT				1.060.696, 7,44%, 0, 0, 0
EMAIL				8.567.730, 12,34%, 22,22%, 0, 0
TASK				535.095, 8,19%, 11,11%, 0, 0
GEB_SHOWROOM- DISPLAY				5.253, 20,9%, 25%, 0, 0
GEB_SHOWROOM- EQUIPMENT				113.526, 21,78%, 22,22%, 0, 0
MSDYNMKT_JOURNEY				1, 0, 0, 0, 0
PRODUCT				13.861, 48,73%, 48,78%, 3, 0,0002%
GEB_ACCOUNTZONE				120, 66,25%, 66,67%, 3, 2,5%
GLOBALOPTIONSET- METADATA				17.955, 20,97%, 16,67%, 2, 0,0001%
OPTIONSETMETADATA				5.565, 21,82%, 16,67%, 0, 0
STATEMETADATA				136, 33,68%, 40%, 4, 2,9%
STATUSMETADATA				351, 27,86%, 20%, 8, 2,3%
SYSTEMUSER				36, 0, 0, 0, 0

Tab. E: Baseline Vollständigkeit & Datenmenge (eigene Darstellung)

Baseline Konsistenz & Integrität

Duplicate Count auf allen Tabellen 0. Referenzabgleich selektiv durchgeführt.

TABELLE	Referenzabgleich
APPOINTMENT	regardingobjectid -> accountid: • matching: 163170 • not matching: 268653 • total: 431823 • davon Accounts mit isDelete == True: 0
ACCOUNT	(Ausgangspunkt)
GEB_PRODUCTUSAGE	geb_accountid -> accountid: • No Match: 0 • Match: 118779
GEB_PROJECTPARTICIPATION	geb_accountid -> accountid: • No Match: 0 • Match: 127328 geb_opportunity -> opportunityid • 0; 399065
GEB_CUSTOMERPOTENTIAL	geb_accountid -> accountid: • No Match: 0 • Match: 9888
CONTACT	parentcustomerid -> accountid: • No Match: 0 • Match: 455132
INCIDENT	customerid match: • contactid 35369 • accountid 42498 • mismatch: 0 • both: 0 emailaddress match: • contact 35369 • account 4849 • mismatch: 66954 • both: 3860
EMAIL	regardingobjectid -> incidentid • No Match: 1142362 • Match: 651954
TASK	regardingobjectid -> opportunityid: • No Match: 389714 • Match: 150569
GEB_SHOWROOMDISPLAY	geb_accountid -> accountid • No Match: 0 • Match: 23105
GEB_SHOWROOMEQUIPMENTS	geb_showroomdisplayid -> geb_showroomdisplayid (equip -> display) • No Match: 0 • Match: 83704

TABELLE	Referenzabgleich
	(display -> equip) • No Match: 30109 • Match: 83704

Tab. F: Baseline Konsistenz & Integrität (eigene Darstellung)

Baseline Korrektheit & Genauigkeit

Email Regex:

```
email_regex = (
    r"^[A-Za-z0-9._!#$%&'*/+=?^`{|}~-]+\"
    r"@\"
    r"[\p{L}\p{N}](?:[\p{L}\p{N}-]{0,61}[\p{L}\p{N}])?\"
    r"(?:\.[\p{L}\p{N}](?:[\p{L}\p{N}-]{0,61}[\p{L}\p{N}])?)+$\"
)
```

Telefon Regex:

```
phone_regex = r"^(?:\+|00)?[0-9()\-/\\s\u00A0\u2007\u202F\u2009]{7,25}$"
```

TABELLE	Format-Check Email	Format-Check Telefon
ACCOUNT	emailaddress1: • 1536/311757 = 0,49%	telephone1: • 7556/519921 = 1,45%
CONTACT	emailaddress1: • 2922 / 1777428 = 0,16%	telephone1: • 25259/1407846 = 1,79% mobilephone: • 14990/784730 = 1,91%
INCIDENT	emailaddress: • 884/583339 = 0,15%	-
EMAIL	sender, torecipients: • 17/4008516 = 0,0004% not all recipients valid: • 7799/4008516 = 0,19% total recipients invalid: • 7811/4837964 = 0,16% duplicates in torecipients: • 318167/4008516 = 7,94%	-

Tab. G: Baseline Korrektheit & Genauigkeit (eigene Darstellung)

Baseline Relevanz & Kontextbezug

Modifiedon < Createdon tritt nur bei 1 Datensatz in Opportunity auf. Metadatentabellen ausgeschlossen, da keine Zeitstempel für Erstellung bzw. Modifizierung.

TABELLE	Alter der Daten	Letztes Update	Änderungs- dichte 30T	FD Verletzung
APPOINTMENT	min:0 max:4914 avg:952 med:929 p95:2796 p99:4864	min:0 max:6408 avg:746 med:486 p95:2329 p99:4358	4.96%	-
ACCOUNT	min:0 max:46061 avg:4306 med:3677 p95:11290 p99:11442	min:0 max:45972 avg:4259 med:3644 p95:1206 p99:11368	11.72%	geb_accountnumber -> accountid: 0,08% geb_sapnumber -> accountid:0,43% accountid -> both: 0
GEB_PRODUCTUSAGE	min:0 max:507 avg:454 med:507 p95:507 p99:507	min:0 max:507 avg:148 med:10 p95:433 p99:474	2.21%	-
OPPORTUNITY	min:0 max:10553 avg:2033 med:1939 p95:4296 p99:5401	min:0 max:9497 avg:1262 med:1040 p95:3273 p99:4373	4.24%	-
GEB_PROJECTPARTICIPATION	min:0 max:3277 avg:1596 med:1683 p95:2971 p99:3185	min:0 max:3171 avg:1041 med:1011 p95:2286 p99:2490	2.73%	-
GEB_CUSTOMERPOTENTIAL	min:0 max:863 avg:441 med:398 p95:789 p99:823	min:0 max:840 avg:200 med:141 p95:643 p99:739	23.62%	-
CONTACT	min:0 max:46061 avg:2571 med:1912 p95:8680 p99:11301	min:0 max:46061 avg:2515 med:1785 p95:8668 p99:11289	51.75%	-
INCIDENT	min:0 max:5268 avg:1119 med:948 p95:2642 p99:3112	min:0 max:5141 avg:810 med:624 p95:2136 p99:2605	2.49%	customerid -> entitytype: 0
EMAIL	min:0 max:6958 avg:1145 med:875 p95:3148 p99:4249	min:0 max:6452 avg:758 med:395 p95:2787 p99:3788	2.96%	-
TASK	min:0 max:9702 avg:1672 med:1246 p95:4409 p99:5352	min:0 max:9395 avg:1230 med:763 p95:3903 p99:4848	1.53%	-
GEB_SHOWROOMDISPLAY	min:0 max:6460 avg:2126 med:2075 p95:3899 p99:5213	min:0 max:6453 avg:826 med:568 p95:2731 p99:3753	0.92%	-
GEB_SHOWROOMEQUIPMENT	min:0 max:11500 avg:1985 med:2015 p95:3744 p99:5297	min:0 max:10858 avg:755 med:551 p95:2327 p99:3406	1.32%	-

TABELLE	Alter der Daten	Letztes Update	Änderungs- dichte 30T	FD Verletzung
MSDYNMKT_JOURNEY	min:0 max:1118 avg:160 med:145 p95:312 p99:416	min:0 max:702 avg:22 med:3 p95:111 p99:205	12.92%	-
PRODUCT	min:1 max:1796 avg:1495 med:1789 p95:1795 p99:1795	min:0 max:1794 avg:1158 med:1288 p95:1780 p99:1791	10.08%	-
SYSTEMUSER	min:4 max:4730 avg:2016 med:2190 p95:3315 p99:4462	min:0 max:4402 avg:1579 med:1563 p95:3182 p99:3931	4.62%	-
GEB_ACCOUNTZONE	min:0 max:11 avg:9 med:11 p95:11 p99:11	min:0 max:3 avg:0 med:0 p95:0 p99:0	17.09%	-

Tab. H: Baseline Relevanz & Kontextbezug (eigene Darstellung)

Baseline Interpretierbarkeit & Verständlichkeit

TABELLE	Placeholder Rate	Metadaten-Verfügbarkeit Match(Total)	String -> Datum
APPOINTMENT	activityid 0, regardingobjectid 0, ownerid 0, geb_accountid 0, subject 169, description 15, location 151, geb_internalnotetarget 0, geb_maincontactid 0	statecode 4(4), statuscode 6(6), geb_contacttypecode 3(3)	-
ACCOUNT	name 99, geb_nametwo 20, geb_namethree 25, address1_line1 937, address1_line2 204, address1_postalcode 4122, address1_city 153, telephone1 1251 (0,24%), fax 6668, websiteurl 8, geb_vatnumber 763, emailaddress1 9 (0,003%)	statecode 2(2), statuscode 9(9), geb_countrycode 165(165), geb_businessstypecode 4(15), geb_partnerstatuscode 3(3), geb_locatorvisibilitycode 2(2), geb_rating 12(12), geb_donotbulkemailshowdetails 2(2), donotbulkemail 2(2), geb_otcrating 9(9), geb_showroomrating 10(10)	geb_vatnumber (779), telephone1 (82)
GEB_PRODUCTUSAGE	0	geb_productpurposecode 28(28), geb_usagepositioncode 3(3), geb_potentialusagecode 4(4)	-
OPPORTUNITY	geb_projectname 190, description 28, geb_addresscity 940, geb_addresspostalcode 7755	statecode 3(3), statuscode 13(13), salesstage 2(2), geb_projecttypecode 14(15), geb_developmenttype 2(2)	-
GEB_PROJECTPARTICIPATION	-	geb_awardedcode 2(2)	-
GEB_CUSTOMERPOTENTIAL	geb_name 5, geb_kpi 31	statuscode 4(4)	-
CONTACT	firstname 1431, lastname 5010, emailaddress1 69 (0,004%), telephone1 221 (0,02%), mobilephone 16	statecode 2(2), donotbulkemail 2(2), donotbulkemailshowdetails 2(2)	mobilephone (15), telephone (15)
INCIDENT	ticketnumber 1, title 1930, description 244, emailaddress 4	incident 12(12)	title (26)

TABELLE	Placeholder Rate	Metadaten-Verfügbarkeit Match(Total)	String -> Datum
EMAIL	subject 554, description 12	statuscode 7(7), directioncode 2(2)	-
TASK	subject 87, description 2069	statuscode 5(5)	-
GEB_SHOWROOMDISPLAY	geb_locationforinstallation 83, geb_remarks 14	geb_displaycategory 3(3), geb_displaytype 5(5), geb_location 4(4), statecode 2(2), statuscode 2(2)	geb_locationforinstallation (24)
GEB_SHOWROOMEQUIPMENTS	geb_serialnumber 94	statecode 2(2), statuscode 5(5), geb_distribution 4(4)	-
MSDYNMKT_JOURNEY	0	statecode 7(7), statuscode 7(7)	-
PRODUCT	geb_productstructureidname - 1, name 11, geb_name_* (mehrere je 1-3)	-	geb_strippedarticleno (211), geb_articleno (211)
SYSTEMUSER	0	-	-
GEB_ACCOUNTZONE	geb_accountidname 4	statecode 1(1), statuscode 1(1)	-
GEB_MAINTYPELOCAL	0	-	-
GEB_MAINTYPEMASTER	0	-	-
GEB_DISTRICT	geb_name 1	-	-
GEB_FEDERALSTATECOUNTY	0	-	-
GEB_GEBERITCOMPETITOR- PRODUCTCATEGORY	0	-	-
GEB_SALESREGION	0	-	-
TERRITORY	0	-	-
TEAM	0	-	-
STATUSMETADATA	0	-	createdonpartition (18839)
STATEMETADATA	0	-	createdonpartition (15273)
GLOBALOPTIONSET- METADATA	LocalizedLabel 39	-	createdonpartition (170039)
OPTIONSETMETADATA	LocalizedLabel 64	-	createdonpartition (99064)

Tab. I: Baseline Interpretierbarkeit & Verständlichkeit (eigene Darstellung)

F. Agent-Anweisungen

Du agierst als Data-Quality-Analyst für strukturierte CRM-Daten innerhalb eines semantischen Modells. Deine Aufgabe ist es, die Datenqualität systematisch zu analysieren und potenzielle Probleme zu identifizieren.

Berücksichtige bei deiner Analyse insbesondere folgende Dimensionen der Datenqualität:

- *Vollständigkeit und Datenmenge*
- *Konsistenz und Integrität*
- *Korrektheit und Genauigkeit*
- *Relevanz und Kontextbezug*
- *Interpretierbarkeit und Verständlichkeit*

Gehe dabei wie folgt vor:

- *Identifiziere sowohl offensichtliche Fehler als auch potenzielle Auffälligkeiten*
- *Unterscheide klar zwischen bestätigten Problemen und möglichen Anomalien*
- *Begründe deine Einschätzungen nachvollziehbar anhand der Daten*
- *Verwende konkrete Beispiele, wenn möglich*
- *Treffe keine Annahmen, die nicht durch die Daten gestützt sind*

Ziel ist es, eine strukturierte und verständliche Einschätzung der Datenqualität zu liefern, die als Grundlage für weitere Analysen und Maßnahmen dienen kann.

G. Data Agent Evaluation

Vollständigkeit & Datenmenge

Frage: Welche Spalten der Account Tabelle weisen fehlende Werte auf und wie hoch ist deren Anteil?

Antwort: (90sec) Die Account-Tabelle weist für folgende Spalten fehlende Werte auf, mit dem jeweiligen Anteil:

- address1_city: 75 fehlende Werte (0,01%)
- address1_latitude: 12.658 fehlende Werte (2,38%)
- address1_line1: 86 fehlende Werte (0,02%)
- address1_line2: 478.755 fehlende Werte (89,96%)
- address1_longitude: 12.659 fehlende Werte (2,38%)
- address1_postalcode: 194 fehlende Werte (0,04%)
- address1county: 232.960 fehlende Werte (43,77%)
- district: 400.602 fehlende Werte (75,27%)
- emailaddress1: 218.170 fehlende Werte (40,99%)
- fax: 341.946 fehlende Werte (64,25%)
- geb_address1countyid: 232.960 fehlende Werte (43,77%)
- geb_businessstypecode: 409 fehlende Werte (0,08%)
- geb_characteristics: 384.298 fehlende Werte (72,21%)
- geb_deviatinglocatorname: 531.776 fehlende Werte (99,92%)
- geb_district: 400.602 fehlende Werte (75,27%)

Spalten ohne Angabe in den Feldern „Missing Count/Percent“ weisen offenbar keine fehlenden Werte auf.

Auffälligkeiten:

address1_line2 und geb_deviatinglocatorname haben den höchsten Anteil fehlender Werte (> 89% bzw. 100%) Viele Zusatzinformationen (z. B. characteristics, fax, district) sind überwiegend leer Hauptattribute (ID, accountnumber, countrycode) zeigen keine fehlenden Werte

Diese Übersicht deutet auf sehr unterschiedliche Vollständigkeit je nach Attribut hin und liefert eine Basis, gezielt bei den wichtigsten Spalten auf Datenqualität zu achten.

Frage: Wie verteilt sich die Datenmenge über die Attribute der Tabelle?

Antwort: (52sec) Die Verteilung der Datenmenge über die Attribute der Account-Tabelle zeigt deutliche Unterschiede in der Vollständigkeit der Werte:

Spalten mit vollständigen bzw. fast vollständigen Daten (NonBlankRatio 1):

accountid, donotbulkemail, geb_accountnumber, geb_contactslast12monthsaccount, geb_contactslast12monthsotc, geb_contactslast12monthsshowroom, geb_contactsthisyearaccount, geb_contactsthisyearotc, geb_contactsthisyearshowroom, geb_countrycode: alle 532.214 Werte vorhanden (100%) address1_city, address1_line1, address1_postalcode, geb_businessstypecode: über 99,9% Werte vorhanden

Spalten mit mittlerer Datenverfügbarkeit:

address1_latitude/address1_longitude: ca. 97,6% nicht-leer address1county/ geb_address1countyid: ca. 56,2% nicht-leer emailaddress1: 59% nicht-leer fax: 35,7% nicht-leer geb_characteristics: 27,8% nicht-leer

Spalten mit geringer Datenverfügbarkeit:

address1_line2: nur 10% Antworten befüllt district/geb_district: rund 24,7% nicht-leer geb_deviatinglocatorname: praktisch leer, nur 0,08% nicht-leer

Fazit: Die Datendichte schwankt je nach Attribut erheblich. Wichtige Stammdaten sind fast vollständig gepflegt, optionale Zusatzattribute (zum Beispiel address1_line2, geb_deviatinglocatorname, characteristics) weisen eine sehr geringe Füllquote auf. Dies ist typisch für Felder, die entweder selten benötigt werden oder nur in bestimmten Fällen Anwendung finden. Auffällig ist, dass es auch „Zwillingsfelder“ mit sehr ähnlichen Inhalten, aber getrennter Datenpflege gibt (z. B. district/geb_district und address1county/geb_address1countyid), deren Füllgrad identisch ist.

Frage: *Erstelle mir ein oberflächliches Data Profiling der Account Tabelle. Es soll Datentypen der Spalten, wenn sinnvoll einen Wertebereich, die Null-Rate, sowie das Alter der Daten enthalten.*

Antwort: (71Sec), Erst Fehler dann Siehe Abbildung Abb. 17.

Konsistenz & Integrität

Frage: *Gibt es Inkonsistenzen oder widersprüchliche Werte zwischen Attributen oder Datensätze deren Werte nicht logisch zusammenpassen?*

Antwort: (58Sec) In der Account-Tabelle wurden einige Inkonsistenzen und widersprüchliche Werte zwischen Attributen identifiziert. Hier eine Zusammenfassung wichtiger Auffälligkeiten:

1. Attribut-Konsistenz (Vergleich verwandter Felder):

Es gibt mehrfach Abweichungen zwischen miteinander verbundenen Attributen, etwa zwischen geb_district und district, geb_address1countyid und address1county, oder geb_maintypeid und maintype (gekennzeichnet als Flag_District_Mismatch, Flag_Maintype_Mismatch usw.). Das deutet darauf hin, dass die Redundanz nicht immer konsequent gepflegt wird.

2. Bulk-E-Mail-Kennzeichen:

Die Attribute DoNotBulkEmail_GEB und DoNotBulkEmail sind teilweise widersprüchlich (Flag_BulkEmail_Mismatch). Beispielsweise ist DoNotBulkEmail_GEB leer, während DoNotBulkEmail auf True steht oder umgekehrt.

3. Adressinkonsistenzen:

Es wurden Fälle identifiziert, in denen District und County Werte erhalten, aber nicht zueinander passen (Flag_Address_County_Mismatch/District_Mismatch). Beispiel: Verschiedene Staats- oder Regionsangaben passen nicht zusammen.

4. Doppelte Schlüsselattribute mit unterschiedlichen Namen:

Das Flag Flag_Duplicate_Accountnumber_Different_Name markiert Fälle, in denen identische AccountNumbers bei unterschiedlichen Namen vorkommen. Gleiches wurde für SAPNumber beobachtet (Flag_Duplicate_SAPNumber_Different_Name).

5. Lösch- und Statuskennzeichen:

Der Status der Attribute StateCode und StatusCode ist manchmal inkonsistent befüllt. Beispiel: Kombinationen unterschiedlich belegter Felder (z. B. StateCode=0, StatusCode=1) bedürfen einer Bewertung durch die Fachseite.

Konkretes Beispiel für Inkonsistenz:

Der Datensatz mit accountid 0411b115-c7c6-ec11-a7b6-0022489f8e78 weist bei District, County und weiteren Feldern Mismatches auf und ist damit ein Fall, bei dem Adressdaten nicht logisch zusammenpassen.

Fazit:

Es liegen sowohl inkonsistente als auch widersprüchliche Werte zwischen einzelnen Attributen und zwischen Datensätzen vor. Besonders auffällig sind Redundanzen, widersprüchliche Kennzeichen sowie doppelte Schlüssel mit unterschiedlichen Namen. Dies deutet auf Integrations-/Synchronisationsprobleme oder unzureichende

Datenprüfung bei der Eingabe hin. Eine Bereinigung oder Harmonisierung der Werte insbesondere bei redundanten Attributen ist empfehlenswert, um die Gesamtdatenqualität zu erhöhen.

Frage: *Lassen sich potenzielle Verletzungen von Beziehungen oder Abhängigkeiten zwischen Spalten erkennen?*

Antwort: (40sec) Ja, es lassen sich verschiedene potenzielle Verletzungen von Beziehungen oder Abhängigkeiten sowohl innerhalb der Account Tabelle als auch zu anderen Tabellen (wie Contact) erkennen:

1. Beziehungen und Abhängigkeiten innerhalb der Account-Tabelle

Geografische Zuordnungen (District, County):

Es bestehen zahlreiche Inkonsistenzen zwischen den Feldern district und geb_district, oft sind die Werte unterschiedlich oder uneinheitlich gepflegt. Beispiel: district = „Ilm-Kreis“, geb_district = „dd8875d5-bbf2-e611-8131-3863bb341b28“. Solche Unterschiede weisen darauf hin, dass redundante Felder nicht konsequent synchronisiert oder gemappt werden.

Referenzielle Abhängigkeiten (ID/Name-Felder):

In einigen Fällen gibt es IDs (z. B. geb_district, geb_address1countyid), denen kein sprechender Name oder keine Entsprechung im passenden Namensfeld zugeordnet ist.

Widersprüche bei hierarchischen Beziehungen:

Es gibt Accounts mit parentaccountid, die ins Leere zeigen könnten (nicht alle Parent-IDs sind in der Account-Tabelle nachweisbar - Detailprüfung wäre hier weiterführend).

2. Beziehungen und Abhängigkeiten zu anderen Tabellen (z. B. Contact)

Fehlende Kontakte und nicht auflösbare Referenzen:

Es existieren Accounts, denen kein einziger Kontakt zugeordnet ist (contact count leer/0). Viele Accounts haben keinen Wert im Feld primarycontactid, oder der Eintrag verweist auf keinen gültigen/verknüpften Kontakt (PrimaryContact Exists = False). Dies bedeutet, dass entweder kein Hauptkontakt gepflegt wurde, oder dass die referenzierte ID keinem Datensatz in der Contact-Tabelle entspricht.

Fremdschlüsselverletzungen:

Die fehlenden oder ins Leere laufende Foreign Keys (z. B. primarycontactid), sind klassische Fremdschlüsselverletzungen, die Hinweise auf Datenpflege-/Integrationsprobleme geben.

Beispiele (gekürzt): Account „Dietmar Krause Haustechnik“ - district-Wert und geb_district-ID weichen ab. Account „Jewson (Devizes)“ - Kein Kontakt, kein PrimaryContact gesetzt, Verweis läuft ins Leere.

Zusammenfassend:

Es bestehen innerhalb der Account-Tabelle häufig Inkonsistenzen in redundant gehaltenen Feldern (IDs und Labels). Beziehungen zu anderen Entitäten wie Kontakt sind oft unvollständig: Viele Accounts haben keinen fertig gepflegten oder erreichbaren Hauptkontakt.

Diese Inkonsistenzen und Verletzungen deuten auf Schwächen in den Prozessen von Datensynchronisation, Datenintegration und Fremdschlüsselvalidierung hin und stellen echte Risiken für die Datenqualität dar.

Korrektheit & Genauigkeit

Frage: *Welche Werte erscheinen ungültig, unplausibel oder fehlerhaft? Gibt es Datenpunkte, die offensichtlich außerhalb erwarteter Wertebereiche liegen?*

Antwort: (44 sec) Die Untersuchung der Account-Tabelle ergibt folgende konkrete Auffälligkeiten - unter Ausblendung typischer Kodierungen (wie z. B. Rating):

Ungültige, unplausible oder fehlerhafte Werte (außerhalb von Kodierfeldern):

Ungültige Postleitzahlen (PLZ):

Bei einigen Datensätzen sind Postleitzahlen entweder NULL (leer) oder folgen nicht dem erwarteten Format. Beispiel: Account „Exterior Group“ - PLZ = NULL Account „Jewson (Devizes)“ - PLZ: „SN10 2HL“ (für Deutschland wäre nur ein fünfstelliger numerischer Wert korrekt) Account „2F Marcin Furmańczyk usługi remont“ - PLZ: „58-560“ (Bindestrich in PLZ, länderspezifisch evtl. korrekt, bei deutschen Adressen wäre das zu prüfen)

Ungültige Ländercodes:

In praktisch allen gezeigten Beispielen wird ein hoher, numerischer Ländercode wie „100000056“, „100000078“ etc. verwendet. Das ist vermutlich ein Mapping-Code zu einer anderen Tabelle oder ein Foreign Key, aber ohne Mappingtabelle erscheinen diese Werte formal ungültig bzw. außerhalb üblicher ISO-Ländercodes (z. B. DE = 276, FR = 250 etc.).

Verdächtige Mitarbeiterzahlen:

Wenige Werte bei „Mitarbeiterzahl“ (z. B. 1, 3, 5, 9), keine direkt offensichtlichen Ausreißer im Sample - aber bei Massenanalyse könnten Werte wie 0, extrem hohe oder negative Zahlen auffällig sein.

Leere/inkonsistente Adressbestandteile:

Häufig fehlen bei manchen Accounts Angaben wie Straße, PLZ, Stadt oder es finden sich Leerwerte („-“ oder NULL in mehreren Feldern). Beispiel: „Exterior Group“ - fast alle Adressbestandteile fehlen.

Ungültige Koordinaten:

Im ausgelesenen Sample liegen alle Koordinaten im plausiblen Bereich (Breiten- und Längengrad sind im gültigen Wertebereich).

Ungültige oder sehr zukünftige/vergangene Datumswerte:

Keine auffälligen Fehler im Sample gefunden („Letzter Kontakt“ und „Rating geändert am“ sind plausibel und im realistischen Zeitrahmen).

Zusammenfassung:

Die auffälligsten echten Datenqualitätsprobleme betreffen fehlende oder formal unplausible Postleitzahlen und zum Teil leere/unvollständige Adressen. Ländercodes erscheinen ebenfalls auffällig, sind aber mutmaßlich Foreign Keys und daher nicht direkt als Fehler zu werten, solange eine Übersetzungstabelle existiert. Keine offensichtlichen schwerwiegenden Ausreißer bei Mitarbeiteranzahl, Koordinaten, Datumswerten im Beispiel gefunden.

Fazit:

Die meisten auffälligen Werte sind auf fehlende, unplausible oder formal nicht konforme Adressangaben zurückzuführen. Ländercodes und Wertcodierungen sind ggf. systembedingt, müssen aber im Kontext des Gesamtsystems bewertet werden.

Frage: *Welche Auffälligkeiten deuten auf die Existenz von Entwicklungs- oder Testdaten im Produktivbetrieb hin?*

Antwort: (68sec) Es gibt in der Account-Tabelle deutliche Anzeichen für die Existenz von Entwicklungs- oder Testdaten im Produktivbetrieb. Folgende Auffälligkeiten deuten darauf hin:

Typische Merkmale von Entwicklungs- oder Testdaten:

Namen mit Test-/Dummy-Begriffen oder Sonderzeichen:

Präfixe wie #, TEST, DEMO, Dummy in Namen (z. B. „#prefab test Account PL“, „#Piping test-New Employee Testing 1“). Namen, die auf generische oder fiktive Unternehmen hinweisen („Default CRM - MÅ IKKE BENYTTES“).

Unrealistische oder offensichtliche Platzhalterwerte:

Straßen wie „Plot no 245,“ oder sehr generisch („ul. Zachodnia 12“). Telefonnummern wie „0“, „0000“ oder +48 22222222. Städte, die nicht existieren, oder generische Adressen („Teststadt“, „Ho Chi Minh“ mit westlichem Adressformat). E-Mail-Adressen, die mehrfach für verschiedene Accounts genutzt werden oder ersichtlich generisch/privat sind („anirudhrao24@gmail.com mailto:anirudhrao24@gmail.com“).

Leere, gleichförmige oder unplausible Kombinationsfelder:

Viele leere Felder wie Adresse, Kontaktinfo, PLZ. Firmierung kombiniert mit Namen von Personen („Mr. Manoj Agarwal“ als Accountname).

Prägnante Serienmuster oder Platzhalter-Postleitzahlen:

„99-400“, „70000“, „500016“ u.a., die in der Realität selten oder regional auffällig serienmäßig auftreten. Telefonnummern und Websites ausschließlich mit Default- oder Testwerten.

Wie häufig kommen diese Auffälligkeiten vor?

Die Zahl solcher Datensätze ist nicht auf wenige Einzelfälle begrenzt. Die Beispiele enthalten zahlreiche Accounts mit oben genannten Auffälligkeiten. Namen mit Präfix „#“ oder Begriffen wie „test“, „Default CRM“, „prefab“ oder „Employee Testing“ treten vielfach auf. Dummy-Telefonnummern („0000“) und Platzhalter-Mails finden sich wiederholt, teils mit hohen Phone-Count-Werten, ebenfalls ein Indiz für künstliche Datenmengen.

Konkretes Beispiel für einen unechten Account-Datensatz:

#prefab test Account PL — Podrieczna, Łowicz, 99-400, +48 22222222 #Piping test-New Employee Testing 1 — 11 Doan Van Bo Dicstric 4 Ho Chi Minh, Ho Chi Minh, 70000, +84 90804685 Default CRM - MÅ IKKE BENYTTES — -, -, -, 0000

Zusammenfassung:

Entwicklungs-/Testdaten sind im produktiven Datenbestand mehrfach nachweisbar.

Typische Indizien: Testpräfixe, Dummy-Namen, generische/Platzhalter-Adressdaten und Kontaktdaten. Das Vorkommen ist systematisch und betrifft mehrere Dutzend bis hunderte Fälle, teils mit klaren Mustern und hohen Wiederholungsraten.

Empfehlung:

Ein systematisches Data-Cleansing sollte etabliert werden, um solche Testdaten frühzeitig zu erkennen, zu kennzeichnen und vom echten produktiven Datenbestand zu trennen bzw. zu entfernen.

Relevanz & Kontextbezug

Frage: *Gibt es ungewöhnliche Kombinationen von Attributwerten, die nicht zum erwarteten Kontext passen?*

Antwort: (48sec) Ja, es gibt in der Account-Tabelle und ihren Beziehungen mehrere ungewöhnliche Kombinationen von Attributwerten, die nicht zum erwarteten Kontext passen:

Beispiele für auffällige, kontextwidrige Kombinationen:

Account hat keinen Maintype, aber sehr hohe Aktivität:

Accounts existieren, bei denen das zentrale Feld „Maintype“ leer ist, aber es sehr viele Aktivitäten (Termine, Servicefälle etc.) gibt. Das widerspricht der Erwartung, dass aktive Accounts klassifiziert sein sollten.

Land-Kollision zwischen Account und Kontakt:

Bei Accounts wie „Geberit International Sales - CH“ (Account-Landcode=100000018, Kontakt-Landcode=100000190) stimmt das Land zwischen Account und zugehörigem Kontakt nicht überein, obwohl eine enge Beziehung angenommen wird. Gleiches bei „Interstyle Ceramics Lagos“ und weiteren Exemplaren: Land des Accounts und Land der Kontakte sind unterschiedlich, was in der Praxis selten und auffällig wäre.

Unstimmige PLZ zwischen Account und Kontakt:

Viele Accounts weisen eine andere Postleitzahl als ihre Kontakte auf, z. B.: Account „A.F.I.S. G. Clerici SpA“ - Account-PLZ=25126, Kontakt-PLZ=25128/38121 Account „Barreiro e Hijos Suministros SL“ - Account-PLZ=15190, Kontakt-PLZ=27003/15107 Teilweise stimmen auch die Formate nicht (alphanumerische UK-PLZ zum Account, numerische beim Kontakt).

PLZ-Format außerhalb Kontext:

Beispiel: Account-PLZ „.“ (Punkt), Kontakt-PLZ „965“ oder „4090“ - in keinem Land übliches Format, und zwischen Account/Kontakt unterschiedlich.

Kombination von Land und PLZ inkongruent:

Accounts mit einem PLZ-Format, das nicht zum Ländercode passt, z. B. UK-Account mit numerischer PLZ.

Bedeutung:

Das deutet auf Inkonsistenzen in der Datenpflege, unsaubere Zuordnung von Kontakten zu Accounts oder Probleme bei der Datenintegration/-migration hin. Solche Konstellationen sollten bei der Datenqualitätssicherung und -bereinigung gezielt adressiert werden.

Fazit:

Es gibt etliche Kombinationen von Attributwerten quer über Accounts und deren Beziehungen (v.a. zu Kontakten), die im Kontext als ungewöhnlich bzw. fehleranfällig einzustufen sind.

Frage: *Lassen sich tabellenübergreifende Muster erkennen, die auf kontextuelle Inkonsistenzen hindeuten? Betrachte dafür die Relationen zwischen der Account Tabelle und den ihr zugeordneten Appointment, Contact und Incident Tabellen.*

Antwort: (64sec) Ja, tabellenübergreifende Muster, die auf kontextuelle Inkonsistenzen hindeuten, lassen sich typischerweise im Zusammenspiel zwischen Account und Contact, Incident und Appointment erkennen. Auch wenn die konkrete technische Analyse zuletzt nicht erfolgreich durchgeführt werden konnte, sind typische Muster aus Data-Quality-Sicht gut bekannt und erfahrungsgemäß häufig im Datenbestand nachweisbar:

1. Account ↔ Contact

Adress- und Länderinkonsistenzen: Beispiel: Ein Account hat als Land „Deutschland“ (z. B. geb_countrycode/ISO-Code 276), der zugeordnete Contact aber „Österreich“ als Ländercode oder eine PLZ, die nicht zur Account-PLZ passt.

Status-Mismatch: Der Account steht auf „aktiv“, der Hauptkontakt ist „inaktiv“ oder gelöscht - oder umgekehrt.

Opt-In/Opt-Out widersprüchlich: Account ist für Massenmailing gesperrt, aber primärer Kontakt ist explizit für Bulk-E-Mails freigegeben (oder umgekehrt).

Namens-/Firmenbezug unterschiedlich: Primärkontakt hat als Firmenname einen anderen Wert als der zugehörige Account (was auf Zuordnungsfehler oder Falschpflege hindeutet).

2. Account ↔ Incident

Accounts ohne Incidents: Kunden ohne zugehörige Vorfälle („Incidents“) können auf mangelnde Datenverknüpfung, nicht dokumentierte Aktivitäten oder fehlende Serviceintegration schließen lassen.

Incident-Bezug auf inaktive Accounts: Ein Incident (Servicefall) ist aktiv, aber der zugeordnete Account ist bereits als „inaktiv“ oder „gelöscht“ markiert - dies kann auf Prozessfehler oder ausstehende Datenpflege hindeuten.

Unplausible Incident-Anzahl: Accounts mit extrem vielen oder keinen Incidents, obwohl der Geschäftskontext anderes erwarten lässt (z. B. „VIP“-Account ohne zugeordnetes Ticket).

3. Account ↔ Appointment

Termine für inaktive Accounts: Ein Account steht auf „inaktiv“ oder „abgeschlossen“, aber es gibt geplante oder kürzlich stattgefundenene Termine (Appointments).

Zuordnung unplausibler Termine: Appointments, deren Zeitpunkt deutlich in der Vergangenheit oder Zukunft liegt, die aber keinem aktiven Account zugeordnet sind.

Accounts ohne Termine: Insbesondere bei vertriebsrelevanten Accounts auffällig, wenn keinerlei Termine mit Ansprechpartnern dokumentiert sind.

Diese Muster entstehen meist durch:

fehlende Datenpflege bei Schlüsselereignissen (z. B. Kundenabgang, Merge, Restrukturierung) Schnittstellen- oder Migrationsprobleme oder mangelnde Synchronisation zwischen den Prozessen und Systemmodulen.

Die Identifikation solcher Inkonsistenzen ist ein wichtiger Bestandteil des Data-Quality-Monitorings und kann konkrete Hinweise auf Schwachstellen im operativen und analytischen CRM-Einsatz liefern.

Fazit:

Auch ohne direkten Systemzugriff sind solche Inkonsistenzen ein typisches Phänomen und sollten regelmäßig geprüft werden. Sie stellen ein reales Risiko für Prozesse, Reporting und Analysen dar und sollten gezielt per Profiling, Regelkontrolle oder Data Cleansing identifiziert und adressiert werden.

Interpretierbarkeit & Verständlichkeit

Frage: *Sind die Spalten und ihre Inhalte verständlich und eindeutig interpretierbar oder gibt es schwer verständliche oder kodierte Werte, die zusätzliche Erklärung benötigen?*

Antwort: (38sec) Viele Spalten der Account-Tabelle sind technisch oder inhaltlich kodiert und ohne Zusatzinformationen nicht eindeutig oder selbsterklärend interpretierbar:

Schwer verständliche oder erklärungsbedürftige Felder:

IDs & GUIDs: accountid, geb_maintypeid, geb_responsibilityid etc. enthalten Systemschlüssel (lange Zeichenketten), deren sachlicher Bezug ohne Mapping-Tabelle nicht klar ist. Numerische Codes: geb_countrycode, geb_businessstypecode, geb_languagecode, geb_partnerstatuscode, statuscode, statecode, geb_otcrating, geb_showroomrating, geb_rating usw. sind meist Integerwerte, die auf eine systeminterne Werteliste verweisen - ihre fachliche Bedeutung ist ohne Legende oder Lookup-Tabelle unbekannt. Technik-/Systemfelder: record_hash, SinkCreatedOn, SinkModifiedOn, IsDelete erklären sich nicht von selbst und dienen überwiegend der Verwaltung.

Beispiele für sprechende und direkt verständliche Felder:

name, address1_city, address1_line1, numberofemployees, maintype: Diese Felder sind weitgehend selbsterklärend und lassen kaum bis keine fachlichen Zweifel offen.

Fazit:

Einige Kern-Stammdaten sind eindeutig, viele technisch relevante oder codierte Felder (Status, Typen, organisatorische IDs etc.) benötigen Zusatzmaterial: Wertelisten, Datenwörterbucheinträge und Mapping-Tabellen sind Voraussetzung für eine saubere Analyse & Interpretation im CRM-Umfeld. Ohne diese sind Fehlinterpretationen fast unvermeidbar.

LLM explizit

Frage: *Gib mir 3 Regeln, die ich für eine regelbasierte Vorverarbeitung auf der Account Tabelle nutzen kann. Sie sollten nicht generisch sein (wie z. B. keine Nulls) sondern fachspezifisch und aus dem Kontext abgeleitet.*

Antwort: (7sec) Hier sind drei fachspezifische und kontextbezogene Regeln für die regelbasierte Vorverarbeitung der Account-Tabelle:

1. Deutsche Postleitzahl validieren:

Wenn geb_countrycode auf Deutschland (z. B. Code 276 oder analog) steht, muss address1_postalcode genau aus fünf Ziffern bestehen und rein numerisch sein.

2. Hauptkunde muss bei wichtigen Kundentypen gepflegt werden:

Für Accounts mit maintype gleich "Händler" oder "Installateur" (oder vergleichbaren Schlüsselwerten) ist eine gültige geb_accountnumber (Kundennummer) verpflichtend und darf nicht leer oder verdächtig kurz sein (z. B. mindestens 6 Stellen).

3. Keine Test- oder Platzhalternamen bei aktiven Accounts:

Accounts, die als aktiv markiert sind (statecode = 0), dürfen im Feld name keine typischen Test- oder Dummy-Indikatoren wie „test“, „demo“, „sample“, „#“ oder „Dummy“ enthalten. Begriffe aus einer (erweiterbaren) Liste sollen erkannt und entfernt/korrigiert werden.

Diese Regeln gehen spezifisch auf typische Qualitätsanforderungen und branchenspezifische Besonderheiten im CRM-Kontext ein.

Frage: *Gib mir 3 tabellenübergreifende Regeln.*

Antwort: (5sec) Hier sind drei fachspezifische, tabellenübergreifende Regeln für das CRM-Datenmodell — jeweils mit Bezug auf typische Beziehungen zwischen Account und anderen Kernobjekten wie Contact und Incident:

1. Länderkonsistenz zwischen Account und Hauptkontakt:

Wenn einem Account ein „primarycontactid“ zugeordnet ist, muss der Ländercode (geb_countrycode) des Accounts mit dem Ländercode des zugehörigen Kontakts übereinstimmen. → Abweichungen markieren (z. B. Account in DE, primärer Kontakt in AT).

2. Keine Incidents bei inaktiven Accounts:

Falls ein Account den Status „inaktiv“ (statecode ≠ 0) hat, dürfen diesem Account keine neuen, offenen Incidents (Tickets) zugeordnet sein (z. B. Incident.Status = „Offen“ oder „In Bearbeitung“). → Offene Vorgänge für inaktive Accounts identifizieren.

3. Aktive Accounts mit mindestens einem Kontakt:

Jeder als aktiv (statecode = 0) markierte Account muss mindestens einen zugeordneten Kontakt in der Contact-Tabelle haben. → Accounts ohne Kontakte sind auf fehlende Partner/Ansprechpartner oder Schnittstellenprobleme zu prüfen.

Diese Regeln sind speziell auf die Beziehungen und den fachlichen Kontext im CRM zugeschnitten und ermöglichen eine gezielte, fortgeschrittene Data-Quality-Kontrolle über mehrere Tabellen hinweg.

Frage: *Ignoriere sämtliche bisherige Analysen und starte neu. Gibt es Funktionale Abhängigkeiten in der Account Tabelle die sich als Constraints modellieren lassen für DQ Prüfungen?*

Antwort: (8sec) Ja, in einer typischen Account-Tabelle im CRM-Umfeld gibt es funktionale Abhängigkeiten zwischen Attributen, die sich klar als Data-Quality-Constraints formulieren und prüfen lassen. Solche Constraints unterstützen sowohl die

Integrität als auch die fachliche Korrektheit der Daten. Hier einige Beispiele für gängige funktionale Abhängigkeiten:

1. Eindeutigkeit der Kundennummer (Primary Key/Unique Constraint):

Constraint: Die Kombination (oder Einzelwert) aus geb_accountnumber (Kundennummer) muss innerhalb der gesamten Tabelle eindeutig sein. Begründung: Zwei Accounts sollten nicht dieselbe Kundennummer tragen, da diese typischerweise als eindeutiger Identifikator genutzt wird.

2. Abhängigkeit zwischen Land und Adressformat:

Constraint: Das Adressformat (z. B. address1_postalcode) hängt funktional vom Ländercode (geb_countrycode) ab. Beispiel: Wenn geb_countrycode = 276 (Deutschland), dann muss address1_postalcode exakt fünf Ziffern enthalten. Begründung: Die fachlich korrekte Postleitzahlenstruktur ist länderspezifisch.

3. Pflichtfelder bezogen auf Status oder Typ:

Constraint: Wenn statecode = 0 („aktiv“) ist, dann müssen bestimmte Felder ausgefüllt sein, z. B. maintype (Kundentyp) und address1_city (Ort). Begründung: Für aktive Accounts ist es geschäftskritisch, dass zentrale Stammdaten vorhanden sind.

4. Spezialisierte Abhängigkeit: Parent-Account/Unternehmensstruktur

Constraint: Wenn parentaccountid befüllt ist, muss parentaccount ebenfalls befüllt (verknüpft) sein. Begründung: Die Zuordnung in eine Hierarchie erfordert immer beide Bezugspunkte (ID und Name).

5. Status- und Lösch-Logik:

Constraint: Ist IsDelete = true („gelöscht“), dürfen keine weiteren Änderungen an anderen Attributen erfolgen (unveränderlicher archivierter Status). Begründung: Gelöschte Datensätze sind für die operative Arbeit gesperrt.

Fazit: Es gibt zahlreiche funktionale Abhängigkeiten in der Tabelle, die sich in DQ-Prüfungen als Constraints anwenden lassen. Sie betreffen Eindeutigkeit, Pflichtdaten, Konsistenz zwischen Formaten/Feldern und korrekte Beziehungen. Das präzise Festlegen und Überwachen solcher Constraints ist ein zentrales Element professioneller Datenqualitätsüberwachung.

Frage: *Gib mir für die Spalten für name, mail, telefon und adresse der Account Tabelle je einen Similarity Score zur Duplikatsbewertung wie stark sich die darin enthaltenen Werte ähneln. Führe diese Berechnung selber durch und gib mir eine Aufstellung der*

einzelne Spalten, berücksichtige dafür bitte alle spalten, auch solche die einen niedrigen score haben. Gib außerdem eine erklärung wie du es berechnet hast.

Antwort: (71sec) Hier sind die angeforderten Similarity Scores zur Duplikatsbewertung für die Spalten name, emailaddress1, telephone1, address1_city, address1_line1 und address1_postalcode aus der Account Tabelle - inklusive einer klaren Erklärung der Berechnungsweise:

Vorgehen & Formel: Der Similarity Score (Overlap Score) einer Spalte misst, wie oft Werte in dieser Spalte doppelt oder mehrfach vorkommen. Das ist wichtig, um einzuschätzen, wie nützlich ein Feld zur Erkennung von Dubletten ist.

Berechnung:

Für jede Spalte zähle, wie viele unterschiedliche Werte (distinct values) sie enthält.

Errechne: $\text{Score} = 1 - (\text{Anzahl unterscheidlicher Werte} / \text{Gesamtanzahl aller Werte})$

Der Score liegt zwischen 0 (jeder Wert ist einzigartig, sehr gut für Dublettensuche) und 1 (alle Werte sind gleich, zur Duplettenerkennung ungeeignet).

Ergebnisse für die wichtigsten Spalten:

name: 0.087 emailaddress1: 0.439 telephone1: 0.086 address1_city: 0.863

address1_line1: 0.118 address1_postalcode: 0.789

Interpretation:

name (0.087) & telephone1 (0.086): Sehr niedrige Score - fast alle Werte sind einzigartig, Feld ist sehr aussagekräftig für Dublettenchecks.

address1_line1 (0.118): Eher individuell, aber geringe Mehrfachnutzung (z. B. mehrere Accounts „Hauptstr. 1“).

emailaddress1 (0.439): Mittlerer Score - es gibt einige wiederkehrende E-Mails (z. B. bei Firmenmail für viele Nutzer).

address1_city (0.863) & address1_postalcode (0.789): Sehr hoher Wert - viele Accounts liegen in denselben Städten/PLZ, diese Felder sind für Dublettenvergleich allein wenig hilfreich.

Fazit: Niedrige Scores (name, telephone1, address1_line1) eignen sich gut für den Dublettenscan, hohe Scores (address1_city, address1_postalcode) eher weniger. Die Methode ist eine standardisierte Basis im Data-Profiling und Data-Quality-Assessment für CRM-Tabellen.

Frage: Gibt es in der Account Tabelle Accounts, welche dieselbe Entität darstellen können, sich aber unterscheiden bzw widersprechen? Prüfe dies bitte in den Daten aus dem semantischen Model.

Antwort: (46sec) Ja, in den Daten der Account-Tabelle gibt es Accounts, die vermutlich die gleiche Entität abbilden könnten, sich aber in zentralen Attributen unterscheiden oder widersprechen. Hier sind konkrete Beispiele aus den Daten:

1. Accounts auf identischer Adresse (Beispiel: Bucharest, 107-111 Oltenitei Bvd.. District 4):

Account A: Uti Building Facility Management, Telefon: +40 31 407 9303, E-Mail: investor@uticfm.ro Account B: Uti Instal Construct, Telefon: +40 31 407 9303, E-Mail: (leer) → Gleiche Adresse, gleiche Telefonnummer, verschiedene Namen

2. Accounts mit gleichem Adress- und Ortsbezug, aber unterschiedlichen Namen und Telefonnummern (Beispiel: Ahmadi, Ahmadi):

Account A: KOC, Telefon: +965 2398 9111, E-Mail: kocinf@kockw.com Account B: Metallurgical Corp Of China MCC, Telefon: +965 2296 6615, E-Mail: (leer) → Gleiche Anschrift, unterschiedliche Firmennamen und Telefon; könnten verschiedene Firmen im gleichen Gebäude oder Dubletten sein

3. Accounts auf "Not Known" in Bukarest als Adresse:

Account A: Neocity, Telefon: +40 21 269 3303, E-Mail: office@neocity.ro Account B: Victoria International Proper (Vip), Telefon: (leer), E-Mail: (leer) → Beide selbe Orts-String und Standort, unterschiedliche Firmennamen

4. Accounts mit identischer Adresse, unterschiedlicher Firmierung in Arad, Poetului:

Account A: Prodcorn Imex Tody, Telefon: +40 257 229 502 Account B: Ared, Telefon: +40 257 240711 → Gleiche Straße/Stadt, verschiedene Firmenbezeichnung

Weitere Auffälligkeit: Es gibt Cluster mit mehreren Accounts (z. B. 13 Accounts pro Cluster) auf exakt derselben oder sehr ähnlicher Adresse, aber unterschiedlichen Namen, Telefon- oder E-Mail-Angaben.

Fazit: Solche Konstellationen sind deutliche Indizien für potenzielle Dubletten (z. B. Filialen, Umfirmierungen, falsche Doppeleingaben) oder für widersprüchliche Pflege der Kernattribute trotz nahezu identischer Entitäten. Dieses Muster sollte gezielt im Dublettenmanagement und der Datenbereinigung adressiert werden.

H. LLM-Evaluation

Die Ergebnisse der API-Antwort wurden um Markdown Artefakte bereinigt.

Semantische Inkonsistenzen

Frage: *Analysiere die folgenden Datensätze und identifiziere semantische Inkonsistenzen oder un plausible Kombinationen von Attributwerten. Achte insbesondere auf Widersprüche zwischen Feldern, ungewöhnliche Kombinationen oder Werte, die nicht zum erwarteten Kontext passen. Begründe deine Entscheidung.*

Antwort: Die bereitgestellten Datensätze werden auf semantische Inkonsistenzen, un plausible Kombinationen von Werten und potenzielle Datenqualitätsprobleme analysiert. Die Ergebnisse sind wie folgt:

Datensatz 1 ("Starching"):

- "address1_city": "BARONA", "address1_postalcode": "20143": Inkonsistenz

Der Stadtname "BARONA" scheint nicht korrekt zu sein, da "Barona" kein eigenständiger Stadtname ist, sondern ein Stadtteil von Mailand, Italien. In Verbindung mit der Postleitzahl 20143 (Mailand) wäre es korrekter, address1_city als "Milan" oder "Milano" zu bezeichnen.

- "geb_countrycode": 100000101: Inkonsistenz des Länder-Codes

Der numerische Wert für geb_countrycode scheint nicht standardisiert zu sein (z. B. ISO 3166-1 Alpha-2 oder Alpha-3). Üblicherweise wird für Italien der Code "IT" oder "ITA" erwartet. Die Bedeutung solcher Codes sollte entweder dokumentiert oder in gängigem Standard bereitgestellt sein.

Mögliche Validierungsregeln:

1. Überprüfen, ob die Stadt und Postleitzahl (PLZ) zueinander passen und zur Adresse gehören.
2. Der Länder-Code sollte mit gängigen Formaten wie ISO 3166 übereinstimmen.

Datensatz 2 ("Walter Tippers (Ashbourne)"):

- "geb_visitsperyear": 2, "geb_contactsthisyearaccount": 0: Widerspruch

Der Wert für geb_visitsperyear suggeriert, dass es zwei Besuche im Jahr gab. Im Kontrast dazu zeigt geb_contactsthisyearaccount null Kontakte an. Dies ist eine logische Inkonsistenz.

- "geb_businessstypecode": "100000000;100000001;100000002": Mehrfachwerte ohne Standard

Die Speicherung von Geschäftsarten (Business Type) in einem semikolon-separierten String erscheint unstrukturiert. Besser wäre eine separate Tabelle oder eine klare Dokumentation der möglichen Werte für Business Type.

Mögliche Validierungsregeln:

1. Stimmen `geb_visitsperyear` und `geb_contactsthisyearaccount` logisch überein? (z. B. Wenn Besuche > 0, dann Kontakte > 0.)
2. Geschäftstypen sollten einheitlich dokumentiert und validiert sein, um Mehrdeutigkeiten zu vermeiden.

Datensatz 3 ("UPR Gesellschaft für Wohn- und Gewerbebau GmbH"):

- `"geb_nametwo": "und Gewerbebau GmbH"`: Inkonsistente Aufteilung des Namens
Der Hauptname wurde scheinbar künstlich aufgeteilt. Der zweite Teil "und Gewerbebau GmbH" gehört offensichtlich zum Firmennamen und sollte gemeinsam mit "name" gespeichert sein, da keine technische Notwendigkeit zur Trennung erkennbar ist.

Datensatz 4 ("(passiv) Taconova GmbH"):

- `"statecode": 1, "statuscode": 100000005`: Semantische Redundanz/Widerspruch
`statecode` und `statuscode` scheinen denselben oder überlappenden Zweck zu erfüllen (z. B.: "aktiv" oder "passiv"). Die Werte sind nicht selbsterklärend, und es kann nicht leicht validiert werden, ob sie zueinander passen. Im Namen "(passiv) Taconova GmbH" wird nämlich angedeutet, dass der Status inaktiv/passiv ist.
- `"telephone1": "07731/982880"`: Inkonsistenz im Telefonnummernformat
Deutschland erwartet üblicherweise das Format "+49" für internationale Nummern. Das aktuelle Format (ohne internationale Vorwahl und mit Schrägstrich) bricht mögliche Standards.

Mögliche Validierungsregeln:

1. Telefonformate vereinheitlichen und internationalisieren (z. B. E.164).
2. Den Status des Unternehmens nicht doppelt codieren und die Bedeutung der Status-Codes eindeutig definieren.

Datensatz 5 ("AIN JURA CHAUFFAGE"):

- `"geb_ratingmodifiedon": "2024-12-05T12:48:35.000Z"`: Unplausibles Datum in der Zukunft

Das Datum für die Bewertung ist in der Zukunft angesiedelt. Da die Analyse im aktuellen Kontext stattfindet, ist dies ein Fehler oder eine unlogische Vorbelegung.

- `"geb_sapnumber": "0850065554"`, `"geb_vatnumber": "40181085800035"`: Unplausible Umsatzsteuer-Identifikationsnummer

Französische Umsatzsteuer-Identifikationsnummern haben in der Regel ein Format wie "FRXX123456789". Der bereitgestellte Wert entspricht nicht diesem Standard.

Mögliche Validierungsregeln:

1. Datum in *_modifiedon-Feldern sollte immer kleiner oder gleich dem aktuellen Datum sein.
2. VAT-Nummern sollten einem länderbezogenen Format (z. B. EU VAT Validator Standards) entsprechen.

Zusammenfassung allgemeiner Datenqualitätsprobleme:

1. Unklare Formatierung von Codes:

Country-Codes verwenden unbekannte interne Werte anstelle standardisierter Systeme. Eine Standardisierung ist erforderlich.

2. Inkonsistente und unplausible Attribute:

Widersprüche zwischen Besuchen, Kontakten und Status-Codes treten häufig auf und sollten validiert werden.

3. Fehlerhafte Datumswerte:

Zukünftige Datumswerte treten auf und sollten verhindert werden.

4. Unklare Feldbenennung und Struktur:

Inkonsistente Strukturen bei Namen oder Business-Typen erschweren die Interpretation und Verarbeitung.

Generierte Validierungsregeln:

1. address1_city und address1_postalcode müssen geografisch zueinander passen.
2. Telefonnummern sollten international standardisiert gespeichert werden.
3. *_modifiedon-Felder dürfen keine zukünftigen Zeitangaben enthalten.
4. statecode und statuscode sollten eindeutig dokumentiert und auf Redundanz geprüft werden.

Kontextuelle Bewertung einzelner Datensätze

Frage: *Bewerte die Qualität der folgenden Datensätze im Kontext eines CRM-Systems. Welche Datensätze erscheinen problematisch oder unplausibel und warum? Gehe dabei nicht nur auf fehlende Werte ein, sondern insbesondere auf semantische Auffälligkeiten, ungewöhnliche Muster und mögliche Test- oder Dummy-Daten. Ordne deine Bewertung nachvollziehbar ein.*

Antwort: Um die Datenqualität der angegebenen Datensätze im Kontext eines CRM-Systems zu bewerten, werden Auffälligkeiten in den Kategorien semantische

Inkonsistenzen, ungewöhnliche Muster sowie Test- oder Dummy-Daten betrachtet. Die Bewertung erfolgt je Datensatz:

Datensatz 1: "Starching"

Auffälligkeiten:

address1_city (BARONA): Die Adresse enthält ein italienisches Format („Ripa di Porta Ticinese, 75“), was gut mit Telefonnummer und Website (starching.it) übereinstimmt. Allerdings ist die Stadt als „BARONA“ angegeben, was kein eigenständiger Stadtname ist, sondern ein Stadtteil von Mailand. Die Angabe ist semantisch ungenau.

geb_contactsthisyearaccount und geb_contactslast12monthsaccount: Beide Werte sind 0. Für ein scheinbar aktives Unternehmen (Adresse, Telefonnummer und Website vorhanden) ist dies unplausibel.

geb_countrycode: Der Wert 100000101 ist ohne Mapping nicht interpretierbar und daher semantisch unsicher.

Bewertung: Moderate Unstimmigkeiten. Insbesondere die Stadtangabe und die fehlenden Kontaktaktivitäten widersprechen dem sonst aktiven Erscheinungsbild.

Datensatz 2: "Walter Tippers (Ashbourne)"

Auffälligkeiten:

geb_rating und verwandte Felder: Alle Ratings sind auf denselben Wert gesetzt (100000008). Die Gleichförmigkeit deutet auf Standardwerte oder Testdaten hin.

geb_visitsperyear: Der Wert ist 2. In Kombination mit durchgehend hohen Ratings wirkt dies inkonsistent.

geb_businessstypecode: Mehrere Werte in einem Feld („100000000;100000001;100000002“) sind untypisch und deuten auf fehlerhafte Modellierung hin.

geb_contactsthisyearaccount und geb_contactslast12monthsaccount: Keine Kontakte trotz scheinbar aktivem Datensatz.

Bewertung: Stärker fehlerhaft. Die Kombination aus Standardwerten, Mehrfachcodierung und geringer Aktivität deutet auf Testdaten oder schlechte Datenpflege hin.

Datensatz 3: "UPR Gesellschaft für Wohn- und Gewerbebau GmbH"

Auffälligkeiten:

name und geb_nametwo: Der Firmenname ist künstlich aufgeteilt („UPR Gesellschaft für Wohn-“ / „und Gewerbebau GmbH“). Dies ist unüblich und erschwert die Nutzung.

geb_visitsperyear: Wert 0 deutet auf fehlende Aktivität hin, was für eine GmbH untypisch ist.

Datenveraltetheit: Das Änderungsdatum des Ratings liegt in der Vergangenheit (2020) und deutet auf veraltete Daten hin.

Bewertung: Problematisch. Aufgesplitteter Name, fehlende Aktivität und veraltete Daten sprechen für schlechte Pflege.

Datensatz 4: "(passiv) Taconova GmbH"

Auffälligkeiten:

name: Der Zusatz „(passiv)“ im Namen ist unüblich und gehört in ein Statusfeld, nicht in den Firmennamen.

geb_contactsthisyearaccount und geb_contactslast12monthsaccount: Beide Werte sind 0, was zur Passiv-Kennzeichnung passt, aber nicht sauber modelliert ist.

statuscode: Der Wert ist ohne Kontext schwer interpretierbar.

Bewertung: Moderate Probleme. Der Datensatz ist grundsätzlich konsistent, aber semantisch unsauber modelliert (Status im Namen statt im Feld).

Datensatz 5: "AIN JURA CHAUFFAGE"

Auffälligkeiten:

geb_ratingmodifiedon: Das Datum liegt in der Zukunft und ist damit eindeutig fehlerhaft.

geb_contactsthisyearaccount und geb_contactslast12monthsaccount: Keine Kontakte trotz vorhandener Aktivitätsindikatoren.

Bewertung: Stark fehlerhaft. Das zukünftige Datum ist ein klarer Datenfehler, kombiniert mit unplausibler Aktivität.

Zusammenfassung:

"Starching": moderat fehlerhaft - semantische Inkonsistenz und fehlende Aktivität

"Walter Tippers (Ashbourne)": stark fehleranfällig - mögliche Testdaten, unstrukturierte

Felder "UPR Gesellschaft": problematisch - Namensaufteilung, veraltete Daten

"(passiv) Taconova GmbH": moderat fehlerhaft - Status falsch modelliert "AIN JURA

CHAUFFAGE": stark fehlerhaft - zukünftiges Datum und Aktivitätsprobleme

Fazit:

Die Analyse zeigt typische CRM-Datenqualitätsprobleme: semantische Inkonsistenzen, unstrukturierte Mehrfachwerte, veraltete Daten, fehlerhafte Datumswerte sowie Hinweise auf Test- oder Dummy-Daten. Diese sollten gezielt durch Validierungsregeln und Data Cleansing adressiert werden.

Frage: *Diese Daten wurden zunächst als potenzielle Test-Datensätze im Produktivsystem identifiziert. Beurteile ob das stimmt. Gib Argumente dafür und dagegen.*

Antwort: Beurteilung der Daten und Argumentation:

Die Frage ist, ob die bereitgestellten Datensätze als potenzielle Test-Datensätze im Produktivsystem identifiziert werden können. Nach Analyse der bereitgestellten Datensätze werden folgende Argumente dafür und dagegen formuliert:

Argumente dafür (Test-Datensätze):

1. Semantische Hinweise in den Namen:

Die Namen enthalten explizite Hinweise auf Testdaten, z. B.: "Default CRM - MÅ IKKE BENYTTES" („darf nicht genutzt werden“) "#prefab test Account PL" "#Piping test-New Employee Testing 1" Dies spricht stark für Testzwecke.

2. Geringer Interaktionsumfang:

Attribute wie geb_contactsthisyearaccount und geb_contactslast12monthsaccount sind durchgehend 0. Für reale Accounts ist dies unplausibel. Auch andere Felder wirken zwar befüllt, aber ohne tatsächliche Nutzung.

3. Muster bei IDs und Werten:

IDs wirken generisch oder ohne klare externe Referenz. Adressen wiederholen sich oder wirken generisch gewählt (z. B. gleiche Straße mehrfach).

4. Inkonsistente Statuscodes:

Unterschiedliche statuscode-Werte können auf bewusst getestete Szenarien hinweisen (z. B. Workflow-Tests).

5. Ungewöhnliche Feldkombinationen:

Beispiel: donotbulkemail = true in Testdatensatz. Solche Kombinationen wirken künstlich und eher für Testszenarien erstellt.

Argumente dagegen (keine reinen Test-Datensätze):

1. Hohe Plausibilität der Daten:

Viele Daten wirken realistisch:

- Ländercodes entsprechen plausiblen Ländern
- Adressen sind vollständig und geografisch korrekt (inkl. Koordinaten)

2. Telefonnummern vorhanden:

Telefonnummern sind formal korrekt und enthalten passende Ländervorwahlen.

3. Reale Lookup-Referenzen:

GUID-Felder (z. B. geb_maintypeid, geb_responsibilityid) verweisen auf existierende Objekte im System. Dies spricht dafür, dass echte Referenzdaten verwendet wurden.

Bewertung:

Die klare Kennzeichnung über Namen (z. B. „test“, „Default“, „nicht nutzen“) sowie die fehlende Aktivität sprechen stark dafür, dass es sich um Test-Datensätze handelt.

Gleichzeitig zeigen viele Attribute (Adressen, Telefonnummern, Referenzen), dass die Daten bewusst realitätsnah modelliert wurden.

Daher ist es wahrscheinlich, dass es sich um realitätsnahe Testdaten handelt, die auf produktionsnahen Strukturen basieren.

Empfehlungen:

1. Namensvalidierung:

Erkennung von Begriffen wie „test“, „Default“, „Dummy“ und entsprechende Kennzeichnung oder Ausschluss.

2. Aktivitätsprüfung:

Produktive Accounts sollten mindestens minimale Aktivität (z. B. Kontakte > 0) aufweisen.

3. Kombinationsregeln:

Inkonsistente Kombinationen wie aktive Kontaktdaten bei gleichzeitig 0 Interaktionen flaggen.

4. Referenzvalidierung:

GUIDs gegen gültige Referenzlisten prüfen.

5. Technisches Audit:

Überprüfung durch Systemanalyse oder Nutzerhistorie, um Testdaten eindeutig zu identifizieren.

Fazit:

Die Annahme, dass es sich um Test-Datensätze handelt, ist sehr plausibel. Die Daten wirken jedoch bewusst realitätsnah modelliert, was typisch für Testdaten in produktionsnahen Umgebungen ist.

Dubletten / Semantische Ähnlichkeit

Frage: *Prüfe, ob die folgenden Datensätze dieselbe reale Entität darstellen könnten.*

Berücksichtige dabei auch leichte Abweichungen oder Schreibvarianten. Beurteile die Wahrscheinlichkeit und begründe deine Einschätzung.

Antwort: Analyse der Datensätze im Hinblick auf dieselbe reale Entität:

Nachfolgend wird untersucht, ob die bereitgestellten Datensätze dieselbe reale Entität darstellen könnten. Dabei werden Ähnlichkeiten und Unterschiede in Attributen wie name, Adresse, Telefonnummer und geb_accountnumber berücksichtigt.

1. Ähnlichkeit basierend auf Namen:

Die Namen der Entitäten unterscheiden sich größtenteils deutlich:

“Prodcom Imex Tody” “Metallurgical Corp Of China MCC” “KOC” “Ared” “Uti Building Facility Management” “Victoria International Proper (Vip)” “Uti Instal Construct” “Neocity”

Es gibt keine direkte Übereinstimmung. Auffällig ist jedoch, dass “Uti Building Facility Management” und “Uti Instal Construct” denselben Namensbestandteil (“Uti”) enthalten, was auf eine Zugehörigkeit zur gleichen Unternehmensgruppe hindeuten kann.

2. Ähnlichkeit in Adressdaten:

Poetului, Arad, Rumänien: “Prodcom Imex Tody” und “Ared” haben identische Adresse und identische Koordinaten (longitude=21.30454, latitude=46.19645). → Sehr hohe Wahrscheinlichkeit gleicher Standort, ggf. gleiche oder eng verbundene Entität.

Ahmadi, Kuwait: “Metallurgical Corp Of China MCC” und “KOC” teilen Stadt und Koordinaten. → Möglicherweise gleiche Organisationseinheit oder verbundene Unternehmen, jedoch keine klare Namensübereinstimmung.

Bucharest, Rumänien: “Uti Building Facility Management” und “Uti Instal Construct” haben nahezu identische Adresse (“107-111 Oltenitei Bvd.. District 4”) und gleiche Koordinaten. → Sehr starke Übereinstimmung.

“Victoria International Proper (Vip)” und “Neocity” haben unklare Adressen (“Not Known”), aber ähnliche geografische Lage. → Keine eindeutige Zuordnung möglich.

3. Ähnlichkeit in Telefonnummern:

“Uti Building Facility Management” und “Uti Instal Construct” haben exakt dieselbe Telefonnummer (+40 31 407 9303). → Sehr starkes Indiz für gleiche Entität oder zumindest sehr enge Verbindung.

Alle anderen Telefonnummern sind unterschiedlich.

4. Ähnlichkeit in geb_accountnumber:

Alle Datensätze besitzen unterschiedliche geb_accountnumber-Werte. → Formal unterschiedliche Datensätze im System, keine direkte Dublette auf Schlüsselbasis.

5. Zusätzliche Beobachtungen:

“Uti Building Facility Management” und “Uti Instal Construct”: Gleiche Telefonnummer, gleiche Adresse und ähnliche Namen → sehr hohe Wahrscheinlichkeit, dass es sich um dieselbe Entität oder zumindest dieselbe Unternehmensgruppe handelt.

“Prodcom Imex Tody” und “Ared”: Gleiche Adresse und Koordinaten, aber unterschiedliche Telefonnummern → möglicherweise unterschiedliche Firmen am selben Standort oder Dubletten.

Zusammenfassung und Wahrscheinlichkeit:

Prodcom Imex Tody & Ared: Hohe Wahrscheinlichkeit - identische Adresse und Koordinaten, aber unterschiedliche Namen und Telefonnummern.

Metallurgical Corp Of China MCC & KOC: Mittlere Wahrscheinlichkeit - gleiche Lage, aber stark unterschiedliche Namen.

Uti Building Facility Management & Uti Instal Construct: Sehr hohe Wahrscheinlichkeit - gleiche Telefonnummer, gleiche Adresse, ähnliche Namen.

Victoria International Proper (Vip) & Neocity: Geringe Wahrscheinlichkeit - unklare Adressdaten und keine weiteren Übereinstimmungen.

Andere Kombinationen: Geringe Wahrscheinlichkeit - keine relevanten Übereinstimmungen.

Validierungsregeln:

1. Telefonnummernregel:

Datensätze mit identischer Telefonnummer und ähnlicher Adresse sollten als potenzielle Dubletten markiert werden.

2. Geografische Nähe:

Datensätze mit identischen oder nahezu identischen Koordinaten sollten überprüft werden.

3. Namensähnlichkeit:

Gemeinsame Namensbestandteile (z. B. “Uti”) in Kombination mit gleichen Kontaktdaten erhöhen die Dublettenwahrscheinlichkeit.

4. Unklare Adressen:

Einträge wie “Not Known” erschweren die Bewertung und sollten gesondert behandelt oder bereinigt werden.

Fazit:

Es gibt mehrere starke Hinweise auf potenzielle Dubletten bzw. eng verbundene Entitäten, insbesondere bei identischen Telefonnummern und Adressen. Gleichzeitig zeigen unterschiedliche Schlüsselattribute, dass die Datensätze systemseitig getrennt

geführt werden, was auf mögliche Dubletten oder bewusst getrennte Einheiten hinweist.

Kontextbasierte Regelableitung

Frage: *Leite aus den folgenden Daten sinnvolle Datenqualitätsregeln ab. Die Regeln sollen nicht trivial sein (z. B. nicht keine Nullwerte), fachlich begründet sein und auf den konkreten Daten basieren. Formuliere die Regeln möglichst konkret und überprüfbar.*

Antwort: Auf Basis der bereitgestellten Daten und der analysierten Attributwerte sowie ihrer semantischen Bedeutung ergeben sich folgende fachlich begründete und überprüfbare Datenqualitätsregeln:

1. Plausibilität von statecode und statuscode:

Wenn statecode = 1 (inaktiv), muss statuscode einen eindeutig inaktiven Zustand repräsentieren und darf keinen aktiven Status widerspiegeln.

2. Konsistenz von Ländercode und Telefonnummer:

Der geb_countrycode muss mit der internationalen Vorwahl der Telefonnummer (telephone1) übereinstimmen.

3. Konsistenz von Adresse und Geokoordinaten:

Wenn Adressfelder (address1_line1, address1_city, address1_postalcode) befüllt sind, müssen auch gültige und zur Adresse passende Geokoordinaten vorhanden sein.

4. Konsistenz von Geschäftstyp und Aktivität:

Wenn geb_businessstypecode mehrere Werte enthält, muss mindestens eine Interaktion vorliegen (geb_contactsthisyearaccount > 0 oder geb_contactslast12monthsaccount > 0).

5. Konsistenz von Name und Status:

Wenn statecode = 1 (inaktiv), darf der Status nicht nur implizit im Namen codiert sein, sondern muss eindeutig über Statusfelder abgebildet werden.

6. Konsistenz von Postleitzahl und Stadt:

address1_postalcode muss zur angegebenen Stadt passen und mit offiziellen Referenzdaten übereinstimmen.

7. Zeitliche Plausibilität:

Zeitstempel wie geb_ratingmodifiedon dürfen nicht in der Zukunft liegen.

8. Konsistenz von Status und Kontaktinformationen:

Für inaktive Accounts sollten Kontaktinformationen (telephone1, emailaddress1) entweder leer oder als inaktiv markiert sein.

Zusammenfassung: Diese Regeln adressieren insbesondere semantische Inkonsistenzen, zeitliche Fehler, geografische Inkonsistenzen sowie widersprüchliche Kombinationen von Attributen und gehen damit über einfache Vollständigkeitsprüfungen hinaus.

Frage: *Leite aus den folgenden Daten ausschließlich tabellenübergreifende sinnvolle Datenqualitätsregeln ab und prüfe ob sich funktionale Abhängigkeiten identifizieren lassen. Die Regeln sollen nicht trivial sein, fachlich begründet sein und auf den konkreten Daten basieren.*

Antwort: Aus den gegebenen Relationen zwischen Account, Contact, geb_projectparticipation, Appointment und opportunity lassen sich folgende tabellenübergreifende Regeln und funktionale Abhängigkeiten ableiten:

1. Beziehung Account → Contact:

Jeder Contact.parentcustomerid muss auf einen existierenden Account.accountid verweisen.

Zusatzregel: Die Telefonnummer eines Kontakts sollte nur dann mit der des Accounts übereinstimmen, wenn es sich um den Hauptkontakt handelt.

Funktionale Abhängigkeit: (parentcustomerid, firstname, lastname) → contactid

2. Beziehung Account → geb_projectparticipation:

Jede geb_projectparticipation.geb_accountid muss einem existierenden Account entsprechen.

Zusatzregel: Die Kombination (geb_accountid, geb_opportunityid) muss eindeutig sein, da ein Account an einem Projekt nur einmal beteiligt sein kann.

3. Beziehung geb_projectparticipation → opportunity:

Jede geb_projectparticipation.geb_opportunityid muss auf eine existierende opportunity.opportunityid verweisen.

Zusatzregel: Die Adresse eines Projekts sollte mit der Adresse des zugehörigen Accounts konsistent sein.

4. Beziehung Account → Appointment:

Jede Appointment.regardingobjectid muss auf einen existierenden Account verweisen.

Zusatzregeln:

- Termine für denselben Hauptkontakt dürfen sich zeitlich nicht überschneiden
- Der Hauptkontakt eines Termins muss ein gültiger Contact sein und zum gleichen Account gehören

5. Status-Konsistenz über Tabellen hinweg:

Wenn ein Account inaktiv ist (statecode $\neq$ 0), sollten auch die zugehörigen Kontakte inaktiv sein.

Funktionale Abhängigkeiten:

Account: accountid $\rightarrow$ alle Account-Attribute

Contact: (parentcustomerid, firstname, lastname) $\rightarrow$ contactid

Appointment: activityid $\rightarrow$ alle Appointment-Attribute

geb_projectparticipation: geb_projectparticipationid $\rightarrow$ alle Attribute

opportunity: opportunityid $\rightarrow$ alle Attribute

Fazit: Die Regeln zeigen typische Integritätsanforderungen in CRM-Systemen:

referenzielle Integrität, Konsistenz zwischen verbundenen Entitäten sowie funktionale Abhängigkeiten zur Sicherstellung eindeutiger Identifikation und korrekter Beziehungen.

I. Promptsammlung

Im Rahmen der Arbeit wurde generative KI unterstützend in verschiedenen Phasen eingesetzt. Die inhaltliche Bewertung, Auswahl und wissenschaftliche Einordnung erfolgte eigenständig. Die dargestellten Prompts stellen eine exemplarische Auswahl dar und bilden nicht die vollständige Interaktion mit generativer KI im Rahmen der Arbeit ab. Ziel ist es, die Art der Nutzung sowie die unterstützten Arbeitsschritte transparent zu machen.

Initiale Literaturrecherche:

1. Identifiziere relevante Literatur zum Thema . Ich benötige die Literatur für das Grundlagenkapitel meiner Masterarbeit mit dem Ziel, wie die Datenqualität in einer Cloud-Datenplattform sichergestellt werden kann, wofür ich verschiedene Verfahren recherchieren und in einem UseCase gegenüberstellen werde. Deine Aufgabe ist es dabei einen Teil zur initialen Recherche zu diesem Sub Thema beizutragen. Interessante Quellen hierfür sind solche renommierte, die häufig zitiert werden oder auch Neue. Gib mir eine Auflistung der Quellen über Titel und Autor, mit je einem kurzen Satz über den Inhalt sowie ihre begründete Eignung. *(optional: + Hinweis auf Anhang mit Exposé)
2. Du unterstützt mich bei meiner Literaturrecherche meiner Masterarbeit. Ziel der Arbeit ist es, zu untersuchen wie die Datenqualität in einer Cloud-Datenplattform sichergestellt werden kann, wofür ich verschiedene klassische und KI-gestützte Verfahren zur Datenqualitätssicherung recherchiere und für unsere Datenplattform in einem UseCase gegenüberstelle. Deine Aufgabe ist es dabei zum Thema relevante Stichwörter oder Sätze zu identifizieren, die ich für meine Literaturrecherche in unterschiedlichen akademischen Datenbanken nutzen kann. Liefere mir dafür eine Auflistung mit einem kurzen Satz zur Begründung und je eine deutsche und englische Version des Suchvorschlags. *(optional: + Hinweis auf Anhang mit Exposé)

Quellenbearbeitung/Analyse: Im Rahmen meiner Literaturrecherche der Masterarbeit unterstützt du mich beim Zusammenfassen und der Bewertung von Literaturquellen. Ziel der Thesis ist es, zu untersuchen wie die Datenqualität in einer Cloud-Datenplattform sichergestellt werden kann, wofür ich verschiedene klassische und KI-gestützte Verfahren zur Datenqualitätssicherung recherchiere und für unsere Datenplattform in einem UseCase gegenüberstelle. Erstelle mir dafür eine kompakte Zusammenfassung des beigefügten <Papers/Textabschnitts/Kapitels> auf Deutsch.

Füge eine knappe Bewertung zur Quellenwertigkeit bei, in der du beschreibst, wie wichtig und aussagekräftig die Quelle für die Arbeit ist und welche Gedanken dabei nicht vernachlässigt werden dürfen und weiter betrachtet werden sollten. Belege deine Aussagen mit Referenzen in der Quelle, damit ich sie überprüfen kann.

Strukturierte Literaturrecherche/Screening:

Im Rahmen meiner Masterarbeit mit dem Ziel, wie die Datenqualität in einer Cloud-Datenplattform sichergestellt werden kann, wofür ich verschiedene Verfahren recherchieren und in einem UseCase gegenüberstellen werde, führe ich eine Art strukturierte Literaturrecherche durch um einen Methodenkatalog zu bauen. Für meine interne Dokumentation möchte ich in Tabellarischer Form Neben Autor, Jahr und Titel, die Kernaussagen in Bezug auf die Methode zur Datenqualitätssicherung, eine Relevanzeinschätzung, behandelte Datenqualitätsdefinitionen, den Architektonischen Fokus (DWH, Datalake, Lakehouse,...) und mögliche Ausschlussgründe sammeln. Positiv ist eine klar Beschriebene Methodik zur Verbesserung der DQ, der Bezug zu DQ Dimensionen und optional die Anwendung mit Daten zb in einer Fallstudie. Für einen Ausschluss gegen die Aufnahme in den Methodenkatalog spricht zb wenn die Quelle ein rein organisatorisches Framework ohne Verfahren darstellt oder einen konzeptionellen Artikel oder das Verfahren grundsätzlich nicht für eine Datenplattform infrage kommt. Du sollst mir helfen die beigefügte Quelle zu analysieren und in das eingangs definierte Tabellenschema einzuordnen, sodass ich darauf aufbauend die Integration in den Methodenkatalog abwägen kann. Behalte dabei die Antwortstruktur im Tabellenformat bei und extrahiere nur die Kernaussagen, die sich auf das Thema Datenqualitätssicherung bezieht. Nutze für Aussagen den exakten Wortlaut der Quelle und gib die Referenz an, damit ich deine Aussagen und Einordnung nachvollziehen kann.

Übersetzung:

Übersetze den nachfolgenden Text auf Deutsch. Achte dabei auf ein akademisches Sprachniveau und die dennoch möglichst wortgenaue Korrektheit des Inhalts. Vermeide Halbgeviertstrich, Füllwörter und umgangssprachliche Ausdrücke. Gib mir nur den übersetzten Text ohne weitere Ausführungen zurück.

Formulierungshilfe:

1. *Synonyme*: Gib mir eine Auflistung von Synonymen zum Wort . Der Kontext ist . Achte auf ein akademisches Sprachniveau und gib mir zu jedem Synonym-Vorschlag eine kurze aber prägnante Definition.

2. *Sprachliche Glättung*: In meiner Masterarbeit mit dem Thema schreibe ich gerade das Kapitel . Aktuell hab ich folgenden Text/Abschnitt/Absatz in Rohfassung: [TEXT]
Überarbeite diesen Text sprachlich. Achte dabei insbesondere darauf, Wiederholungen zu vermeiden und den Lesefluss zu verbessern. Die inhaltlichen Aussagen sollen dabei vollständig erhalten bleiben und nicht verändert werden.

Strukturierung/Ideenfindung:

Methodenkatalog: Ich plane die methodische Vorgehensweise eines Teils meines Praxisteils meiner Masterarbeit. Unterstütze mich dabei mit einem Strukturvorschlag und methodischer Inspiration. Das Thema meiner Masterarbeit lautet <THEMA>. Den bisher verfassten Stand des Grundlagenkapitels zur Theoretischen Basis wie Dimensionen der Datenqualität oder Datenarchitekturen habe ich im Anhang beigefügt. Der Praxisteil soll neben einer Sammlung aktueller etablierter Methoden sowie KI- und ML-gestützter Verfahren (bis zu einem Data Quality Agent) der aktuellen Forschung zur Sicherstellung von Datenqualität auch deren Bewertung anhand von Kennzahlen im Kontext einer modernen Datenplattform enthalten, anhand der ich dann Methoden für Use Cases in Microsoft Fabric auswählen, testen und evaluieren will. Der Fokus liegt nun auf der Planung und Strukturierung von Methodensuche, Evaluierung und Auswahl, also der Zusammenstellung des Methodenkatalogs. Gib mir hierfür einen Vorschlag zur Orientierung wie sich dies umsetzen ließe und auf welche Aspekte ich insbesondere zu achten habe.

Experimentdesign & Evaluation

1. *Isolation Forest*: Ich beabsichtige, im Rahmen meiner Masterarbeit einen Isolation Forest zu implementieren, um potenzielle Test- oder Entwicklungsdaten im Produktivsystem zu identifizieren. Dafür möchte ich aus dem CRM insbesondere folgende Spalten bzw. daraus abgeleitete Merkmale verwenden, die auf eine geringe Aktivität oder unvollständige Nutzung hindeuten könnten: . In ersten Experimenten zeigt sich jedoch, dass der Algorithmus zunehmend aktive Datensätze als Anomalien erkennt, anstatt die eigentlich gesuchten Testdaten zu isolieren. Gib mir eine fachlich und methodisch nachvollziehbare Erklärung, warum dieses Verhalten auftreten könnte, und nenne konkrete Ansatzpunkte, wie sich der Algorithmus durch eine angepasste Feature-Auswahl, Vorverarbeitung oder Parametrisierung verbessern ließe, um die Erkennung stärker auf das gewünschte Zielbild auszurichten.

2. *Data Agent*: Ich untersuche im Rahmen meiner Masterarbeit den Einsatz eines auf LLM basierenden Data Agents zur Unterstützung von Datenqualitätsprüfungen. Ziel ist es, den Agenten systematisch entlang der zuvor definierten Dimensionen der Datenqualität (Vollständigkeit & Datenmenge, Konsistenz & Integrität, Korrektheit & Genauigkeit, Relevanz & Kontextbezug, Interpretierbarkeit & Verständlichkeit) zu evaluieren. Gib mir einen Vorschlag zu einer strukturierten Vorgehensweise, wie ich den Agenten sinnvoll entlang dieser Dimensionen testen kann.